TidesDB

TIDESDB MANUAL

A manual for the storage engine

Version 10.0.0

August 2026

CONTENTS

CHAPTER 1

Preface

TidesDB is a transactional key-value storage engine written in C. It is a **library**, linked into your process, storing data in a directory you name.

1.1 What it is not

Being clear about this early saves time:

- **Not a database server.** There is no daemon, no port, no wire protocol, no authentication.
- **Not a query engine.** There is no SQL, no query planner, no secondary indexes. You get keys, values, ordered iteration, and transactions.
- **Not distributed.** No replication, no consensus, no sharding.

It is the layer such systems are built *on*. If you are building a database, a message queue, a metadata store, or a storage layer for an application, TidesDB is intended to be the part that makes bytes durable and gets them back in order.

1.2 What it is

- **Transactional**, with five isolation levels from read-uncommitted to serializable, and two-phase commit for participating in distributed transactions.
- **Multi-version**, so readers never block writers and writers never block readers.
- **Crash-safe**, with a write-ahead log and a recovery path exercised on every open rather than only after a crash.
- **Log-structured**, optimised for write throughput, with compaction keeping read cost bounded.
- **Portable**, across Linux, macOS, Windows, the BSDs, and Illumos, on x86, ARM, RISC-V, and PowerPC.

1.3 Who this manual is for

If you are embedding TidesDB, read Getting Started (ch. 2), then Concepts (ch. 3), then keep the C API Reference (ch. 10) open.

If you are operating something built on it, Administration (ch. 6) covers building, configuration, operations, and monitoring.

If you are modifying the engine, Internals (ch. 16) is written for you. Start with the two narrative chapters — the life of a write and the life of a read — before any subsystem chapter, and read Invariants (ch. 31) before changing anything.

1.4 Conventions

Error sets are exhaustive. Where the reference lists the errors a function returns, that list is what the implementation actually produces, verified against the code rather than transcribed from a header comment. “Or an error code” is not a contract and does not appear.

Ownership is always stated. A pointer you receive is either *borrowed* — owned by the engine, valid for a stated period, never freed by you — or *owned*, in which case the reference says what frees it. Memory the engine allocated is released with `tidesdb_free`, never your own `free`.

One threading model covers the whole API: a database handle and a column family handle are shared freely across threads, while a transaction or an iterator belongs to one thread at a time — neither carries a lock of its own, so concurrent calls on one of them are undefined rather than serialized. Where a function departs from that, it says so. Two do: `tidesdb_close` requires that no other

thread is inside a call on the handle, and `tidesdb_txn_request_abort` may be called on a transaction another thread is running, because it stores a flag and leaves that thread to act on it.

Every code sample in this manual is compiled against the real public header as part of the build. A sample that drifts from the API breaks the build rather than misleading you.

1.5 Versions

This manual describes the release it ships with. The version is in `CMakeLists.txt` and `vcpkg.json`; the documentation for a release is the documentation in that release's tree.

Compatibility policy — what a major, minor, and patch release may change, and what the on-disk format guarantees — is in `VERSIONING.md`.

1.6 A note on the writing

Chapters explain *why* before *what*, because a rule you understand is one you can apply to a case the manual did not anticipate. Where a design decision has a cost, the cost is stated; where something is a known weakness, it says so. A manual that only lists strengths is an advertisement, and you cannot operate a system from an advertisement.

Part II

Getting Started

CHAPTER 2

Your First Database

This chapter is one working program, built up a piece at a time. Everything here compiles against the real header — the samples in this manual are compiled as part of the build.

If you have not built TidesDB yet, see Building (ch. 6). The short version:

```
cmake -S . -B build && cmake --build build -j && cmake --install build
```

2.1 The model in five sentences

A **database** is a directory. Inside it are **column families**, each an independent ordered keyspace. All reads and writes happen inside a **transaction** — there is no non-transactional path. Keys and values are arbitrary bytes, ordered by `memcmp`. A transaction sees a consistent snapshot and commits atomically, including across several families.

2.2 Opening

```
#include <tidesdb/db.h>
#include <stdio.h>

int main(void)
{
    tidesdb_config_t config = tidesdb_default_config();
    config.db_path = "/tmp/mydb";

    tidesdb_t *db = NULL;
    int rc = tidesdb_open(&config, &db);
    if (rc != TDB_SUCCESS)
    {
        fprintf(stderr, "open failed: %d\n", rc);
        return 1;
    }

    tidesdb_close(db);
    return 0;
}
```

Start from `tidesdb_default_config()` and change what you need. Only `db_path` has no default. The directory is created if it does not exist, and opening an existing one recovers it.

2.3 Creating a column family

```
tidesdb_column_family_config_t cf_config = tidesdb_default_column_family_config();

int rc = tidesdb_create_column_family(db, "users", &cf_config);
if (rc != TDB_SUCCESS && rc != TDB_ERR_EXISTS) return 1;

tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "users");
```

`TDB_ERR_EXISTS` on the second run is expected, not a failure — creating a family is idempotent if you treat it that way. The handle is **borrowed**: owned by the database, valid until the family is dropped or the database closes. Do not free it.

2.4 Writing

```
tidesdb_txn_t *txn = NULL;
if (tidesdb_txn_begin(db, &txn) != TDB_SUCCESS) return 1;

const char *key = "user:1";
const char *val = "alice";

tidesdb_txn_put(txn, cf, (const uint8_t *)key, 6, (const uint8_t *)val, 5, 0);

int rc = tidesdb_txn_commit(txn);
tidesdb_txn_free(txn);
```

Three things worth noticing:

- The write is **buffered** until commit. Nothing is durable or visible until `commit` returns success.
- The final `0` is the expiry: a **lifetime in seconds**, where zero or negative means never. To expire in an hour, pass `3600`.
- `tidesdb_txn_free` is required **even after a successful commit**. Committing does not free.

2.5 Handling conflicts

The default isolation level is **read-committed**, which does not check for conflicts. A commit at that level succeeds if the I/O succeeds, and a write that races another transaction's write to the same key simply overwrites it. For appending independent records that is exactly what you want. For read-modify-write — a counter, a balance, anything derived from what you just read — it silently loses updates.

Ask for the checking by naming the level:

```
for (int attempt = 0; attempt < MAX_RETRIES; attempt++)
{
    tidesdb_txn_t *txn = NULL;
    if (tidesdb_txn_begin_with_isolation(db, TDB_ISOLATION_SNAPSHOT, &txn) != TDB_SUCCESS) break;

    tidesdb_txn_put(txn, cf, (const uint8_t *)"user:1", 6, (const uint8_t *)"alice", 5, 0);

    int rc = tidesdb_txn_commit(txn);
    tidesdb_txn_free(txn);

    if (rc != TDB_ERR_CONFLICT) break;   /* success, or a real failure */
}
```

At snapshot and serializable, **a commit can fail because another transaction got there first**. That is not an error to log and move past; it is a normal outcome you retry, and the retry must redo the reads as well as the writes, since the reads it was validated against are now stale.

Write this loop once, early. Code that ignores `TDB_ERR_CONFLICT` works perfectly in single- threaded testing and drops writes the moment it is used concurrently.

2.6 Reading

```
tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

uint8_t *value = NULL;
size_t value_size = 0;

int rc = tidesdb_txn_get(txn, cf, (const uint8_t *)"user:1", 6, &value, &value_size);
if (rc == TDB_SUCCESS)
{
    printf("%.*s\n", (int)value_size, value);
    tidesdb_free(value);
}

tidesdb_txn_commit(txn);
tidesdb_txn_free(txn);
```

The value is **yours** and is released with `tidesdb_free` — not your own `free`, because the engine may be built against a different allocator.

Two error codes to distinguish carefully:

- `TDB_ERR_NOT_FOUND` — the key genuinely is not there.
- `TDB_ERR_LOCKED` — transient contention. **Retry.** Treating it as absent is a correctness bug, not a missed optimisation.

2.7 Iterating

```
tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

tidesdb_iter_t *it = NULL;
if (tidesdb_iter_new(txn, cf, &it) == TDB_SUCCESS)
{
    for (tidesdb_iter_seek_to_first(it); tidesdb_iter_valid(it); tidesdb_iter_next(it))
    {
        uint8_t *k = NULL, *v = NULL;
        size_t klen = 0, vlen = 0;

        if (tidesdb_iter_key_value(it, &k, &klen, &v, &vlen) != TDB_SUCCESS) break;

        printf("%.*s = %.*s\n", (int)klen, k, (int)vlen, v);
```

```
            tidesdb_free(k);
            tidesdb_free(v);
        }
        tidesdb_iter_free(it);
    }

    tidesdb_txn_commit(txn);
    tidesdb_txn_free(txn);
```

The iterator reads at the transaction's snapshot, so it is stable — concurrent commits, flushes, and compactions do not disturb it. `tidesdb_iter_valid` is the loop condition; the return value of `next` is not.

Free the iterator before the transaction it came from.

2.8 Prefix scans

There is no separate prefix API. Seek to the prefix and walk while it still matches:

```
tidesdb_iter_seek(it, (const uint8_t *)"user:", 5);

while (tidesdb_iter_valid(it))
{
    uint8_t *k = NULL;
    size_t klen = 0;
    if (tidesdb_iter_key(it, &k, &klen) != TDB_SUCCESS) break;

    const int matches = (klen >= 5 && memcmp(k, "user:", 5) == 0);
    tidesdb_free(k);
    if (!matches) break;

    tidesdb_iter_next(it);
}
```

This works because keys are ordered by `memcmp`, so everything sharing a prefix is contiguous. That same property is why there are no custom comparators: encode your keys to sort correctly as bytes — big-endian integers, sign-flipped signed values — and ordering follows.

2.9 Rules worth learning now

Rule	Why
Free every transaction, even after a successful commit	Commit does not free the handle
Free iterators before their transaction	The iterator borrows it
Use `tidesdb_free` for engine-returned buffers	The engine may use a different allocator
Choose an isolation level deliberately	The default is read-committed, which does not detect conflicts
Retry `TDB_ERR_CONFLICT`	It is the normal outcome of a write race at snapshot isolation
Retry `TDB_ERR_LOCKED`, never treat it as absent	It means "ask again", not "not there"
`ttl` is a lifetime in seconds	The clock starts at the `put`, not at the commit

2.10 Where to go next

Concepts (ch. 3) explains the model properly — isolation levels, durability modes, and key/value separation. The C API Reference (ch. 10) documents every function. Administration (ch. 6) covers running it.

Part III

Concepts

Chapter 3

Data Model

3.1 Keys and values are bytes

A key is a byte string; a value is a byte string. The engine attaches no meaning to either — no types, no schema, no encoding. A key may be any length above zero; a value may be empty.

This is the whole model, and everything else is a property of how those bytes are ordered and when they are visible.

3.2 Ordering is `memcmp`, always

Keys are ordered by unsigned byte comparison. There are no custom comparators, and this is a deliberate design decision rather than a missing feature.

A pluggable comparator would have to be supplied identically on every open, forever, by every process that touches the database. Supply a different one — a fixed bug, a changed locale, a second application — and the sstables are no longer mergeable and the data is silently misordered. The failure is catastrophic, delayed, and unattributable.

Byte ordering removes that class of failure entirely, and costs only that **you encode keys to sort correctly**:

To sort by	Encode as
Unsigned integer	Big-endian, fixed width
Signed integer	Big-endian with the sign bit flipped
Descending anything	Invert the bytes
Timestamp then id	Big-endian timestamp, then the id, concatenated
Text, case-insensitive	Normalise before storing

A composite key is just concatenation, and because ordering is lexicographic, a prefix scan over the leading component works with no extra machinery — which is what makes prefix iteration (ch. 2) fall out for free.

3.3 Column families

A database holds one or more **column families**, each an independent ordered keyspace. The same key may exist in several families with different values.

Each family has its own on-disk levels, its own sstables, and its own compaction policy and triggers, so families with different access patterns do not degrade each other: a write-heavy family's compaction does not inflate a read-heavy family's read amplification.

What families **share** is what makes them cheap and what makes cross-family transactions atomic: one memtable, one write-ahead log, one value log, one block cache, one sequence clock.

That sharing has visible consequences worth knowing before you design around families:

- A flush writes out **every** family's data, because there is one memtable.
- **Cloning** a family costs a database-wide flush, because a clone copies files and everything it should copy has to be in a file first. **Renaming** costs none: a family's key logs are named for its id rather than its name, so a rename moves nothing on disk and changes only the registry index and the persisted configuration.
- Configuration is per family; a family is the unit at which you tune node size, bloom filters and compaction triggers. Value separation is the exception: the size threshold is database-wide, because the value log is one shared store and the decision keys on how large a value is rather than on which family holds it. A family can opt out of it entirely with `keep_values_inline`.

Use families to separate data with **different shapes or access patterns** — small hot metadata apart from large cold blobs, for instance. Do not use them as a substitute for key prefixes: thousands of families are not the intended shape, and a prefix inside one family costs nothing.

Removing one of those prefixes costs nothing to write either. `tidesdb_txn_delete_prefix` (ch. 12) deletes every key under a prefix as a single entry, so a tenant or a partition leaves in one operation rather than one per key. The space it frees comes back when a compaction next rewrites the range, not at the moment of the call.

A range whose bounds fall where no prefix could put them is `tidesdb_txn_delete_range` (ch. 12), which takes the two bounds directly and costs the same single entry. The prefix form is that call with the one bound a prefix implies, and it is the one to reach for when what you are removing genuinely is a prefix, because it cannot get its bounds wrong.

3.4 Versions

A write does not overwrite. It appends a new version of the key, stamped with the sequence its transaction committed at, and older versions remain until compaction can prove nothing needs them.

That is what makes a snapshot cheap — it is a number, not a copy — and it is why a reader never blocks a writer. It is also why deleting data does not immediately free space: a delete writes a **tombstone**, another version, which shadows what is beneath it until a merge can safely drop both. See Compaction (ch. 24).

A tombstone is the only thing that means *absent*. A value of **zero bytes is a value**: the key is present and a read returns it with a size of zero rather than reporting it missing. That distinction matters for anything whose meaning is carried entirely by the key — a secondary index entry, a set member — which would otherwise have to store a filler byte it does not want.

3.5 Application timestamps

The engine versions by **its own commit sequence**, and reading at a past sequence is what `tidesdb_txn_begin_at_seq` (ch. 12) does. That answers "what did the database look like then." It does not answer "what did the application mean at time T," because the engine attaches no meaning to an application clock.

A timestamp you supply is carried **in the key**, and the `memcmp` ordering is what makes it work:

```
<id> | <inverted timestamp>
```

Invert the stamp — `UINT64_MAX - ts`, big-endian — so a newer version sorts **earlier**. Then the newest version of an id is the first key under `<id>|`, and reading as of a moment is a plain seek:

To read	Seek to
The latest version	`<id>|`
The version as of T	`<id>|` + `invert(T)`

A seek lands on the first key at or after its target, so seeking to `invert(T)` lands on the newest version whose stamp is at or before T. Seeking past the oldest version runs into whatever sorts next, so **check that what you landed on still carries the id's prefix** before reading it as a version — otherwise a neighbouring id answers in its place.

Retiring an id is then one operation, since all its versions share a prefix:

```
/* every version of this id, however many accumulated */
tidesdb_txn_delete_prefix(txn, cf, (const uint8_t *)"doc|", 4);
```

Dropping versions older than a moment is a range delete (ch. 12) over that id's versions, since the bound falls wherever the moment does rather than on a prefix boundary.

3.5.1 What the two axes each give you

They compose, and it is worth being deliberate about which one answers which question. An iterator is created against a transaction, and a transaction can be opened at a past sequence — so an "as of T" seek can run **inside** a consistent point-in-time view of the store:

```
/* the store as it stood at a sequence, and within it the application's own sense of time */
uint64_t seq = tidesdb_oldest_readable_seq(db);
tidesdb_txn_begin_at_seq(db, seq, &txn);
tidesdb_iter_new(txn, cf, &it);
```

The sequence gives cross-key consistency, which an application timestamp cannot on its own — two keys stamped independently were not necessarily written together. The stamp gives meaning the engine's sequence does not carry.

What this costs, against an engine that understood the timestamp natively:

- Reading the latest version is a **seek rather than a get**, so it does not take the bloom filter's exact-key path.
- Old versions live until you delete them. The engine will not collapse them by stamp, only by sequence once nothing can read them.
- A scan across many ids at one moment has to **fold by id as it walks**, because the engine merges by key and each version is a distinct key.

3.6 Expiry

An entry may carry an expiry. Once the clock passes it, the entry reads as absent everywhere, and the space is reclaimed by a later compaction — visibility is prompt, reclamation is lazy.

The clock is a second-granular one the engine publishes on a tick, so an entry stops being visible within about a second of its deadline rather than at the instant it passes. Every source compares against that same published second, so a key never appears to expire in one place and not another.

The expiry is given as a **lifetime in seconds**, and the engine converts it to an absolute deadline at the moment of the `put`. Zero or negative never expires.

3.7 Limits

Limit	Value
Column family name	128 bytes including the terminator
Levels per family	8
Encoding pipeline stages	8
Key size	Non-zero, and under 4 GiB — the sstable node records a key's length in a `uint32`
Value size	Under 4 GiB, and otherwise bounded by memory. A separated value is bounded by the block frame instead — its stored bytes plus a small header must fit a `uint32` — and an inline one by the same node field a key uses

CHAPTER 4

Transactions and Isolation

Every read and every write happens inside a transaction. A single put is a transaction with one operation in it — there is no cheaper path that skips the machinery, because the machinery is what makes the put atomic and durable.

4.1 What a transaction gives you

Atomicity. Either every operation in the transaction becomes visible, or none does — across column families as well as within one.

A stable view. Reads see a consistent snapshot rather than a moving target.

Isolation you choose. Five levels, trading strictness against the cost of enforcing it.

Writes are **buffered** until commit. Your own transaction sees its own writes immediately; nothing else sees any of them until commit succeeds.

4.2 Snapshots

A snapshot is a sequence ceiling: a version is visible if it committed at or below it. Taking one costs nothing — it is a number.

Holding one is not free, though the cost is indirect. The oldest live snapshot is the floor below which compaction may not discard old versions and tombstones. A transaction left open for minutes prevents reclamation of everything that changed since it began, which appears as disk usage that will not fall.

Keep transactions short. This is the main operational rule for using them.

4.3 The five levels

Level	Reads see	Can conflict
`TDB_ISOLATION_READ_UNCOMMITTED`	Everything, including uncommitted writes	No
`TDB_ISOLATION_READ_COMMITTED` *(default)*	The latest committed version, re-read per operation	No
`TDB_ISOLATION_REPEATABLE_READ`	A snapshot frozen at begin	**Yes**, on what it read
`TDB_ISOLATION_SNAPSHOT`	A snapshot frozen at begin	**Yes**, on what it wrote
`TDB_ISOLATION_SERIALIZABLE`	A snapshot frozen at begin	**Yes**, on both

`tidesdb_txn_begin` uses read-committed, and so does a family whose `default_isolation_level` is left unset. **The default therefore does not detect conflicts** — see the division below, and choose a level deliberately if a lost update would matter.

The meaningful division is the last column.

The lower two never fail on conflict. A commit succeeds if the I/O succeeds. A write is a blind overwrite: if another transaction wrote the same key first, that write is simply lost. For appending independent records — logs, events, metrics — that is exactly right, and paying for conflict detection would be waste.

The upper three check, but not all for the same thing, and the difference decides which of them you want:

- **Repeatable read** validates **what it read**. Every key it recorded a read for is checked for a newer committed version, which is what stops a value moving under it. It takes no reservation on what it writes, so two transactions blindly writing the same key still both succeed.

- **Snapshot** validates **what it wrote**, on a first-committer-wins basis. It is the level that stops a lost update, and it does not check reads.
- **Serializable** does both, and adds the check that catches write skew.

Whichever check fires, the loser gets `TDB_ERR_CONFLICT` at commit, before anything durable is written.

> Caution—**Conflict detection is opt-in, and ignoring it loses writes**
>
> The default level does not check, so a race silently overwrites. Once you choose any level above read-committed, code that ignores `TDB_ERR_CONFLICT` passes single-threaded tests and fails under concurrency. Write the retry loop (ch. 2) once and use it everywhere.

4.4 Reading a point in the past

A transaction's snapshot lasts as long as the transaction. When a point in time is wanted for longer — a consistent export, a comparison against a known-good state — name it instead: `tidesdb_snapshot_create` captures the current sequence, and a transaction opened against that snapshot reads as of it, point reads and scans alike.

A snapshot holds the reclamation floor for as long as it lives, which is what keeps the versions it names readable, and is also what it costs. Treat one exactly as you would a long-running transaction: release it when the point in time is no longer wanted, or it pins space the same way.

A sequence can also be read directly, with `tidesdb_txn_begin_at_seq` (ch. 12). That form refuses rather than approximates: an older point stays readable only while something holds the floor under it, and once a merge has run past that sequence the versions it named are gone. Asking for one that has been collected returns `TDB_ERR_TOO_OLD` instead of an answer assembled from whatever survived. `tidesdb_oldest_readable_seq` (ch. 12) says where that boundary currently sits.

4.5 What a write is validated against

This is the part worth understanding, because it determines which reads you should use.

This applies to the levels that validate writes — snapshot and serializable. A write is not validated against your snapshot. It is validated against **the version you actually read for that key** — recorded when you called `tidesdb_txn_get`. A write with no prior read falls back to the snapshot.

So a read-modify-write is checked properly: you read version 5, someone else commits version 6, your write is rejected because the value you based it on is stale.

The consequence is that **reads have costs beyond their own latency**. A read you record widens what your commit must validate, so reading keys you do not base writes on produces conflicts that are not real. Three read entry points exist for this reason:

Call	Records the read	Use for
`tidesdb_txn_get`	Yes	A value that determines what you write
`tidesdb_txn_get_notrack`	No	A probe whose answer does not feed a write
`tidesdb_txn_contains`	No	Existence only; allocates nothing

A uniqueness check before an insert is the canonical `notrack` case: you care that the key is absent, and a concurrent insert of a *different* key is irrelevant to you.

4.6 Savepoints

A savepoint marks a position in the buffered sequence. Rolling back to it discards everything after it and leaves the transaction active; releasing it forgets the mark without discarding anything. They nest.

Useful for speculative work inside a larger transaction — attempt something, and if it does not work out, unwind just that part without losing the rest.

4.7 Two-phase commit

For transactions spanning TidesDB and something else, commit splits in two.

Prepare runs the full conflict check and durably logs the batch under a transaction id you supply, but leaves it invisible and unapplied. If prepare succeeds, the transaction *can* commit — that is the

vote you give your coordinator.

Commit-prepared or **rollback-prepared** applies the decision.

Between the two the transaction holds its snapshot and its key reservations. That is real backpressure: anything contending for those keys is blocked, and the reclamation floor is held down. **Decide promptly.**

A prepared transaction survives a restart. On the next open, `tidesdb_recover_prepared` hands back everything that was prepared and never decided, so a coordinator can finish resolving them. A transaction whose decision *was* logged is settled during open and never appears.

4.8 Practical guidance

Pick a level per workload, not per database. `tidesdb_txn_begin_with_isolation` takes it per transaction, and a column family carries a default for transactions begun against it. Event ingestion at read-committed and account updates at snapshot can coexist in one database.

Keep transactions short, for the reclamation floor. An open transaction holds a snapshot, and nothing above that snapshot can be reclaimed while it lives — so a transaction that is never resolved costs disk for as long as the database is open. Where that cannot be guaranteed by construction, bound it: `txn_timeout_seconds` for every transaction, or `tidesdb_txn_set_timeout` (ch. 12) for one. Expiry is lazy, so the next operation on a stale transaction aborts it and returns `TDB_ERR_TXN_EXPIRED`.

Retry on conflict, retry on locked. Both mean “try again”; neither means “it failed”.

Do not reuse a handle after a failed commit without resetting it. `tidesdb_txn_reset` re-arms a handle without the free-and-allocate cycle, which is worth it in a tight loop.

CHAPTER 5

Durability

When `tidesdb_txn_commit` returns success, something has happened. Exactly *what* is the most consequential configuration choice you will make, and it is one setting: `memtable_sync_mode`.

5.1 The three modes

Mode	On return from commit, the batch is	Survives a process crash	Survives a machine crash
`TDB_SYNC_NONE` *(default)*	Staged in the log's ring, still in this process	No	No
`TDB_SYNC_INTERVAL`	In the operating system's page cache; on the device within the interval	**Yes**	Within the interval
`TDB_SYNC_FULL`	On the device	**Yes**	**Yes**

Read that as a ladder of three questions, each mode answering one more of them. Has the record left this process? Has it left the operating system? Has it left the device? `TDB_SYNC_NONE` answers none of them, `TDB_SYNC_INTERVAL` the first, `TDB_SYNC_FULL` all three.

This is the same ladder every mature engine offers under different names. InnoDB's `innodb_flush_log_at_trx_commit` takes `0`, `2` and `1` for those three rungs, and Berkeley DB takes `DB_TXN_NOSYNC`, `DB_TXN_WRITE_NOSYNC` and its default. If you have tuned either, the mapping is exact.

A clean close loses nothing, in any mode. Closing the database drains the log's ring and writes everything staged before the file is closed, so the weakest mode is not a risk you take on shutdown — only on a crash.

What `TDB_SYNC_NONE` buys is that a commit never waits for a write. That matters more than it sounds: the log is written by one thread sharing a device with flush and compaction, so when that device is busy a single write can take hundreds of milliseconds, and under any mode that waits, every committer inherits that latency. Not waiting removes the coupling outright.

For a great many applications that is the correct trade. A cache, a derived index, an analytics store, anything reconstructible from an upstream source — losing the last moment of writes to a crash costs a rebuild, and waiting on every commit costs throughput continuously.

If you need an acknowledged commit to survive your process dying, use `TDB_SYNC_INTERVAL`. It is the mode that makes that promise, and it does not fsync on the commit path either.

5.2 What `TDB_SYNC_FULL` costs

Commit throughput, because every commit waits for the device rather than returning once the bytes reach the operating system. How much depends entirely on the storage.

The engine narrows the gap: commits coalesce in the log's staging ring, so concurrent committers share the cost of one write rather than each paying separately. The more concurrent your workload, the better `TDB_SYNC_FULL` amortises.

`TDB_SYNC_INTERVAL` is the middle: the log is forced on a timer, so the exposure is bounded by `memtable_sync_interval_us` rather than being unbounded, without a barrier on every commit.

NOTE—**Interval mode syncs the log, not the whole database**

Interval mode batches the **write-ahead log**. The durable base — sstables, value log, manifest — is fsynced under both interval and full modes; only `TDB_SYNC_NONE` skips it. So a crash under interval risks at most the last interval of *commits*, never the integrity of what was already flushed.

NOTE—**A barrier that fails, fails the operation**

Under a syncing mode the fsync is the whole of what durability means, so a barrier that returns an error is reported rather than absorbed: a catalogue commit whose sync fails is a failed commit, and the flush or compaction behind it withdraws what it had recorded instead of going on as though the bytes had landed. Swallowing it would be the worst available outcome — the engine would report the write durable, and a compaction would go on to unlink the inputs a crash would then need.

NOTE—**A large value is durable before the record that names it**

A value at or above `value_separation_threshold` goes to the value log during the commit, and the log record carries only its id. The value is written first, so a crash can leave a value nothing names — reclaimable garbage — but never a record naming a value that is not there. Under a syncing mode both are barriered before the commit is acknowledged.

5.3 Stronger guarantees on demand

The sync mode is a standing policy. Two calls give a stronger guarantee at a moment you choose:

`tidesdb_sync_wal` forces the log to the device. Every commit acknowledged before it becomes machine-crash durable. Cheap, and covers the log only.

`tidesdb_checkpoint` flushes the memtable and forces the value log, the write-ahead log, *and* the manifest, regardless of sync mode. When it returns, everything committed beforehand is on the device. This is the barrier to use before a planned restart, a storage migration, or a filesystem snapshot.

CAUTION—**Closing is not a durability barrier**

`tidesdb_close` cannot report a failure — the shutdown path returns no status, so an I/O error while writing the last of the data is logged, not returned. If you need to know your data reached the device, call `tidesdb_checkpoint` first and check *its* result.

5.4 What a crash actually costs

Recovery replays the write-ahead logs, so the boundary is *what reached the log*, not what reached an sstable.

Situation	Outcome
Process killed, `TDB_SYNC_FULL` or `TDB_SYNC_INTERVAL`	Nothing acknowledged is lost
Process killed, `TDB_SYNC_NONE`	Commits acknowledged but still staged in the ring — this mode acknowledges before the record leaves the process, so a kill takes them with it
Machine lost, `TDB_SYNC_FULL`	Nothing acknowledged is lost
Machine lost, `TDB_SYNC_INTERVAL`	At most the last interval of commits
Machine lost, `TDB_SYNC_NONE`	Commits not yet written out by the OS
Crash mid-flush	Nothing; the data is still in the log, and the partial sstable is an orphan the manifest never named
Crash mid-compaction	Nothing; inputs remain until their replacement is committed

The guarantee recovery makes is an **exact prefix**: everything durable is present, and nothing else is. Not a superset — a torn final record is discarded rather than half-applied. The crash fuzzer tests precisely this property, and it runs at `TDB_SYNC_FULL`.

Under `TDB_SYNC_FULL` and `TDB_SYNC_INTERVAL`, durable and acknowledged are the same set once a barrier has passed, so the prefix is the acknowledged one. Under `TDB_SYNC_NONE` they are not: acknowledgement happens a step earlier, while the record is still staged in this process, so the prefix recovery restores can stop short of what commit already returned success for.

5.5 Backups

`tidesdb_backup` writes a consistent, directly-openable copy into a directory you name, taken at a single manifest snapshot with compaction held off, so it can never reference a file a merge deleted mid-copy. The database stays writable throughout; writes during the backup are not included.

The result is a database directory, not an archive. Point `tidesdb_open` at it.

5.6 Choosing

Ask what losing the last second of writes costs you.

- **Reconstructible from elsewhere** — `TDB_SYNC_NONE`, with periodic `tidesdb_checkpoint` if you want bounded exposure at moments of your choosing.
- **Painful but survivable** — `TDB_SYNC_INTERVAL`, sized to what you can afford to lose.
- **Unacceptable** — `TDB_SYNC_FULL`, and design for concurrency so the coalescing works for you.

Whatever you pick, `tidesdb_checkpoint` before anything deliberately risky, and do not read a successful `tidesdb_close` as evidence your data is safe.

Part IV

Administration

Chapter 6

Building and Installing

TidesDB is a CMake project with no mandatory external dependencies. The compression backends are optional and default on where a package exists; turn all three off and the library builds against nothing but the C standard library and the platform's threads.

6.1 Requirements

- **CMake** 3.25 or newer
- A **C11** compiler with atomics
- A threads implementation (pthreads, or the Win32 equivalent)

Compiler	Minimum	Notes
GCC	7.0	Linux, MinGW, cross-compilation
Clang	6.0	macOS, Linux, BSD
MSVC	2019 16.8	Requires `/experimental:c11atomics`

6.2 Quick build

```
cmake -S . -B build
cmake --build build -j
ctest --test-dir build
```

That gives you a build with tests, the default compression backends, and the platform's default library type. `cmake --install build` installs it.

6.3 Build options

Every option below is set with `-DNAME=VALUE` at configure time.

6.3.1 What gets built

Option	Default	Effect
`TIDESDB_BUILD_TESTS`	`ON`	Unit and integration tests, plus the manual's sample-compile check
`TIDESDB_BUILD_FUZZERS`	`OFF`	Model-based differential, crash/recovery, and concurrency fuzz harnesses
`TIDESDB_BUILD_BENCH`	`OFF`	The `tidesdb_bench` driver — see Testing and tools (ch. 32)
`BUILD_SHARED_LIBS`	Platform	Shared everywhere except Windows, which defaults to static to avoid DLL export complexity

Everything is built position-independent, so a static archive still links into a shared module without a rebuild.

6.3.2 Compression backends

Option	Default	Notes
`TIDESDB_WITH_SNAPPY`	`ON`, `OFF` on SunOS	No OmniOS/Illumos package exists; still overridable
`TIDESDB_WITH_LZ4`	`ON`	
`TIDESDB_WITH_ZSTD`	`ON`	

A build with all three off has no compression dependencies and supports `TDB_COMPRESS_NONE` only. Which backends were compiled in is visible to consumers as `TIDESDB_HAVE_SNAPPY`, `TIDESDB_HAVE_LZ4`,

and `TIDESDB_HAVE_ZSTD` in `<tidesdb/tidesdb_version.h>`.

If you vendor a compression library under a target name the auto-probe cannot guess — via `FetchContent` or `add_subdirectory` — name it explicitly:

```
cmake -S . -B build -DTIDESDB_ZSTD_TARGET=zstd::libzstd_static
```

`TIDESDB_SNAPPY_TARGET` and `TIDESDB_LZ4_TARGET` work the same way. Empty (the default) means probe the known target names, then fall back to `-l<name>`.

6.3.3 Allocators

Option	Default
`TIDESDB_WITH_MIMALLOC`	`OFF`
`TIDESDB_WITH_TCMALLOC`	`OFF`
`TIDESDB_WITH_JEMALLOC`	`OFF`

These are mutually exclusive in practice — pick at most one. Enabling mimalloc also enables the corresponding vcpkg feature.

> Caution—**Allocator choice is part of your ABI**
>
> Buffers returned by TidesDB must be released with `tidesdb_free` (ch. 10), never your own `free`. That is true always, but building against a non-default allocator is what turns “should” into “will crash”.

6.3.4 Development builds

Option	Default	Effect
`TIDESDB_WITH_SANITIZER`	`OFF`	Address + undefined sanitizer
`TIDESDB_WITH_TSAN`	`OFF`	Thread sanitizer
`TIDESDB_WARN_STRICT`	`ON`	The extended warning set on the library target
`TIDESDB_WARN_MAYBE_UNINIT`	`OFF`	`-Wmaybe-uninitialized`; GCC only, needs an optimized build
`TIDESDB_WITH_ALLOC_FAULT`	`OFF`	Refuse a chosen allocation, for walking the out-of-memory paths

`TIDESDB_WITH_SANITIZER` and `TIDESDB_WITH_TSAN` cannot be combined — the two runtimes are incompatible and CMake rejects the combination rather than producing a broken build.

`TIDESDB_WITH_ALLOC_FAULT` redirects the library’s allocations so a chosen one can be refused, which needs a linker that can rewrite a symbol — GNU `ld` and `lld` through `--wrap`. CMake fails the configure on toolchains that cannot, rather than building something that silently never fires. It adds `alloc_fault_tests`, which measures how many allocations a small workload makes and then runs it once per allocation, refusing a different one each time and requiring that the database still opens and still owes back every commit it acknowledged. Pair it with the address sanitizer, where a partial structure abandoned on the way out fails the round instead of passing quietly:

```
cmake -S . -B build-allocfault -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DTIDESDB_WITH_ALLOC_FAULT=ON -DTIDESDB_WITH_SANITIZER=ON
cmake --build build-allocfault -j && ./build-allocfault/alloc_fault_tests
```

6.4 Installing dependencies

Only needed for the compression backends you leave enabled.

6.4.1 Linux

```
# Debian/Ubuntu
sudo apt install cmake build-essential libsnappy-dev liblz4-dev libzstd-dev

# Fedora/RHEL/CentOS
sudo dnf install cmake gcc snappy-devel lz4-devel libzstd-devel

# Arch
sudo pacman -S cmake gcc snappy lz4 zstd
```

6.4.2 macOS

Homebrew is auto-detected — `/opt/homebrew` on Apple Silicon, `/usr/local` on Intel — and only used when a `brew` binary is actually there. MacPorts, pkgsrc, and Fink work by naming their prefix with `MACOS_DEPENDENCY_PREFIX`, which takes precedence over Homebrew detection:

```
brew install cmake snappy lz4 zstd
cmake -S . -B build

# MacPorts
cmake -S . -B build -DMACOS_DEPENDENCY_PREFIX=/opt/local
```

Option	Default	Effect
`USE_HOMEBREW`	`ON`	Auto-detect Homebrew paths. `OFF` to keep them off the include and link lines entirely
`MACOS_DEPENDENCY_PREFIX`	unset	A custom dependency prefix; when set, Homebrew detection is skipped

6.4.3 Windows

```
vcpkg install snappy lz4 zstd
cmake -S . -B build -DCMAKE_TOOLCHAIN_FILE=<vcpkg>/scripts/buildsystems/vcpkg.cmake
```

The repository ships a `vcpkg.json` manifest, so a manifest-mode build resolves the dependencies itself.

6.4.4 BSD and Illumos

FreeBSD, OpenBSD, and DragonFlyBSD keep packages in `/usr/local`; NetBSD uses `/usr/pkg`.

```
# FreeBSD / OpenBSD / DragonFlyBSD
cmake -S . -B build -DCMAKE_PREFIX_PATH=/usr/local

# NetBSD
cmake -S . -B build -DCMAKE_PREFIX_PATH=/usr/pkg
```

On Illumos/OmniOS/Solaris, Snappy is off by default because no package exists.

6.5 Supported platforms

Operating system	Architectures
Linux	x86, x64, ARM64, PowerPC 32-bit, RISC-V 64-bit
macOS	x64, ARM64 (Apple Silicon)
Windows	x86, x64 (MSVC and MinGW)
FreeBSD 14.0+	x64
OpenBSD 7.4+	x64
NetBSD	x64
DragonFlyBSD	x64
Illumos/OmniOS, Solaris	x64

> Note—**Platform notes**
>
> - **PowerPC and 32-bit targets** need `libatomic` for 64-bit atomics, and multilib for 32-bit.
> - **SunOS/Illumos** has no Snappy package, so that backend defaults off.
> - **MSVC** needs `/experimental:c11atomics`, which the build adds for you.

6.6 Using TidesDB from your project

6.6.1 With CMake

The install exports a package, so a consumer needs three lines:

```
find_package(TidesDB REQUIRED)
add_executable(myapp main.c)
target_link_libraries(myapp PRIVATE TidesDB::tidesdb)
```

Linking `TidesDB::tidesdb` brings the include directory with it, so `<tidesdb/db.h>` resolves without any `include_directories` of your own. If TidesDB is installed somewhere unusual, point CMake at it with `-DCMAKE_PREFIX_PATH=/your/prefix`.

6.6.2 Without CMake

```
cc main.c -ltidesdb -o myapp
```

The install ships three headers and nothing else: `<tidesdb/db.h>`, the generated `<tidesdb/tidesdb_version.h>` (which carries `TIDESDB_VERSION` and the `TIDESDB_HAVE_*` codec defines), and `<tidesdb/xxhash.h>`. `db.h` is self-contained — it includes nothing else from the package — so a stale installation of an older TidesDB in the same prefix sits beside it harmlessly.

The engine's internal headers are deliberately not installed. Nothing outside the library is meant to reach them, and shipping them would make every one of them public API under the rule in `VERSIONING.md`.

```
#include <tidesdb/db.h>
```

6.7 Verifying your build

Building is not the same as working. The project ships several checks, and running them is how you know an unusual platform or an unusual option combination actually produced a sound library.

6.7.1 The test suite

```
cmake -S . -B build -DTIDESDB_BUILD_TESTS=ON
cmake --build build -j
ctest --test-dir build --output-on-failure
```

This includes `doc_samples`, which compiles every C sample in this manual against the real public header. A sample that drifts from the API it documents fails the build rather than misleading you.

6.7.2 Under the sanitizers

Worth doing on any platform not in the supported table above, and after any change to the concurrency-sensitive paths.

```
cmake -S . -B build-asan -DCMAKE_BUILD_TYPE=RelWithDebInfo -DTIDESDB_WITH_SANITIZER=ON
cmake --build build-asan -j && ctest --test-dir build-asan

cmake -S . -B build-tsan -DCMAKE_BUILD_TYPE=RelWithDebInfo -DTIDESDB_WITH_TSAN=ON
cmake --build build-tsan -j && ctest --test-dir build-tsan
```

6.7.3 The fuzzers

The strongest check, and the one that exercises durability. The crash harness forks a child that commits under a sync barrier, kills it, reopens the database, and verifies the recovered state is an exact prefix of what was acknowledged.

```
cmake -S . -B build-fuzz -DCMAKE_BUILD_TYPE=RelWithDebInfo -DTIDESDB_BUILD_FUZZERS=ON
cmake --build build-fuzz -j

TIDESDB_FUZZ_ITERS=100 ./build-fuzz/fuzz_standalone   # logical model
TIDESDB_FUZZ_ITERS=100 ./build-fuzz/fuzz_crash        # crash and recovery
TIDESDB_FUZZ_ITERS=50  ./build-fuzz/fuzz_conc         # concurrency
TIDESDB_FUZZ_ITERS=100 ./build-fuzz/fuzz_decode       # decoders, on untrusted bytes
```

What each one verifies, and how to reproduce a failure from its seed, is in Testing and tools (ch. 32).

`TIDESDB_FUZZ_DIR` picks where they work; point it at fast local storage, since each iteration creates and tears down a database.

Caution—**Put fuzz and benchmark data on fast storage**

Per-iteration database teardown is dominated by filesystem work. On spinning disks these harnesses run more than ten times slower, which reads as a hang rather than a slow run.

6.8 Troubleshooting

`find_package(TidesDB)` cannot find the package. Pass the install prefix: `-DCMAKE_PREFIX_PATH=/your/prefix`. The package files land in `<prefix>/lib/cmake/tidesdb`.

Undefined references to atomic builtins. Link `libatomic`. This affects 32-bit targets and PowerPC, where 64-bit atomics are not native.

A compression library is installed but not found. The auto-probe looks for known target and library names. Name yours explicitly with `-DTIDESDB_ZSTD_TARGET=...` (or the Snappy or LZ4 equivalent), or drop the backend with `-DTIDESDB_WITH_ZSTD=OFF`.

Sanitizer build fails to configure. `TIDESDB_WITH_SANITIZER` and `TIDESDB_WITH_TSAN` are mutually exclusive. Use two build directories.

Crashes at the first free of a returned buffer. The buffer was released with the wrong `free`. Use `tidesdb_free` (ch. 10) for anything TidesDB allocated on your behalf.

Operations

TidesDB compacts, flushes, and syncs on its own. Everything here forces work that would otherwise happen on its own schedule, or establishes a guarantee stronger than the configured sync mode.

A well-configured database under normal load needs none of it routinely. Reach for these to take a backup, to make a barrier before something risky, or to move expensive work into a window you choose rather than one the engine chooses.

7.1 Taking a backup

`tidesdb_backup` writes a consistent, directly-openable copy into a directory you name.

It flushes the memtable, then copies the manifest, the value log, and every sstable the manifest references — at a single manifest snapshot, with compaction held off, so the copy can never reference a file a merge deleted mid-copy.

Write-ahead logs are deliberately not copied. The flush at the start put every committed write into the sstables being copied, so a log would add nothing, and a log still being appended to would copy torn. That is also why the copy has nothing to replay when you open it.

- The database **stays writable** throughout.
- Writes committed during the backup are **not** included; the copy is consistent as of the flush at the start.
- Cost and free space needed are proportional to the live on-disk size.

The result is a database directory, not an archive. Restore by pointing `tidesdb_open` at it, or by copying it into place.

```
if (tidesdb_backup(db, "/backups/2026-07-30") != TDB_SUCCESS)
{
    /* the copy is incomplete -- do not treat it as a usable backup */
    fprintf(stderr, "backup failed\n");
}
```

Verify a backup by opening it. That exercises the whole recovery path and is the only check that proves the copy is usable.

7.2 Checkpoints

`tidesdb_checkpoint` flushes the memtable and forces the value log, the write-ahead log, and the manifest to disk **regardless of sync mode**. When it returns, everything committed beforehand is on the device.

Use it before anything that could take the machine down: a planned reboot, a storage migration, a filesystem-level snapshot, a hypervisor operation.

CAUTION—**Close is not a barrier**

`tidesdb_close` returns `TDB_SUCCESS` for any non-`NULL` handle: the shutdown itself reports nothing, so an I/O failure while draining cannot reach you through it. Checkpoint first and check *that* result if you need to know the data landed.

`tidesdb_sync_wal` is the cheaper, narrower version: it forces the log only. It matters most under `TDB_SYNC_NONE`, where a commit is acknowledged while its record is still staged inside this process — forcing the log is what carries those commits out to the device. Under `TDB_SYNC_INTERVAL` it brings the next barrier forward, and under `TDB_SYNC_FULL` it is redundant.

7.3 Forcing compaction

`tidesdb_compact` runs one pass on a family synchronously, merging even when no trigger is due.

It is I/O-heavy in proportion to what it merges and competes with the background pool for the same device. **Forcing it during peak load makes latency worse, not better.** The legitimate uses are a known-quiet window, and after a bulk delete.

It fails fast with `TDB_ERR_LOCKED` if the family is already compacting — checking `tidesdb_is_compacting` first does not prevent that, since a background compaction can start in between. Handle the error instead.

`tidesdb_compact_range` compacts only the sstables overlapping a key range, which is the right tool after deleting a contiguous span: a full compaction would do far more work than the space being reclaimed justifies. Either endpoint may be `NULL` for unbounded, but **both `NULL` is rejected** rather than silently becoming a full compaction.

7.4 Forcing a flush

`tidesdb_flush_memtable` seals the active memtable and writes it out, returning when done.

Because the memtable is shared, this flushes **every** family — there is no per-family flush. Writers are not blocked throughout: a fresh memtable is installed immediately and writes continue into it while the sealed one is written.

Useful before a backup you are taking by other means, or to bound recovery time before a planned restart.

7.5 Reclaiming space after deletes

Deleting does not immediately free space. A delete writes a tombstone, which shadows older versions until a merge can prove nothing survives beneath it.

To reclaim promptly after a bulk delete:

1. `tidesdb_flush_memtable` so the tombstones reach L1.
2. `tidesdb_compact_range` over the deleted span, or `tidesdb_compact` for the whole family.

If space still does not fall, check `min_snapshot_seq` against `global_seq` in the database statistics. A long-running transaction holds the reclamation floor down, and no amount of compaction will release what it pins.

7.6 Growing the descriptor budget

`max_open_sstables` is **cut at open to fit the process descriptor ceiling**, leaving a fixed reserve for the manifest, stdio and temporaries. A budget above what the process allows therefore costs you descriptors you never get rather than `EMFILE` retries, and the cut says so in the log at warn. This bites at the defaults, not only at the extremes: the default budget is 1024 and the usual POSIX default `ulimit -n` is also 1024.

So raising the ceiling is what actually buys the descriptors. Raise it first, then set the budget from what it returns.

```
long ceiling = tidesdb_raise_open_file_limit(8192);

tidesdb_config_t config = tidesdb_default_config();
config.db_path = "/var/lib/myapp/data";
config.max_open_sstables = (size_t)(ceiling / 2);
```

Raising it after opening does not widen an already-open database's budget.

7.7 Column family operations

Cloning flushes the whole database memtable, because it copies files and everything it should capture has to be in one first. It is heavyweight — proportional to the source family's on-disk size — and it stalls flushes database-wide while running.

Renaming does not. A family's files are named for its immutable id rather than its name, so a rename moves nothing, drains no flush, and stalls no committer; it is on the order of tens of microseconds and writes keep landing across it.

Both claim the family against the compaction scheduler so a concurrent DDL or planner pass cannot act on it mid-update, and both report `TDB_ERR_LOCKED` if the family stays busy with compaction for the whole quiesce window.

Dropping destroys data irreversibly and waits out any in-flight compaction on that family. Any handle you hold for it is invalid afterwards.

Monitoring

The statistics calls expose several dozen fields across seven structures. Most are diagnostic detail you will look at once while investigating something. A much smaller set tells you whether the database is healthy, and those are the ones to sample continuously.

Every call fills a caller-owned struct by value. Nothing allocates, nothing needs freeing.

8.1 The five numbers to watch

If you graph nothing else, graph these.

8.1.1 1. `write_stall_ceiling_hits` — from `tidesdb_get_db_stats`

Healthy: zero. Alert on any sustained increase.

This counts writers admitted *only because the backpressure wait ceiling expired* — the engine gave up waiting for flush to drain and let the write through anyway, to keep a stuck flush from becoming a stuck database.

A non-zero value means ingest has outrun flush badly enough that the safety valve is operating. It is the clearest single signal that the write path is in trouble.

8.1.2 2. `immutable_memtable_count` — from `tidesdb_get_db_stats`

Healthy: near zero, spiking briefly.

Sealed memtables waiting to be flushed. Brief spikes under load are normal. A count that sits persistently above zero means flush is not keeping up with ingest, and it is the leading indicator for the stalls above — you will see this climb before writers start being held.

8.1.3 3. Cache `hit_rate` — from `tidesdb_get_cache_stats`

Healthy: depends on your workload; watch the trend, not the value.

A falling hit rate on a stable workload means the working set has outgrown the cache. Because it is cumulative since open, it moves slowly — **compare deltas between two samples** rather than reading the absolute number.

8.1.4 4. `read_amp` — from `tidesdb_get_cf_stats`, per family

Healthy: low and stable.

An estimate of how many sstables a point lookup consults. Rising `read_amp` means compaction is falling behind flush for that family, and it translates directly into read latency.

8.1.5 5. `min_snapshot_seq` versus `global_seq` — from `tidesdb_get_db_stats`

Healthy: a small and stable gap.

`min_snapshot_seq` is the oldest snapshot any live transaction holds. Compaction may not reclaim anything above it. A widening gap means a long-running transaction is pinning old versions, and it is the usual cause of disk usage that will not fall despite deletes.

This is the one people miss, because the symptom — space not reclaiming — looks like a compaction problem rather than an application one.

A **named snapshot** holds the floor the same way and is not distinguishable from a transaction here, because it is one. If the gap is wide and no transaction is outstanding, look for a snapshot that was never released — `tidesdb_snapshot_seq` says which sequence each one is holding, and one of them will match `min_snapshot_seq`.

If the application cannot guarantee every transaction is resolved, bound them rather than watch for it: `txn_timeout_seconds` on the database, or `tidesdb_txn_set_timeout` (ch. 12) on the ones that

might be left open. A bounded transaction is aborted by the next operation on it once its deadline has passed, which releases the snapshot it was holding.

8.2 Write amplification

From `tidesdb_get_cf_stats`, all cumulative since open:

```
write_amp = (wal_bytes_written + flush_bytes_written + compaction_bytes_written)
          / user_bytes_written
```

This is the multiplier between what you wrote and what reached the device. An LSM tree always has some; what matters is the trend and whether it is proportionate to your workload. Rising write amplification on a stable workload usually means compaction triggers are set too eagerly.

8.3 Backpressure, in full

Four fields, and the order tells a story:

Field	Meaning
`writes_throttled`	Writers made to dwell before admission
`writes_blocked`	Writers made to wait for the queue to drain
`write_stall_us`	Total time writers spent held
`write_stall_ceiling_hits`	Writers admitted only because the ceiling expired

All zero means ingest is comfortably within what flush absorbs. Throttling alone is the system working as designed. Blocking means it is at the limit. Ceiling hits mean it has been past the limit long enough that the safety valve fired.

8.4 Space

Field	From	Watch for
`total_data_size_bytes`	db stats	Key-log bytes across every family. The value log is separate — add `vlog_file_size` for the whole store
`vlog_file_size` vs `vlog_used_bytes`	db stats	The gap is reclaimable value-log space
`tombstone_ratio`	cf stats	Persistently high means deletes are outrunning compaction
`unflushed_key_count`	cf stats	Distinct keys only in memory — what a crash would replay

8.5 Sampling

Sample every 10 to 30 seconds. The counters are cumulative, so store deltas as well as values — `hit_rate` and the write-amplification ratio are meaningless as instantaneous readings.

Samples are taken without stopping the engine, so fields can be very slightly inconsistent with each other; a flush may land between two of them being read. They are for monitoring and capacity planning, not for making transactional decisions.

8.6 A minimal health check

```c
/* queue depth above this is a backlog worth reporting, and a snapshot this far behind the
   current sequence means something is holding reclamation back */
#define QUEUE_BACKLOG      4
#define SNAPSHOT_LAG_LIMIT 100000

static uint64_t last_ceiling_hits = 0;

tidesdb_db_stats_t s;
if (tidesdb_get_db_stats(db, &s) == TDB_SUCCESS)
{
    if (s.write_stall_ceiling_hits > last_ceiling_hits)
        fprintf(stderr, "flush is not keeping up with ingest\n");

    if (s.immutable_memtable_count > QUEUE_BACKLOG)
        fprintf(stderr, "flush queue backing up: %d\n", s.immutable_memtable_count);

    if (s.global_seq - s.min_snapshot_seq > SNAPSHOT_LAG_LIMIT)
        fprintf(stderr, "a long-running transaction is holding back reclamation\n");
```

```
    last_ceiling_hits = s.write_stall_ceiling_hits;
}
```

CHAPTER 9

Troubleshooting

Organised by what you observe rather than by subsystem, because that is what you have when something is wrong.

9.1 Writes have become slow or are stalling

Start with `tidesdb_get_stall_stats` (ch. 15). It names where writers waited, so a tail is attributable without a debugger. Compare each reason's longest single wait against the tail you measured — whichever matches is where the time went.

If `wal_append` dominates, compare the two classes in `tidesdb_get_io_stats` (ch. 15). When `sstable` is moving several times the bytes the `wal` is, the log's writes are queued behind flush and compaction, and the lever is write amplification rather than anything in the log itself.

Then check whether the storage can keep up at all, which is worth answering before touching any engine setting. A quick control:

```
dd if=/dev/zero of=<db_dir>/ddtest bs=1M count=12288 conv=fdatasync
```

Compare that figure against the engine's own throughput. Consumer SSDs commonly sustain a fraction of their rated speed once their write cache is exhausted, and a database achieving most of what `dd` achieves is not the thing to fix. If the device is the limit, the lever is **write amplification** — `flush_bytes_written` plus `compaction_bytes_written` over `user_bytes_written` — since halving it halves the traffic competing for the same device.

Look at `write_stall_ceiling_hits`, `writes_blocked`, and `immutable_memtable_count` from `tidesdb_get_db_stats`.

Non-zero stall counters mean ingest is outrunning flush, and writers are being paced deliberately. The queue depth confirms it.

Cause	Fix
Flush cannot keep up	Raise `num_flush_threads`, but only if cores are available
Memtable too small, so rotation is constant	Raise `memtable_write_buffer_size`
Device saturated by compaction	Lower compaction eagerness, or move compaction to a quiet window
Genuinely more write load than the device can take	Nothing configuration can fix

If the counters are all zero, the write path is not the problem — look at your transaction pattern instead. Long transactions, or conflicts causing repeated retries, look like slowness from outside.

9.2 Reads have become slow

Check the block cache is the size you asked for. `tidesdb_get_cache_stats` reports `total_bytes` — compare it against the `block_cache_size` you configured after the cache has had time to fill. A cache settling well under its budget means something other than the budget is binding, and the symptom is a miss rate that no amount of extra configured memory improves.

Look at cache `hit_rate` and per-family `read_amp`.

A falling hit rate means the working set outgrew `block_cache_size`. Remember the cache also holds filter partitions, so a cache too small makes every lookup re-read its filter.

Rising `read_amp` means compaction is falling behind for that family, so a lookup consults more sstables. Check whether `l1_file_count_trigger` is too high, or whether compaction is starved of threads or device bandwidth.

If a workload reads large separated values, a point-lookup-heavy pattern pays a second read per value. Raising `value_separation_threshold` keeps values inline at the cost of a larger btree.

9.3 Disk usage will not fall after deleting

Look at `min_snapshot_seq` against `global_seq`, then `tombstone_ratio`.

The usual cause is not compaction. It is a **long-running transaction** holding the reclamation floor down: compaction may not discard anything above `min_snapshot_seq`, so old versions and tombstones survive no matter how much you compact. A widening gap between the two is the signature.

A transaction that is never resolved holds that floor forever, and nothing else in the engine can decide it is safe to let go of.

A **named snapshot** holds it the same way, by design — it is a registered transaction — so one that is never released is indistinguishable from a leaked one here. Compare `tidesdb_snapshot_seq` on the snapshots you hold against `min_snapshot_seq`; if one of them matches, that is what the floor is waiting on, and no timeout will clear it because a snapshot has none.

If your callers can leak one, bound them: set `txn_timeout_seconds` on the database, or `tidesdb_txn_set_timeout` (ch. 12) on the transactions that might be left open. Expiry is lazy, so a bounded transaction is resolved by the next operation on it rather than by a background sweep.

If the gap is small, the tombstones simply have not been merged deeply enough yet. Flush, then `tidesdb_compact_range` over the deleted span.

Also check `vlog_file_size` against `vlog_used_bytes` — the gap is value-log space held by values nothing references.

If the store is far larger than the data and the `.klog` files are the bulk of it, check what the individual files are sized at. A store whose sstables all sit at exactly the same size is reporting its preallocation extent rather than its data, which points at a build that did not trim itself. Compare a few file sizes against `total_data_size`; a healthy store has files of varied sizes well under the 64 MiB extent, and only the builds currently in flight sit at a whole chunk.

If instead the `.log` files are the bulk of it, look for a transaction left prepared. A prepared batch is durable only in the log it was written to, so that generation's log survives its flush until the batch is decided. Deciding it releases the log. This is worth checking specifically when TidesDB runs under a coordinator that prepares on every commit — MariaDB's internal two-phase commit with the binary log enabled does exactly that.

If none of those explain it, look for orphaned files. A crash part way through writing an sstable leaves a `.klog` the manifest never named, and possibly a `.klog.lstmp` beside it. These are swept at the next open, so a restart reclaims them; a build that fails without the process dying already cleans up after itself. The sweep is deliberately skipped when the log records a self-healed manifest, since that catalogue was rebuilt from the files themselves — if you see that warning and the space does not come back, the leftovers are sstables the rebuild could not read, and they are worth looking at rather than deleting.

A store written by a version before the sweep existed keeps whatever it accumulated until it is reopened once.

9.4 `TDB_ERR_LOCKED` appearing in reads

Reads should rarely produce this. Transient contention — a rotating memtable, a momentarily exhausted descriptor budget — is waited out inside the engine, which wakes the descriptor reaper and rechecks before giving up. Reaching a caller means the pressure outlasted every recheck, so treat it as a signal about the system rather than ordinary noise.

Seeing it at all usually means the **descriptor budget** is genuinely exhausted: an sstable read cannot open its key log even after the reaper has had its chances. Raise the process ceiling with `tidesdb_raise_open_file_limit` before opening and set `max_open_sstables` accordingly, or reduce the number of sstables by compacting.

From the maintenance calls — compact, rename, clone, reconfigure — it means something else entirely, and is ordinary: another exclusive operation holds that family. Those return at once rather than parking you behind a compaction that may run for minutes.

CAUTION—**Never treat it as "not found"**
Code that folds `TDB_ERR_LOCKED` into an absence returns silently wrong answers under load. It means "ask again", not "not there".

9.5 Commits failing with `TDB_ERR_CONFLICT`

Expected at snapshot and serializable isolation whenever two transactions write the same key. Retry the whole transaction.

If the rate is high enough to hurt:

- **Are you reading more than you need?** A tracking `tidesdb_txn_get` widens what the commit validates. Probes that do not feed a write should use `tidesdb_txn_get_notrack` or `tidesdb_txn_contains`.
- **Are transactions too long?** A longer transaction has a wider window to lose a race.
- **Do you need the guarantee?** Append-only workloads are usually correct at `TDB_ISOLATION_READ_COMMITTED`, which never conflicts.

9.6 The database will not open

`TDB_ERR_IO` — check permissions on the directory, and that it is not on a filesystem lacking the operations the engine needs.

`TDB_ERR_CORRUPTION` — a column family configuration blob did not decode. A damaged manifest does *not* produce this; it self-heals and logs at warn level.

It opens but logs a self-heal warning — the manifest was damaged and was rebuilt from the sstables. The data is intact. Investigate the storage; check nothing else is writing into the directory.

9.7 Recovery is slow on open

Recovery replays surviving write-ahead logs. Replay time is proportional to what had not been flushed when the process stopped.

`memtable_write_buffer_size` is the trade: larger buffers mean less frequent flushing and more to replay. If recovery time matters more than write throughput, lower it, or checkpoint before planned restarts.

9.8 Crashes on freeing a returned buffer

The buffer was released with the wrong `free`. Anything TidesDB allocated on your behalf is released with `tidesdb_free` — the engine may be built against a different allocator, and on Windows a library and its caller routinely link separate heaps.

Statistics structures are filled by value and must **not** be freed at all.

9.9 Process memory grows while the engine's numbers stay flat

If resident memory climbs steadily but `memtable_bytes`, the block cache figures and `txn_memory_bytes` are all flat, the growth is in something the engine is holding but not counting. The engine's memory is meant to be bounded by the write buffer, the cache capacity and live transactions, so the three of them flat while the process grows is the signal that one of those bounds is not being enforced rather than that a limit is set too high.

Check `immutable_memtable_count` alongside it. Zero, with writes flowing, means the memtable is not sealing — the rotation threshold reads `memtable_bytes`, so if that number is not moving the seal never fires and the write buffer grows past its configured size.

Worth knowing which shape of workload provokes it, because the shape narrows the cause. Writes spread over distinct keys behave differently from writes concentrated on a few hot keys, since only the second builds long version chains in the write buffer. Delete-heavy traffic differs again, as a tombstone occupies a version while carrying no value. So does a workload writing empty or very small values, which is what secondary index maintenance produces.

A load that stays flat under a uniform key distribution and grows under a skewed one points at version retention rather than at the volume of data. One that stays flat with large values and grows

with small or absent ones points at something being counted by the bytes a caller supplied rather than by what the engine allocated to hold it.

9.10 Getting more detail

Raise `log_level` to `TDB_LOG_TRACE` — the most detailed level — and set `log_to_file`, which routes output to `tidesdb.log` inside the database directory instead of stderr. Bound it with `log_truncation_at` if you leave it on, since a traced engine under load writes steadily. The engine logs recovery decisions, self-heals, slow rotations, flush and compaction completions, and backpressure events.

The sink is process-wide, not per-database. If your process opens several, the last one opened with `log_to_file` owns the file and the others log into it too, until that handle closes.

A slow rotation — over 20 ms — is logged specifically, because it runs on a committing thread and lands directly in write latency where it is otherwise invisible.

Part V

C API Reference

CHAPTER 10

Database Operations

A TidesDB database is a directory. Opening one recovers everything it contains — the manifest, the sstables it lists, and any write-ahead logs left behind by the last run — and starts the background workers. Closing one stops the workers and releases the handle. Everything else in this manual happens between those two calls.

The handle, `tidesdb_t *`, is opaque and thread-safe. One handle serves every thread in the process, and a directory is held by **one handle at a time**.

That is enforced rather than left to the caller. Opening takes an exclusive lock file in the database directory, and a second open — from this process or from another — is refused with `TDB_ERR_LOCKED` rather than being allowed to corrupt the store. Both axes are covered: the lock itself keeps other processes out, and a recorded owner pid keeps the same process out, since `fcntl` locks are per-process and would otherwise let one caller re-lock a directory it already holds. The lock is released last of all on close, so the directory is available again as soon as `tidesdb_close` returns.

10.1 tidesdb_default_config

Return a database configuration filled with defaults.

Synopsis

```
tidesdb_config_t tidesdb_default_config(void);
```

Description

Returns the configuration by value. There is nothing to free. The intended use is to take the defaults and adjust the few fields that matter to you, rather than to zero a struct and fill it in field by field:

```
tidesdb_config_t config = tidesdb_default_config();
config.db_path = "/var/lib/myapp/data";
config.memtable_sync_mode = TDB_SYNC_FULL;
```

`db_path` has no default and must be set. Every other field is usable as returned.

Return Value

Field	Default
`db_path`	`NULL` — **must be set by the caller**
`num_flush_threads`	2
`num_compaction_threads`	2
`log_level`	`TDB_LOG_INFO`
`block_cache_size`	64 MiB
`max_open_sstables`	1024
`log_to_file`	0 (stderr; 1 writes `tidesdb.log` inside `db_path`)
`log_truncation_at`	0 (no truncation)
`memtable_write_buffer_size`	64 MiB
`memtable_skip_list_max_level`	12
`memtable_skip_list_probability`	0.25
`memtable_idle_flush_seconds`	30 (0 disables idle rotation)
`memtable_sync_mode`	`TDB_SYNC_NONE`
`memtable_sync_interval_us`	1000000 (1 second)

Field	Default
`memtable_l0_queue_stall_threshold`	16
`vlog_segment_size`	256 MiB
`txn_timeout_seconds`	0 (no timeout)

Thread Safety

Safe to call from any thread at any time. It reads no shared state.

See Also

`tidesdb_open`, `tidesdb_default_column_family_config` (ch. 11)

10.2 tidesdb_open

Open or create a database.

Synopsis

```
int tidesdb_open(const tidesdb_config_t *config, tidesdb_t **db);
```

Description

Opens the database at `config->db_path`, creating the directory if it does not exist. On an existing database this replays the manifest to rebuild the column family registry and the level sets, reopens the value log, replays every surviving write-ahead log generation back into the memtable, and reseeds the sequence clock past everything durable. Prepared but undecided transactions are recovered into an in-doubt set; see `tidesdb_recover_prepared` (ch. 12).

The configuration is **borrowed, not retained**. `tidesdb_open` copies what it needs, including `db_path`, so the caller may free or reuse the struct as soon as the call returns.

Fields left at zero are resolved to their defaults rather than being taken literally. This applies to `num_flush_threads`, `num_compaction_threads`, `block_cache_size`, `max_open_sstables`, `vlog_segment_size`, `memtable_write_buffer_size`, `memtable_skip_list_max_level`, and `memtable_skip_list_probability`. A zero in any of those means “choose for me”, so a caller who wants a specific small value must state it — there is no way to request zero flush threads, for example. Fields not on that list are used exactly as given, including `memtable_sync_mode`, where zero is the meaningful value `TDB_SYNC_NONE`, and `memtable_idle_flush_seconds`, where zero disables idle rotation.

A database whose manifest is damaged does not fail to open, and which repair runs depends on the damage. Corruption **partway through** keeps the readable prefix and rewrites the log as a clean snapshot. A header that will not validate leaves nothing to keep, so the log is discarded and the catalogue is **rebuilt from the sstables on disk** — the database directory is scanned and each `.klog` whose footer reads back is re-registered under the family id its filename carries, which is what keeps those files reachable rather than orphaned.

A rebuilt family comes back named `cf_` followed by its zero-padded id, with the default configuration, and its sstables are placed at L1, because a file records neither its family’s name nor the level it sat at. See Manifest (ch. 23) for the full limits.

Either repair is reported through the log at `TDB_LOG_WARN`, not through the return value.

Parameters

Parameter	Description
`config`	Database configuration, borrowed. `config->db_path` must be non-`NULL` and non-empty.
`db`	Out parameter. Receives the handle on success; untouched on failure.

Return Value

`TDB_SUCCESS` on success, with `*db` set to a handle the caller closes with `tidesdb_close`.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`config` or `db` is `NULL`, `db_path` is `NULL` or empty, or a path built from `db_path` would not fit the internal path buffer
`TDB_ERR_LOCKED`	Another handle already holds this directory — a second open from this process, or an open from another process. Close the first handle, or point at a different directory
`TDB_ERR_MEMORY`	Allocation failed while building the handle, the block cache, the arena pool, the memtable, or the worker pools
`TDB_ERR_IO`	The database directory could not be created, or the manifest, value log, or a write-ahead log could not be opened
`TDB_ERR_CORRUPTION`	A recovered column family configuration blob did not decode

On any failure the partially built handle is released internally and `*db` is not written, so the caller has nothing to clean up.

Thread Safety

Safe to call concurrently for *different* directories. Two concurrent opens of the **same** directory resolve rather than race: one takes the lock and succeeds, the other is refused with `TDB_ERR_LOCKED`. That holds whether the second caller is this process or another one, so a directory is never opened twice and there is no corruption to guard against by convention.

Notes

`max_open_sstables` is a budget, not a guarantee. Call `tidesdb_raise_open_file_limit` **before** opening if you intend to raise it, because the engine sizes the budget against the ceiling in effect at open time.

Examples

```
tidesdb_config_t config = tidesdb_default_config();
config.db_path = "/var/lib/myapp/data";

tidesdb_t *db = NULL;
int rc = tidesdb_open(&config, &db);
if (rc != TDB_SUCCESS)
{
    fprintf(stderr, "open failed: %d\n", rc);
    return 1;
}
/* ... use db ... */
tidesdb_close(db);
```

See Also

`tidesdb_close`, `tidesdb_default_config`, `tidesdb_raise_open_file_limit`

10.3 tidesdb_close

Close a database and release its handle.

Synopsis

```
int tidesdb_close(tidesdb_t *db);
```

Description

Stops accepting new work, drains the flush and compaction pools, writes out what is still in memory, closes every open file, and frees the handle. After it returns the handle is invalid and must not be used again.

Closing does not wait for a compaction that would only reduce read amplification; it finishes what is needed for durability and stops. Data already acknowledged under the database's sync mode remains durable, and anything still in a write-ahead log is recovered by the next open.

Parameters

Parameter	Description
`db`	The handle to close. Must not be in use by any other thread.

Return Value

`TDB_SUCCESS`, or `TDB_ERR_INVALID_ARGS` if `db` is `NULL`.

Close cannot report a failure. It always succeeds for a non-`NULL` handle; the shutdown path returns no status. An I/O error while writing out the last of the data is logged, not returned. A caller that needs to know its data reached the device must not rely on the close return value — use `tidesdb_sync_wal` (ch. 14) or `tidesdb_checkpoint` (ch. 14) beforehand, or run under `TDB_SYNC_FULL`, where a commit is already durable when it returns.

Thread Safety

Not thread-safe with respect to its own handle. The caller must ensure no other thread is inside any TidesDB call on this handle, and that every transaction and iterator derived from it has been freed, before calling.

See Also

`tidesdb_open`, `tidesdb_checkpoint` (ch. 14)

10.4 tidesdb_raise_open_file_limit

Raise the process's open-file ceiling.

Synopsis

```
long tidesdb_raise_open_file_limit(long desired);
```

Description

Raises this process's descriptor ceiling toward `desired` so that a database can keep more sstables open at once. This is an explicit operator action: **TidesDB never raises the limit on its own.**

On POSIX it raises the `RLIMIT_NOFILE` soft limit toward the hard limit, so it can only reach what the hard limit already permits. On Windows it raises the C runtime's stdio cap, which tops out at 8192.

A failed or partial raise is not an error. The function reports the ceiling actually in effect afterwards, which may be lower than `desired` and may be unchanged.

Call it **before** `tidesdb_open`. The engine sizes `max_open_sstables` against the ceiling in effect at open time, so raising the limit afterwards does not widen an already-open database's budget.

Parameters

Parameter	Description
`desired`	Target descriptor count. A value `<= 0` changes nothing and just reports the current ceiling.

Return Value

The open-file ceiling in effect after the attempt. Compare it against `desired` to find out whether the raise was granted in full.

Thread Safety

Safe to call from any thread, but it changes process-wide state. Call it once during startup rather than from several threads.

Examples

```
long ceiling = tidesdb_raise_open_file_limit(8192);
if (ceiling < 8192)
    fprintf(stderr, "only got %ld descriptors\n", ceiling);

tidesdb_config_t config = tidesdb_default_config();
```

```
config.db_path = "/var/lib/myapp/data";
config.max_open_sstables = (size_t)(ceiling / 2);
```

See Also

`tidesdb_open`

10.5 tidesdb_free

Free a buffer the engine allocated.

Synopsis

```
void tidesdb_free(void *ptr);
```

Description

Releases memory handed back by a TidesDB call that allocates on the caller's behalf — values returned by `tidesdb_txn_get` (ch. 12), the name array from `tidesdb_list_column_families` (ch. 11), and so on. Passing `NULL` is safe and does nothing.

Use this rather than `free` from your own C runtime. The engine may be built against a different allocator — jemalloc, mimalloc, or tcmalloc are all supported build options — and on Windows a library and its caller routinely link separate C runtime heaps. Freeing an engine allocation with the wrong `free` is undefined behaviour in both cases.

Statistics structures are **not** freed this way. `tidesdb_get_cf_stats`, `tidesdb_get_db_stats`, and `tidesdb_get_cache_stats` all fill a caller-owned struct by value and allocate nothing.

Parameters

Parameter	Description
`ptr`	A pointer returned by a TidesDB call that documents the caller as owning it, or `NULL`.

Thread Safety

Safe to call from any thread.

See Also

`tidesdb_txn_get` (ch. 12), `tidesdb_list_column_families` (ch. 11)

Chapter 11

Column Family Operations

A column family is an independent keyspace inside one database. Each has its own levels, its own sstables, and its own compaction policy, so families with different access patterns do not interfere with each other's read amplification. They share the database's write-ahead log, memtable, value log, and block cache, which is what makes a transaction spanning several families atomic.

Keys are namespaced by family, so the same key may appear in several families with different values. Ordering is byte-wise (`memcmp`) in every family; a caller who wants a different order encodes keys to be memcomparable.

Handles returned by `tidesdb_get_column_family` are **borrowed**. They are owned by the database, live until the family is dropped or the database is closed, and must never be freed.

11.1 tidesdb_default_column_family_config

Return a column family configuration filled with defaults.

Synopsis

```
tidesdb_column_family_config_t tidesdb_default_column_family_config(void);
```

Description

Returns the configuration by value; there is nothing to free. The `name` field is left empty, and it does not matter what you put there — every call that takes a config also takes the name separately, and the separate argument wins. See `tidesdb_create_column_family`.

Return Value

Field	Default	Meaning
`name`	empty	Ignored; the `name` argument is authoritative
`level_size_ratio`	10	Each level holds this many times the one above it
`min_levels`	1	Floor on the level count. A family already this deep is never shed further, however little its largest level holds
`dividing_level_offset`	1	How far above the largest level the dividing level sits, so 1 means `X = L - 2`. Lower rewrites more data per dividing merge; higher opens more files at once
`keep_values_inline`	0	Hold every value in the key log whatever its size, ignoring the database's `value_separation_threshold`
`btree_klog_block_size`	4096	Target btree node size in the key log
`encoding_pipeline`	empty	Encodings applied in order to btree klog nodes and separated values — see Configuration (ch. 36)
`encoding_count`	0	Entries in the pipeline
`enable_bloom_filter`	1	Build a partition range filter per sstable
`bloom_fpr`	0.01	Target false positive rate
`default_isolation_level`	`TDB_ISOLATION_READ_COMMITTED`	Level used by `tidesdb_txn_begin_cf`
`l1_file_count_trigger`	4	L1 file count that triggers compaction

Field	Default	Meaning
`tombstone_density_trigger`	0.5	Tombstone fraction that triggers compaction
`tombstone_density_min_entries`	4096	Entry count below which the density trigger is ignored
`commit_hook_fn`	`NULL`	Optional commit hook, registered at create time
`commit_hook_ctx`	`NULL`	Context passed to the hook

Notes

Which values get separated is a database setting, `value_separation_threshold`, not a family one. The value log is a single shared structure and the decision keys on the size of a value rather than on which family holds it, so one cut-off serves a family of small metadata and a family of large blobs alike. See Configuration (ch. 36).

`keep_values_inline` is the family-level exception. Values above the threshold are normally written to the shared value log with the key log holding a reference, which keeps the btree small and makes scans over keys fast, but a point read of a separated value costs a second I/O and a range scan pays it per row. A family that is scanned far more often than it is merged can set this and keep its values whole whatever their size. The cost is the one the threshold exists to avoid, that compaction rewrites those bytes on every merge.

See Also

`tidesdb_create_column_family`, `tidesdb_cf_update_runtime_config`

11.2 tidesdb_compression_available

Report whether this build can use a compression algorithm.

Synopsis

```
int tidesdb_compression_available(tidesdb_compression_algorithm_t type);
```

Description

Every `tidesdb_compression_algorithm_t` enumerator is defined in every build, because the numeric values are written into sstable and value-log metadata and a file written elsewhere has to stay readable. Whether a given backend is actually linked in is a build-time choice — the `-DTIDESDB_WITH_SNAPPY`, `-DTIDESDB_WITH_LZ4` and `-DTIDESDB_WITH_ZSTD` options — so the enum being complete says nothing about what this binary can do.

Algorithm	Value	Requires	Notes
`TDB_COMPRESS_NONE`	`0`	—	Always available; stores the data verbatim
`TDB_COMPRESS_SNAPPY`	`1`	`TIDESDB_WITH_SNAPPY`	
`TDB_COMPRESS_LZ4`	`2`	`TIDESDB_WITH_LZ4`	
`TDB_COMPRESS_ZSTD`	`3`	`TIDESDB_WITH_ZSTD`	Strongest ratio of the four backends
`TDB_COMPRESS_LZ4_FAST`	`4`	`TIDESDB_WITH_LZ4`	The LZ4 backend at a higher acceleration — faster, weaker ratio

The values are the on-disk contract, written into sstable and value-log metadata, so they are fixed and never reused. `TDB_COMPRESS_LZ4_FAST` is the same backend as `TDB_COMPRESS_LZ4` and differs only in its acceleration setting, so the two are available together or not at all.

This is how a caller finds out. It matters because `tidesdb_create_column_family` and `tidesdb_cf_update_runtime_config` both **reject** an `encoding_pipeline` naming an algorithm this build lacks, and both report it as `TDB_ERR_INVALID_ARGS` — the same code a malformed configuration produces. Without this call there is no way to tell those two apart.

Opening a database written by a build that *did* have the codec is a different matter and is allowed: the family's stored configuration is accepted so the database opens, and the gap surfaces as a read error only if a node encoded with that algorithm is actually read.

Parameters

Parameter	Description
`type`	The algorithm to query.

Return Value

`1` if this build can both compress and decompress with the algorithm, `0` otherwise.

Thread Safety

Safe to call from any thread at any time. It reads only build-time state and takes no locks.

Examples

```
tidesdb_column_family_config_t cf_config = tidesdb_default_column_family_config();

if (tidesdb_compression_available(TDB_COMPRESS_ZSTD))
{
    cf_config.encoding_pipeline[0] = (uint8_t)TDB_COMPRESS_ZSTD;
    cf_config.encoding_count = 1;
}

int rc = tidesdb_create_column_family(db, "events", &cf_config);
```

See Also

`tidesdb_create_column_family`, `tidesdb_cf_update_runtime_config`

11.3 tidesdb_create_column_family

Create a column family.

Synopsis

```
int tidesdb_create_column_family(tidesdb_t *db, const char *name,
                                 const tidesdb_column_family_config_t *config);
```

Description

Serializes the family's configuration and registers it in the manifest before publishing it in memory. No directory is created – a family's key logs live in the database directory and name their family by id. The order matters: the manifest commit happens first, so a crash cannot leave a live family the manifest does not know about.

The `name` argument is authoritative. Whatever `config->name` holds is overwritten with `name` before the config is stored, so the two can never disagree on disk.

If `config->commit_hook_fn` is set, the hook is registered as part of the create. See `tidesdb_cf_set_commit_hook`.

The config is borrowed and may be freed as soon as the call returns.

Parameters

Parameter	Description
`db`	Database handle
`name`	Family name, non-empty, at most `TDB_MAX_CF_NAME_LEN` (128) bytes including the terminator
`config`	Configuration, borrowed. Its `name` field is ignored.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db`, `name`, or `config` is `NULL`; `name` is empty; or the configuration failed validation
`TDB_ERR_INVALID_DB`	The database is closing
`TDB_ERR_EXISTS`	A family with this name already exists
`TDB_ERR_MEMORY`	Allocation failed

Code	Cause
TDB_ERR_IO	The manifest record could not be written or committed

An invalid configuration is reported as TDB_ERR_INVALID_ARGS, not as a distinct code, so it is indistinguishable from a NULL argument by return value alone. The log records which field was rejected.

Thread Safety

Safe to call concurrently with other operations on the same database, including creates of differently named families.

Examples

```
tidesdb_column_family_config_t cfg = tidesdb_default_column_family_config();
cfg.keep_values_inline  = 1;   /* scanned constantly, keep values whole */
cfg.enable_bloom_filter = 1;

int rc = tidesdb_create_column_family(db, "events", &cfg);
if (rc == TDB_ERR_EXISTS)
    rc = TDB_SUCCESS;            /* already there, fine */
```

See Also

tidesdb_drop_column_family, tidesdb_get_column_family

11.4 tidesdb_drop_column_family

Drop a column family and delete its data.

Synopsis

```
int tidesdb_drop_column_family(tidesdb_t *db, const char *name);
```

Description

Removes the family from the registry and the manifest and deletes its sstables. **This destroys data and is not reversible.**

The call **waits** for any in-progress compaction on that family to finish before proceeding, polling until the family is quiet. A family under heavy compaction can therefore make this call take a while.

Any column family handle previously obtained for this family is invalid once the drop returns. Using one afterwards is undefined behaviour.

Parameters

Parameter	Description
db	Database handle
name	Family to drop

Errors

Code	Cause
TDB_ERR_INVALID_ARGS	db or name is NULL
TDB_ERR_NOT_FOUND	No family with that name
TDB_ERR_IO	The manifest could not be updated, or files could not be removed

Thread Safety

Safe to call concurrently, but the caller is responsible for ensuring no transaction or iterator is still using the family being dropped.

See Also

tidesdb_create_column_family

11.5 tidesdb_rename_column_family

Rename a column family.

Synopsis

```
int tidesdb_rename_column_family(tidesdb_t *db, const char *old_name, const char *new_name);
```

Description

Renames the family, atomically with respect to readers.

It moves no files. A family's files are named for its immutable id rather than its name (see Formats (ch. 35)), so a rename changes the registry index and the persisted configuration and nothing else. Nothing on disk moves, there is no flush to drain first, and no rebuilding of the family's open sstables — writers and readers run through it untouched. Measured on a live database it is on the order of tens of microseconds, and commits keep landing across it.

It still claims the family against the compaction scheduler, so that a concurrent DDL or planner pass cannot read the configuration while it is replaced. That is a bounded wait, not an indefinite one: if a compaction holds the family for the whole window (up to about 60 seconds), the call gives up and reports `TDB_ERR_LOCKED`.

Parameters

Parameter	Description
`db`	Database handle
`old_name`	Current name
`new_name`	New name, at most `TDB_MAX_CF_NAME_LEN` bytes

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db`, `old_name`, or `new_name` is `NULL`
`TDB_ERR_NOT_FOUND`	No family named `old_name`
`TDB_ERR_EXISTS`	A family named `new_name` already exists
`TDB_ERR_LOCKED`	The family stayed busy with compaction for the whole quiesce window
`TDB_ERR_IO`	The manifest update failed

Thread Safety

Safe to call concurrently, and it does not stall writers. Because nothing on disk has to be kept in step with the name, a rename drains no flush and blocks no committer; it claims the family only so a second DDL and the compaction scheduler cannot act on it mid-update.

See Also

`tidesdb_clone_column_family`

11.6 tidesdb_clone_column_family

Copy a column family to a new name.

Synopsis

```
int tidesdb_clone_column_family(tidesdb_t *db, const char *src_name, const char *dst_name);
```

Description

Creates `dst_name` as a copy of `src_name`, copying the source's sstables. It claims the source family and, unlike a rename, **flushes the database memtable** first — it copies files, so everything written before the call has to be in one, rather than only what had already reached disk.

The destination inherits the source's configuration, minus the commit hook: a hook is a runtime registration rather than persisted state, so a clone does not acquire one.

The clone is a point-in-time copy. Writes to the source after it returns do not appear in the destination.

Because it copies sstables, the cost is proportional to the source family's on-disk size and it needs that much free space.

Parameters

Parameter	Description
`db`	Database handle
`src_name`	Family to copy from
`dst_name`	Name for the new family; must not already exist

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db`, `src_name`, or `dst_name` is `NULL`
`TDB_ERR_NOT_FOUND`	No family named `src_name`
`TDB_ERR_EXISTS`	A family named `dst_name` already exists
`TDB_ERR_LOCKED`	The source stayed busy with compaction for the whole quiesce window
`TDB_ERR_IO`	The flush, a file copy, or the manifest update failed
`TDB_ERR_MEMORY`	Allocation failed

Thread Safety

Safe to call concurrently. Unlike rename, a clone does copy files, so it flushes the database memtable first and stalls flushes while it runs.

See Also

`tidesdb_rename_column_family`, `tidesdb_backup` (ch. 14)

11.7 tidesdb_get_column_family

Look up a column family handle.

Synopsis

```
tidesdb_column_family_t *tidesdb_get_column_family(tidesdb_t *db, const char *name);
```

Description

Returns the handle for `name`, or `NULL` if there is no such family. The handle is **borrowed** — owned by the database, valid until that family is dropped or the database is closed. Do not free it.

`NULL` is returned both for "no such family" and for a `NULL` argument; the two are not distinguishable.

Parameters

Parameter	Description
`db`	Database handle
`name`	Family name

Return Value

The handle, or `NULL`.

Thread Safety

Safe to call concurrently. The returned handle is safe to use from several threads, subject to the rules of whatever call you pass it to.

See Also

`tidesdb_list_column_families`, `tidesdb_txn_begin_cf` (ch. 12)

11.8 tidesdb_list_column_families

List every column family name.

Synopsis

```
int tidesdb_list_column_families(tidesdb_t *db, char ***names, int *count);
```

Description

Allocates an array of newly allocated name strings. **The caller owns both levels** — free each name, then the array itself, using `tidesdb_free` (ch. 10) rather than your own `free`, because the engine may be built against a different allocator.

A database with no families yields `TDB_SUCCESS`, `*names == NULL`, and `*count == 0`. Guard the free loop accordingly.

Parameters

Parameter	Description
`db`	Database handle
`names`	Out. Receives the array.
`count`	Out. Receives the number of names.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db`, `names`, or `count` is `NULL`
`TDB_ERR_MEMORY`	Allocation failed; nothing is returned and there is nothing to free

Thread Safety

Safe to call concurrently. The snapshot is taken under a read lock, so the list is consistent, but a family may be created or dropped before you act on it.

Examples

```
char **names = NULL;
int count = 0;

if (tidesdb_list_column_families(db, &names, &count) == TDB_SUCCESS)
{
    for (int i = 0; i < count; i++)
    {
        printf("%s\n", names[i]);
        tidesdb_free(names[i]);
    }
    tidesdb_free(names);
}
```

See Also

`tidesdb_get_column_family`

11.9 tidesdb_cf_update_runtime_config

Change a column family's configuration while it is open.

Synopsis

```
int tidesdb_cf_update_runtime_config(tidesdb_t *db, tidesdb_column_family_t *cf,
                                     const tidesdb_column_family_config_t *new_config,
                                     int persist_to_disk);
```

Description

Applies a new configuration to a live family. **Every field may change**, including ones that affect how data is written, because byte-wise key ordering means sstables written under different settings remain mergeable. Existing sstables are not rewritten; the new settings apply to what is written from now on.

The family's name and id are preserved — the incoming `name` is ignored. Renaming is `tidesdb_rename_column_family`.

With `persist_to_disk` non-zero the new configuration is written to the manifest and survives a restart. With zero it applies in memory only and is lost on close.

The new configuration is published whole. A flush or compaction already building an sstable continues against the configuration it started with, and the next one picks up the new one — readers of the configuration never see a mixture of the two.

The call **fails fast** rather than waiting: if another exclusive operation holds the family it returns `TDB_ERR_LOCKED` immediately. That is about ordering two writers, not about protecting readers — two reconfigures racing could otherwise install one configuration and persist the other. It returns rather than waits because the holder may be a compaction that runs for minutes, and parking the caller behind one is worse than telling it the family is busy. Retry.

Parameters

Parameter	Description
`db`	Database handle
`cf`	Family handle from `tidesdb_get_column_family`
`new_config`	Configuration to apply, borrowed; its `name` is ignored
`persist_to_disk`	1 to persist in the manifest, 0 for in-memory only

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	A `NULL` argument, or the configuration failed validation
`TDB_ERR_LOCKED`	Another exclusive operation holds the family — retry
`TDB_ERR_IO`	`persist_to_disk` was set and the manifest write failed

Thread Safety

Safe to call concurrently, though two concurrent updates to the same family will see one of them take `TDB_ERR_LOCKED`.

See Also

`tidesdb_default_column_family_config`

11.10 tidesdb_cf_set_commit_hook

Set or clear a column family's commit hook.

Synopsis

```
int tidesdb_cf_set_commit_hook(tidesdb_t *db, tidesdb_column_family_t *cf,
                               tidesdb_commit_hook_fn fn, void *ctx);
```

Description

Installs a callback invoked when a transaction touching this family commits, which is how change data capture is built on TidesDB. Passing `NULL` for `fn` disables it.

The hook receives the committed operations as an array of `tidesdb_commit_op_t` along with the context pointer given here. It runs on the committing thread, so it is on the write path — work done inside it is latency every writer pays.

A hook may also be supplied at create time through the configuration's `commit_hook_fn`, which is equivalent to calling this immediately after `tidesdb_create_column_family`. It is a runtime registra-

tion rather than persisted state, so it does not survive a reopen and a clone does not inherit one.

A hook cannot fail the commit. It fires after the batch is durable and visible, so there is nothing left to undo; a non-zero return is logged and otherwise ignored. Anything the hook must not lose has to be made durable by the hook itself.

Each touched family's hook fires once, with only that family's operations, so a transaction spanning several families invokes several hooks.

Callback

```
typedef int (*tidesdb_commit_hook_fn)(const tidesdb_commit_op_t *ops, int num_ops,
                                      uint64_t commit_seq, void *ctx);
```

It fires after the WAL write, the memtable apply, and the commit-status marking have completed, receiving that family's whole batch atomically. Returning non-zero is logged as a warning and changes nothing else.

Argument	Description
`ops`	The committed operations, valid only for the duration of the call
`num_ops`	Number of entries in `ops`
`commit_seq`	Monotonic commit sequence number assigned to the batch
`ctx`	The context pointer registered alongside the hook

Each entry is a `tidesdb_commit_op_t`:

Field	Description
`key`	Key data
`key_size`	Size of `key` in bytes
`value`	Value data, `NULL` for a delete
`value_size`	Size of `value` in bytes, `0` for a delete
`ttl`	Absolute expiry as a Unix timestamp, or `-1` when the pair never expires
`is_delete`	`1` for a delete, `0` for a put

Two details matter for anything forwarding these operations elsewhere. The key and value pointers address engine memory that is reused once the callback returns, so a hook that retains either must copy it. And `ttl` is the deadline the engine stored, not the lifetime in seconds that `tidesdb_txn_put` (ch. 12) was given — forwarding it verbatim reproduces the same expiry instant instead of restarting the clock at the far end.

Parameters

Parameter	Description
`db`	Database handle
`cf`	Family handle
`fn`	Callback, or `NULL` to disable
`ctx`	Context passed through to the callback; the engine does not interpret or free it

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` or `cf` is `NULL`

Thread Safety

Safe to call concurrently. Clearing a hook does not wait for an in-flight invocation to finish, so the callback and its context must remain valid until the caller has established by its own means that no commit is in progress.

See Also

`tidesdb_txn_commit` (ch. 12)

CHAPTER 12

Transaction Operations

Every read and every write in TidesDB happens inside a transaction. There is no non-transactional path — a single put is a transaction with one operation in it.

Writes are buffered until commit. A transaction that has put a key can read it back and will see its own write, but nothing else will until the commit succeeds. Commit writes the batch to the write-ahead log and then applies it to the memtable, in that order, so a crash between the two leaves the batch recoverable rather than lost.

A transaction handle is **not thread-safe**. One transaction belongs to one thread at a time. Several threads may each hold their own transaction against the same database, which is the normal way to use the engine.

`tidesdb_txn_request_abort` is the single exception, and it is narrow by construction: it stores a flag and changes nothing else, leaving the thread that owns the transaction to act on it. Everything else here still belongs to one thread.

12.1 Isolation levels

The level fixes two things: which committed versions a read can see, and what makes a commit fail.

Level	Value	Reads see	Commit can return `TDB_ERR_CONFLICT`
`TDB_ISOLATION_READ_UNCOMMITTED`	0	Everything, including uncommitted writes	No
`TDB_ISOLATION_READ_COMMITTED`	1	The latest committed version, re-read per operation	No
`TDB_ISOLATION_REPEATABLE_READ`	2	A sequence frozen at begin	**Yes**, if a key it read changed
`TDB_ISOLATION_SNAPSHOT`	3	A sequence frozen at begin	**Yes**, if a key it wrote was written first
`TDB_ISOLATION_SERIALIZABLE`	4	A sequence frozen at begin	**Yes**, on either

`TDB_ISOLATION_READ_COMMITTED` is the default, so **conflict detection is opt-in**. Only the lower two never conflict, and a blind write at `TDB_ISOLATION_READ_COMMITTED` — the default — behaves like an unchecked overwrite, so a lost update is the expected outcome of a race there rather than a defect.

The three above it each check something different. Repeatable read validates its **read set**: a commit fails if any key it recorded a read for has a newer committed version. Snapshot instead takes a first-committer-wins reservation on every key it **writes**, validated against the version the transaction actually read, so a write-write race loses at commit rather than silently overwriting — but it does not check reads. Serializable does both and adds the check for write skew. **Any code using a level above read-committed must handle `TDB_ERR_CONFLICT` by retrying the whole transaction.**

12.2 tidesdb_txn_begin

Begin a transaction at the database default isolation level.

Synopsis

```
int tidesdb_txn_begin(tidesdb_t *db, tidesdb_txn_t **txn);
```

Description

Begins at `TDB_ISOLATION_READ_COMMITTED`, which does not detect conflicts. This is a fixed library default, not a configurable one — the database configuration has no isolation field. To choose a level use `tidesdb_txn_begin_with_isolation`, or `tidesdb_txn_begin_cf` to inherit a column family's default.

The handle must be released with `tidesdb_txn_free` whatever the outcome, including after a successful commit.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` or `txn` is `NULL`
`TDB_ERR_INVALID_DB`	The database is closing, so no new transaction is admitted
`TDB_ERR_MEMORY`	Allocation failed

See Also

`tidesdb_txn_commit`, `tidesdb_txn_free`

12.3 tidesdb_txn_begin_with_isolation

Begin a transaction at an explicit isolation level.

Synopsis

```
int tidesdb_txn_begin_with_isolation(tidesdb_t *db, tidesdb_isolation_level_t isolation,
                                     tidesdb_txn_t **txn);
```

Description

As `tidesdb_txn_begin`, with the level given explicitly. See Isolation levels for what each one costs and guarantees.

Errors

Same as `tidesdb_txn_begin`, plus `TDB_ERR_INVALID_ARGS` for a level outside the enum.

See Also

`tidesdb_txn_reset`

12.4 tidesdb_txn_begin_cf

Begin a transaction at a column family's default isolation level.

Synopsis

```
int tidesdb_txn_begin_cf(tidesdb_t *db, tidesdb_column_family_t *cf, tidesdb_txn_t **txn);
```

Description

Uses `cf`'s `default_isolation_level`. This only picks the level — the transaction is not confined to that family and may read and write any family.

Errors

Same as `tidesdb_txn_begin`, plus `TDB_ERR_INVALID_ARGS` if `cf` is `NULL`.

12.5 tidesdb_snapshot_create

Name the database as it stands now, so it can be read again later.

Synopsis

```
int tidesdb_snapshot_create(tidesdb_t *db, tidesdb_snapshot_t **snapshot);
```

Description

Captures the current sequence and **holds the reclamation floor at it**. That hold is the feature: compaction may not discard versions above the floor, and a flush carries them into L1 for the same reason, so the state the snapshot names stays readable for as long as it lives.

It is also the cost, and it is the same cost a long-running transaction has. A snapshot left open pins `min_snapshot_seq` exactly as a forgotten transaction does, and shows up the same way — as disk that will not reclaim. Release it as soon as the point in time is no longer wanted.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` or `snapshot` is `NULL`
`TDB_ERR_INVALID_DB`	The database is closing
`TDB_ERR_MEMORY`	Allocation failed

See Also

`tidesdb_txn_begin_at_snapshot`, `tidesdb_snapshot_release`

12.6 tidesdb_snapshot_release

Release a snapshot and the reclamation floor it held.

Synopsis

```
void tidesdb_snapshot_release(tidesdb_snapshot_t *snapshot);
```

Description

Every transaction opened against the snapshot must be freed first. They read versions only this snapshot keeps alive, and releasing it lets the next merge collect them.

Passing `NULL` does nothing.

12.7 tidesdb_snapshot_seq

The sequence a snapshot reads at.

Synopsis

```
uint64_t tidesdb_snapshot_seq(const tidesdb_snapshot_t *snapshot);
```

Description

Returns 0 for a `NULL` snapshot. Compare it against `min_snapshot_seq` from `tidesdb_get_db_stats` (ch. 15) to see which snapshot is holding the floor down when reclamation stalls.

12.8 tidesdb_txn_begin_at_snapshot

Begin a transaction that reads as of a snapshot.

Synopsis

```
int tidesdb_txn_begin_at_snapshot(tidesdb_t *db, tidesdb_snapshot_t *snapshot,
                                  tidesdb_txn_t **txn);
```

Description

The same keys and the same families, answered as they stood when the snapshot was taken. Point reads and scans agree, because both resolve at the transaction's read snapshot.

The snapshot must **outlive** the transaction — it is what holds the floor under the versions being read.

For a point in time no snapshot was taken at, `tidesdb_txn_begin_at_seq` takes the sequence directly. A snapshot is the stronger form: it holds the floor from the moment it is taken, so the point stays readable, where a bare sequence is readable only while something else happens to be holding the floor under it.

Expiry is judged against the clock at the moment of the read, not against the snapshot, so a snapshot does not bring back an entry whose lifetime has since elapsed. Time travel applies to versions,

not to deadlines.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db`, `snapshot` or `txn` is `NULL`
`TDB_ERR_INVALID_DB`	The database is closing
`TDB_ERR_MEMORY`	Allocation failed

Examples

```
tidesdb_snapshot_t *snap = NULL;
if (tidesdb_snapshot_create(db, &snap) == TDB_SUCCESS)
{
    /* the database moves on */

    tidesdb_txn_t *past = NULL;
    if (tidesdb_txn_begin_at_snapshot(db, snap, &past) == TDB_SUCCESS)
    {
        uint8_t *value = NULL;
        size_t value_size = 0;
        if (tidesdb_txn_get(past, cf, key, key_size, &value, &value_size) == TDB_SUCCESS)
        {
            /* ... the value as of the snapshot ... */
            tidesdb_free(value);
        }
        (void)tidesdb_txn_rollback(past);
        tidesdb_txn_free(past);
    }

    /* the versions it was holding become collectable again here */
    tidesdb_snapshot_release(snap);
}
```

See Also

`tidesdb_snapshot_create`, `tidesdb_txn_read_snapshot`

12.9 tidesdb_txn_begin_at_seq

Begin a transaction that reads as of an explicit sequence.

Synopsis

```
int tidesdb_txn_begin_at_seq(tidesdb_t *db, uint64_t seq, tidesdb_txn_t **txn);
```

Description

For a point in time no snapshot was taken at — a sequence read back from `tidesdb_txn_read_snapshot`, or one recorded elsewhere.

It refuses rather than approximates. The versions an older point resolves to survive only while the reclamation floor sits at or below it. Once a collection has run past that sequence a merge has kept one version of each key and dropped the rest, so a read there would answer from what survived instead of from what was true — most often reporting a key that existed as absent. That case returns `TDB_ERR_TOO_OLD` and reads nothing.

A sequence is therefore readable while something holds the floor under it: an open transaction, or a snapshot taken in advance. On a database with no reader registered the floor is free to rise, and a sequence stops being reconstructable as soon as it does — which is why a snapshot is the stronger tool when the point in time is known ahead of time.

The boundary is not guessed at. `tidesdb_oldest_readable_seq` reports it, and a sequence at or above it is exact.

A reopened database cannot speak for what an earlier run collected, so the boundary starts at the sequence recovery resumed from. Points from before a restart are refused.

Expiry is judged against the clock at the moment of the read, so this does not bring back an entry whose lifetime has elapsed. It travels over versions, not over deadlines.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` or `txn` is `NULL`
`TDB_ERR_TOO_OLD`	A collection has already run below `seq`; the point in time cannot be reconstructed
`TDB_ERR_INVALID_DB`	The database is closing
`TDB_ERR_MEMORY`	Allocation failed

See Also

`tidesdb_oldest_readable_seq`, `tidesdb_txn_begin_at_snapshot`

12.10 tidesdb_oldest_readable_seq

The oldest sequence still exactly readable.

Synopsis

```
uint64_t tidesdb_oldest_readable_seq(const tidesdb_t *db);
```

Description

The highest reclamation floor any collection has taken. A sequence at or above it names a state the database can still reconstruct exactly; below it a merge has already dropped versions, and `tidesdb_txn_begin_at_seq` refuses.

It only ever rises. Holding a snapshot or an open transaction is what keeps it from rising past a point still wanted, which is the same mechanism — and the same cost — as pinning `min_snapshot_seq`.

Returns 0 for a `NULL` database.

12.11 tidesdb_txn_put

Buffer a put.

Synopsis

```
int tidesdb_txn_put(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, const uint8_t *key,
                    size_t key_size, const uint8_t *value, size_t value_size, time_t ttl);
```

Description

Buffers a write. Key and value bytes are **copied**, so the caller's buffers may be reused or freed immediately.

Nothing is durable and nothing is visible to other transactions until `tidesdb_txn_commit` succeeds.

An empty value is a value

`value_size` may be zero. The key is then **present carrying nothing**, which is a different state from being absent: a read returns it with a zero size rather than reporting it missing, and only `tidesdb_txn_delete` makes a key absent.

```
/* present, with nothing in it -- the key carries the meaning */
tidesdb_txn_put(txn, cf, key, klen, NULL, 0, 0);
```

This is the natural shape for anything whose meaning is entirely in the key — a secondary index entry, a set member — which would otherwise have to store a filler byte it does not want.

The `ttl` parameter

`ttl` is a **lifetime in seconds from now**. Zero or negative never expires.

```
/* expires one hour from now */
tidesdb_txn_put(txn, cf, key, klen, val, vlen, 3600);

/* never expires */
tidesdb_txn_put(txn, cf, key, klen, val, vlen, 0);
```

The engine converts the lifetime to an absolute deadline **once, at this call**, and stores that. Two consequences worth knowing:

- The clock starts when you call `put`, not when the transaction commits. A transaction held open for a minute after the put has already spent a minute of the entry's life.
- A slow recovery cannot extend an entry's life. Because the write-ahead log carries the deadline rather than the lifetime, replay does not restart the clock.

Expiry is evaluated on read and at compaction. An expired entry is invisible before it is physically removed, so space is reclaimed lazily but visibility is immediate.

Parameters

Parameter	Description
`txn`	Transaction, must be active
`cf`	Target column family
`key` / `key_size`	Key bytes, copied. `key_size` must be non-zero.
`value` / `value_size`	Value bytes, copied. `value_size` may be zero — see below
`ttl`	Lifetime in seconds from now; `<= 0` never expires

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	A `NULL` argument or a zero-length key
`TDB_ERR_TXN_EXPIRED`	The transaction is no longer active
`TDB_ERR_TXN_ABORTED`	Another thread called `tidesdb_txn_request_abort` on the transaction
`TDB_ERR_MEMORY`	Allocation failed buffering the operation

See Also

`tidesdb_txn_get`, `tidesdb_txn_delete`

12.12 tidesdb_txn_get

Read a key, recording the read for conflict detection.

Synopsis

```
int tidesdb_txn_get(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, const uint8_t *key,
                    size_t key_size, uint8_t **value, size_t *value_size);
```

Description

Reads at the transaction's snapshot and **records the read into the conflict footprint**. Under snapshot and serializable isolation this is what makes a later write to the same key validate against the version actually read rather than against the snapshot, so a read-modify-write is checked properly.

Sees the transaction's own buffered writes first, then the committed state at the snapshot.

`*value` is newly allocated and **owned by the caller** — release it with `tidesdb_free` (ch. 10), not your own `free`.

Reading a key you do not intend to base a write on inflates the footprint and causes avoidable conflicts. For existence probes use `tidesdb_txn_contains` or `tidesdb_txn_get_notrack`.

Errors

Code	Cause
`TDB_ERR_NOT_FOUND`	No visible version, or the newest visible one is a tombstone or expired
`TDB_ERR_INVALID_ARGS`	A `NULL` argument or a zero-length key
`TDB_ERR_TXN_EXPIRED`	The transaction is no longer active

Code	Cause
TDB_ERR_TXN_ABORTED	Another thread called tidesdb_txn_request_abort on the transaction
TDB_ERR_MEMORY	Allocation failed
TDB_ERR_LOCKED	Transient contention — retryable, not a miss
TDB_ERR_IO / TDB_ERR_CORRUPTION	Reading an sstable or the value log failed

TDB_ERR_LOCKED is **not** "absent". Treating it as a miss is a correctness bug; retry.

Examples

```
uint8_t *value = NULL;
size_t value_size = 0;

int rc = tidesdb_txn_get(txn, cf, key, key_size, &value, &value_size);
if (rc == TDB_SUCCESS)
{
    /* ... use value ... */
    tidesdb_free(value);
}
else if (rc != TDB_ERR_NOT_FOUND)
{
    /* a real error, not an absence */
}
```

See Also

tidesdb_txn_get_notrack, tidesdb_txn_contains

12.13 tidesdb_txn_get_notrack

Read a key without recording it for conflict detection.

Synopsis

```
int tidesdb_txn_get_notrack(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, const uint8_t *key,
                            size_t key_size, uint8_t **value, size_t *value_size);
```

Description

Identical to tidesdb_txn_get except that the read is **not** added to the conflict footprint. For probes whose result does not justify aborting the transaction — a primary-key uniqueness check, for example, where the answer you care about is "absent" and a concurrent insert of a *different* key is irrelevant.

Ownership is the same: free *value with tidesdb_free (ch. 10).

Using this where the read *does* feed a write weakens the guarantee to blind-write semantics at snapshot isolation. If the value read decides what you write, use tidesdb_txn_get.

Errors

Same as tidesdb_txn_get.

12.14 tidesdb_txn_read_snapshot

Report the sequence the transaction's reads filter at.

Synopsis

```
uint64_t tidesdb_txn_read_snapshot(const tidesdb_txn_t *txn);
```

Description

Returns the sequence ceiling this transaction reads at, so a caller can reason about which committed versions are and are not visible. Diagnostic; nothing in the API consumes it.

Level	Value returned
TDB_ISOLATION_READ_UNCOMMITTED	UINT64_MAX
TDB_ISOLATION_READ_COMMITTED	The current sequence, moving with each read

Level	Value returned
Repeatable read and stronger	The sequence frozen at begin

Returns 0 for a `NULL` transaction, which is otherwise not a valid snapshot.

Thread Safety

Reads transaction state; observe the same one-thread-per-transaction rule.

12.15 tidesdb_txn_contains

Existence check without allocating or tracking.

Synopsis

```
int tidesdb_txn_contains(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, const uint8_t *key,
                         size_t key_size);
```

Description

Reports whether a visible, unexpired, non-tombstoned version exists at the snapshot. Does not allocate and does not record the read into the conflict footprint.

Cheaper than `tidesdb_txn_get` when the value is not needed, especially for a value that lives in the value log — the check is answered without dereferencing it.

Return Value

`TDB_SUCCESS` if present, `TDB_ERR_NOT_FOUND` if absent, otherwise an error as for `tidesdb_txn_get` — including `TDB_ERR_TXN_EXPIRED` when a timeout has passed and `TDB_ERR_TXN_ABORTED` when another thread has requested an abort. `TDB_ERR_LOCKED` again means retry, not absent.

12.16 tidesdb_txn_delete

Buffer a delete.

Synopsis

```
int tidesdb_txn_delete(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, const uint8_t *key,
                       size_t key_size);
```

Description

Buffers a tombstone. Deleting a key that does not exist is not an error — the tombstone is written regardless, since the engine cannot know at buffer time whether a version exists in some lower level.

The tombstone occupies space until a compaction can prove no older version of the key survives beneath it. Delete-heavy workloads are what `tombstone_density_trigger` exists to compact.

Errors

Same as `tidesdb_txn_put`.

See Also

`tidesdb_txn_single_delete`, `tidesdb_txn_delete_prefix`

12.17 tidesdb_txn_single_delete

Buffer a delete that supersedes at most one put.

Synopsis

```
int tidesdb_txn_single_delete(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, const uint8_t *key,
                              size_t key_size);
```

Description

A tombstone carrying a **promise from the caller**: this key was written at most once, so the tombstone and that single put annihilate as soon as a compaction sees them together, instead of the tombstone being retained until every level below has been proven clear.

Breaking the promise resurrects data. If the key was written more than once, the tombstone and the newest put cancel, and an older put underneath becomes visible again. Use it only for keys written exactly once — insert-once identifiers, for example. When in doubt use `tidesdb_txn_delete`, which is always safe.

Errors

Same as `tidesdb_txn_put`.

12.18 tidesdb_txn_delete_prefix

Buffer a delete of every key under a prefix.

Synopsis

```
int tidesdb_txn_delete_prefix(tidesdb_txn_t *txn, tidesdb_column_family_t *cf,
                              const uint8_t *prefix, size_t prefix_size);
```

Description

Buffers one entry that deletes every key in the family beginning with `prefix`, however many keys that is. It shadows keys written before it as well as keys written after it, so it is not the same as deleting the keys that happen to exist when you call it.

This is what a family-per-tenant shape does not need and a prefix-per-tenant shape does. The data model steers you toward prefixes inside one family rather than thousands of families, and this is the operation that makes removing one of those prefixes cheap to write.

A write to a key under the prefix survives the delete when it is newer — buffered after it in the same transaction, or committed at a later sequence:

```
/* both land, and user:1 is alive afterwards while its siblings are not */
tidesdb_txn_delete_prefix(txn, cf, (const uint8_t *)"user:", 5);
tidesdb_txn_put(txn, cf, (const uint8_t *)"user:1", 6, val, vlen, 0);
```

The write is O(1); the reclamation is not. The keys stay on disk until a compaction rewrites the range, and reads pay a bounded interval lookup until then. What you buy is the write and the atomicity, not immediate space.

`prefix_size` must be greater than zero and at most `TDB_MAX_RANGE_BOUND_SIZE` (256), for the reason given under `tidesdb_txn_delete_range`. Dropping a whole family is `tidesdb_drop_column_family` (ch. 11), which unlinks its files rather than walking its keys.

Choosing between this and a range

A prefix names the interval `[prefix, successor(prefix))`, so this is `tidesdb_txn_delete_range` with the one bound a prefix implies. Reach for the prefix form when the thing you are removing genuinely *is* a prefix — a tenant, a partition, an id's versions — because it says so at the call site and cannot get its bounds wrong. Reach for the range form when the boundary falls somewhere a prefix cannot put it.

Isolation

At `TDB_ISOLATION_SNAPSHOT` and above the commit is refused with `TDB_ERR_CONFLICT` when any key under the prefix was written after this transaction drew its snapshot. The reverse is refused too: a write to a key under a prefix another transaction is deleting conflicts, which is what makes the two orderings symmetric.

A commit carrying a prefix delete runs alone against other committing transactions, so the check is still true by the time it commits. A two-phase transaction takes that exclusion only for its prepare — the window before phase two resolves it has no bound, and holding it there would stall every other committer for as long as the transaction stayed undecided. What covers that window instead is the prefix itself, held from the prepare until phase two, so a write under it is refused for as long as it stays in doubt.

Errors

Code	Meaning
TDB_ERR_INVALID_ARGS	txn, cf or prefix is NULL, prefix_size is zero, or it exceeds TDB_MAX_RANGE_BOUND_SIZE
TDB_ERR_TXN_EXPIRED	The transaction is no longer active
TDB_ERR_TXN_ABORTED	Another thread called tidesdb_txn_request_abort on the transaction
TDB_ERR_MEMORY	Allocation failed buffering the operation

See Also

tidesdb_txn_delete

12.19 tidesdb_txn_delete_range

Buffer a delete of every key in a range.

Synopsis

```
int tidesdb_txn_delete_range(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, const uint8_t *lo,
                             size_t lo_size, const uint8_t *hi, size_t hi_size);
```

Description

Buffers one entry that deletes every key from lo inclusive to hi exclusive, however many keys that is. Like a prefix delete it shadows keys written before it as well as keys written after it, and a newer write to a key inside the range survives it.

The bounds fall wherever you put them. They do not have to line up with any component of the key, which is what separates this from tidesdb_txn_delete_prefix:

```
/* a window inside a tenant, where no prefix covers the range and nothing else */
tidesdb_txn_delete_range(txn, cf, (const uint8_t *)"acme|20260105", 13,
                         (const uint8_t *)"acme|20260120", 13);
```

The upper bound is exclusive. A key equal to hi survives. Passing NULL with hi_size of zero leaves the range open above, so it runs to the end of the column family.

lo_size must be greater than zero. Keys are never empty, so a single zero byte is a lower bound below every key there can be:

```
/* everything in the family below a bound */
tidesdb_txn_delete_range(txn, cf, (const uint8_t *)"\x00", 1, (const uint8_t *)"m", 1);
```

Each bound is at most TDB_MAX_RANGE_BOUND_SIZE (256) bytes. A range delete holds its bounds in a fixed slot for the length of its commit — that slot is what stops a concurrent write landing inside the range while the commit is still undecided, and there is no slot wide enough for an arbitrary bound. A longer one is refused here with TDB_ERR_INVALID_ARGS rather than narrowed, because a narrowed bound would delete a range the caller never asked for. A range too wide to name in one call is expressed as several.

The write is O(1); the reclamation is not. The keys stay on disk until a compaction rewrites the range, and reads pay a bounded interval lookup until then. What you buy is the write and the atomicity, not immediate space. Dropping a whole family is tidesdb_drop_column_family (ch. 11), which unlinks its files rather than walking its keys.

Isolation

The same as tidesdb_txn_delete_prefix: at TDB_ISOLATION_SNAPSHOT and above, a commit is refused with TDB_ERR_CONFLICT when any key in the range was written after this transaction drew its snapshot, and a write to a key inside a range another transaction is deleting is refused the same way.

Errors

Code	Meaning
TDB_ERR_INVALID_ARGS	txn, cf or lo is NULL, lo_size is zero, or either bound exceeds TDB_MAX_RANGE_BOUND_SIZE
TDB_ERR_TXN_EXPIRED	The transaction is no longer active

Code	Meaning
TDB_ERR_TXN_ABORTED	Another thread called tidesdb_txn_request_abort on the transaction
TDB_ERR_MEMORY	Allocation failed buffering the operation

See Also

tidesdb_txn_delete_prefix, tidesdb_txn_delete

12.20 tidesdb_txn_commit

Commit a transaction.

Synopsis

```
int tidesdb_txn_commit(tidesdb_txn_t *txn);
```

Description

Validates conflicts if the isolation level requires it, draws a commit sequence, reserves the written keys, writes the batch to the write-ahead log, and applies it to the memtable. The log write happens before the memtable apply, so a crash in between recovers the batch rather than losing it.

What "durable" means when this returns depends on the database sync mode: under TDB_SYNC_FULL the batch is on the device; under TDB_SYNC_INTERVAL it has left the process into the operating system's page cache, so it survives process death, and reaches the device within the configured interval; under TDB_SYNC_NONE it is only staged in the log's ring, still inside this process, so a kill takes it with it. See Durability (ch. 5).

Commit may **block** on write admission if the flush queue is backed up. This is backpressure, not a fault.

On failure the transaction is aborted. It cannot be retried by calling commit again — begin a new transaction, or reuse this one with tidesdb_txn_reset.

The handle still has to be freed with tidesdb_txn_free after a successful commit.

Errors

Code	Cause
TDB_ERR_CONFLICT	Another transaction committed a conflicting write first. Only at snapshot and serializable. **Retry the whole transaction.**
TDB_ERR_INVALID_ARGS	txn is NULL or not active
TDB_ERR_TXN_EXPIRED	The transaction is no longer active
TDB_ERR_TXN_ABORTED	Another thread called tidesdb_txn_request_abort on the transaction
TDB_ERR_IO	The write-ahead log append failed
TDB_ERR_MEMORY	Allocation failed

Examples

```
for (int attempt = 0; attempt < MAX_RETRIES; attempt++)
{
    tidesdb_txn_t *txn = NULL;
    if (tidesdb_txn_begin(db, &txn) != TDB_SUCCESS) break;

    tidesdb_txn_put(txn, cf, key, key_size, value, value_size, 0);

    int rc = tidesdb_txn_commit(txn);
    tidesdb_txn_free(txn);

    if (rc != TDB_ERR_CONFLICT) break;   /* done, or a real failure */
}
```

See Also

tidesdb_txn_rollback, tidesdb_txn_prepare

12.21 tidesdb_txn_rollback

Discard a transaction's buffered writes.

Synopsis

```
int tidesdb_txn_rollback(tidesdb_txn_t *txn);
```

Description

Throws away everything buffered. Nothing was written to the log or the memtable, so there is no undo to perform and rollback cannot fail for I/O reasons.

The handle still has to be freed with `tidesdb_txn_free`.

Errors

`TDB_ERR_INVALID_ARGS` if `txn` is `NULL` or not in a rollbackable state.

12.22 tidesdb_txn_set_timeout

Bound how long a transaction may stay active.

Synopsis

```
int tidesdb_txn_set_timeout(tidesdb_txn_t *txn, int64_t seconds);
```

Description

Sets a deadline `seconds` from now, overriding the database's `txn_timeout_seconds` for this transaction alone. Pass `0` or less to clear any bound.

Why this exists. An active transaction holds its snapshot, and at snapshot and serializable isolation its write reservations too. That keeps the reclamation floor down, so compaction cannot drop the old versions below it and the value log cannot reclaim their bytes. A transaction that is never resolved therefore costs disk for as long as the database stays open. A timeout is what bounds that for a caller that might leak one.

Expiry is lazy and reported once. Nothing aborts the transaction in the background. The next operation on it notices the deadline has passed, aborts it, and returns `TDB_ERR_TXN_EXPIRED`. After that the transaction is resolved rather than expired, so further operations report `TDB_ERR_INVALID_ARGS`. The handle still has to be freed with `tidesdb_txn_free`.

The deadline is measured from the call, not from the begin, so calling it again on a live transaction extends it rather than accumulating.

The clock behind it advances once a second, so a bound is accurate to about a second and is not suited to sub-second deadlines.

Parameters

Parameter	Description
`txn`	Transaction
`seconds`	Seconds from now at which it expires, or `<= 0` to clear the bound

Errors

`TDB_ERR_INVALID_ARGS` if `txn` is `NULL` or is no longer active.

12.23 tidesdb_txn_request_abort

Abort a transaction another thread is running.

Synopsis

```
void tidesdb_txn_request_abort(tidesdb_txn_t *txn);
```

Description

The one call in this API that may be made on a transaction owned by a different thread.

Why this exists. The engine decides between two transactions racing on the same key by first-committer-wins, and reports the loser `TDB_ERR_CONFLICT`. But a caller can itself be the authority on whether a transaction may proceed — a replication plugin whose cluster has already certified against this one, say — and the engine has no way to know a ruling has been made elsewhere. This is how that ruling gets in.

It stores a flag and does nothing else. The transaction is not rolled back here, and no state changes on the calling thread — which is what keeps the one-thread-per-transaction rule intact. The thread running the transaction sees the flag when it next enters an operation, aborts it there, and returns `TDB_ERR_TXN_ABORTED` (ch. 34). That thread then frees the handle as it normally would.

Every operation afterwards reports the same code, the commit included. That matters more than it looks: if a read happened to be the operation that noticed, a later commit reporting only "transaction finished" would lose the very fact the caller needs at the point it most needs to report it.

Phase two is the exception. A transaction already in the prepared state has voted, and `tidesdb_txn_commit_prepared` and `tidesdb_txn_rollback_prepared` proceed regardless of a request. Once a participant has voted, only the coordinator's decision may resolve it — letting a local flag override that would leave one participant abandoning a transaction the others committed. Abort a transaction before it prepares, or abandon it through the coordinator afterwards.

The code is deliberately not `TDB_ERR_CONFLICT`. A caller layered over the engine has to tell an outside ruling from the engine's own verdict, because the two mean different things to whatever it reports upwards.

Caution—**The abort takes effect at the next operation, not instantly**

A request landing just after the running thread has checked lets the operation in progress finish and stops the one after it. A caller that needs the abort to take effect before a particular operation must not be racing that operation — it is expected to be holding the transaction still by its own means, exactly as it would to abort a transaction anywhere else.

Calling it more than once is harmless, as is calling it on a transaction that has already resolved, where it has no effect. A `NULL` transaction is a no-op.

Parameters

Parameter	Description
`txn`	Transaction to abort, or `NULL`

Thread Safety

Safe to call from any thread, including one that does not own the transaction. That is the point of it.

12.24 tidesdb_txn_prepare

Two-phase commit, phase one.

Synopsis

```
int tidesdb_txn_prepare(tidesdb_txn_t *txn, const uint8_t *xid, size_t xid_size);
```

Description

Runs the same conflict checks as commit and durably logs the write batch under `xid`, but leaves the writes **invisible and unapplied** so a coordinator can collect votes from every participant before deciding.

On success the transaction is `TDB_TXN_STATE_PREPARED` and holds its snapshot and its key reservations until resolved with `tidesdb_txn_commit_prepared` or `tidesdb_txn_rollback_prepared`. Holding reservations blocks conflicting commits, so an undecided prepared transaction applies backpressure to everything contending for its keys — decide promptly.

A read-only transaction prepares with nothing durable and needs no phase two.

The `xid` is copied. It is the coordinator's identifier and the engine only stores and returns it.

Once prepared, the transaction survives a restart: `tidesdb_recover_prepared` hands it back in doubt.

Errors

Code	Cause
`TDB_ERR_CONFLICT`	Conflict detected at phase one — the transaction is aborted, not prepared
`TDB_ERR_INVALID_ARGS`	`NULL` `txn` or `xid`, `xid_size` of 0, or a transaction not in the active state
`TDB_ERR_TXN_EXPIRED`	The transaction is no longer active
`TDB_ERR_TXN_ABORTED`	Another thread called `tidesdb_txn_request_abort` on the transaction
`TDB_ERR_IO` / `TDB_ERR_MEMORY`	The prepare record could not be logged

See Also

`tidesdb_txn_state`

12.25 tidesdb_txn_commit_prepared

Two-phase commit, phase two — commit.

Synopsis

```
int tidesdb_txn_commit_prepared(tidesdb_txn_t *txn);
```

Description

Durably logs the decision, then applies the batch and makes it visible. Valid **only** on a prepared transaction.

The batch lands at a sequence drawn now, when the decision is made, not at the sequence it held when it prepared. That is what keeps replay ordering correct: a key written by another transaction between the prepare and the decision is correctly superseded by this one, rather than being shadowed by a stale sequence.

A transient I/O failure leaves the transaction **prepared**, not aborted, so the coordinator can retry the same decision.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`txn` is `NULL` or not prepared
`TDB_ERR_IO` / `TDB_ERR_MEMORY`	The decision could not be logged; the transaction stays prepared and the call may be retried

12.26 tidesdb_txn_rollback_prepared

Two-phase commit, phase two — roll back.

Synopsis

```
int tidesdb_txn_rollback_prepared(tidesdb_txn_t *txn);
```

Description

Durably logs the decision to abandon the prepared transaction and releases its reservations. Nothing had been applied, so nothing is undone. The decision is logged rather than merely dropped so that a restart does not resurrect the transaction as in-doubt.

A transient I/O failure leaves it prepared for retry, as with `tidesdb_txn_commit_prepared`.

Errors

Same as `tidesdb_txn_commit_prepared`.

12.27 tidesdb_recover_prepared

List transactions left in doubt by a restart.

Synopsis

```
int tidesdb_recover_prepared(tidesdb_t *db, tidesdb_prepared_txn_t *out, int max, int *out_count);
```

Description

Returns the transactions durably prepared before the last shutdown that were never decided, so a coordinator can finish them. One whose decision *was* logged is settled during `tidesdb_open` (ch. 10) and never appears here.

The set is fixed when the database opens, so the two-call pattern is safe: call once with `out` as `NULL` to learn the count, allocate, then call again to fill.

Each entry's `txn` is a real handle in the prepared state — resolve it with `tidesdb_txn_commit_prepared` or `tidesdb_txn_rollback_prepared` and free it with `tidesdb_txn_free` like any other. The `xid` is **borrowed** from the database and valid until it is closed; copy it if you need it longer.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` or `out_count` is `NULL`
`TDB_ERR_TOO_LARGE`	`out` was given and `max` is below the real count. `*out_count` is still set, **nothing was written, and no handles were created** — so there is nothing to free before retrying with a bigger buffer.
`TDB_ERR_MEMORY`	Allocation failed

Examples

```
int count = 0;
if (tidesdb_recover_prepared(db, NULL, 0, &count) == TDB_SUCCESS && count > 0)
{
    tidesdb_prepared_txn_t *pending = malloc(sizeof(*pending) * (size_t)count);

    if (tidesdb_recover_prepared(db, pending, count, &count) == TDB_SUCCESS)
    {
        for (int i = 0; i < count; i++)
        {
            /* ask the coordinator what was decided for pending[i].xid */
            tidesdb_txn_rollback_prepared(pending[i].txn);
            tidesdb_txn_free(pending[i].txn);
        }
    }
    free(pending);   /* your allocation, your free */
}
```

12.28 tidesdb_txn_state

Report a transaction's lifecycle state.

Synopsis

```
int tidesdb_txn_state(const tidesdb_txn_t *txn, tidesdb_txn_state_t *out_state);
```

Description

Reports one of `TDB_TXN_STATE_ACTIVE`, `TDB_TXN_STATE_PREPARED`, `TDB_TXN_STATE_COMMITTED`, or `TDB_TXN_STATE_ABORTED`. Mainly for coordinators distinguishing a prepared transaction from a resolved one.

Errors

`TDB_ERR_INVALID_ARGS` if `txn` or `out_state` is `NULL`.

12.29 tidesdb_txn_reset

Reset a transaction for reuse.

Synopsis

```
int tidesdb_txn_reset(tidesdb_txn_t *txn, tidesdb_isolation_level_t isolation);
```

Description

Discards buffered state and returns the handle to active at `isolation`, avoiding a free/allocate cycle in a loop that runs many small transactions. A new snapshot is taken, so a reset transaction sees everything committed up to that moment.

Resetting an uncommitted transaction discards its writes — the effect of a rollback.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`txn` is `NULL`, or `isolation` is outside the enum
`TDB_ERR_MEMORY`	Allocation failed re-arming the transaction

12.30 tidesdb_txn_free

Free a transaction handle.

Synopsis

```
void tidesdb_txn_free(tidesdb_txn_t *txn);
```

Description

Releases the handle, **rolling it back first if it is still open**. Passing `NULL` is safe.

Must be called for every handle from any `begin`, and after a successful commit as well — committing does not free.

Freeing a **prepared** transaction without deciding it abandons the handle while the prepared batch stays durable on disk; it comes back as in-doubt from `tidesdb_recover_prepared` after the next open. Decide before freeing unless that is what you intend.

12.31 tidesdb_txn_savepoint

Mark a savepoint.

Synopsis

```
int tidesdb_txn_savepoint(tidesdb_txn_t *txn, const char *name);
```

Description

Names a point in the buffered write sequence to roll back to later. Savepoints nest: marking several and rolling back to an earlier one discards the later ones too.

Names are the caller's; reusing a name shadows the earlier mark.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`txn` or `name` is `NULL`, or the transaction is already resolved
`TDB_ERR_TXN_EXPIRED`	A timeout has passed; the transaction is aborted by this call
`TDB_ERR_TXN_ABORTED`	Another thread called `tidesdb_txn_request_abort` on the transaction
`TDB_ERR_MEMORY`	Allocation failed

12.32 tidesdb_txn_rollback_to_savepoint

Roll back to a savepoint.

Synopsis

```
int tidesdb_txn_rollback_to_savepoint(tidesdb_txn_t *txn, const char *name);
```

Description

Discards every operation buffered after `name` was marked, leaving the transaction active and everything before it intact. The savepoint itself remains, so the same name can be rolled back to again.

Errors

Code	Cause
`TDB_ERR_NOT_FOUND`	No savepoint by that name
`TDB_ERR_INVALID_ARGS`	`txn` or `name` is `NULL`, or the transaction is already resolved
`TDB_ERR_TXN_EXPIRED`	A timeout has passed; the transaction is aborted by this call
`TDB_ERR_TXN_ABORTED`	Another thread called `tidesdb_txn_request_abort` on the transaction

12.33 tidesdb_txn_release_savepoint

Release a savepoint.

Synopsis

```
int tidesdb_txn_release_savepoint(tidesdb_txn_t *txn, const char *name);
```

Description

Forgets the mark, keeping every buffered operation. Releasing a savepoint does **not** discard work — it only makes that point no longer available to roll back to, along with any savepoints nested inside it.

Errors

Same as `tidesdb_txn_rollback_to_savepoint`.

Iterator Operations

An iterator walks one column family in byte order at a transaction's snapshot. It merges every source that can hold a version of a key — the transaction's own buffered writes, the active memtable, the sealed memtables awaiting flush, and each level's sstables — and presents the newest version visible at the snapshot, hiding tombstones and expired entries.

An iterator is stable: sstables merged or deleted by a compaction while you are iterating do not disturb it, and writes committed by other transactions after its snapshot do not appear.

Which snapshot it reads at follows the transaction's isolation level, matching what a point read in the same transaction would see:

Isolation	The iterator reads at
`TDB_ISOLATION_REPEATABLE_READ`, `TDB_ISOLATION_SNAPSHOT`, `TDB_ISOLATION_SERIALIZABLE`	The transaction's snapshot, taken at begin — every scan in the transaction sees one instant
`TDB_ISOLATION_READ_COMMITTED`	The current sequence, taken when the iterator is created — everything committed before that moment, so two scans in one transaction may legitimately differ
`TDB_ISOLATION_READ_UNCOMMITTED`	Everything, including versions other transactions have written but not committed

Iterators are **not thread-safe**. One iterator belongs to one thread, and it must be freed before the transaction it was created from is freed.

13.1 Direction changes are supported but not free

The iterator is bidirectional: you may call `tidesdb_iter_prev` after `tidesdb_iter_next` and the sequence stays correct. Reversing forces every source to re-seek to the current position, because a source that was exhausted in the old direction has to be brought back into play. Scans that flip direction repeatedly pay that cost each time; scans that go one way do not.

13.2 tidesdb_iter_new

Create an iterator.

Synopsis

```
int tidesdb_iter_new(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, tidesdb_iter_t **iter);
```

Description

Creates an iterator over `cf` at `txn`'s snapshot. The iterator starts **unpositioned** — call one of the seek functions before reading. Calling `tidesdb_iter_valid` first returns 0.

The iterator borrows the transaction. Free it with `tidesdb_iter_free` before `tidesdb_txn_free` (ch. 12).

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`txn`, `cf`, or `iter` is `NULL`
`TDB_ERR_MEMORY`	Allocation failed

Code	Cause
TDB_ERR_LOCKED	The descriptor budget or a source set moving under a compaction left the scan unopenable. The engine waits such pressure out, so this is the rare case where it did not clear — retry, and never read it as an empty family
TDB_ERR_IO / TDB_ERR_CORRUPTION	A source could not be opened or did not decode

See Also

tidesdb_iter_seek_to_first

13.3 tidesdb_iter_new_range

Create an iterator over a known key range.

Synopsis

```
int tidesdb_iter_new_range(tidesdb_txn_t *txn, tidesdb_column_family_t *cf, const uint8_t *lower,
                           size_t lower_size, const uint8_t *upper, size_t upper_size,
                           tidesdb_iter_t **iter);
```

Description

Creates an iterator over the part of cf between lower and upper, at txn's snapshot. Like tidesdb_iter_new it starts **unpositioned**.

The difference is what it costs. An iterator holds one open cursor per sstable that could answer it, descends each of them on every seek, and compares each of them on every step — so an unbounded scan of a family holding hundreds of sstables pays for all of them even when it reads three rows. Given the range up front, the iterator leaves out every sstable whose own key range cannot meet it. On a family of 32 sstables holding disjoint bands, a four-row scan went from 128 block cache lookups to 4.

This is the difference between range scans that scale with concurrency and range scans that do not: many narrow scans running at once otherwise each contend for the whole store.

> Caution—**The range is a promise, not a fence**
>
> Results are defined **only inside the range**. The sstables that could answer outside it were never opened, so seeking or stepping past either end may report a key absent that exists. The iterator does not stop you at the bound and does not report one — the caller stops itself. Use tidesdb_iter_new when the extent is not known in advance.

upper is treated as inclusive when choosing sstables, so a caller holding an exclusive end may pass it unchanged; at worst one sstable is kept that the scan never reads from.

Errors

Code	Cause
TDB_ERR_INVALID_ARGS	txn, cf, iter, lower, or upper is NULL — both ends are required, since one end alone leaves nothing to test an sstable against
TDB_ERR_MEMORY	Allocation failed
TDB_ERR_LOCKED	As tidesdb_iter_new — retry, and never read it as an empty range
TDB_ERR_IO / TDB_ERR_CORRUPTION	A source could not be opened or did not decode

Examples

```
/* rows from "order:1000" up to "order:2000" */
tidesdb_iter_t *it = NULL;
if (tidesdb_iter_new_range(txn, cf, (const uint8_t *)"order:1000", 10,
                           (const uint8_t *)"order:2000", 10, &it) == TDB_SUCCESS)
{
    tidesdb_iter_seek(it, (const uint8_t *)"order:1000", 10);

    while (tidesdb_iter_valid(it))
    {
        uint8_t *k = NULL; size_t klen = 0;
        if (tidesdb_iter_key(it, &k, &klen) != TDB_SUCCESS) break;
```

```
        /* the caller stops at its own upper bound, the iterator does not */
        const int past_end = (klen >= 10 && memcmp(k, "order:2000", 10) > 0);
        tidesdb_free(k);
        if (past_end) break;

        tidesdb_iter_next(it);
    }
    tidesdb_iter_free(it);
}
```

See Also

`tidesdb_iter_new`, `tidesdb_range_stats` (ch. 15)

13.4 tidesdb_iter_seek

Position at the first key >= the given key.

Synopsis

```
int tidesdb_iter_seek(tidesdb_iter_t *iter, const uint8_t *key, size_t key_size);
```

Description

Positions at the first key greater than or equal to `key`. If no such key exists the iterator becomes invalid, which is the normal way a forward scan ends.

Combined with a prefix check on each key, this is how prefix scans are done — seek to the prefix, then walk forward while the key still starts with it.

Errors

Code	Cause
`TDB_ERR_NOT_FOUND`	No key qualifies; the iterator is left invalid. This is how a scan ends, not a fault
`TDB_ERR_INVALID_ARGS`	A `NULL` argument or a zero-length key
`TDB_ERR_LOCKED`	Descriptor pressure kept a source from being read — retry
`TDB_ERR_IO` / `TDB_ERR_CORRUPTION` / `TDB_ERR_MEMORY`	A source read failed

Examples

```
/* every key beginning with "user:" */
tidesdb_iter_seek(it, (const uint8_t *)"user:", 5);

while (tidesdb_iter_valid(it))
{
    uint8_t *k = NULL; size_t klen = 0;
    if (tidesdb_iter_key(it, &k, &klen) != TDB_SUCCESS) break;

    const int in_prefix = (klen >= 5 && memcmp(k, "user:", 5) == 0);
    tidesdb_free(k);
    if (!in_prefix) break;

    tidesdb_iter_next(it);
}
```

13.5 tidesdb_iter_seek_for_prev

Position at the last key <= the given key.

Synopsis

```
int tidesdb_iter_seek_for_prev(tidesdb_iter_t *iter, const uint8_t *key, size_t key_size);
```

Description

The mirror of `tidesdb_iter_seek`, for starting a backward scan at or before a key. If every key is greater than `key` the iterator becomes invalid.

Errors

Same as `tidesdb_iter_seek`.

13.6 tidesdb_iter_seek_to_first

Position at the first key.

Synopsis

```
int tidesdb_iter_seek_to_first(tidesdb_iter_t *iter);
```

Description

Positions at the smallest visible key. An empty column family leaves the iterator invalid and reports `TDB_ERR_NOT_FOUND`.

Errors

As `tidesdb_iter_seek`, less the key arguments: `TDB_ERR_NOT_FOUND` when the family has no visible key, `TDB_ERR_INVALID_ARGS` if `iter` is `NULL`, `TDB_ERR_LOCKED` under descriptor pressure, and `TDB_ERR_IO`, `TDB_ERR_CORRUPTION` or `TDB_ERR_MEMORY` from a source read.

13.7 tidesdb_iter_seek_to_last

Position at the last key.

Synopsis

```
int tidesdb_iter_seek_to_last(tidesdb_iter_t *iter);
```

Description

Positions at the largest visible key, for a backward scan. An empty column family leaves the iterator invalid.

Errors

Same as `tidesdb_iter_seek_to_first`.

13.8 tidesdb_iter_next

Advance to the next key.

Synopsis

```
int tidesdb_iter_next(tidesdb_iter_t *iter);
```

Description

Moves forward one key. Advancing past the last key makes the iterator invalid and reports `TDB_ERR_NOT_FOUND`; this is the normal end of a scan rather than a fault, and `tidesdb_iter_valid` is how you detect it. Do not use the return value as the loop condition — check `valid` instead.

Errors

Same as `tidesdb_iter_seek`, less the argument checks that do not apply: `TDB_ERR_NOT_FOUND` at the end of the stream, `TDB_ERR_INVALID_ARGS` if `iter` is `NULL`, `TDB_ERR_LOCKED` under descriptor pressure, and `TDB_ERR_IO`, `TDB_ERR_CORRUPTION` or `TDB_ERR_MEMORY` from a source read.

13.9 tidesdb_iter_prev

Step to the previous key.

Synopsis

```
int tidesdb_iter_prev(tidesdb_iter_t *iter);
```

Description

Moves back one key, becoming invalid when stepped before the first. See Direction changes for the cost of alternating with `tidesdb_iter_next`.

Errors

Same as `tidesdb_iter_next`.

13.10 tidesdb_iter_valid

Report whether the iterator is on a live key.

Synopsis

```
int tidesdb_iter_valid(tidesdb_iter_t *iter);
```

Description

Returns 1 when the iterator is positioned on a readable key and 0 otherwise — before the first seek, after running off either end, or for a `NULL` iterator. This is the loop condition for every scan.

Return Value

1 if valid, 0 otherwise. It cannot fail and reports no errors, so a source read failure during a `next` shows up as invalidity here rather than as a distinct signal; check the return value of the movement call if you need to tell the two apart.

13.11 tidesdb_iter_key

Read the key at the current position.

Synopsis

```
int tidesdb_iter_key(tidesdb_iter_t *iter, uint8_t **key, size_t *key_size);
```

Description

Returns a **newly allocated** copy of the current key. Free it with `tidesdb_free` (ch. 10). The copy is independent of the iterator and stays valid after moving or freeing it.

Errors

Code	Cause
`TDB_ERR_NOT_FOUND`	The iterator is not positioned on a key — before the first seek, or after running off either end
`TDB_ERR_INVALID_ARGS`	A `NULL` argument
`TDB_ERR_MEMORY`	Allocation failed

Note that “not positioned” is `TDB_ERR_NOT_FOUND`, not `TDB_ERR_INVALID_ARGS`. Ordinarily `tidesdb_iter_valid` is checked first and neither arises.

13.12 tidesdb_iter_value

Read the value at the current position.

Synopsis

```
int tidesdb_iter_value(tidesdb_iter_t *iter, uint8_t **value, size_t *value_size);
```

Description

Returns a **newly allocated** copy of the current value, freed with `tidesdb_free` (ch. 10).

A value stored in the value log rather than inline is fetched here, so this call can cost an extra read that `tidesdb_iter_key` does not. A scan that only needs keys should not call it — that is most of the benefit of key/value separation.

Errors

Same as `tidesdb_iter_key`, plus `TDB_ERR_IO` or `TDB_ERR_CORRUPTION` if a separated value cannot be read back.

13.13 tidesdb_iter_key_value

Read both in one call.

Synopsis

```
int tidesdb_iter_key_value(tidesdb_iter_t *iter, uint8_t **key, size_t *key_size,
                           uint8_t **value, size_t *value_size);
```

Description

Equivalent to calling `tidesdb_iter_key` and `tidesdb_iter_value`, returning both as newly allocated buffers. **Both** must be freed with `tidesdb_free` (ch. 10).

On failure neither is written, so there is nothing to free after an error.

Errors

Same as `tidesdb_iter_value`.

Examples

```
tidesdb_iter_t *it = NULL;
if (tidesdb_iter_new(txn, cf, &it) != TDB_SUCCESS) return TDB_ERR_MEMORY;

for (tidesdb_iter_seek_to_first(it); tidesdb_iter_valid(it); tidesdb_iter_next(it))
{
    uint8_t *k = NULL, *v = NULL;
    size_t klen = 0, vlen = 0;

    if (tidesdb_iter_key_value(it, &k, &klen, &v, &vlen) != TDB_SUCCESS) break;

    /* ... use k and v ... */

    tidesdb_free(k);
    tidesdb_free(v);
}

tidesdb_iter_free(it);
return TDB_SUCCESS;
```

13.14 tidesdb_iter_free

Free an iterator.

Synopsis

```
void tidesdb_iter_free(tidesdb_iter_t *iter);
```

Description

Releases the iterator and the source handles it holds. `NULL` is safe.

Free it before the transaction it came from. An iterator holds references that keep sstables and sealed memtables alive, so one left open pins resources a flush or compaction would otherwise reclaim — long-lived iterators hold back space.

CHAPTER 14

Maintenance Operations

TidesDB compacts and flushes on its own. Everything here forces work that would otherwise happen when a trigger fired, or establishes a durability barrier stronger than the configured sync mode.

These calls exist for operators and for tests. A well-configured database under normal load should not need them on a schedule — reach for them to take a backup, to make a barrier before something risky, or to compact during a known-quiet window rather than letting it land during a busy one.

14.1 Thread safety

Every call in this chapter is safe from any thread, and the database stays open to readers and writers while one runs. What they do not do is queue behind each other: each can report `TDB_ERR_LOCKED`, which always means the work was not done because something else held what it needed, never that anything was lost or corrupted. Asking again later is the whole remedy.

What is held differs by call. `tidesdb_compact` and `tidesdb_compact_range` take one family exclusively and report locked immediately if a compaction already holds it, rather than parking the caller behind a merge that may run for minutes — and in that case the work asked for is usually already under way. `tidesdb_backup` claims every family the same way and reports locked if any one of them will not come free. `tidesdb_flush_memtable`, and `tidesdb_checkpoint` through it, instead wait a bounded time for the immutable queue to drain and report locked only if it has not, which says flush is not keeping up rather than that anything is wrong with the call.

14.2 tidesdb_compact

Force one compaction pass on a column family.

Synopsis

```
int tidesdb_compact(tidesdb_t *db, tidesdb_column_family_t *cf);
```

Description

Runs one compaction pass synchronously, merging even when no trigger is due. Returns when the pass is finished.

This is I/O-heavy in proportion to how much data the pass merges, and it competes with the background compaction pool for the same device. Forcing it during peak load makes latency worse, not better.

Fails fast rather than queueing. If the family is already being compacted — by the background scheduler or another caller — it returns `TDB_ERR_LOCKED` immediately.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` or `cf` is `NULL`
`TDB_ERR_LOCKED`	A compaction is already running on this family
`TDB_ERR_IO` / `TDB_ERR_CORRUPTION`	Reading inputs or writing outputs failed
`TDB_ERR_MEMORY`	Allocation failed

See Also

`tidesdb_compact_range`, `tidesdb_is_compacting`

14.3 tidesdb_compact_range

Compact only the sstables overlapping a key range.

Synopsis

```
int tidesdb_compact_range(tidesdb_t *db, tidesdb_column_family_t *cf,
                          const uint8_t *start_key, size_t start_key_size,
                          const uint8_t *end_key, size_t end_key_size);
```

Description

Compacts every sstable overlapping `[start_key, end_key)`, merging toward the largest level affected. Useful after deleting a contiguous span of keys, where a full compaction would do far more work than the space being reclaimed justifies.

Either endpoint may be `NULL` for unbounded. **Both `NULL` is rejected** with `TDB_ERR_INVALID_ARGS` rather than silently becoming a full compaction — use `tidesdb_compact` when that is what you mean.

The range is half-open: `start_key` is included, `end_key` is not.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` or `cf` is `NULL`, or both endpoints are `NULL`
`TDB_ERR_LOCKED`	A compaction is already running on this family
`TDB_ERR_IO` / `TDB_ERR_CORRUPTION`	Reading inputs or writing outputs failed
`TDB_ERR_MEMORY`	Allocation failed

14.4 tidesdb_flush_memtable

Rotate and flush the memtable.

Synopsis

```
int tidesdb_flush_memtable(tidesdb_t *db);
```

Description

Seals the active memtable, installs a fresh one, and writes the sealed one out to each column family's L1, returning when that is done.

The memtable is **shared by every column family**, so this flushes all of them — there is no per-family flush. This is why `tidesdb_rename_column_family` (ch. 11) and `tidesdb_clone_column_family` (ch. 11) are database-wide in their cost.

Writers are not blocked for the whole call: the rotation installs a new active memtable immediately, and writes continue into it while the sealed one is written out.

The call waits for the **immutable queue to drain**, so on return every generation sealed before it is in L1 — which is what backup and clone (ch. 11) need, since they copy files and anything still only in memory would be missed.

Under sustained writes from other threads the queue may never be observed empty, and the call then reports `TDB_ERR_LOCKED` even though this caller's own data has landed. That is deliberate: the alternative — returning as soon as the caller's own generation lands — lets a copy silently omit data that was still queued.

Note—**A flush wakes compaction**

Landing sstables in L1 is what the compaction scheduler waits for, so a flush wakes it and the scheduler claims the families it plans. A `tidesdb_compact` issued straight after a flush will often find the family claimed and return `TDB_ERR_LOCKED`, which is retryable contention rather than an error — see Error codes (ch. 34). The same applies to `tidesdb_is_compacting`, which reads the same claim and may be set for planning that has nothing to do with a merge you requested.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` is `NULL`
`TDB_ERR_IO`	Writing an sstable or committing the manifest failed
`TDB_ERR_NO_SPACE`	The device is full; the data stays in the memtable and its log until space is freed
`TDB_ERR_LOCKED`	The immutable queue did not drain within the wait — either the flush pool is stuck, or writes are arriving faster than it drains
`TDB_ERR_MEMORY`	Allocation failed

See Also

`tidesdb_is_flushing`, `tidesdb_checkpoint`

14.5 tidesdb_is_flushing

Report whether a flush or rotation is in progress.

Synopsis

```
int tidesdb_is_flushing(tidesdb_t *db);
```

Description

Returns 1 while the memtable is rotating or a sealed memtable is being written out, 0 otherwise. A sample of a moving value — it may be stale the instant it returns, so it is for monitoring, not for synchronisation. Waiting on it in a loop is a bug.

Return Value

1 if flushing, 0 otherwise, including for a `NULL` handle.

14.6 tidesdb_is_compacting

Report whether a family is being compacted.

Synopsis

```
int tidesdb_is_compacting(tidesdb_column_family_t *cf);
```

Description

Returns 1 while a compaction is running on `cf`. The same caveat applies: a sample, not a lock. Checking it before `tidesdb_compact` does not prevent `TDB_ERR_LOCKED`, since a background compaction can start in between — handle the error instead.

Return Value

1 if compacting, 0 otherwise, including for a `NULL` handle.

14.7 tidesdb_sync_wal

Force the write-ahead log to disk.

Synopsis

```
int tidesdb_sync_wal(tidesdb_t *db);
```

Description

Forces an fsync of the write-ahead log, making every commit acknowledged before this call durable on the device regardless of the configured sync mode.

This is the cheap barrier. Under `TDB_SYNC_NONE` it converts “staged in the log’s ring, still inside this process” into “on the device” for the WAL alone — it waits for the ring to drain before the fsync, because syncing the descriptor alone would report as durable a record still held in memory. Under

`TDB_SYNC_INTERVAL` it makes the barrier now instead of at the next tick. Under `TDB_SYNC_FULL` it is redundant, since each commit is already durable when it returns.

It does **not** cover the sstables, the value log, or the manifest. For a barrier across all of those use `tidesdb_checkpoint`.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` is `NULL`
`TDB_ERR_IO`	The fsync failed

14.8 tidesdb_backup

Write a consistent, directly-openable copy of the database.

Synopsis

```
int tidesdb_backup(tidesdb_t *db, const char *dir);
```

Description

Flushes the memtable, then copies the manifest, the shared value log, and every sstable the manifest references — all at a single manifest snapshot, with compaction held off, so the copy can never reference a file a merge deleted mid-copy.

The result is a database directory, not an archive. Point `tidesdb_open` (ch. 10) at it and it opens.

The database stays writable throughout. Writes committed during the backup are not included: the copy is consistent as of the flush at the start.

Cost and free space required are proportional to the live on-disk size.

Parameters

Parameter	Description
`db`	Database handle
`dir`	Destination directory, created if absent. Must not be the live database directory.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` or `dir` is `NULL`
`TDB_ERR_LOCKED`	A column family stayed under compaction for the whole freeze window, so it could not be copied at a consistent point. Retryable
`TDB_ERR_MEMORY`	Allocation failed
`TDB_ERR_IO`	The directory could not be created, or a file could not be read or written

See Also

`tidesdb_checkpoint`, `tidesdb_clone_column_family` (ch. 11)

14.9 tidesdb_checkpoint

Establish a full durability barrier.

Synopsis

```
int tidesdb_checkpoint(tidesdb_t *db);
```

Description

Flushes the memtable to L1, then forces the value log, the write-ahead log, and the manifest to disk **regardless of the configured sync mode**. When it returns, everything committed beforehand is on the device.

This is the strong barrier, and the one to use before anything risky — a host reboot, a storage migration, a filesystem-level snapshot. It differs from `tidesdb_sync_wal` in covering the whole durable base rather than the log alone, and from `tidesdb_backup` in making the live database durable rather than producing a second copy.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`db` is `NULL`
`TDB_ERR_LOCKED`	The flush it begins with could not drain the immutable queue inside its bounded wait, which says flush is behind rather than that anything is held. Retryable
`TDB_ERR_IO`	The flush or one of the fsyncs failed

Examples

```
/* make everything durable before a planned restart */
if (tidesdb_checkpoint(db) != TDB_SUCCESS)
    fprintf(stderr, "checkpoint failed; do not assume the data is on disk\n");

tidesdb_close(db);   /* close cannot report a failure -- see its entry */
```

Statistics

Every statistics call fills a **caller-owned struct by value**. Nothing is allocated and nothing is freed — declare the struct on the stack, pass its address, and read the fields.

```
tidesdb_db_stats_t stats;
if (tidesdb_get_db_stats(db, &stats) == TDB_SUCCESS)
    printf("%d sstables\n", stats.total_sstable_count);
/* nothing to free */
```

The values are samples taken without stopping the engine, so counters can be slightly inconsistent with each other — a flush may land between two fields being read. They are for monitoring and capacity planning, not for making transactional decisions.

15.1 tidesdb_get_cf_stats

Collect one column family's statistics.

Synopsis

```
int tidesdb_get_cf_stats(tidesdb_column_family_t *cf, tidesdb_cf_stats_t *stats);
```

Description

Fills `stats` with the shape and history of one family. Note the signature takes no database handle — the family handle is enough.

Shape of the tree. `num_levels` is how many levels hold data, and the four `level_*` arrays are indexed by level up to `TDB_MAX_LEVELS` (8). `read_amp` estimates how many sstables a point read must consult; it is the number to watch when reads slow down.

Field	Meaning
`num_levels`	Levels currently holding data
`level_sizes[]`	Bytes per level
`level_num_sstables[]`	Sstable count per level
`level_key_counts[]`	Keys per level
`level_tombstone_counts[]`	Tombstones per level
`total_keys` / `total_data_size`	Totals across levels. `total_data_size` is the family's key logs and nothing else, so it equals the sum of `level_sizes`. Values below `value_separation_threshold` are already inside those bytes; what spilled lives in the shared value log, reported at database level as `vlog_file_size`
`avg_key_size` / `avg_value_size`	Averages in bytes
`read_amp`	Estimated sstables consulted per point read

The btree. `btree_total_nodes`, `btree_max_height`, and `btree_avg_height` describe the key logs. Height rising over time on a family whose key count is stable usually means the node size is too small for the keys being stored.

Tombstones. `total_tombstones` and `tombstone_ratio` are what the `tombstone_density_trigger` acts on. A ratio that stays high means deletes are outrunning compaction. `max_sst_density` and `max_sst_density_level` locate the worst single sstable.

Write volume. `wal_bytes_written`, `flush_bytes_written`, `compaction_bytes_written`, `compaction_bytes_read`, and `user_bytes_written` are cumulative since open.

A family's `wal_bytes_written` is an **attribution**, not a measurement. One log serves the whole database, so this is the encoded size of this family's own entries; the batch header and the block fram-

ing belong to no single family, and these do not sum to what the log wrote. The database-level field of the same name is measured at the append and is the one to use.

None of them count a separated value, because the log does not carry one — the record holds an id of a few bytes and the bytes themselves are in `vlog_bytes_written`. A family whose values are all above the threshold therefore reports a small `wal_bytes_written` against a large `user_bytes_written`, and that is the arrangement working rather than an accounting gap.

Caution—**Write amplification has to include the value log**

Compute it as `(wal + flush + compaction_written + vlog_written) / user_bytes_written`, all four from the **database** statistics.

Leaving `vlog_bytes_written` out is not a rounding error on a store that separates its values, it is most of the answer. A workload writing 4 KiB values against the default threshold puts nearly every byte in the value log and almost nothing in the key logs, and the three-term formula reported a write amplification of **0.11** on a run that had just written eight gigabytes to the log.

What the four-term formula reports on that workload is now close to **1.06**. A separated value goes to the value log once and the write-ahead log carries only its id, so `wal_bytes_written` collapses to the records themselves — on one 30 second run, 90 MB of log against 7.9 GB of user data. A store whose values all stay inline still reports the shape you would expect from one log copy plus one copy per flush and merge.

In memory. `unflushed_key_count` is the distinct keys belonging to this family that are still only in the memtable. It is the family's share of what a flush would write and what a crash would have to recover. It is reported as an unsigned figure and floored at zero: the underlying count can dip below it for an instant when a flush's decrement is observed before the matching apply increment, and that dip is an artefact of reading two independent updates rather than a real figure.

Configuration. `config` is a copy of the family's current configuration (ch. 36), including any runtime changes applied since it was created. It is filled in here so a caller reading the numbers has the settings that produced them in the same struct, rather than having to correlate two calls that could disagree.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`cf` or `stats` is `NULL`
`TDB_ERR_LOCKED`	Descriptor pressure kept a level from being read. Transient — retry rather than reporting the family as empty
`TDB_ERR_MEMORY`	Allocation failed while collecting the level snapshot

15.2 tidesdb_cf_estimate_cardinality

Estimate a family's distinct key count.

Synopsis

```
int tidesdb_cf_estimate_cardinality(tidesdb_column_family_t *cf, uint64_t *out_estimate);
```

Description

Estimates how many **distinct** keys the family holds. This is not the same as `total_keys` from `tidesdb_get_cf_stats`, which counts entries across levels and therefore counts a key once per level that holds a version of it.

It is an estimate. Use it for planning, not for anything that must be exact — there is no exact distinct count short of a full scan.

Errors

Code	Cause
`TDB_ERR_INVALID_ARGS`	`cf` or `out_estimate` is `NULL`
`TDB_ERR_LOCKED`	Descriptor pressure kept a level from being read. Transient — retry
`TDB_ERR_MEMORY`	Allocation failed while collecting the level snapshot

15.3 tidesdb_get_db_stats

Collect database-level statistics.

Synopsis

```
int tidesdb_get_db_stats(tidesdb_t *db, tidesdb_db_stats_t *stats);
```

Description

Fills `stats` with figures spanning every family plus the shared write path.

Inventory. `num_column_families`, `total_sstable_count`, `total_data_size_bytes`, `num_open_sstables` (against the `max_open_sstables` budget), and `next_cf_index`.

Write path pressure. `immutable_memtable_count` is sealed memtables waiting to be flushed — the queue that backpressure watches. `compaction_pending_count`, `memtable_bytes`, and `is_flushing` complete the picture. A persistently non-zero `immutable_memtable_count` means flush is not keeping up with ingest.

`memtable_bytes` counts every version resident in the write buffer, not one per distinct key, and counts each by what it occupies rather than by the value it carries. A key overwritten repeatedly adds a version each time rather than replacing one, because a reader at an older snapshot may still need the version being superseded. A delete adds one too — a tombstone carries no value but is still a resident allocation — as does a put of an empty value, which is what a secondary index entry usually is. It is also the number the rotation threshold reads, so it is the one to watch if a memtable seems not to be sealing.

Backpressure. These four are the ones to alert on:

Field	Meaning
`writes_throttled`	Writers made to dwell before being admitted
`writes_blocked`	Writers made to wait for the queue to drain
`write_stall_us`	Total microseconds writers spent held in admission
`write_stall_ceiling_hits`	Writers admitted only because the wait ceiling expired

All zero means ingest is comfortably within what flush can absorb. `write_stall_ceiling_hits` climbing is the serious one: it means flush is not draining and writes were let through anyway to avoid turning a slow database into a stuck one.

Two separate pressures feed these four: the immutable memtable queue, and the write-ahead log's staging ring. That is worth knowing when reading them, because the two look different — a rising `write_stall_us` with a flat `writes_blocked` is the ring band doing its job, applying many small dwells rather than one long stop.

MVCC. `global_seq` is the current sequence; `min_snapshot_seq` is the oldest snapshot any live transaction holds, and it is what gates tombstone reclamation — a long-running transaction holds it back and keeps garbage alive. `active_txn_count` and `txn_memory_bytes` cover the live transactions themselves.

Value log. `vlog_file_size`, `vlog_value_count` and `vlog_used_bytes` describe the contents; `vlog_segment_count` is how many files they are spread across, one of which is taking appends.

`vlog_bytes_written` is cumulative rather than a description of the contents, counting every byte ever appended including the rewrites reclamation performs. It is the term write amplification needs from the log, and it is the only value-log field that answers what the device was asked to write rather than what is held now.

`vlog_stored_bytes` is the on-disk length of the indexed values. It is only a meaningful ratio against `vlog_used_bytes` when the whole store was written under one pipeline — use `tidesdb_get_vlog_encoding_stats` instead, which keeps the chains apart.

`vlog_live_bytes` is what the installed sstables still reference, summed from what each records about the segments its separated values landed in. **It is the figure space amplification is against.** `vlog_used_bytes` is not: the index names every value whose segment has not been dropped, reachable or not, so it converges on the file size and reports a store full of garbage as entirely live.

`vlog_dead_bytes` is the gap between file size and used bytes — space held by values no live key references, and what a reclaim can recover.

The rest describe reclamation itself, and reset when the handle reopens because they measure what this process has done rather than what is on disk. `vlog_reclaim_calls` counts every reclaim attempted; `vlog_reclaim_passes` counts only those that actually drained a segment, and `vlog_segments_retired` counts the files freed. **Read calls against passes**: a reclaim that is never called and one that is called and finds nothing to do are different faults with different fixes, and the passes count alone cannot tell them apart. `vlog_segments_drainable` counts the sealed segments holding so little live data that emptying them is worthwhile — those are emptied by the next compaction that carries one of their values, not by the store itself.

Read `vlog_dead_bytes` against `vlog_segments_drainable`. The first is the work outstanding and the second is how much of it is currently actionable. Dead bytes climbing while the drainable count stays high means compaction is not reaching those segments; dead bytes climbing with nothing drainable means the garbage is spread thin across segments that are still mostly live.

Encoding stats

```
int tidesdb_get_klog_encoding_stats(tidesdb_t *db, tidesdb_encoding_stats_t *out, size_t max,
                                    size_t *out_count);
int tidesdb_get_vlog_encoding_stats(tidesdb_t *db, tidesdb_encoding_stats_t *out, size_t max,
                                    size_t *out_count);
```

`out` is caller-allocated with capacity `max`, and `out_count` receives how many entries were written; sizing `out` beyond `TDB_MAX_ENCODING_CHAINS` gains nothing, since that is the most that can exist. Both return `TDB_SUCCESS`, or `TDB_ERR_INVALID_ARGS` on a `NULL` argument.

`tidesdb_get_klog_encoding_stats` and `tidesdb_get_vlog_encoding_stats` report one row per encoding chain, up to `TDB_MAX_ENCODING_CHAINS`. Each row names the chain in `ids` (the codec ids in the order applied, with `id_count` of them, and empty when the data was stored verbatim) and then reports `logical_bytes`, `stored_bytes`, and `item_count` — values for the value log, sstables for the key logs. **`logical_bytes / stored_bytes` is that codec's realised ratio on the data it actually wrote.**

They are reported per chain rather than per column family for a reason. A family can change its codec, and compaction rewrites data under whichever pipeline is merging it, so a single figure for a family averages across settings that no longer apply. A key log is attributed by the pipeline recorded in its own footer; a value is attributed by the chain recorded in its own block header. A family mid-migration shows two rows, which is the truth, rather than one number that is not.

Durability. `wal_generation` is the current write-ahead log generation, incremented on each rotation. `flush_count` and `compaction_count` are cumulative, alongside the same byte counters as the per-family stats.

Errors

`TDB_ERR_INVALID_ARGS` if `db` or `stats` is `NULL`.

15.4 tidesdb_get_stall_stats

Report where writers have been made to wait.

Synopsis

```
int tidesdb_get_stall_stats(tidesdb_t *db, tidesdb_stall_stats_t *stats);
const char *tidesdb_stall_reason_name(tidesdb_stall_reason_t reason);
```

Description

A write latency tail is one of the hardest things to explain from the outside: the p50 is fine, a handful of commits take a thousand times longer, and nothing in the ordinary statistics says which part of the engine held them. This answers that directly. Every place a caller's own thread can be made to wait reports a **count**, a **total**, and the **longest single wait** — and it is the last one that a tail is made of, since a total cannot tell many short waits from one long one.

Reason	Constant	The caller was
`wal_append`	`TDB_STALL_WAL_APPEND`	waiting on the write-ahead log — for staging-ring space, or under a syncing mode for its record to reach the file
`rotate_lock`	`TDB_STALL_ROTATE_LOCK`	waiting to take the rotation lock, so another committer was rotating

Reason	Constant	The caller was
`rotate_work`	`TDB_STALL_ROTATE_WORK`	performing the rotation itself, which one committer pays on every other's behalf
`admission`	`TDB_STALL_ADMISSION`	held by write admission because the unflushed backlog was too deep

The short name is what `tidesdb_stall_reason_name` returns; the constant is how you index `reasons[]` for one particular reason. `TDB_STALL_COUNT` is the number of reasons, not itself a reason.

Read it by comparing each reason's `max_us` against the tail you measured. If one of them matches the tail, that is where the time went. If `wal_append` dominates, the log is the constraint and the next question is whether the device can keep up — compare the bytes written against what the device can sustain, because a saturated disk and a stalled engine look identical from the application.

The totals are cumulative since the database opened and never reset, so sample twice and subtract to attribute a particular window.

Return Value

`TDB_SUCCESS`, or `TDB_ERR_INVALID_ARGS` if `db` or `stats` is `NULL`. `tidesdb_stall_reason_name` returns a stable short name, or `"unknown"` outside the enum.

Thread Safety

Safe from any thread at any time. The counters are relaxed atomics on the write path — they are reported, never decided on — so reading them neither blocks writers nor perturbs what it measures.

Examples

```
tidesdb_stall_stats_t stalls;
if (tidesdb_get_stall_stats(db, &stalls) == TDB_SUCCESS)
    for (int i = 0; i < TDB_STALL_COUNT; i++)
        printf("%-12s count=%llu total_ms=%.1f max_ms=%.1f\n",
               tidesdb_stall_reason_name((tidesdb_stall_reason_t)i),
               (unsigned long long)stalls.reasons[i].count,
               (double)stalls.reasons[i].total_us / 1000.0,
               (double)stalls.reasons[i].max_us / 1000.0);
```

See Also

`tidesdb_get_db_stats`

15.5 tidesdb_get_io_stats

Report what each class of file asked of the device.

Synopsis

```
int tidesdb_get_io_stats(tidesdb_t *db, tidesdb_io_stats_t *stats);
const char *tidesdb_io_class_name(tidesdb_io_class_t cls);
```

Description

This is the other half of `tidesdb_get_stall_stats`. That one says writers waited on the log; this one says whether the device was the reason, and how much traffic the rest of the engine was putting beside it.

Class	Constant	Written by
`sstable`	`TDB_IO_SSTABLE`	flush and compaction, writing key logs
`wal`	`TDB_IO_WAL`	the write-ahead log's own single flush thread

As with the stall reasons, the short name is what `tidesdb_io_class_name` returns and the constant is how you index `classes[]`; `TDB_IO_COUNT` is the number of classes, not itself a class.

Each class reports `ops` writes issued, `bytes` written, `total_us` summed inside those writes, and `max_us` for the slowest single one.

Read `bytes` over `total_us` as the throughput that class achieved, and `bytes` over `ops` as the average write size — a class issuing many small writes pays per-call overhead that the throughput figure alone hides. The comparison that matters is between the two classes: if `sstable` is moving several

times the bytes the `wal` is, the log's writes are queued behind that traffic, and the lever is **write amplification** rather than anything in the log.

> Caution—**What the timings measure**
> These time the write call, not the platter. Under `TDB_SYNC_NONE` a write returns once it reaches the page cache, so the reported rates can far exceed what the storage can sustain and the device catches up afterwards. Under `TDB_SYNC_FULL` they are device time. In either mode compare the **bytes** between classes — that ratio is real — and cross-check absolute throughput against a `dd` control on the same filesystem.

Only handles opened through the engine's descriptor manager are counted, which is every key log and every write-ahead log. The value log and the manifest are not, so this measures the two classes that compete for the device under load rather than every byte the database writes.

Totals are cumulative since open and never reset, so sample twice and subtract for a window.

Return Value

`TDB_SUCCESS`, or `TDB_ERR_INVALID_ARGS` if `db` or `stats` is `NULL`. `tidesdb_io_class_name` returns a stable short name, or `"unknown"` outside the enum.

Thread Safety

Safe from any thread at any time; the counters are relaxed atomics on the write path.

See Also

`tidesdb_get_stall_stats`, `tidesdb_get_db_stats`

15.6 tidesdb_get_cache_stats

Collect block cache statistics.

Synopsis

```
int tidesdb_get_cache_stats(tidesdb_t *db, tidesdb_cache_stats_t *stats);
```

Description

Fills `stats` for the database-wide block cache, which every column family shares.

Field	Meaning
`enabled`	1 when a cache is configured
`total_entries` / `total_bytes`	Current residency
`hits` / `misses`	Cumulative since open
`hit_rate`	`hits / (hits + misses)`, 0 when neither has happened
`num_partitions`	Shards, derived from the configured size and the CPU count

`hit_rate` is the number that matters. A low rate on a read-heavy workload usually means `block_cache_size` is too small for the working set. Because it is cumulative since open, it is slow to reflect a recent change — compare deltas between two samples rather than the absolute value.

Errors

`TDB_ERR_INVALID_ARGS` if `db` or `stats` is `NULL`.

15.7 tidesdb_range_stats

Describe a key range for a query planner.

Synopsis

```
typedef struct
{
    uint64_t sstables_overlapping;
    uint64_t estimated_keys;
```

```
    int keys_exact;
} tidesdb_range_stats_t;

int tidesdb_range_stats(tidesdb_t *db, tidesdb_column_family_t *cf, const uint8_t *key_a,
                        size_t key_a_size, const uint8_t *key_b, size_t key_b_size,
                        tidesdb_range_stats_t *out);
```

Description

Answers the two questions a planner asks about `[key_a, key_b)` — what a scan of it would cost, and how many rows it would return. Both come from **one layout snapshot**, so they describe the same instant rather than two moments the caller cannot distinguish.

`sstables_overlapping` is the number of sorted runs a scan would merge. That is the shape of the scan's cost, and it is what decides between a range scan and a series of point lookups.

`estimated_keys` is the live key count. It is **memtable-aware**: a range whose data has not been flushed yet reports a real cardinality rather than zero, and a key that was flushed and then rewritten counts once rather than twice. Tombstoned keys are excluded, so a heavily deleted range reports what a reader would actually see.

`keys_exact` is what makes the count usable. A range small enough to walk is **counted**, and this flag is set. A wider one is estimated from sstable metadata without walking, and the flag is clear. A planner can commit to an exact figure and hedge on an estimated one — which it cannot do if both arrive as the same opaque number.

The gating is what keeps the call cheap at plan time. The metadata pass runs first and decides whether walking is worth it, so a wide range costs no more than the overlap count alone, and the walk only happens when it is provably short.

The estimate takes each overlapping sstable's keys **in proportion to the share of its own key span the range covers**, interpolating both against the bounds the file records. Counting an overlapping file whole would be badly wrong in the shape that matters most: a file flushed from a memtable that held interleaved writes spans the entire key space, so every narrow range meets it, and every narrow range would be told it holds the entire store. That error compounds — an inflated estimate also pushes the range past the threshold under which it would have been counted exactly, so the caller loses the precise answer as well as getting a wrong approximate one.

Interpolation assumes keys are spread evenly within a file, which is rough for skewed key distributions. It is calibrated for being in the right order of magnitude rather than being exact, and the narrow ranges where precision matters most are the ones that fall under the threshold and get counted.

Errors

`TDB_ERR_INVALID_ARGS` if `db`, `cf`, either key, or `out` is `NULL`. Note that unlike the range functions elsewhere in the API, **neither key may be `NULL`** — there is no unbounded form.

`TDB_ERR_LOCKED` if the level layout changed while it was being read; the call is safe to retry.

`TDB_ERR_MEMORY` if the working set of sstable entries could not be allocated.

Part VI

Internals

Architecture

TidesDB is a log-structured merge-tree. Writes go to memory and a log; memory fills and is written out as a sorted file; files accumulate and are merged. Everything else in this part is detail on those three sentences.

This chapter is the map. The two that follow it walk the whole system twice — once following a write (ch. 17), once following a read (ch. 18) — and after that each subsystem gets a chapter of its own. Read the three in order and the rest can be read in any order.

16.1 One rule shapes the whole codebase

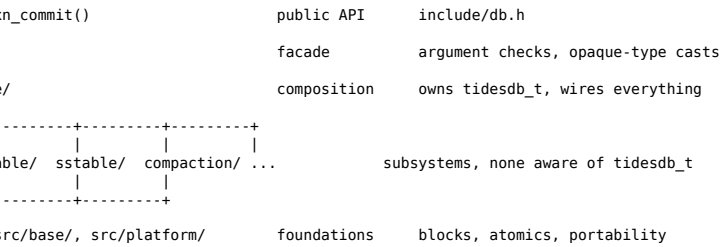
No module below the engine takes a `tidesdb_t`.

The database handle is the composition root's own type. The memtable does not know what a column family is; the block manager does not know what a memtable is; the compaction planner does not know what a file is. Each module takes the narrow set of things it actually needs and is constructed by the layer above it.

The immediate payoff is testability. The compaction planner is a pure function over a snapshot of level sizes — no I/O, no handles, no clock — so it is tested by handing it numbers. The block manager is tested against a temp file with no database anywhere. The fd manager is tested with a stack-allocated instance. A module that reached for a global database handle could not be tested this way, and in practice would not be tested at all.

The longer-term payoff is that the seams stay honest. When a module needs something it was not given, that is a design question surfaced at the function signature rather than a field quietly read from a global.

16.2 The layers



`src/db.c` is deliberately thin. It validates arguments, casts the opaque public types to their internal ones, and forwards. Nothing decides anything there, which is why the public API can be read as a list of intentions and the engine as a list of mechanisms.

16.3 Module map

Directory	Responsibility
`src/engine/`	The composition root. Owns `tidesdb_t`, open and close, recovery, the worker pools, and the orchestration of flush and compaction.
`src/txn/`	Transactions, MVCC sequences and snapshots, write and read sets, conflict reservations, two-phase commit, WAL record encoding, and the L0 adapter binding the txn core to the memtable.
`src/memtable/`	The one shared in-memory write buffer and its write-ahead log, rotation, the immutable queue, and reader-pinned reclamation.
`src/sstable/`	The on-disk sorted table: a btree key log, the shared value log, the partition range filter, and a self-describing footer.

Directory	Responsibility
`src/column_family/`	Family configuration, the registry, and the level set that organises a family's open sstables into tiers.
`src/compaction/`	The Spooky planner (pure, deterministic) and the executor that runs the jobs it emits.
`src/flush/`	Turning one sealed memtable into per-family L1 sstables under a single atomic manifest commit.
`src/manifest/`	The durable catalogue of what exists, shared by every family so a flush across families is atomic.
`src/iter/`	The k-way merge over a versioned source vtable that backs every iterator and every compaction.
`src/cache/`	The database-wide block cache every on-disk block read goes through.
`src/fdmanager/`	A descriptor budget that waits out file-descriptor pressure inside the engine, so exhaustion becomes backpressure instead of failure.
`src/io/`	The block manager: framing, durability, preallocation, and the buffered append ring.
`src/datastructures/`	The skip list backing the memtable, and the queue backing the work queues.
`src/base/`	Errors, logging, serialization, arena allocation, and the lock-free reclamation primitives everything else uses.
`src/internal/`	The memtable type and the two tombstone flag bits the write path carries from the memtable down into an sstable.
`src/platform/`, `src/threadmanager/`	Portability and thread pools.

16.4 What is shared and what is per-family

This distinction explains a great deal of behaviour that is otherwise surprising, so it is worth fixing early.

Shared by the whole database: the memtable, the write-ahead log, the value log, the manifest, the block cache, the descriptor budget, and the sequence clock.

Per column family: the level set and its sstables, the compaction policy and triggers, and the family's configuration.

One shared memtable is why a transaction spanning several families is atomic — there is one log and one apply. It is also why `tidesdb_flush_memtable` (ch. 14) flushes everything rather than one family, and why cloning a family costs a database-wide flush — the clone copies files, so every committed write has to be in one first. Renaming costs no flush, because a family has no directory of its own — every key log sits in the database directory carrying its family's **id** in its file name, so a new name moves nothing.

Keys are namespaced inside the shared structures by a four-byte big-endian family index prefix, so one skip list and one log hold every family's data in one ordered space.

16.4.1 The family registry lock

The families themselves live in a registry behind a read-write lock, and which side of it a thread takes is not evenly matched.

Almost everything takes it for **reading**: every flush install, every compaction claim, every reaper sweep, every statistics fold. Under load these arrive continuously, and none of them is the caller's work. Only `create`, `drop`, `rename` and `clone` take it for **writing**, and those are exactly the operations a user issues and waits on.

That asymmetry makes the default lock policy the wrong one. glibc's rwlock prefers readers, so a new reader is admitted even while a writer is queued – and a database whose background work never stops never produces the gap the writer needs. The writer is not slow; it never runs. A `CREATE TABLE` hangs for minutes while the process sits at a full core, which reads as a deadlock and is really starvation.

So the registry lock is initialised to prefer a waiting writer, and the read side is kept short enough that the writer's wait is bounded by one operation rather than by the load. The one rule that buys is that **no reader may take the lock again while holding it**, and none does; the attribute deadlocks a reader that tries.

Two things follow from the same reasoning. A flush waits for its install ticket *before* taking the read lock, never while holding it – waiting under it would both lengthen the hold to include every flush ahead and close a cycle, since the worker holding the earlier ticket needs the same read lock to install. And a flush never waits for a reader to leave in order to free a memtable; it hands the memtable to the reaper instead. See Memtable and WAL (ch. 19).

16.5 Threads

Thread	Work
Caller threads	Everything on the transaction path: buffering, conflict checks, WAL append, memtable apply. A commit does its own work rather than handing it to a worker.
WAL flush thread	One per open write-ahead log. The only thread that writes that file.
Flush pool	Turning sealed memtables into sstables. Sized by `num_flush_threads`.
Compaction pool	Merging sstables. Sized by `num_compaction_threads`.
Reaper	Closing idle file descriptors back down to the budget, and reclaiming what the paths that produced it could not free inline – drained level-set layouts, and sealed memtables a flush had to leave pinned.
Compaction scheduler	Reconsidering every family for a merge once a second, so a family still becomes due without a flush to wake it.
Idle flush	Rotating an active memtable nothing has written to, unless `memtable_idle_flush_seconds` is zero.
Transaction clock	Publishing the current second, once a second, for transaction timeouts to age against.
Sync ticker	Under `TDB_SYNC_INTERVAL`, the periodic durability barrier.

Rotation is the interesting case: it runs on **whichever committing thread finds the memtable full**, not on a worker. That thread pays the latency, and every other committer that agrees waits behind the same lock — which is why a slow rotation shows up directly in the write latency tail, and why the engine logs one when it exceeds 20 ms.

16.6 Where the durability guarantees come from

Three files carry everything, and the order they are written in is the whole recovery story:

1. **The write-ahead log** (`NNNNNNN.log`) takes a committed batch before the memtable does. A crash between the two recovers the batch.
2. **The sstables** (`CCCCCCCCCC.SSSSSSS.klog`, named for the owning family's id and the table's) and the **value log** (`NNNNNNN.vlog` segments) hold what has been flushed. They are durable before the manifest names them.
3. **The manifest** (`MANIFEST`) is the catalogue. A file it does not name does not exist as far as recovery is concerned, so a crash mid-flush leaves orphans rather than corruption.

Each of those is a block manager file with the same framing, so one recovery discipline covers all of them. The appendix (ch. 35) has the byte layouts.

CHAPTER 17

The Life of a Write

This chapter follows a single key from `tidesdb_txn_put` to its final resting place in a compacted sstable. It crosses most of the engine. Nothing here is described in full — each subsystem has its own chapter — but the order of events is exact, because almost every durability property falls out of that order.

17.1 Stage 1 — Buffering

`tidesdb_txn_put` copies the key and value into the transaction's write set and returns. Nothing else happens. The write is invisible to every other transaction and not durable in any sense.

Copying rather than borrowing is deliberate: a caller may free its buffers the moment the call returns, and a transaction may live a long time before it commits.

The write set preserves insertion order, which is what makes savepoints work — a savepoint is a position in that sequence, and rolling back to it discards the tail.

This is also where a transaction that has outlived a timeout finds out. Nothing ages it in the background; the check happens on the way into an operation, so a stale transaction is aborted here and this `put` returns `TDB_ERR_TXN_EXPIRED` rather than buffering anything.

17.2 Stage 2 — Commit begins: conflict detection

A read-only transaction stops here: with nothing in the write set there is nothing durable to do, so it is marked committed and leaves.

For everything else, the isolation level decides what is validated, and the two checks are not stacked one on top of the other — each level runs the one that suits how it prevents anomalies.

- Below `TDB_ISOLATION_REPEATABLE_READ` there is no check at all; commit proceeds.
- `TDB_ISOLATION_REPEATABLE_READ` and `TDB_ISOLATION_SERIALIZABLE` validate the **read footprint**: every key the transaction read is probed for a newer committed version, which is what stops a non-repeatable read or a phantom.
- `TDB_ISOLATION_SNAPSHOT` and `TDB_ISOLATION_SERIALIZABLE` scan their **write keys** for a version newer than the snapshot — a writer that already applied and so is invisible to the reservation taken in stage 4.
- `TDB_ISOLATION_SERIALIZABLE` alone then runs the dangerous-structure check that catches write skew.

A range delete (ch. 12) is scanned over its interval rather than as a key: *does any key in these bounds sit above my snapshot*. Probing its lower bound as though it were a key would clear a commit about to delete data another transaction had just written.

NOTE—**A batch carrying an interval commits alone**

At snapshot and above, a commit that deletes a range takes the database's commit gate exclusively; every other commit at those levels holds it shared and never waits on another. The gate is held for the **whole commit**, not just the reservation, because a batch that has reserved but not yet marked its sequence visible is precisely the write the scan above would otherwise miss.

A two-phase transaction takes it only for its prepare — the in-doubt window that follows has no bound, and the interval reservation in stage 4 covers that window instead.

Snapshot deliberately does not validate reads: first-committer-wins reservation is how it prevents lost updates, and validating the read set on top of it would abort transactions the level is defined to allow.

A conflict aborts here, before anything durable has been written.

17.3 Stage 3 — Sequencing

The transaction draws a commit sequence from the database clock and immediately marks it **in progress**.

The mark matters. A sequence that has been drawn but not completed must be invisible: a reader whose snapshot is above it must not see a half-applied batch. Marking it in progress first, and committed only at the very end, is what makes the batch atomic to readers without a lock.

17.4 Stage 4 — Reservation

Under snapshot and serializable isolation, every key in the batch is reserved on a first-committer-wins basis.

The subtlety is what each key is validated *against*. Not the transaction's snapshot, but **the version this transaction actually read for that key** — recorded when `tidesdb_txn_get` (ch. 12) was called. A blind write with no prior read falls back to the snapshot. This is why a read-modify-write is checked correctly while a blind overwrite is not penalised for reading nothing, and it is why `tidesdb_txn_get_notrack` (ch. 12) exists: a probe that does not feed the write should not narrow the window the write is validated over.

An interval has no key to hash, so a range delete claims the interval itself in a second, small table, and every point write checks that table before taking its own slot. The table is almost always empty, so the check is a single relaxed load. The claim is released when the transaction resolves, which is what carries a two-phase range delete through its in-doubt window.

A lost reservation is `TDB_ERR_CONFLICT`, and still nothing durable has happened.

17.5 Stage 5 — Admission

Before the durable write, the batch is paced once per distinct column family it touches.

This is where a writer waits when the engine is not keeping up. **Three signals feed it, and the strongest wins** — none is allowed to mask another:

- **The queue of sealed memtables awaiting flush.** As it fills, writers are first made to dwell, then held at the limit. This is the one `memtable_l0_queue_stall_threshold` configures.
- **The write-ahead log's staging ring.** Its lag is bounded whatever the queue is doing, so the ring paces ingest on its own. Its capacity comes from `memtable_write_buffer_size`, not from the queue threshold.
- **The depth of overlapping runs in L1.** A tier that has run past its slow mark paces writers so compaction can catch up.

The practical consequence is that **raising the queue threshold does not necessarily reduce stalling**, because the dwell may not be coming from the queue at all. Raising it sixteenfold on a write-heavy run here changed the total stall not at all; the memtable size did, by a factor of three, because it sizes the ring and lowers the flush rate at once.

There is a ceiling on the wait — a writer held too long is admitted regardless, because a flush that never drains would otherwise turn a slow database into a stuck one. Each of those outcomes is counted, and `write_stall_ceiling_hits` climbing is the serious one.

17.6 Stage 6 — Separating the large values

Before the record is written, every value at or above the database's `value_separation_threshold` is appended to the value log (ch. 22), and its entry keeps the returned id and the value's logical length in place of the bytes. A family that set `keep_values_inline` is left out.

The order is what makes it safe. It runs after the reservation and the admission wait have both let this commit through, so a conflict or a refused admission leaves nothing behind in the value log; and it runs before the record naming the values is appended, so the bytes are on the device before anything points at them. Under a syncing mode both are barriered before the commit is acknowledged.

What this buys is that a large value reaches the device **once**. Written inline, it goes into the log and then again into the sstable the flush builds; written here, the log record carries an id of a few bytes and the flush carries the same id forward. On a 4 KiB-value workload that is the difference between a write amplification of 2.10 and 1.06.

A commit that fails after this point leaves its value behind as garbage, which the value log's own reclamation is what clears.

17.7 Stage 7 — The write-ahead log

The batch is encoded as one self-describing record — a version byte, a kind, an optional transaction id, then the entries — and appended to the active log.

The append does not write to the file directly. It reserves space in the log's **staging ring** with a single atomic fetch-and-add, copies its bytes into that reservation, and marks its slot complete. A dedicated flush thread — the only thread that ever writes this file — drains the contiguous run of completed records at the frontier and writes it out in one call. The committer then waits for the frontier to pass its own record.

Three properties come out of this arrangement:

- **Concurrent committers do not serialize.** Space allocation is one atomic; the copy is parallel.
- **The log is written in offset order with no gaps.** The flush thread only ever emits the *contiguous* completed run, so a record can never be written before one that precedes it. This is load-bearing — see below.
- **Durability is the sync mode's business, not the caller's.** Waiting for the frontier means the bytes reached the file; whether that means the page cache or the device depends on how the file was opened.

Caution—**The log must be gap-free**

Recovery replays the log by reading blocks forward and **stops at the first one it cannot read**. A hole would not be skipped — everything after it would be silently discarded. The single-writer-contiguous-run rule is what guarantees no hole exists, and any change to the append path has to preserve it.

17.8 Stage 8 — Apply

Only now does the batch enter the memtable, at the commit sequence drawn in stage 3.

The order is the point. The log has the batch before memory does, so a crash between the two recovers it. The reverse order would make the write visible before it was recoverable.

If the apply fails, the batch is already durable, and replay treats a write batch's presence as its commitment — so left alone the transaction would come back whole on the next open, after its caller was told it failed. Three things happen instead, in this order:

1. An **abort record** naming the sequence is appended, which is what replay consults to leave the batch out.
2. The backend **abandons** the sequence, keeping whatever the failed apply already landed out of reads and out of any flush before that next open.
3. The reservations are released and the transaction is marked aborted.

The abort record is best effort by construction: if appending it fails there is nothing further to try, so the failure is logged rather than returned, and the batch would replay on the next open. That is the one case where a caller told its transaction failed can still see it applied, and it is why the failure is logged loudly rather than swallowed.

17.9 Stage 9 — Publication

The sequence is marked committed, and the batch becomes visible to every reader whose snapshot is at or above it. The transaction leaves the registry, which releases the snapshot it was holding — and with it, possibly, the floor that was keeping older versions alive.

Any commit hooks registered on the affected families fire here, synchronously, on the committing thread. Work done in a hook is latency every writer pays.

17.10 Stage 10 — Rotation

The committing thread then checks whether its write filled the memtable. If so, **that thread** performs the rotation: it seals the active memtable, enqueues it, and installs a fresh one.

The order within the rotation is exact: the sealed memtable is **enqueued before** the active slot is swapped. For a moment it is reachable both ways. The alternative — swap then enqueue — leaves a window where a reader sees the new active memtable and cannot see the sealed one, and reads a key that exists as absent.

Rotation does **not** wait for the sealed memtable to be written out, and does not drain its log's staging ring. The sealed memtable keeps its log alive, which is why a committer waiting on that log must hold a pin on the memtable across the wait.

17.11 Stage 11 — Flush

A flush worker takes a sealed memtable and demultiplexes it: one shared skip list holding every family's keys, prefixed by family index, becomes one sstable per family that has data in it.

The flush walks each key's **version chain** and keeps what the reclamation floor still protects: every version above it, and the newest at or below it as the base. That is the rule a merge applies. With nothing registered the floor sits at the top, so the flush writes the newest version alone and this costs nothing until a reader holds a snapshot under an overwrite. See Memtable and WAL (ch. 19).

Each output is built, then made durable, and only then are they all recorded in the manifest in **one atomic commit**. That is why the manifest is database-level rather than per-family: a flush that produced sstables for five families publishes all five or none.

The install that follows the commit can still fail — it allocates, and nothing else about it can go wrong — so an output that never reaches a level set has its catalogue entry **withdrawn**, and its file is unlinked only once that withdrawal is itself durable. Unlinking first would leave a committed catalogue naming a file that is already gone. The memtable is left for a retry either way, its data still in the log, and the withdrawal is what stops the retry from writing the same keys a second time beside files nothing will ever read.

Installs are ordered by a ticket. Workers may build in parallel — building is the expensive part — but each waits its turn to install, so the level sets are mutated in the same order the memtables were sealed. Out-of-order installs would put an older sstable above a newer one.

That wait happens **before** the worker takes the family registry read lock, not while holding it. Both the build and the install need that lock, so the families they address cannot be dropped underneath them — but holding it across the wait as well would stretch one worker's hold to cover every flush queued ahead of it, and a `create` or `drop` waits behind exactly that. It would also deadlock outright once writers are preferred, since the worker holding the earlier ticket needs the same read lock to make progress. The families are therefore re-resolved by id after the wait, and an output whose family was dropped in the gap is discarded rather than installed.

Once installed, the sealed memtable is reclaimed and its log can be unlinked. Reclamation needs every reader that had it pinned to leave first, and the flush does not wait for that indefinitely: it rechecks a bounded number of times, and if a reader is still inside it hands the memtable to a pending list the reaper sweeps. The flush is holding locks a family create or drop needs, so a wait there is a wait the whole database feels — see Memtable and WAL (ch. 19).

17.12 Stage 12 — Compaction

The key now lives in L1, alongside every other key flushed at the same time. From here the compaction planner decides when it moves down.

The planner is a **pure function**: given a snapshot of level sizes and file counts plus the family's configuration, it computes the level capacities, the dividing level, the target of the next merge, the cap on how large a single output file may grow, and whether any trigger is due. No I/O, no handles, no clock — the same snapshot always produces the same plan, which is what makes compaction policy testable by handing it numbers.

Three things make a merge due: L1 accumulating too many files, a level exceeding its capacity, and tombstone density crossing its threshold.

The merge reads its inputs through the same k-way merge iterator that serves user iterators, but asks it for a raw stream — every version of every key — and applies its own retention to it. The newest version survives; older ones are dropped, but only those older than the **oldest live snapshot**. A long-running transaction holds that floor down and keeps garbage alive; this is visible as `min_snapshot_seq`.

A tombstone is the hard case. It can only be dropped when the merge can prove no older version of that key survives beneath it. Drop it too early and the older version is resurrected. This is why deletes cost space until a merge deep enough to prove it happens, and why a single-delete — where the caller promises the key was written at most once — can annihilate immediately.

17.13 The whole path

```
tidesdb_txn_put         copy into the write set
      |
tidesdb_txn_commit
      |
   conflict check       snapshot+ only, aborts before anything durable
      |
   draw sequence        marked in progress -> invisible
      |
   reserve keys         first-committer-wins, vs the version actually read
      |
   admission            paced against the flush queue
      |
   separate values      large ones to the value log; the record carries the id
      |                 after the reservation, before the record naming them
   WAL append           ring reservation -> parallel copy -> ordered drain
      |                                    ^ gap-free, or recovery truncates
   memtable apply       durable before visible
      |
   mark committed       now visible; hooks fire
      |
   maybe rotate         on this thread; enqueue before swap
      |
   flush                demux to per-family sstables, one atomic manifest commit,
      |                 installs ordered by ticket
   compaction           merge down; drop versions below min_snapshot_seq;
                        drop a tombstone only when nothing survives beneath it
```

CHAPTER 18

The Life of a Read

A read has to answer one question — *what is the newest version of this key visible at this snapshot?* — against data spread across a transaction's own buffer, memory, and several tiers of files. This chapter follows one `tidesdb_txn_get` down through all of them.

The whole design turns on a single idea: **the sources are ordered newest-first, so the first hit is the answer.** Everything else is the consequences of making that true and keeping it true while other threads are writing, flushing, and merging.

18.1 The snapshot decides what "visible" means

Before anything is searched, the transaction's snapshot fixes a sequence ceiling. A version is visible if its sequence is at or below it.

Isolation	Ceiling
`TDB_ISOLATION_READ_UNCOMMITTED`	Unbounded — everything, including sequences still in progress
`TDB_ISOLATION_READ_COMMITTED`	The current sequence, re-read per operation
Repeatable read and above	Frozen when the transaction began

A sequence that has been drawn but not yet marked committed is invisible at every level except read-uncommitted. That is what makes a multi-key batch atomic to readers without any lock: the batch's sequence flips from in-progress to committed in one step, and until it does, none of it can be seen.

18.2 Stage 1 — The transaction's own writes

The transaction's write set is consulted first. A transaction always sees its own uncommitted writes, and a key it has deleted reads as absent to it even though the tombstone is not durable anywhere.

If the read is a tracking `tidesdb_txn_get` (ch. 12), the key and the version found are recorded in the read footprint here. That record is what a later commit validates against — see The life of a write (ch. 17).

18.3 Stage 2 — The source stack

Past the transaction's own state, the read walks an ordered list of sources:

```
sources[0]   L0    the active memtable, then the sealed ones awaiting flush
sources[1]   cf    dispatch into this column family's sstable levels
```

The walk stops on the first source that answers definitively:

```
if (r == TDB_SOURCE_FOUND || r == TDB_SOURCE_BUSY) return r;
```

`FOUND` stopping is the newest-first rule paying off. **`BUSY` stopping is the subtle part.** A busy source is one that could not answer right now — a memtable slot being rotated out from under the reader, an sstable whose descriptor budget is exhausted. Falling through to an older source would find a *stale* version and return it as the answer, because the busy source might have held a newer one. So contention is reported to the caller as `TDB_ERR_LOCKED` and retried, rather than silently answered wrong.

This is why treating `TDB_ERR_LOCKED` as "not found" is a correctness bug and not merely a missed retry.

18.4 Stage 3 — L0: memory

The L0 source searches the active memtable, then each sealed memtable in the queue, newest first.

Two hazards live here, both about a rotation happening underneath the reader.

The memtable must not be freed while being read. A reader pins what it loads: the active slot is guarded by a reader epoch, and a sealed memtable carries a reference count. Reclaiming a memtable waits for every reader to drop out. The pin is also what keeps a rotated memtable's *write-ahead log* alive — rotation seals the memtable but does not drain its log's ring, so a committer still waiting on that log holds the memtable to hold the log.

The set of memtables must not appear to shrink. Rotation enqueues the sealed memtable before swapping the active slot, so at worst a reader sees it twice, never zero times. A reader that walks the active slot and the queue non-atomically can still observe the set move under it — there is a visible-change counter for exactly this, and the read re-snapshots and retries rather than reporting a false absence.

Within one memtable the skip list holds a version chain per key. The read takes the newest version at or below the snapshot ceiling. **A memtable version may hold a reference rather than the bytes** — the commit separated it into the value log — and the L0 source resolves that before answering, so what the layers above it see is always bytes. It asks the value log directly rather than going back through this stack, which during recovery is reading the very memtables being rebuilt.

If that version is a tombstone, or its expiry deadline has passed, the answer is a definitive **absent** — not a fall-through to older sources. A tombstone shadowing an older value is the whole point of a tombstone.

One kind of version is stepped over rather than taken: a commit whose batch reached the log but failed to enter memory leaves entries at a sequence that never committed. The chain walk skips those and resolves to the next visible version beneath them, at every isolation level including read-uncommitted, so a batch its caller was told had failed cannot be read back as the key's value.

18.5 Stage 4 — The level set

If memory has nothing, the read dispatches into the family's levels. Their shapes differ, and the difference is not incidental:

- **L1 holds flush outputs, which may overlap in key range.** Two memtables flushed minutes apart can both contain the same key. So every L1 sstable whose key range covers the key must be consulted, newest first. L1 overlap depth is therefore a direct read-amplification cost, and it is what the L1 file-count trigger exists to bound.
- **L2 and below are non-overlapping runs sorted by minimum key.** At most one sstable per level can contain the key, and it is found by search rather than by scanning the level.

Levels are searched shallowest first, because shallower is newer. This is an invariant the whole read path rests on, and it is why flush installs are strictly ordered — an out-of-order install would place an older sstable above a newer one and quietly invert the answer.

The level set is read lock-free. Readers see an immutable layout under an epoch guard, and the layout itself holds a reference on every sstable in it, so a compaction that replaces the layout cannot free a table a reader is still inside.

A bitmap published with each layout says which levels hold anything, so a read skips the empty ones without touching them. The mask is a **snapshot**, though, and the levels it does visit are scanned against the live layout — so a merge that moves a table into a level the mask called empty would be skipped while the level it came from reads as already emptied.

That is harmless when a **flush** did it: a flush installs its sstables and only retires the memtable they came from afterwards, so through that whole window the keys are still in L0, which every read consults first. It is **not** harmless when a compaction did it, because a compaction's keys left L0 long ago and nothing backs them up. So the read records the layout generation before the walk and checks it again on a miss; a shape that moved underneath reports a retryable busy rather than an absence that was never true.

18.6 Stage 5 — Inside one sstable

Three steps, arranged so the expensive ones are usually skipped.

The partition range filter first. A definite miss means the key is not in this table, and the key log is never opened. Membership is version-agnostic — the filter answers about the key, not about any particular version — so a miss is conclusive regardless of the snapshot.

Then the descriptor budget. Opening the key log needs a file descriptor, and the budget is a soft ceiling. When it is exhausted the read does not force the open, and it does not hand the shortage straight back either: the gate wakes the reaper and rechecks a bounded number of times, so ordinary descriptor pressure is waited out inside the read. Only if it never clears does the read report busy — the `BUSY` from stage 2, retryable backpressure rather than an error, and by then the rare case rather than the routine one.

Then the btree key log. A search descends to the leaf holding the key. Leaves are prefix-compressed with an indirection table over the key suffixes, and sequences are stored as deltas from a per-node base, so a node holds many entries in little space. The search resolves the newest version at or below the snapshot ceiling — the MVCC filter is native to the btree rather than layered over it.

Every block read on this path goes through the database-wide block cache, keyed by the owning file's id and the block's byte offset. Cache reads are lock-free: probe a slot, pin it with a reference count, then recheck that the slot still holds what was probed. Entry slots are never freed while pinned; only payloads are reclaimed, exactly once, by whichever thread drops the last reference — evictor or reader, whoever is last.

The root is the exception, because every descent of a given key log starts at the same node. Pinning it per descent would put every reader of that sstable on one reference count's cache line, so the sstable reads its root once, holds that pin for as long as it is open, and lends the node to each descent unpinned. The btree the descent runs on is built per read, and lives far less long than the sstable holding the pin.

Only an internal root is held this way. A tree small enough that its root is also its leaf hands that node straight back to a caller who releases what it is given, so it takes the ordinary per-descent path instead. The cost of holding it is one node per open sstable that eviction cannot reclaim until the sstable closes.

18.6.1 A lapsed entry reads as a tombstone

A deadline is judged against the clock at the moment of the read, not against the reader's snapshot: a transaction older than the deadline still sees the key gone. That is the rule the memtable has always applied, and the sstable path applies the same one, because two rules would make a key flicker depending on which source happened to answer it.

The clock they compare against is not `time(NULL)`. A ticker publishes the current second once a second, and both the memtable's list and every sstable read that one published value. Reading the system clock instead would mean a call on **every entry a scan touches**, which is the wrong place to spend a syscall. The consequence is that a deadline is honoured within about a second of passing rather than instantly — expiry is a visibility rule, not a timer — and that the second in question is the same one for every source, which is what keeps them from disagreeing.

What it reads as matters as much as when. A lapsed entry answers like a **tombstone** — no value, nothing to resolve out of the value log — and not like an absence. Reading it as absent would let the source stack fall through to an older version of the same key in a deeper level, handing back a value the overwrite had already replaced. Shadowing is also what carries it into a merge as a tombstone, which is what eventually reclaims the bytes: the read only hides it.

18.7 Stage 6 — Fetching a separated value

Small values are held inline. Larger ones — at or above the database's `value_separation_threshold` — live in the shared value log, and whatever names them holds a reference instead: a memtable version, an sstable entry, or a prepared batch waiting on its decision. The committing transaction is what put them there, so a value is separated from the moment it is written rather than from the flush that first persists it.

So a point read of a separated value costs a second I/O, while a scan that only needs keys costs none at all. That asymmetry is the entire trade of key/value separation: it keeps the btree small and scans fast, and it makes large-value point reads more expensive. `tidesdb_txn_contains` (ch. 12) exists so an existence check need not pay it.

18.8 What the read returns

The first definitive answer wins:

- a **live version** at or below the snapshot — its value, copied for the caller

- a **tombstone** or an **expired** entry — `TDB_ERR_NOT_FOUND`
- nothing anywhere — `TDB_ERR_NOT_FOUND`
- a **busy** source — `TDB_ERR_LOCKED`, meaning *ask again*, not *absent*. Rare, because the pressure that produces it is waited out inside the read before it is reported

18.9 The whole path

```
snapshot ceiling fixed by isolation level
        |
transaction write set            own writes, own tombstones; read recorded here
        |
+-- source stack, newest first -- first FOUND or BUSY wins --------------------+
|                                                                              |
|  L0   active memtable            epoch-pinned; version chain, newest <= snap |
|       sealed memtables           ref-counted; enqueue-before-swap            |
|            |                                                                 |
|            +-- value log                only if the value was separated      |
|                                                                              |
|  cf   L1  overlapping            every covering table, newest first          |
|       L2+ non-overlapping runs   at most one table per level                 |
|            |                                                                 |
|            +-- partition range filter   definite miss -> skip the table      |
|            +-- descriptor budget        waits out pressure; BUSY if it stays |
|            +-- btree key log            newest version <= snapshot           |
|            +-- value log                only if the value was separated      |
+------------------------------------------------------------------------------+
```

Read amplification is the number of sources this walk actually touches, and the filters are what keep it near one. `read_amp` in `tidesdb_get_cf_stats` (ch. 15) estimates it, and it is the number to watch when reads slow down.

CHAPTER 19

Memtable and Write-Ahead Log

19.1 The problem

A write must become durable quickly and become queryable immediately, and those two demands pull in opposite directions. Durability wants sequential appends to a file. Queryability wants an ordered structure that can answer a lookup. Doing both per write would mean writing a sorted file on every commit.

The standard resolution is to do each separately: append to a log for durability, insert into an in-memory ordered structure for queryability, and periodically turn the accumulated memory into one sorted file. The log exists only to survive the gap between "committed" and "written out as a sorted file".

TidesDB adds one decision on top: **there is exactly one memtable for the whole database**, not one per column family.

19.2 Why one shared memtable

A transaction may touch several column families and must be atomic across all of them. With one memtable and one log, that atomicity is free — a single log record, a single apply. With per-family memtables it would require a commit protocol across them.

The cost is that the memtable is a shared resource. Flushing is database-wide because there is only one thing to flush, which is why `tidesdb_flush_memtable` (ch. 14) has no per-family form and why cloning a family pays a full flush – a clone copies files, so everything it should copy has to be in a file first. Renaming does not: a family's files are named for its immutable id, so a rename moves nothing and needs no flush at all.

Families are kept apart inside the shared structures by prefixing every key with a **4-byte big-endian family index**. One ordered keyspace holds every family, and each family's keys form a contiguous run within it. Flush exploits exactly this: demultiplexing the shared memtable into per-family sstables is a walk in order, splitting at prefix boundaries.

19.2.1 A flush retains against the same floor a merge does

A memtable holds a version chain per key. The flush walks all of it, and the **reclamation floor** decides what survives: every version above the floor, and the newest at or below it as the base. That is the rule a merge already applies, asked here too.

With no transaction registered the floor sits at the top, nothing is above it, and the newest version takes the base. An ordinary flush therefore writes one version per key and costs nothing extra. It carries more only while a reader holds a snapshot underneath one.

CAUTION—**Taking only the newest version silently breaks a frozen snapshot**

The floor exists because a reader can be sitting below an overwrite. A flush that wrote the newest version alone would drop every older one whatever the floor said, and a repeatable-read transaction whose snapshot sat below an overwrite would then find the key **absent** — not stale, absent, which is a live key reading as missing.

It is easy to miss because the floor is computed, published and honoured by compaction, so the protection looks like it is already there. Flush is the other half of the write path and has to ask the same question. The default isolation level re-reads on every operation and so never notices, which is why this survived: it needs a frozen snapshot, an overwrite, and a flush together.

One difference from a merge: a flush writes into L1, never the largest level, so it never drops a base tombstone. A tombstone at the shallowest tier is what shadows older versions in the levels below it, and dropping one there would bring a deleted key back.

19.3 Structure

A memtable is a pair — an ordered structure and the log that makes it recoverable — plus the state needed to retire it safely:

```
skip_list        the ordered structure; a version chain per key
wal              the block manager for this generation's log
id, generation   identity; the generation names the log file
refcount         structural + transient reader references
writers          an epoch guarding in-flight writers
flushed          set once the data is durable in L1
claimed          set when a flush worker has taken it
vlog_token       a value log floor, for the values only this memtable names
range_tombstones the intervals deleted in this generation, with their own lock and a
                 published fragment count so a read that needs none never takes it
```

The range tombstones sit beside the skip list rather than in it, because they are not keyed by anything the list is sorted on — an interval covers keys that may not exist yet. They are stored over the same family-prefixed keyspace, so one set serves every family, and a memtable that has never taken a range delete (ch. 12) costs a count of zero that every read short-circuits on.

The skip list stores a **version chain per key** rather than one value: several versions of the same key coexist, each with its sequence, and a read takes the newest at or below its snapshot. That is what makes MVCC work in memory without copying the structure per snapshot.

A version holds either the value or a reference to it. A value at or above the database's `value_separation_threshold` is written to the value log (ch. 22) by the committing transaction, and the version keeps the id and the value's logical length in place of the bytes. A read that lands on one resolves it through the value log; the flush that follows carries the id into the sstable rather than writing the bytes a second time.

That makes the memtable the only thing naming those values until its flush installs, so it takes a value log floor over their segments — `vlog_token` above — held from the moment it becomes the active memtable until that install. Taken then rather than at its first reference, because a value reaches the value log before the commit that produced it reaches any memtable.

19.3.1 The active memtable and the queue

At any moment there is one **active** memtable taking writes, and a queue of **sealed** memtables that are full and awaiting flush. Reads consult the active one first, then the queue newest-first — see The life of a read (ch. 18).

The log file is named for the generation, `NNNNNNN.log`, zero-padded, so the set on disk sorts into replay order by name alone. Recovery needs no sidecar index to know what to replay or in what order.

19.4 Rotation

When a write fills the active memtable, the committing thread that noticed rotates it. Not a background worker — the thread that filled it.

What "fills" means is every version, not every key. A memtable holds a version chain per key, and nothing prunes a chain before the whole memtable is freed — an overwrite adds a version rather than replacing one, because a reader at an older snapshot may still need the version being superseded. So the size the rotation threshold reads has to count every version resident, and a key written a thousand times occupies a thousand versions' worth of it.

The range tombstones count toward that size too. Without them a memtable holding nothing but interval deletes would measure as empty, and neither the size trigger nor the idle flush would ever seal it — so the deletes would sit in memory with no flush to make them durable.

A version's cost is its own structure plus whatever value it carries, so a tombstone is not free — it carries no value but is still a resident allocation, and a key deleted repeatedly grows its chain just as surely as one overwritten repeatedly. The same holds for a put of an empty or tiny value, which is what a secondary index entry usually is.

And what it reads is memory, not data. A distinct key costs a skip list node, two pointer arrays sized by the level its coin flips gave it, and the key bytes, laid down in one allocation; its first version costs a version struct and the value bytes, in another. That comes to about 100 bytes an entry beyond the key and the value, and it does not move with entry size — so it is most of an entry holding an 8 byte value and under 3% of one holding a 4 KiB value.

A referenced version carries eight bytes of id where an inline one carries the value, so a memtable of separated values holds vastly more keys before it fills — which is most of what separation buys on

the write path, beyond the bytes it saves.

Counting only key and value bytes therefore understates a memtable of small entries by about five times, and a store configured for 64 MiB would quietly hold 300 MB and more, multiplied again by however deep the immutable queue is allowed to get. The threshold reads the memory figure, so `memtable_write_buffer_size` is a budget on memory occupied rather than a promise about the size of what a rotation flushes. The two counters are kept apart for that reason: the flush path sizes its key log preallocation from the data bytes, which is what it is about to write, while rotation and backpressure read the memory bytes, which is what the host is actually paying.

Counting only the newest version per key, or counting a version by its value alone, would be the same bug in a different disguise every time: the memtable would report a size that never reaches the threshold, never seal, and grow until the host ran out of memory — while every statistic the engine reports stayed flat, because they would all be reading the same understated number. A workload of distinct keys would look perfectly healthy; only repeated writes to the same keys would show it.

That choice is visible in latency: a rotation is real work on the write path, and every other committer that agrees the memtable is full waits behind the same lock. A slow one lands directly in the write latency tail, which is why the engine logs any rotation over 20 ms.

The log the rotation installs is opened before the rotation needs it. Opening one costs a file creation, a staging ring and a flush thread, and paid inline that measured 49 ms — with every committer in the database stopped for the whole of it, since the rotation holds the lock while it works. A log is therefore prepared after each rotation, outside the lock, and the next rotation takes the prepared one and becomes a memtable allocation and two pointer swaps. The preparing thread still pays the cost; the other fifteen no longer do.

Preparing is claimed, not merely checked. Every committer that rotated arrives at the same moment and the empty-slot check is only a hint, so without a claim each one creates a log for a slot that holds one, and all but the winner abandon theirs. A log that loses the race is unlinked as well as closed, and so is a prepared log the database never rotated into before closing. An abandoned log is not inert: the next open scans it, replays it, and gives it a memtable in the L0 queue, so one left behind costs work on every open thereafter for records it never held.

The order inside a rotation is not adjustable:

1. **Enqueue** the sealed memtable.
2. **Swap** in a fresh active memtable.

For an instant the sealed memtable is reachable both as the active one and in the queue. That is harmless — a reader that finds it twice gets the same answer. The reverse order is not harmless: between the swap and the enqueue there is a window where the memtable is in neither place, and a reader would report a live key as absent.

Caution—**Rotation does not drain the log**

Sealing a memtable does not flush or close its write-ahead log. A committer may still be inside that log when the rotation happens — waiting for its record to reach the file under interval and full durability, and waiting for ring space under any mode. The committer therefore holds a **pin on the memtable** across the append — holding the memtable is what holds the log open. Any change that shortens that pin risks operating on a freed block manager.

19.5 Where an append stops waiting

A committer stages its record into the log's ring and one flush thread writes whatever completed as one contiguous run. Where the committer stops waiting is the sync mode's only real difference.

Under `TDB_SYNC_INTERVAL` and `TDB_SYNC_FULL` it waits for the flush thread to carry the durability frontier past its record, so an acknowledged commit has left the process. Under `TDB_SYNC_NONE` it returns as soon as the record is staged, which is what that mode promises and what makes it the weakest rung of the ladder in Durability (ch. 5).

Staging cannot outrun the writer without bound, because reserving ring space still blocks when the slot being claimed holds bytes the flush thread has not written.

That block is a wall: it stops every committer at once, for as long as the flush thread needs to free the slot. So the append path paces **before** reaching it. Once the ring is seven eighths full of unwritten bytes, a committer dwells briefly, rising with the fill toward a small cap, and ingest meets the writer's rate gradually instead of colliding with it.

The pacing is the same backpressure policy the immutable queue uses, given the ring's fill as another pressure input, so there is one place that decides how hard to push back rather than two. It is applied on the append path rather than at admission because the append already holds the log pinned — reading the lag there is two atomic loads and no extra pin, where admission would have to pin the active memtable on every commit just to look.

Measured on a sixteen-family churn workload, pacing cut commits that stalled past forty milliseconds by roughly three quarters and halved p99.9, at the cost of about a hundred microseconds at p99 — the dwell itself, paid by the commits that would otherwise have hit the wall.

Caution—**A clean close is what makes the weakest mode safe**

Nothing waits for a staged record under `TDB_SYNC_NONE`, so the only thing that guarantees it reaches disk on an orderly shutdown is that closing a block manager joins its flush thread and drains the ring before touching the descriptor. Break that and the mode loses data on a clean close, silently, which no crash test would catch.

19.6 The rotation lock is on the commit path

Rotation runs on whichever committing thread finds the memtable full, and every other committer that agrees waits on the same lock. That makes `rotate_lock` unlike any other lock in the engine: **time spent holding it is time every writer in the database is stopped.**

The rotation itself is short — a few hundred microseconds to open the next log and swap the slot. What is not safe is anything else borrowing the lock for convenience.

Caution—**Never hold the rotation lock across a wait or a device barrier**

Anything that sleeps, polls, or issues an `fsync` while holding it stops every committer in the database for the whole of it, and the result is multi-hundred-millisecond commits that no amount of subsystem-level tuning explains — the cost lands on threads that had nothing to do with the work being done.

The rule is kept in the simplest way available: **nothing outside the rotation itself takes the lock.** The only two acquisitions in the engine are `engine_maybe_rotate` and `engine_force_rotate`. The paths that would otherwise be tempted avoid it by construction — the value-log reclaim runs under the column-family registry read lock alone, and the interval sync ticker reaches the log through the same pin an append takes rather than through the lock, so its `fsync` blocks nobody's commit but its own.

The rule this leaves has two halves.

If you need the active log not to swap under you, pin the memtable — do not take the lock. `tidesdb_l0_sync_active_wal` is that pattern: the same pin an append takes, safe because a flush drains the writers epoch before it closes a log.

If you need files not to move under you, claim rather than freeze. Backup and clone copy the whole database and a family's sstables respectively, and holding the lock across either would be the longest hold in the engine. Neither needs it. A copy only requires that nothing *removes* a file it is about to read, and both paths work from a snapshot taken up front: backup copies the length its own commit left behind, and clone copies each sstable to the length the manifest recorded. A flush landing a new file meanwhile is not in either snapshot, which is what a point-in-time copy means.

What removes files is compaction and value-log reclaim, and both are claimed with an atomic flag rather than locked out — the same claim the compaction scheduler already used. Backup takes the value-log claim too, because a reclaim moves live values from one segment into another and unlinks the source, which can strand values a copy has already passed.

19.7 Waiting for a flush

A synchronous flush rotates and then waits for the **immutable queue to drain** — not merely for the generation it just sealed to reach L1.

That distinction is the whole point, and getting it wrong caused a data-loss bug. Waiting on the sealed generation alone looks better: under sustained writes the queue may never be observed empty, so a queue wait can report contention long after the caller's own data has landed. But the callers that matter here — backup (ch. 14) and clone (ch. 11) — **copy files**. Everything they should capture has to be in a file before they start.

And a rotation seals nothing at all when the active memtable is empty, which is an ordinary state. A flush that waited only on what it sealed would then return immediately while generations from ear-

lier rotations were still queued and still only in memory, and the copy would quietly miss their data. It is not a rare interleaving; it needs nothing to fail.

Caution—**Draining the queue is the contract, not an implementation detail**

The cost is real: a database under sustained writes may never show an empty queue, so this call can report `TDB_ERR_LOCKED` when the caller's own data has in fact landed. That is the safe direction to be wrong in. Narrowing the wait to the caller's own generation trades a spurious busy result for a silently incomplete copy, which is not a trade this engine makes.

19.8 Backpressure

Ingest can outrun flush. Left alone, the queue of sealed memtables grows without bound, and memory with it, until the process dies of success.

It can also outrun **merging**, and the two fail independently. A burst can drain out of memory promptly — an empty queue, flush entirely keeping up — and still leave a flush tier of runs behind that every later read has to merge across. Pacing on the queue alone cannot see that: it measures whether the memtables are being written out, not whether what was written is being merged down. So admission weighs both, and an empty queue is a free admit only while the tier is also shallow.

Admission is consulted once per distinct column family a commit touches, and reads the queue depth against the configured limit:

Depth	Decision
Below three quarters of the limit	**Admit** — proceed now
Between three quarters and the limit	**Throttle** — dwell, then proceed
At the limit	**Block** — wait for the queue to drain, then re-evaluate

The throttle dwell rises with depth — a per-slot step for each slot filled past the high-water mark — so pressure is applied gradually rather than as a cliff at the limit.

The flush tier has its own band beside the queue's, on the same three outcomes: below the slow mark a deep tier costs a writer nothing, past it the dwell grows with the depth, and at the stall mark the writer waits for the tier to drain. The compaction trigger sits below both, so merging is already underway before ingestion is slowed and well underway before it is stopped. Whichever band asks for more wins, so neither pressure is masked by the other.

Blocking has a **ceiling**. A writer held too long is admitted regardless, and the event is counted as `write_stall_ceiling_hits`. This is a deliberate choice about failure modes: a flush that has stopped draining is a broken database, and blocking every writer forever turns a recoverable problem into an unrecoverable one. Admitting past the ceiling keeps the database slow rather than stuck, and makes the condition observable instead of silent.

19.9 Reclamation

A sealed memtable can be freed once its data is durable in L1 and no reader is inside it. The second half is the hard part, because readers take no locks.

Two mechanisms cover the two ways a memtable is reached:

- The **active slot** is guarded by a reader epoch. A reader enters the epoch, loads the pointer, and exits when done. A rotation that replaces the slot waits for the epoch to drain before the old memtable can be retired.
- A **queued** memtable is reference counted. A reader takes a reference for as long as it is inside; reclamation waits for the count to fall back to its structural baseline.

`claimed` prevents two flush workers from taking the same memtable, and `flushed` records that its data reached L1 — only then may the log be unlinked, because until that moment the log is the only durable copy.

19.9.1 Reclaiming a sealed memtable

"Waits for the readers to drain" is where a flush would hurt the rest of the database if it were taken literally.

A flush retires the memtable it just installed while holding the column-family registry read lock, and it holds that lock across the whole install because the level sets it mutates belong to families the lock keeps alive. Creating or dropping a family takes the same lock for writing. So a retire that waited for a reader to leave would hold a read lock for as long as that reader stayed — and under glibc's default reader-preference, a waiting writer does not stop new readers arriving, so the write lock is not merely delayed but can be starved outright while the flush spins. The visible symptom is a `tidesdb_create_column_family` that never returns while the process burns a core.

The reclaim is therefore bounded. It rechecks whether the readers have gone a fixed number of times, and if they have not, it puts the memtable on a **pending list** and returns. The deferred-free reaper sweeps that list each second, freeing whatever has since drained. Nothing is leaked — a memtable still pinned at close is freed when the L0 is destroyed — and the flush gives the registry lock back on a bound that does not depend on how long some reader chooses to stay.

The overwhelmingly common case still frees inline: a reader that entered just before the retire is normally gone within a recheck or two, and the deferral is what happens when it is not.

19.9.2 Rotating on idle

A rotation normally runs on the committing thread that filled the memtable, which means a database that stops taking writes never rotates again. The data is not at risk — its log is durable — but nothing else improves either: the log cannot be unlinked until the data reaches L1, so recovery stays long, reads keep paying for a memtable they could be reading from an sstable, and compaction never sees the shape it would act on.

So a ticker rotates an active memtable that has gone a whole `memtable_idle_flush_seconds` without a write. The idle signal is the memtable's size sampled across two ticks: unchanged and non-zero means nothing landed in between. That keeps the cost off the write path entirely — no timestamp stamped per commit, no counter to contend on. Two different writes could in principle leave the size identical, and the cost of being wrong is a rotation one interval early, which is legal at any time.

It is opt-out rather than opt-in, because a database that never drains is the more surprising behaviour. Setting the interval to zero turns it off, which is what a laptop or a metered volume wants — rotating on idle is a write a quiet process would not otherwise make.

19.10 Recovery

On open, every `NNNNNNN.log` still present is replayed in generation order. Each becomes a sealed memtable in the queue, and a fresh active memtable is installed above them, so the recovered state has exactly the shape a running database has. The recovered generations are then flushed normally.

Replay walks a log by reading framed blocks forward until one cannot be read. A torn final record — a crash mid-append — ends the replay there, which is correct: that record was never acknowledged.

Caution—**The log must have no holes**

Because replay stops at the first unreadable block, a **gap** in the middle of a log does not skip one record — it silently discards every record after it. The append path guarantees no gap can exist by only ever writing the contiguous completed run; see Block manager (ch. 20). This is the single most consequential invariant in the write path.

Records carry their own sequence, so replay applies each at the sequence it committed with rather than at its position in the file. After replay the clock is reseeded past the highest sequence anything durable holds, so no sequence is ever reissued.

Not every durable batch is applied. A commit whose batch reached the log but failed to enter the memtable appends an abort record naming its sequence, and replay gathers those first so a batch can be cancelled by a record written after it. Without that, the record's mere presence would bring back a transaction its caller was told had failed.

A `PREPARE` record with no matching `COMMIT` or `ROLLBACK` after it is staged as in-doubt rather than applied, and surfaces through `tidesdb_recover_prepared` (ch. 12).

19.11 Invariants

Invariant	Why
Enqueue before swapping the active slot	Otherwise a reader sees neither copy and reports a live key absent
A pinned memtable keeps its log alive	Rotation does not drain the ring; a committer may still be waiting on that file
A log is unlinked only after `flushed`	Until L1 has the data, the log is the only durable copy
A log that will never be written to is unlinked, not just closed	An abandoned log is scanned, replayed and given a memtable on every subsequent open
Logs are replayed in generation order	Ordering comes from the file name; nothing else records it
An idle non-empty memtable is rotated on a timer	Otherwise a database that stops taking writes holds its last writes in memory for the life of the process, keeping its log unreclaimable and its data out of reach of compaction
A memtable is freed only after readers drain	Epoch for the active slot, refcount for queued ones
A flush retains a key's chain against the reclamation floor, not just its newest version	The floor is what a frozen snapshot rests on. Taking only the newest drops the versions under it whatever the floor says, and a repeatable-read reader below an overwrite then finds a live key absent
A flush never drops a base tombstone	Its output lands in L1, the shallowest tier, where a tombstone is what shadows older versions in the levels beneath it
A memtable takes its value log floor when it becomes active, not at its first reference	A value reaches the value log before the commit that produced it reaches any memtable, so a floor taken later leaves a window where a reclaim drops the segment it just landed in
A memtable lowers that floor onto a reference's own segment	Its floor covers what was written after it became active; a value separated a moment before a rotation is applied here while living below that point
The floor is released only when the flush installs	Until an sstable names those values, this memtable is the only thing that does
A flush never waits out a reader to free a memtable	It holds the registry read lock, so an unbounded wait there starves family create and drop; it defers to the reaper instead
Blocking on backpressure has a ceiling	A stuck flush must degrade to slow, not to deadlocked
Ingestion is paced against merging, not only against flush	They fail independently: an empty queue with a deep tier means flush kept up and merging did not, and every later read pays for it
A synchronous flush waits for the immutable queue to drain, not for its own generation	Backup and clone copy files, and a rotation seals nothing when the active memtable is empty — so waiting on the sealed generation returns while earlier generations are still only in memory, and the copy misses them

Block Manager

20.1 The problem

Every file TidesDB writes — the write-ahead logs, the sstable key logs, the value log, the manifest — needs the same four things: a way to tell where one record ends and the next begins, a way to detect a record that was damaged or half-written, a way to append concurrently without threads serializing on each other, and a defined meaning for "durable".

Solving that once, in one module, is why a single recovery discipline covers every file in the database. It is also why a torn tail in the manifest and a torn tail in a log are handled by the same code.

20.2 Framing

Every record is one framed block:

```
[ size:4 ][ checksum:4 ][ ...payload... ][ size:4 ][ magic:4 ]
 \_____________________/                  \_________________/
      header (8)                               footer (8)
```

The size appears **twice**, at both ends, and the footer carries a magic value. That redundancy is what makes a partial write detectable without a separate log of intentions: a record whose trailing size disagrees with its leading one, or whose footer magic is absent, was never completely written. The checksum then covers the payload itself, catching damage that arrived after a successful write.

The file itself opens with an 8-byte header carrying a magic value and a format version, so a file that is not ours, or is from a format we do not speak, is rejected at open rather than misread.

A zero size field is treated as end-of-file by every reader. A zero-length block is therefore rejected at write time — permitting one would let it truncate iteration for everything after it.

Checksums are XXH3, truncated to 32 bits.

20.3 Reading

A read of an unknown record must first learn its length, which naively means two reads: one for the header, one for the payload.

Instead the first read is speculative — it fetches a fixed 4 KB window, which covers the header and, for most records, the entire payload. One read where the record fits, two where it does not.

The window size is a deliberate trade rather than a maximum. Raising it would capture more records in one read, but every read below the threshold then transfers bytes it does not use. 4 KB matches the page size and the default btree node size, so the common cases land inside it.

NOTE—**The cliff is real**

A record one byte past the window costs a second read for that byte. A btree node sized just above 4 KB pays this on every access, which is why the node size and this window are chosen together rather than independently.

20.4 Durability

Two sync modes, chosen per file by the layer above:

Mode	Meaning
`BLOCK_MANAGER_SYNC_NONE`	Writes go to the page cache. Durable against process death, not machine death.
`BLOCK_MANAGER_SYNC_FULL`	Each write reaches the device before it is reported complete.

Under `SYNC_FULL` the file is opened `O_DSYNC` where the platform has it, so the write syscall itself carries the durability rather than a following `fdatasync`. Where it is unavailable the manager falls back

to an explicit `fdatasync` per write. The distinction matters for performance, not semantics — both mean the bytes are on the device when the call returns.

Truncation is a special case: `ftruncate` is not covered by `O_DSYNC`, so a truncation always syncs explicitly.

20.5 Preallocation

An append that extends a file takes the kernel's per-inode write lock, so extending writes serialize even when their offsets are disjoint. On a busy log this dominates.

The manager therefore extends the file's on-disk allocation ahead of the data, in chunks, so that appends land **inside already-allocated space** and skip the extend path. When the remaining reservation drops below a low-water mark, the next append extends again.

The chunk size is per-file rather than global. Small, short-lived files like the manifest disable preallocation entirely, so that a copy of the file carries no trailing reserved zeros. Both the direct path and the buffered ring extend the same way, the buffered one on the thread that reserved the offset rather than on the flush thread that will write it.

The call underneath differs per platform — `fallocate` on Linux, `F_PREALLOCATE` followed by an `ftruncate` on macOS, `FileAllocationInfo` then `FileEndOfFileInfo` on Windows, and `posix_fallocate` elsewhere. macOS and Windows need that second step because reserving blocks does not by itself advance the logical end of file, and it is the logical end that decides whether a write is an extending one.

Whether `posix_fallocate` exists is **probed by the build**, not inferred from a feature-test macro. `_POSIX_C_SOURCE` is something an application defines to request strict POSIX rather than something a platform advertises, so testing it asks the wrong question and answers no on systems that have the call — which silently removes preallocation from them entirely. OpenBSD genuinely lacks it and falls through to the no-op.

20.5.1 Giving the tail back

Preallocation advances the file's **logical end**, not just its block reservation, and that is deliberate: reserving blocks while leaving `i_size` alone keeps the kernel treating every write as an extending one, which buys nothing. The consequence is that a preallocated file is charged its whole extent, not its data — and on a filesystem that reports allocated blocks, `du` and `df` agree.

So the extent has to be handed back when the file stops growing. There are two moments that can do it, and only one of them is soon enough:

- **On close.** The manager trims to the data on the way out. That covers the write-ahead logs, which are closed once flushed.
- **When a build finishes.** An sstable is closed only when it is evicted or the database shuts down, so a live one holds its handle for as long as it is installed. Waiting for close would leave every installed sstable occupying a full chunk.

A finished sstable is therefore trimmed by its builder, before the durability barrier so the one fsync covers the truncation as well. Skipping it is not a correctness bug and nothing fails — the data is intact, reads are unaffected, and closing the database reclaims it all — which is exactly what makes it easy to miss. What it does is make the store mostly padding: at a 64 MiB chunk, a database of a few hundred sstables reports several gigabytes of live data while occupying tens of gigabytes on disk, and the gap looks convincingly like a compaction that is failing to reclaim.

A platform where no variant is available is not broken, only slower: the first failure disables further attempts for that file and writes take the extending path. Because that cost is large and otherwise invisible, the first such failure in the process is logged once.

Note — **On ext4 this removes less than it appears to**

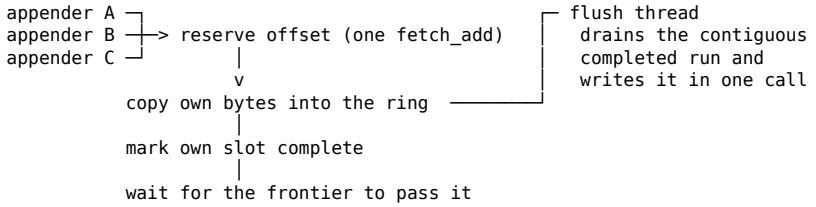
`fallocate` advances the logical end of file, which is what takes writes off the extending path. It does not initialize the blocks — the extents come back marked unwritten, which is why the call is cheap, and the first write into each one converts it. That conversion is itself a journalled metadata operation. Preallocation therefore exchanges one metadata cost for another rather than removing metadata work from the write path, and measured on ext4 it changes neither throughput nor tail latency for the log. It still earns its place on filesystems that behave differently, and it keeps the two write paths consistent.

20.6 The buffered append ring

This is the piece that makes concurrent commits scale, and it is the most intricate part of the module. Its shape follows **Aether** (Johnson et al., PVLDB 2010) — see Design lineage (ch. 33).

The naive arrangement — every appender writes the file itself — puts every committing thread on the same inode lock. Serializing a durable write is the single largest cost on the commit path.

Instead:



1. **Reserve.** One atomic fetch-and-add on the file's size hands each appender a disjoint byte range. This is the only contended point, and it is a single instruction.
2. **Fill in parallel.** Each appender copies its framed record into its own slice of the ring. Appenders never wait for each other to copy.
3. **Release.** Each marks its slot complete.
4. **Drain in order.** One flush thread — the only thread that ever touches the file descriptor — advances over the **contiguous** run of completed records from the frontier and writes it out in a single call, then publishes the new frontier.
5. **Wait.** The appender returns once the frontier has passed its own record.

The properties this buys:

- **No inode-lock contention.** One writer, always.
- **Coalescing without a coordinator.** Records that arrive while a write is in flight are drained together in the next one; the batch size grows exactly when the writer is the bottleneck.
- **Gap-free output, guaranteed by construction.** Only the *contiguous* run is ever written, so a record can never reach the file before one that precedes it.

That last property is what WAL replay (ch. 19) depends on. Replay stops at the first unreadable block, so a hole would discard everything after it.

The ring is bounded, so an appender whose slot still holds unflushed bytes waits for the frontier to pass them. That is backpressure at the I/O layer: it bounds memory, and it is why ring size is sized against the write buffer rather than made arbitrarily large.

A record larger than the whole ring cannot be staged — it would wrap onto itself. Such a record waits to become the frontier, writes itself directly as a single vectored write, and publishes the advanced frontier. Rare, and confined to oversized commit batches.

Note—**The flush thread backs off when there is nothing to write**

The park timeout is only a net for a wake that was somehow missed — `bm_wake_flush` is the real signal, and it is ordered against the appender's completion flag so it cannot be lost. Holding a short timeout while nothing arrives therefore buys nothing and costs a wakeup every half millisecond, which on an otherwise idle database is most of the process's CPU. Consecutive empty parks grow the timeout toward a quarter-second ceiling and any drained run resets it, so an idle database costs nothing while a busy one still reacts on the signal rather than the timer.

20.7 Recovery

Opening an existing file validates the last block. A crash mid-append leaves a torn record, and the two modes differ in what that means:

- **Permissive** truncates back to the last valid block. This is what a write-ahead log and the manifest want: the torn record was never acknowledged, so discarding it is correct and the file is usable again.
- **Strict** refuses. Reserved for cases where silent truncation would hide real damage.

A preallocated file needs this most: its on-disk size runs ahead of its data, so the tail is reserved zeros. Without validation, a reopened preallocated file looks like it ends in a record of size zero — which readers treat as end-of-file, correctly, but which leaves the write cursor in the wrong place unless the real end is established at open.

20.8 Invariants

Invariant	Why
Only the contiguous completed run is written	Guarantees no gap; WAL replay would truncate at one
One thread writes a buffered file's descriptor	The ring's ordering guarantee assumes a single writer
A slot is not reusable until the frontier passes it	Otherwise a live record is overwritten before it is written out
Zero-length blocks are rejected at write	A zero size field reads as EOF and would truncate iteration
The trailing size must match the leading one	This is what makes a partial write detectable
`ftruncate` is synced explicitly	It is not covered by `O_DSYNC`
A file that has stopped growing is trimmed to its data	Preallocation advances the logical end, so an untrimmed file is charged its whole extent. An sstable cannot wait for close, since a live one never closes

Chapter 21

SSTable

21.1 The problem

A flush turns a memtable into a file that will be read many times and never modified. That file has to support point lookups, ordered scans in both directions, and MVCC — several versions of a key, each visible to some snapshots and not others — while staying cheap enough that having many of them is affordable.

Immutability is what makes the rest tractable. Nothing rewrites an sstable in place, so readers need no locks against writers, and the only lifetime question is when the *file* may be deleted.

21.2 What one sstable is

A single `CCCCCCCCCC.SSSSSSS.klog` file — the owning family's id and then the table's, each zero-padded — containing, in order:

```
+-------------------------------------------------+
|  btree nodes          leaves, then the levels   |
|                       above them                |
+-------------------------------------------------+
|  filter partitions    the aux region, written   |
|  + directory          when the build finalizes  |
+-------------------------------------------------+
|  footer               fixed head + variable tail|
+-------------------------------------------------+
```

Values above the database's `value_separation_threshold` are not in this file at all. They live in the **shared, database-wide value log**, and the entry holds a reference. An sstable owns exactly one file — its key log — and borrows the value log per call.

21.3 The footer makes the file self-describing

The last block is a footer holding everything needed to use the file: format version, the btree root and first/last leaf offsets, where the filter directory lives, the key and tombstone counts, the highest sequence, the byte totals, the btree shape, the minimum and maximum key, and the encoding pipeline the file was written through.

Opening an sstable therefore requires only the file. Nothing needs to be reconstructed from a sidecar, and nothing needs to be looked up in the manifest first — which is what lets recovery treat the sstables as ground truth and rebuild an unreadable manifest from them.

The footer has a fixed-size head followed by a variable tail (the two bounding keys and the encoding list), each length-prefixed and bounds-checked against what remains, so a truncated or lying footer is rejected rather than read past.

The **btree node size is recorded in the footer** rather than taken from the current configuration. A file must be read with the geometry it was written with, and configuration can change between the write and the read.

The same applies to the encoding pipeline, and more sharply. The builder copies the family's pipeline once when it is constructed, then both encodes the nodes and writes the footer from that copy. It has to be one copy: the pipeline it is handed belongs to the live column family configuration, a runtime reconfigure replaces that whole configuration, and a builder that read it once to choose its codec and again to fill in the footer could straddle the change — writing nodes under the old chain and recording the new one. The file would open, and every node read would then decode with the wrong chain. Nothing later can repair it, because the footer is the only record of how the bytes were written.

21.4 The btree key log

The sorted store and the index are the same structure. There is no separate index block: a lookup descends the btree to a leaf.

Leaves are laid out to hold as much as possible in as few bytes as possible:

- **Prefix compression.** Each key stores only the suffix it does not share with its predecessor. Sorted keys with common prefixes — which is most real key spaces — collapse.
- **A key indirection table.** Fixed-width offsets to each key's suffix, so a binary search within a leaf is O(1) per probe instead of a walk.
- **Delta-encoded sequences.** Sequences are stored relative to a per-node base, so nearby sequences cost a byte or two rather than eight.
- **Varints throughout** for sizes and metadata.

Leaves are doubly linked, which is what makes scans O(1) per step in both directions once positioned, and backward iteration possible at all.

MVCC is **native to the btree** rather than layered over it: a lookup takes a sequence ceiling and resolves to the newest version at or below it during the descent. There is no read-then-filter step.

NOTE—**Node size and the read window**

The block manager's first read fetches a fixed 4 KB window. A node sized just above it costs a second read on every access. Node size and that window are chosen together — see Block manager (ch. 20).

21.5 The partition range filter

A bloom filter answers "is this key definitely absent?" and lets a lookup skip a file entirely. The difficulty is that one filter over a whole sstable is sized by the entry count, so a large table means a large resident filter, and many tables mean many of them resident at once.

So the filter is **split by key range**. The key space is divided, in sorted write order, into partitions; each partition gets an ordinary bloom filter serialized as its own blob. Only a small directory of partition first-keys stays resident. The partition blobs are fetched on demand and live in the block cache, which means **the filter's memory cost is bounded by the cache rather than by the data size**.

The directory holds each partition's *full* first key, so routing a query to its partition is exact. That matters: an approximate routing could send a query to the wrong partition and get a false negative, and a bloom filter that can report a false negative is worse than no filter.

The partitions are accumulated during the build and written together at the end, so the btree's data blocks stay contiguous rather than being interleaved with filter blobs.

21.6 The shared value log

A value at or above the database's `value_separation_threshold` is not stored in the btree. The entry holds an opaque logical id instead, and the bytes live in the database-wide value log (ch. 22).

That is why a compaction rewriting a key does not rewrite its value: it copies the reference forward. Values are written once and survive any number of merges, which removes what would otherwise be the dominant cost of compacting large-value data.

A flush usually copies a reference rather than making one. The committing transaction already put the value in the value log, so the entry arriving from the memtable carries an id and the builder emits it unchanged. It still separates a value itself when one reaches it inline — a family that opted out with `keep_values_inline`, or a value that was under the threshold when it was written and is being merged under a larger one. The trade is one extra read when a point lookup actually wants a separated value, and the API is shaped so most patterns never pay it — key-only iteration and `tidesdb_txn_contains` (ch. 12) never dereference at all.

21.7 Lifetime

An sstable is immutable, but its *file* and its *descriptor* both need lifetimes, and they are not the same lifetime.

The object is reference counted. A level set holds one reference for as long as the table is installed; readers take transient references while inside. A compaction that replaces a layout drops the old layout's references, and the table is freed by whichever thread drops the last one — which may

be the compaction or a reader still finishing. An installed table rests at one reference, and a reaper that has collected it holds a second, so what tells the reaper no reader is in flight is the table sitting at the level set's reference **plus its own**. The check is an exact match, so a resting count named without the collector's reference never matches and nothing is ever reclaimed.

The descriptor is separate and scarcer. A table may stay installed and readable while its file descriptor is closed to stay within the budget; the next read reopens it. This is why an sstable read can report busy: the object is alive, but opening its file would exceed the budget, and that is retryable backpressure rather than an error.

21.8 How a build fails

The builder writes the btree, finalizes the filter, appends the footer, and makes the file durable — in that order, and the order is the recovery story. A crash at any point before the manifest names the file leaves an orphan: a complete or partial `.klog` that nothing refers to. Recovery ignores it, because the manifest is the catalogue and a file it does not name does not exist.

A build that fails without the process dying cleans up after itself — flush and compaction both unlink the file they were writing before returning the error. What survives is the case where the process died: a `.klog` the manifest never got to name, and any `.klog.lstmp` staging file beside it.

Those are swept at the next open. The manifest is the catalogue, so a key log it does not name is by definition unreachable, and the sweep removes it before any worker starts — while nothing can yet be writing a file the manifest has not caught up with. A database that has crashed repeatedly mid-build therefore gives the space back on restart rather than carrying it until someone deletes the files by hand.

The sweep is skipped entirely when the manifest was **self-healed**. That path rebuilds the catalogue *from* the sstables on disk, so for that one open the files are the source of truth rather than the other way round — and the files it would delete are exactly the ones the rebuild could not adopt, which is the last copy of whatever they hold. They are left for an operator.

The build also **trims the key log to its data** before that barrier, giving back the preallocated extent it grew through. An sstable holds its block manager open for as long as it is installed, so the trim that runs at close would never reach it — see Block manager (ch. 20).

That is why the durability barrier on the key log comes **before** the manifest commit. The reverse would let the manifest name a file whose bytes never landed.

The value log is deliberately *not* synced by flush or compaction. Its segments carry the database's own durability mode instead: under `TDB_SYNC_INTERVAL` and `TDB_SYNC_FULL` they are opened so that each write is durable when it returns — `O_DSYNC` where the platform has it, a per-write `fdatasync` where it does not — so values are already on the device by the time a key log can reference them. Under `TDB_SYNC_NONE` the segments are opened unsynced like everything else, which is that mode's whole promise; the value log is no weaker than the log the keys arrived on.

That inheritance is what removes the need for a barrier, and it is why weakening it is not a local change. Opening segments unsynced under a mode that syncs — without adding an explicit value-log barrier ahead of the manifest commit — would leave a durable key log pointing at value bytes that were never written, and nothing in the test suite would say so.

21.9 Invariants

Invariant	Why
An sstable is never modified after it is finalized	Readers take no locks against writers
The key log is durable before the manifest names it	Otherwise the catalogue can reference bytes that never landed
A finished key log is trimmed to its data	Preallocation advanced the file's logical end; a live sstable never closes, so nothing else would give the extent back
The footer records the geometry the file was written with	Configuration may have changed since
The footer's encoding pipeline is the one the nodes were actually encoded with	The builder takes the family's pipeline once at construction and both encodes and records from that copy; reading it twice lets a reconfigure land in between and produces a file no encoding in it can decode

Invariant	Why
The filter directory holds full first keys	Approximate routing would produce false negatives
A value log id outlives every key log that holds it	Values are shared across merges, and only the value log may retire one
A table is freed by whoever drops the last reference	Evictor or reader — whoever is last
An exhausted descriptor budget is busy, not absent	The table exists; only the descriptor is unavailable
A key log the manifest does not name is swept at open	Unreachable by construction, so the space is pure loss — except after a self-heal, where the files are the catalogue's source and are kept

CHAPTER 22

Value log

22.1 The problem

A value large enough to dominate a btree node should not be stored in the btree. Separating it keeps the tree small — a btree holding references instead of kilobyte values has far fewer, denser nodes, so scans touch less and the cache holds more — and it means a compaction that rewrites a key does not rewrite its value. That removes what would otherwise be the dominant cost of compacting large-value data.

This follows **WiscKey** (Lu et al., FAST '16), though the addressing and the reclamation both differ from it substantially; see Design lineage (ch. 33).

Separation creates its own problem. Values written once and referenced from many sstables accumulate garbage as keys are overwritten and deleted, and reclaiming that garbage means moving bytes that live readers may be reading.

A separated value is written at commit, not at flush. The committing transaction appends the bytes here and the write-ahead log record carries only the id, so a large value reaches the device once instead of once in the log and again when the memtable is flushed. What follows from that is that the value exists before anything durable references it, and stays that way until the memtable holding the reference is flushed – which is what the floors below are for.

22.2 Segments

The store is **database-wide**: one set of files serves every column family, because a value's identity has nothing to do with which family holds the key. It is a series of append-only segment files named `NNNNNNN.vlog`, of which exactly one is open for appends. That segment seals when it reaches its target size and a fresh one takes over.

A sealed segment is immutable for the rest of its life. Nothing rewrites it, shifts it, or truncates it. It is read until it is unlinked, and then it is gone.

A klog entry references a value by an **opaque logical id**, never a physical location. The id names one block for the whole of its life, and nothing in the store ever moves it. What the indirection buys is that no sstable records *where* a value sits, so ordinary compaction carries one forward by id without reading, re-encoding or moving it, and a segment nothing references is unlinked whole without touching a single tree.

Emptying a segment that still holds live values is the one case that rewrites anything, and it does not relocate them either: the next merge to carry such a value writes it **afresh under a new id**, into an output table that merge was already building. See Reclaiming.

NOTE—**Why not one file**

The obvious design is one file, compacted in place by shifting survivors down over the gaps and truncating the tail. It is simpler, and it packs perfectly. It is also impossible to make concurrent: the shift moves bytes underneath readers, and the closing truncate assumes nothing was appended above the region being compacted, so both readers and appenders have to be excluded for the whole rewrite.

The cost of that exclusion is not theoretical. A single pass over a 6.5 GB store takes over two minutes, and every read, write, flush and compaction in the database blocks behind it. It also starves: with readers continuously arriving, the exclusive writer never acquires the lock at all until load stops, and the store grows without bound in the meantime. Segments avoid both because nothing is ever moved within the store — a segment is either unlinked whole or left alone.

22.3 Reading

A read resolves the id to a segment slot, an offset and a length under the index read lock, drops that lock, and only then takes a **reference on the segment** before fetching the block. It holds no lock a reclamation can take.

The reference is acquired *after* the index lock is released rather than under it, which is what makes acquiring able to fail: a reclaim may retire the segment in that window. That is not an error but the answer — the id resolved to a segment that no longer exists, so the read reports the value absent rather than touching a file being unlinked.

The reference is the whole lifetime protocol. It is the same refcount with an evicting window (ch. 21) the sstable uses: a segment rests at a baseline of one, readers transiently raise it, and an unlink waits for it to fall back.

The resting count is one here and two for an sstable, and the difference is not an inconsistency: a segment's evictor works from the slot table and takes no reference of its own, while the reaper that evicts a key log holds one on every candidate it collected. The count the window is claimed at is always "the structural reference, plus whatever the evicting thread itself holds" — which for a segment is nothing.

Acquiring a segment also goes through the form that waits the window out rather than reading the sentinel as a freed object. That is what makes a release inside the window impossible here: a release can only follow an acquire, and an acquire that had already succeeded would have raised the count above the baseline and failed the claim.

Every block carries its own id in its payload header, and the read checks it. That is a correctness check against a stale index rather than a routine one, and it sits alongside the block manager's checksum: a read either returns the value it asked for or an error, never somebody else's bytes.

The header carries one more thing: **the chain of encodings the value was written through**, in the top byte of the length word and the ids following it. The value is decoded with that chain rather than with the pipeline of whichever family is reading, because the two are not always the same. Compaction carries a separated value forward by id without re-reading or re-encoding it, so a value written under one pipeline can end up referenced by an sstable whose footer records another; and one shared store holds values from families that encode differently. A value that described itself by anything but its own bytes would eventually be decoded with the wrong chain.

A reader that cannot resolve the chain a value carries — a build without that codec compiled in — reports the read as corrupt and hands back nothing. That is the one damage a checksum cannot catch: the block is intact and its bytes are exactly what was written, they simply cannot be turned back into the value. Answering with the stored bytes would be a silent wrong read, so it is treated as the same unreadable condition as any other decode failure, and the value reads correctly again from a build that has the codec.

22.4 Reclaiming

The vlog has **no idea which values are still referenced** — that truth lives in the klogs. Asking them for it is not affordable: building the live id set that way means scanning every key of every sstable of every family, on a store shared by all of them, and on a database large enough to need reclaiming that scan does not finish.

So the trees report rather than being read. Every sstable records at build time **which segments its separated values landed in and how many bytes of each it holds**, and carries that histogram in its footer. Summed over the tables installed right now, it is exactly the live bytes of every segment, and it costs no file read at all.

The tables are not the only holder, though, and the two others have to be added to that sum or the segments they hold read as empty:

- **A memtable** holds references from the moment a commit separates a value until its flush installs. Those are covered by a floor rather than by counting, since a memtable's contents move under a reader; see below.
- **A staged prepared batch** holds them for as long as it is undecided. Its entries enter no memtable – that is what a prepare is – and no sstable names them until phase two decides it, so they are counted here alongside the tables.

A pass therefore has two tiers:

1. **A segment nothing holds is unlinked.** Its index entries are dropped and its file is removed. Nothing is read, nothing is copied, and no reader is excluded, because nothing can resolve into a segment nothing references.

2. **A segment at most half live is marked.** The next compaction that carries one of its values forward **writes the value afresh instead of keeping the reference**, so the value lands in a current segment and the marked one falls towards zero — where tier one collects it for nothing.

The second tier is the whole reason the store never moves bytes itself. Emptying a half-dead segment is a rewrite of the tables referencing it, and compaction already rewrites tables; doing it there costs a merge that was going to run anyway. Ordinary compaction is untouched — carrying a reference forward stays free — and only values in a marked segment pay a read and a write.

Liveness is restated, not adjusted. Each pass zeroes what every segment is reported to hold and sums the histograms afresh. Tracking installs and drops incrementally would be cheaper and is the wrong trade: the two error directions are not equal. A missed drop only delays reclaiming a segment, while a missed install makes a segment holding live values look empty, and dropping that loses them. Restating cannot drift.

Caution — **`vlog_used_bytes` is not live data**

The index names every value the store has ever written that has not had its segment dropped, reachable or not. It therefore converges on the file size, and a store holding gigabytes of garbage reports a space amplification near 1. `vlog_live_bytes` is the figure to divide by.

Caution — **The active segment is never reclaimed and never marked**

It is still growing, so the share of it that is live says nothing yet, and rewriting values into the very segment being emptied would not converge.

22.4.1 The windows a floor holds

A holder that is not an installed table protects its values with a **floor** — the segment taking appends when it took one — and nothing at or above the lowest floor in flight is reclaimable. Four holders take one:

- **A flush** writes its values and only then installs the sstable naming them.
- **A compaction** holds one across its swap, since a re-spill writes before it installs.
- **A memtable** takes one when it becomes the active memtable, so the floor is already in place before the first commit that can reach it separates anything. Taking it at the memtable's first reference would be too late: the value reaches this store before the commit that produced it reaches any memtable, and a pass in that window drops the segment it just landed in.
- **A prepared batch** takes one while it is undecided, beside the write-ahead log generation it already pins for the same reason.

A floor is a lower bound on where the holder's values might be, so a holder that adopts a value written before it existed has to reach further back than the segment it started at. A memtable receiving a reference the commit separated a moment before a rotation does exactly that, and lowers its floor onto that value's own segment.

22.4.2 What it costs

Tier one is free: an unlink and an in-memory index purge. Tier two costs one read and one write per value, inside a merge already running, and only for values in a marked segment.

Sweeping segment size from 16 MiB to 1 GiB on a sixteen-family churn workload leaves the store within about 2% of the same size throughout — 3,664 MB at 16 MiB and 3,718 MB at 1 GiB, against 3,745 MB at the 256 MiB default. **Store size is flat across the range**, so segmenting costs no space worth naming, and the size to pick is decided by the trade below rather than by space.

The half-live bar is what stops the second tier costing more than it recovers, and it cannot be much stricter than that. Under a random update load segments decay at about the same rate, so they sit together around half live rather than spreading out; a quarter-live bar is one almost nothing crosses, which leaves the store holding twice its data with nothing marked.

Segment size carries a real trade. A segment can only be dropped when *nothing* in it is live, so larger segments mix more values together and one cold key can hold a whole file open until compaction rewrites it out. Smaller segments reach zero sooner and reclaim faster; larger ones amortise descriptors and rolls.

`vlog_dead_bytes` says how much there is to reclaim and `vlog_segments_drainable` says how much of it is currently actionable. Read together in `tidesdb_get_db_stats` (ch. 15), a drainable count that

stays high is reclamation falling behind the garbage.

22.4.3 Descriptors

A store can hold thousands of segments, so segment files are opened against the **same descriptor budget** as klogs and write-ahead logs, under their own `vlog_segment` label. Exhaustion wakes the reaper and retries rather than failing the store.

An idle segment's descriptor is also **given back on demand**. The reaper closes sealed segments nothing is reading and the next read reopens the file, published by a compare-exchange so two readers racing to reopen cannot both install a handle. A segment with a reader inside it is never closed: the eviction claims the same reference count a reader takes, so it either wins with the segment idle or leaves it alone.

22.4.4 Values larger than a segment

A value is never split. One value is one block, so a value larger than the segment target lands whole in a fresh segment that simply overshoots it — the target is a **roll threshold, not a size bound**. The ceiling is instead the block frame: a value whose payload will not fit a uint32 length is refused with an error rather than truncated.

This is the easiest case for reclamation rather than the hardest. An oversized value effectively gets a segment to itself, so that segment is either entirely live or entirely dead, and the moment nothing references it the whole file is dropped without any rewriting.

22.5 What a codec achieved

A family can change its codec, and the tier-two rewrite deliberately re-spills values under the **merging** family's pipeline — so a store is not written under one encoding, it accumulates values under several, and migrates between them as compaction runs. A single compression figure for the store would average across all of that and describe none of it.

So compression is attributed **per chain**, and the chain comes from the value rather than from any configuration: each value already records the pipeline it was written through in its block header, because that is what lets a value be carried forward by id and decoded correctly by a family that encodes differently. `vlog_get_chain_stats` reports, for each chain seen, the uncompressed bytes, the on-disk bytes, and how many values it accounts for — so `used / stored` is that codec's realised ratio on the values it actually wrote.

This survives a reopen without being persisted anywhere, because it is rebuilt by reading the chain back off each value during recovery.

Caution—**A family's codec describes what it will write next**

It does not describe what the family holds. Any figure reporting on stored bytes has to ask the bytes, or it will credit a codec with data written before it was configured.

22.6 Recovery

On open, every segment in the directory is adopted in ascending number order and its blocks are scanned to rebuild the id index. Appends then go to a **fresh** segment rather than extending the newest recovered one, so a segment sealed before a restart stays immutable across it.

Nothing is ever copied within the store, so **an id names one block for its whole life** and a crash cannot leave the same id in two segments. A pass that dies part way has either unlinked a segment or not; there is no half-drained state to resolve.

Per-segment liveness is not persisted, and does not need to be. It is summed from the footers of the tables the manifest says are installed, so the first pass after an open restates it from what is actually there.

A torn tail, from a crash mid-append, is skipped by the block scan. Nothing else is affected: the segment is unlinked whole when it is eventually dropped, so there is no gap to reason about.

22.7 Invariants

Invariant	Why
A sealed segment is never modified	It is what lets a reader hold an offset without a lock
A klog holds a logical id, never a location	The store owns the id-to-segment mapping, so slots can be retired and reused without any sstable recording a file that no longer exists
A segment is unlinked only after every reader leaves	The index is repointed first, so no new reader can arrive
Of two entries for one id, the higher segment wins	The store copies nothing, so this cannot arise from its own operation; the tiebreak is defensive, resolving a duplicate deterministically rather than by scan order
A reclaim never marks the active segment	It is still growing, so the share of it that is live says nothing yet, and rewriting values into the very segment being emptied would not converge
A reclaim never retires the segment taking appends	A pass decides a slot is reclaimable and only then retires it, and a roll landing between the two makes the slot it chose the active one. Retiring it unlinks the open file, and every later append then spins on a slot that never reopens and reports the store as out of room. The decision reads the active slot per segment rather than once per pass, and the retire itself re-reads it after claiming the eviction window – a slot only becomes active by being claimed out of the table, and a claim takes slots reading absent, so past that point the reading is final
A segment is marked only when at least half of it is dead	Below that, the rewrite it costs the next compaction is more than the space it returns
Liveness is decided by the caller's set, never by the index alone	The index holds dropped values until their segment drains, so it would report garbage as live
Every block carries its own id, and reads check it	A stale index must produce an error, never another value's bytes
Every block carries the encoding chain it was written through	Compaction carries a value forward without re-encoding it, so the sstable referencing it may record a different pipeline
Ids are never reissued	The next id resumes above every id recovered

Chapter 23

Manifest

23.1 The problem

Files appear and disappear constantly: a flush creates several, a compaction creates some and deletes others. A crash can land anywhere in that. Recovery has to answer one question without ambiguity — **which files are part of the database right now?**

Scanning the directory cannot answer it. A `.klog` on disk might be a live table, a half-written flush output, or an input a compaction had finished with but not yet unlinked. The bytes look identical.

So the set is recorded explicitly, and the record is the authority. **A file the manifest does not name does not exist.** A crash mid-flush leaves orphaned files rather than a corrupt database, and the orphans are simply ignored.

23.2 One manifest, not one per family

The manifest is database-level. Every column family's tables are recorded in the same log.

This exists to make cross-family flush atomic. One sealed memtable holds keys for several families, so flushing it produces several sstables that must all become visible together — otherwise a transaction that atomically wrote to two families would be recoverable in one and not the other. One log means one append makes all of them live at once.

23.3 Structure

An append-only block-manager log. Each commit appends **one framed block** holding a batch of records:

```
[ version:1 ][ record ][ record ][ record ] ...
```

Each record is one opcode plus fixed fields, self-delimiting so a batch is walked without an index:

Record	Meaning
`MANIFEST_OP_CF_ADD` / `MANIFEST_OP_CF_DROP`	A column family appears or goes, with its opaque config blob
`MANIFEST_OP_ADD_P`	An sstable joins a level, with its counts, size, and partition
`MANIFEST_OP_MOVE`	An sstable changes level without being rewritten
`MANIFEST_OP_REMOVE`	An sstable leaves the set
`MANIFEST_OP_SEQ` / `MANIFEST_OP_CF_SEQ`	Sequence and family-id high-water marks
`MANIFEST_OP_RANGE_DEL`	One family's whole set of range deletes (ch. 12), as an opaque blob

The config blob is stored **verbatim and never interpreted**. The manifest does not know what a column family configuration contains — that belongs to the configuration module — so the format can change without touching this one.

`MANIFEST_OP_RANGE_DEL` is the one record that **replaces** rather than adds: it carries a family's entire interval set, so the newest one replay meets is the set and a rollover writes one per family that has any. Keeping them here rather than in the sstables is what lets a family that deleted a range but wrote no keys still record it — a flush that builds no sstable has nothing to hang it on, and an sstable with no keys cannot be placed in a level set at all.

`MANIFEST_OP_MOVE` deserves note: promoting an sstable to a deeper level when nothing needs merging is a catalogue edit, not a rewrite. The file stays where it is and the manifest re-points it.

23.4 Committing

A commit is a batch, not a record. Operations accumulate into a pending buffer and are written as one block, which is what makes a multi-file flush atomic — the block is either a complete readable block or it is not there at all, and framing makes that decidable.

Durability follows the database sync mode, except that the manifest is part of the **durable base**: both full and interval durability fsync it, and only `TDB_SYNC_NONE` skips. Interval mode batches the write-ahead log, never the catalogue.

CAUTION**—A failed commit keeps the edit and loses the record**

Only the log record is batched. An add or a remove applies to the **live set immediately** and queues its record alongside; the commit is what writes the queued records out. When the commit fails it drops them and leaves the live set exactly as the edit left it.

So a caller that treats a failed commit as "nothing happened" is wrong in the direction that matters. In memory the catalogue already names the new tables; on disk it does not, and no later commit resends the lost records because they are gone from the pending buffer. The next **rollover** writes the whole live set as a snapshot, at which point the edit becomes durable — so an entry left behind here is not transient, it is waiting to be made permanent.

That is why the flush and compaction paths undo their own edits when their commit fails: the outputs are withdrawn from the live set and, for a compaction, the inputs are named again at the levels they came from. Without it the catalogue would go on naming tables no level set ever took, and a rollover would eventually write that out as the truth.

23.5 Rollover

An append-only log grows forever, and replay time grows with it. So the log is periodically rewritten as a single **snapshot block** holding the whole live set — the family registry, every live sstable, each family's range tombstone set, then the sequence records that close it.

Rollover triggers when records since the last snapshot exceed `max(512, 2 × live entries)`. Tying it to the live set rather than a fixed count is what bounds replay to a small multiple of what is actually there, while amortizing the O(N) snapshot to O(1) per commit.

The rewrite is done safely: the snapshot is written to a temp file beside the manifest, made durable, then **atomically renamed** over it. A crash before the rename leaves the old manifest intact; after it, the new one. There is no window where the file is partially replaced. The temp name carries a thread id and pid so two concurrent rollovers cannot collide, and under a syncing mode the parent directory is fsynced too — otherwise the rename itself could be lost while both files survive.

CAUTION**—A rollover replaces the file, so a reader outside the log needs a hold**

"Append-only" describes the common path, not every path. A rollover renames a different file over the same name, so anything reading the manifest by path — the backup (ch. 14) copy, notably — cannot measure its length at one moment and read the bytes at another. Both have to happen inside one `tidesdb_manifest_hold`, which a commit and a rollover alike wait on. A length measured outside a hold can describe a file that no longer exists, and the bytes read under it are then a truncated snapshot, or a complete but newer one naming files the reader does not have.

CAUTION**—A snapshot must carry the high-water marks**

A snapshot emits only the *live* set. Without the explicit `MANIFEST_OP_SEQ` and `MANIFEST_OP_CF_SEQ` records, replay of a snapshot taken after a family was dropped would derive the next family id from the largest surviving one — and reissue an id that a dropped family had used. The high-water marks are raised, never merely stored, so replaying an older snapshot cannot walk them backwards.

23.6 Recovery and self-heal

Replay reads blocks forward, applying each batch. A batch whose fields would overrun its payload is treated as truncated, which is what a torn final commit looks like.

The interesting decision is what happens when the manifest is damaged. It could refuse to open. It does not — because **the sstables on disk are the real ground truth**, and their footers are self-describing enough to rebuild the catalogue from.

Two cases, both self-healing:

- **The header will not validate** — a garbage or truncated file. The manifest is discarded and a fresh log started, then **rebuilt from the files on disk**: the database directory is scanned, every `.klog` in it reopened, and each one whose footer reads back is re-registered under the family id its filename carries. Without that step the sstables would still be there and still be readable, and nothing would reference them.
- **Corruption partway through** — the readable prefix is kept and the log is immediately rewritten as a clean snapshot, discarding the unreadable remainder.

The rebuild recovers what the files carry and no more, which sets two limits worth knowing before you rely on it:

- **A family comes back named for its id** — the zero-padded `cf_` form rather than the name you gave it — because the name lived only in the manifest, while a key log carries its family's id in its own filename. Rename it back with `tidesdb_rename_column_family` (ch. 11).
- **A family comes back on default configuration**, for the same reason, and **every adopted sstable is placed at L1**, because no file records the level it sat at. L1 is the one tier whose members may overlap, so it is the only placement that cannot be wrong. Versions resolve by sequence rather than by level, so this costs compaction work rather than correctness.

A `.klog` whose footer will not read — a file a crash left half-written — is skipped and left on disk rather than adopted.

Both are reported at warn level in the log, not through the return value, so `tidesdb_open` (ch. 10) succeeds. That is a deliberate trade: refusing to open a database whose data is intact, because its index was damaged, is worse than rebuilding the index.

23.7 Ordering with the rest of the engine

The manifest commit is the **publication point**, and everything else is arranged around it:

- A **flush** builds its sstables and makes them durable, *then* commits the manifest. A crash before the commit leaves orphans; after it, the tables are live.
- A **compaction** commits its outputs and its input removals in one batch, so the set never contains both the inputs and their replacement, nor neither.
- A **create column family** commits before publishing the family in memory, so a crash can never leave a live family the manifest does not know about.

23.8 Invariants

Invariant	Why
A file the manifest does not name does not exist	Makes orphans harmless and recovery unambiguous
One commit is one framed block	Framing is what makes a multi-file change atomic
Data is durable before the manifest names it	Otherwise the catalogue references bytes that never landed
A snapshot carries `MANIFEST_OP_SEQ` and `MANIFEST_OP_CF_SEQ`	Live-set-only snapshots would let ids and sequences be reissued
High-water marks are raised, never stored	Replaying an older snapshot must not walk them back
Rollover is write-temp-then-rename	No window where the catalogue is partially replaced
A damaged manifest is rebuilt from the sstables	The tables are the ground truth; the catalogue is derived, so it can be re-derived

Compaction

24.1 The problem

Flushing produces a file per sealed memtable, all in L1, all potentially covering the same key range. Left alone, a point lookup would have to consult every one of them, and read amplification would grow without bound. Compaction merges files into fewer, larger, deeper ones so that reads stay cheap.

Every LSM engine faces the same three-way trade. Merge aggressively and reads are fast but write amplification is high. Merge lazily and writes are cheap but reads consult many files. Compaction *policy* is where an engine picks its point on that curve, and it is the single biggest determinant of behaviour under load. TidesDB follows a published design here rather than an ad-hoc one, which is what makes the policy separable and testable.

24.2 The planner is a pure function

TidesDB's policy follows **Spooky** (Dayan et al., PVLDB 2022) — see Design lineage (ch. 33) for what is implemented as published and what diverges — and separates the decision-making completely from the doing.

The planner is given a snapshot — the byte size and file count of each level, plus the family's configuration — and returns decisions. It performs **no I/O, opens no files, reads no clock, and touches no handles**. The same snapshot always produces the same plan.

That is not stylistic. Compaction policy is the hardest part of an LSM engine to get right and the hardest to test, because in most implementations it is entangled with the I/O it schedules. Here it is tested by handing it numbers and checking the answer, so its behaviour at level counts and size distributions that would be painful to construct on disk is trivially exercised.

24.3 Level capacities: DCA

Levels are 1-based. The classic scheme gives level i a capacity of `base × T^(i-1)` for a size ratio `T` — a geometric series fixed in advance.

Spooky pairs with **DCA — Dynamic Capacity Adaptation**, introduced by RocksDB and adopted by the paper — which sizes the smaller levels relative to *how much data the largest level actually holds*:

```
C_L  = base × T^(L-1)              the largest level keeps a geometric capacity
C_i  = N_L / T^(L-i)   for i < L   a smaller level is a fraction of the largest level's data
```

where `N_L` is the current byte size of the largest level. The consequence is that capacities **adapt to the data**. A database that is half full does not carry the level capacities of one that is full; the intermediate levels stay proportionally small, so the work of maintaining them stays proportional too.

24.4 The dividing level

```
X = num_levels - 1 - dividing_level_offset      clamped to [1, num_levels - 1]
```

A tree with fewer than two levels has no dividing level at all and yields 0, since there is nothing below to partition against.

`X` is the shallowest level whose merges **partition their output** by the largest level's file boundaries, instead of writing one output file.

The reason is the cost of merging into a large level. A merge that produces one enormous file must rewrite it in full next time anything overlaps it. By splitting output at the boundaries of the level below, each subsequent merge touches only the partitions it actually overlaps. `dividing_level_offset` moves that behaviour shallower or deeper — shallower partitions sooner and keeps individual merges smaller; deeper defers the bookkeeping.

It **defaults to 1**, putting `X` at `L - 2`. Spooky measures both ends of this: at `L - 1` a dividing merge rewrites a level holding a tenth of the data in one go and the system runs out of space at scale, while at `L - 4` it exhausts the open-file budget. `L - 2` is the balance, and it also keeps output files large enough to be written as long sequential runs.

24.5 The tree grows with the data

A family starts at one level and gains levels as it fills. When a merge would land in the largest level and that level is already at its capacity, it lands one level deeper instead — which is how a level is added. The count settles near `log_T(N/B)`, and the capacities above are what bound the space that costs.

The step is easy to leave out and expensive to omit. Without it a family stops deepening, so every merge has to land in a level it already occupies, the dividing level collapses onto the flush tier, and the tier grows one run per flush with nowhere to drain to. Every trigger still fires, every plan still produces jobs, every job still succeeds — and a scan of any range still ends up opening every run in the tier. Growth is evaluated where the target is chosen, **after** a merge has been selected rather than before, so it cannot preempt a dividing merge that was already due.

24.6 Partitioned merges fan out when their inputs allow it

Spooky's partitioned merge compacts **one group of perfectly overlapping files at a time** across the largest levels. When every input in the merge range sits wholly inside one of the largest level's boundary partitions, the plan emits **one job per partition**. Those jobs share no input, so they may run at the same time, and each costs one partition of transient space rather than the whole span of levels it covers.

That is only sound when the inputs really are aligned, and the plan checks rather than assumes it. The largest level is partitioned to these boundaries by construction, and a dividing merge writes its output the same way — so alignment holds once the levels above have been through one. But a run flushed straight into the tier spans whatever its memtable happened to hold, which under interleaved writers is the entire key space.

Caution—**A spanning input forces one job**

If any input straddles a boundary, the merge stays a single job over the whole range. Per-partition jobs would then share that input, and because the executor removes a job's inputs **atomically**, only the first could run — the rest would find the shared input gone and skip. A largest-level tombstone would be dropped without the older versions it shadows, and the deleted key would come back.

24.7 A lone job splits its range across threads instead

The tier drain is exactly the case above: a file flushed into the tier spans the key space, so the plan cannot divide it and emits one job. Left there, one thread carries the whole drain for the family, and a family that cannot drain accumulates overlapping runs that every read then pays for. Adding compaction threads does not help, because there is no second job for them to take.

So the executor divides what the planner could not. **A job that is the only one in its plan splits its key range at the same boundaries its output would have rolled at**, and runs those ranges on separate threads.

The output is the same either way. That single job already carries the boundaries and already seals its current file whenever a key crosses into the next partition — it was producing one file per partition all along, one after another. Handing each thread a contiguous run of those partitions produces the same files; what changes is that several are built at once.

Two properties make the split safe. The merge holds no state across keys — the base-version decision resets on every new key, and tombstone collection is a per-key question — so ranges are independent. And the split keys are real keys from the level above with an exclusive upper bound, so every version of a key stays in one range and retention sees a whole version chain exactly as one pass would.

The inputs are shared, immutable and already referenced; each range opens its own cursors over them. Resolution and commit stay single, which is what keeps a stale job a no-op and the install atomic.

NOTE—**It applies to one job, not to every job**

A plan that emitted several jobs is already spread across the pool, and letting each of those subdivide as well would multiply the thread count by the partition count. Only a plan that produced a single job permits it. Measured on a sustained single-family fill, this took compaction throughput from 1.27x to 1.93x across one to four threads, and the compactions completed per minute from 15-16 to 20-21 — the count being the drain-rate signal, since it was previously pinned regardless of how many threads the family was given.

CAUTION—**It raises the load a family can carry, not the depth it carries a load at**

Read amplification is an equilibrium: the flush tier grows when data arrives faster than compaction takes it away, and settles where those rates meet. Draining faster does not make a load the family already keeps up with any shallower — it raises the load at which it stops keeping up.

Measured at a pinned input rate, with the split as the only variable: at a rate both sustain the two are indistinguishable, same depth and the same level shape. Between the old drain ceiling and the new one, the divided merge reaches the target where the undivided one falls short, and writers spend a third of the time parked — at the *same* read amplification. Past both ceilings neither keeps up.

So the benefit to a read is indirect. Depth grows when a family cannot drain, and this moves the point where that starts.

24.8 And sheds one when the data goes away

The reverse also happens. When the largest level has shrunk to less than a size ratio's worth of its own capacity, that level is merged into the one above it and the tree loses a level.

The two thresholds are deliberately far apart — growth at the capacity, shrink at a `T`-th of it — and that gap is hysteresis. A family sitting near a boundary would otherwise grow and shed the same level repeatedly, paying a full merge each time.

Shedding is judged **on its own**, not behind the backlog triggers. A family whose data was mostly deleted has no level at capacity and no tier over its file count, which is exactly the state an over-deep tree would otherwise be stuck in forever, with every read paying for levels that hold almost nothing.

The merge takes **both** the largest level and the one it lands in. Levels below the flush tier hold non-overlapping runs, so moving the deepest level's files into a level that already holds files would leave two overlapping runs at one level and a read there would have to consult both.

`min_levels` is the floor: a family already at its configured minimum depth stays there however little its largest level holds. The engine's own floor is a flush tier plus one level for merges to land in, since below that the dividing level has nowhere to sit.

24.9 What makes a merge due

Three triggers, checked against the snapshot:

Trigger	Condition
L1 file count	L1 holds at least `l1_file_count_trigger` files
Level capacity	Some level's size has reached its DCA capacity
Tombstone density	An sstable's tombstone fraction has reached `tombstone_density_trigger`, above a minimum entry count

The L1 trigger is the read-amplification guard: L1 files overlap, so every one of them is a file a point lookup may have to consult. The capacity triggers keep the shape of the tree. The tombstone trigger exists because deleted data costs space and read work until a merge can prove it is safe to drop — a family that is mostly deletes would otherwise never compact, since its levels are not growing.

A forced `tidesdb_compact` (ch. 14) sets `force`, which plans a merge whether or not a trigger is due.

24.9.1 When the scheduler decides not to look

Planning a family means referencing every sstable it owns and copying their metadata. Doing that once a second for every family costs more as the database grows and usually produces the plan it produced last time, so the scheduler memoizes: it skips a family whose **level-set generation** and **reclamation floor** both match what it last planned against.

Both inputs are needed, and the second is the one that is easy to omit. A merge's value does not come only from the shape — when the last long-running transaction ends, `min_snapshot_seq` rises

and versions the same layout could not drop become droppable. Keyed on shape alone, the scheduler stops reconsidering a family at exactly the moment the work becomes worth doing, and on an idle database nothing ever moves the shape again.

The memo is recorded **only when the plan came back empty**. A plan that produced jobs is left unrecorded on purpose: if the jobs run, the layout moves and the memo would not have applied anyway, and if they fail the layout does not move — so recording it there would retire the family from scheduling until something else happened to write to it. A single failed compaction would otherwise stop that family compacting for the life of the process.

24.9.2 The merge target is never the flush tier

Spooky's preemptive merge picks the smallest level that could hold everything at and below it, and folds the smaller levels into it. TidesDB numbers the flush tier **L1**, so that search has to start at **L2**: a merge whose target is L1 reads the tier and writes it straight back, consolidating its files while leaving every byte exactly where it was.

That distinction is not academic. L1 is where every flush lands, one run per flush per family, so a merge that does not promote leaves the tier growing at the arrival rate no matter how often it runs. Every trigger fires, every plan produces jobs, every job succeeds — and the tier still climbs to dozens of runs, each spanning the whole key space, until some other path happens to promote it. A scan of any range then opens all of them, which is the read-side collapse reached without a single component failing.

The tier is always an **input** to a merge and never its destination.

24.10 Jobs

The planner emits **jobs**. Each job is a merge unit: its input sstable ids, its target level, and how to split the output.

Split policy	Behaviour
`COMPACTION_SPLIT_NONE`	Outputs are not aligned to anything — the target is below the dividing level
`COMPACTION_SPLIT_BOUNDARIES`	Roll at the largest level's file boundaries — the target *is* the dividing level, or the job is a partitioned merge

Independently of alignment, a job carries a **size cap**. An output rolls once it reaches that many klog bytes, so a partition holding an unusually large share of the data does not yield one correspondingly large file that every later merge touching it must rewrite.

The cap is derived as a fraction of the largest level's data, so it tracks the tree rather than being a fixed number, and it is not applied at all while that fraction would be too small to be meaningful — a young tree would otherwise split every entry into its own file.

> Note—**The cap counts the klog, not the data**
>
> With values separated, a spilled value leaves only a reference in the klog. The cap therefore counts what the klog actually holds, because what a later merge rewrites is the klog — the values stay where they are. Counting logical value bytes would split a run of large values into one file per key.

Splitting a job into per-partition sub-merges is what keeps individual merges bounded. It is also where a subtle correctness hazard lives, discussed below.

24.11 The executor

The executor takes a job and does the I/O. It reads its inputs through the **same k-way merge iterator that serves user iterators**, but asks it for a **raw** stream: every version of every key, ordered by key and then by sequence descending, with no snapshot applied and nothing hidden. What the two share is the hard part — keeping many sources positioned and picking the next key across them — not the resolution, which a compaction has to do for itself because it decides retention across the whole version chain rather than answering at one instant. See Iterators (ch. 29).

For each key it decides what survives:

- The **newest version** always survives.

- **Older versions** are dropped once no live snapshot could still need them. The floor is the oldest snapshot any live transaction holds, exposed as `min_snapshot_seq`. A long-running transaction holds that floor down and keeps garbage alive — a real operational effect, and the reason a forgotten open transaction shows up as space that will not reclaim.
- **Expired entries** are dropped.
- **Versions a range tombstone covers** are dropped once that tombstone sits at or below the floor, the base version included — nothing can resolve to it any more. See below.
- **Tombstones** are the hard case.

24.11.1 Range tombstones, which are not in the sstables

A range delete (ch. 12) is not written into the merge stream at all. It lives in the manifest, one blob per column family, so a compaction neither carries it nor can lose it — the merge simply asks the family whether an interval covers each key it is about to emit.

Retiring the interval itself is the mirror of the tombstone problem below, and `max_seq` cannot answer it: a table whose newest sequence is above the interval can still hold plenty of older versions inside it. So each build records the newest range tombstone it could see at the floor it ran at, and **the minimum of that across the family's tables** is the point below which nothing covered remains on disk. An interval at or below that minimum has nothing left to hide and is dropped from the manifest.

The watermark has to be a sequence the build actually observed, never the floor it ran at. With no live transaction the floor is every sequence there is, so a table built *before* an interval existed would claim to have applied it and the interval would be retired with its data still present. A table built when the family held no intervals records zero and holds the minimum down until it is rewritten, which is the conservative direction.

24.11.2 Why tombstones are hard

A tombstone must be retained until the merge can prove that no older version of that key survives *beneath* it. Drop it while an older value still exists in a deeper level, and that value becomes visible again — the delete is undone.

So a tombstone may only be dropped when the merge includes **every sstable that could hold the key**. If the merge covers a subset, the tombstone is written to the output and lives on.

This interacts directly with partitioned merges. A partitioned sub-merge sees only part of the keyspace, so its notion of "every sstable holding this key" has to be evaluated over the *whole* boundary-split job rather than the sub-range in front of it. Getting that wrong resurrects deleted data, and it is subtle precisely because each sub-merge looks locally correct.

The proof has to fail closed. The check asks each level which sstables overlap the key, into a fixed-size array, and a level that reports more than fits leaves a sibling it never examined — so that case refuses the drop rather than concluding from what it did see. A level it could not read at all refuses too. The tombstone surviving a merge it could have left costs one entry; the opposite mistake costs a resurrected key.

A **single-delete** is the exception: the caller has promised the key was written at most once, so the tombstone and that one put annihilate as soon as a merge sees them together. That promise is unverifiable by the engine, which is why breaking it resurrects data.

24.11.3 A lapsed entry arrives as a tombstone

An entry whose deadline has passed is read as a tombstone rather than as a value, and the merge reads its inputs through that same path — so expiry needs no rule of its own here. A lapsed entry enters the merge already a tombstone, shadows older versions of its key exactly as a delete would, and is collected on precisely the terms above: dropped once the merge can prove nothing older survives beneath it, retained otherwise. It is written out without a deadline, since a tombstone has nothing left to outlive.

This is why the read path hides a lapsed entry rather than skipping it. Skipping would leave the older version underneath reachable, and no merge would ever be asked to collect anything.

24.12 Concurrency

A family carries a claim, taken for a whole **plan** and released when the last of its jobs returns. So only one plan runs on a family at a time — but the jobs that plan emitted run concurrently, on separate workers, which is exactly what the per-partition fan-out above is for. The claim is what `tidesdb_compact` (ch. 14) reports as `TDB_ERR_LOCKED`, and what a runtime configuration change contends with — the scheduler must not read the planner knobs while they are being replaced.

Outputs and input removals commit to the manifest in **one batch**, so the set never contains both the inputs and their replacement, nor neither. Readers already inside a replaced sstable are safe: the level set's layout holds a reference on every table in it, so a table is freed only when the last reader leaves.

24.12.1 Reclaiming the merged-away inputs

Dropping an input from the catalogue and the level set stops anything reading it, but the filesystem is still charged for the file until someone unlinks it. Those are separate acts, and the gap between them is deliberate.

The unlink cannot happen at commit, because a reader may still be inside the file. It also must not happen *before* the commit: a crash in between would leave the manifest naming a file that is gone, and outside L1 — where the data can be replayed from the write-ahead log — that fails the next open outright rather than self-healing.

So each input is **marked** once the swap is published, and the unlink happens where both conditions are already satisfied: when the last reference to that sstable drops. That is the same moment the handle is closed, and by then the manifest edit has committed and no reader is left.

The mark waits for the swap rather than riding on the commit, because a swap that cannot be published leaves the inputs as the live data — still installed, still the only copy anything can read. Marking them at the commit would send their files with them the moment the level set let go, for a merge whose result nothing ever adopted.

A swap that fails after the commit is rolled back rather than left: the inputs are named again at the levels they were removed from and the outputs are withdrawn, so the catalogue matches what is installed. Without it the catalogue keeps naming outputs no level set took, and the next merge of the same inputs names a second set — which a reopen then adopts alongside the first.

Skipping the mark does not corrupt anything and nothing reads the leftover files — which is exactly why it is easy to miss. What it does is grow the store without bound for as long as the database stays open, while the level set goes on reporting a healthy live set, because every round of compaction rewrites its data and keeps the previous copy.

24.13 Invariants

Invariant	Why
The planner performs no I/O and reads no clock	Determinism is what makes policy testable
Versions are dropped only below `min_snapshot_seq`	A live snapshot must still see what it could see
A tombstone drops only when every sstable holding the key is in the merge	Otherwise a deeper value is resurrected
Tombstone safety is evaluated over the whole job, not a sub-range	A partitioned sub-merge sees only part of the keyspace
Outputs and removals commit in one manifest batch	The set must never hold both or neither
A merged-away input is unlinked when its last reference drops	Earlier and a reader is still inside it, or a crash leaves the manifest naming a missing file; never, and the store grows without bound while the live set looks healthy
An input is marked for unlink only once the swap is published	A swap that could not be published leaves the inputs as the live data, and a mark taken at the commit would take their files the moment the level set let go

Invariant	**Why**
A commit the level set does not take is rolled back	Otherwise the catalogue names outputs nothing installed, and the next merge of the same inputs names a second set that a reopen adopts alongside them
One compaction **plan** per family at a time	The claim is taken for a whole plan and released when the last of its jobs returns, so two merges of one family do run at once – on separate workers, over the plan's separate jobs. What it excludes is a second *plan*, and it also protects the config from a torn read
The plan memo is keyed on the shape and the reclamation floor together	A floor that rose makes versions droppable that the same layout could not drop, so shape alone stops reconsidering a family exactly when the work becomes worthwhile
A plan that produced jobs is never memoized	The jobs may fail, leaving the layout unmoved; memoizing there retires the family from scheduling until something else writes to it
Deeper is older	Merging must never place an older table above a newer one

Chapter 25

Block Cache

25.1 The problem

Every read that misses memory reads a block from a file — a btree node, a filter partition, a raw block on the sstable read path. Reading the same block repeatedly from the operating system is wasteful, and the kernel's own page cache does not help as much as it looks: a btree node arrives as bytes and has to be *parsed* before it is useful, and parsing it on every access is much of the cost.

So the engine keeps its own cache, and it caches **parsed objects as readily as raw bytes**. The consequence is that a cache hit skips both the read and the parse.

25.2 One cache, database-wide

Note—**The value log deliberately does not read through it**

The sstable read path uses this cache — btree nodes and filter partitions — and the value log does not. That is a deliberate choice, not an omission. The cache earns its budget on btree nodes because it stores them **parsed**, so a hit skips real work; values are raw bytes the operating system's own page cache already holds, and caching them a second time in the engine displaces the nodes that make lookups fast. Keying is the other half: a value is reached through its logical id rather than by position, and the segment holding it can be unlinked whole by a reclaim, so an entry keyed by file offset would outlive the file it named.

Not per family, not per sstable. One bounded pool shared by everything, because a per-file cache would carve the memory budget by file rather than by usefulness — a hot family with few files would be capped while a cold one with many held memory it was not using.

The key is `(file_id, offset)`: the 64-bit id of the owning file plus the block's byte offset. Together those name one block unambiguously across the whole database, which is what lets one pool serve every file without collisions.

25.3 Structure

The cache is divided into independent **shards**, each with a fixed array of frames and a bucket table mapping keys to them:

```
shard 0   [ frame ][ frame ][ frame ] ...    +  bucket table  +  lock  +  clock hand
shard 1   [ frame ][ frame ][ frame ] ...    +  bucket table  +  lock  +  clock hand
...
```

Sharding is what keeps writers from serializing: a put takes only its own shard's lock. The shard count is derived from the CPU count and the frames per shard from the configured capacity, both rounded to powers of two so the mapping is a mask rather than a division.

Caution—**The frame count must follow the byte budget**

A shard evicts on **bytes**, but it can only hold as many entries as it has **frames**. If the frame count is capped below what the budget can hold, the frames run out first and the cache plateaus at a fraction of its configured size — while still reporting the budget it was given. That failure is silent and reads as a miss-rate problem rather than a sizing one: an eight gigabyte cache capped at eight thousand frames per shard holds two gigabytes, and every read past that point misses and evicts.

So the frame count is derived from the budget and not capped below it. The metadata a frame costs is a fixed fraction of the entry it tracks — about two percent — so the array cannot grow out of proportion to the budget it serves. How long an insert may hold the shard lock hunting a victim is a **separate** limit, bounded absolutely rather than as a multiple of the frame count; conflating the two is what produced the cap.

Caution—**A frame is charged what it reserves, not what it uses**

A cached btree node is decoded into an **arena of its own**, and an arena serves its first request by taking a whole chunk from the shared pool. So the memory a frame holds is the chunk, which is rounded up, and not the bytes the node happened to need.

The budget is charged that reserved figure. Charging the used one — which is what the code did until it was measured — let a shard hold several times the memory it had been given, and the overshoot was invisible, because nothing reported the difference between the two. It was worst on a *small* cache, where the rounding is largest relative to the budget: a 64 MiB cache was measured resident at 908 MiB, while a 1 GiB cache overshot by only a tenth. That shape is why it survived — the failure shrinks exactly as the configuration grows, and the test that guards the budget uses entries near the nominal size, where there is almost nothing to round.

The consequence to carry into sizing: **a byte of budget buys less than a byte of node data**, by whatever the chunk rounds up. The pool's chunk is twice the node size a family is created with, so at the default a frame holds about half its chunk in useful node bytes.

That ratio was once far worse. The chunk was derived from the btree builder's *fallback* node size — the value used only when a caller asks for none — while families are created with a node size sixteen times smaller. Every cached node then reserved a 128 KiB chunk to hold about 4 KiB, and a 256 MiB cache spent its whole budget on 2,007 frames holding 8 MiB of node data. Tying the chunk to the size families actually use took the same budget to 31,677 frames, raised the hit rate from 0.90 to 0.98, and raised read throughput 35%. The lesson is not about the constant: **two numbers that describe the same object have to be defined in terms of each other, or they drift and nothing reports it.**

Frames are **fixed and never freed** while the cache lives. That is the central design choice and everything else follows from it: a reader that has found a frame can hold it without any risk that the frame itself disappears. Only the *payload* a frame points at is ever reclaimed.

A frame is in one of three states:

State	Meaning
`FREE`	Reusable
`LIVE`	Holds a gettable entry
`DYING`	Evicted, but still pinned by at least one reader

`DYING` is what makes eviction non-blocking. An evictor does not wait for readers to leave; it marks the frame and moves on, and the payload is released by whichever thread drops the last reference.

25.4 Reads are lock-free

A get takes no lock at all:

1. **Probe** the bucket chain for the key.
2. **Pin** the frame with an atomic reference count.
3. **Recheck** that the frame still holds the key that was probed.

The recheck is the whole trick. Between the probe and the pin, the frame may have been evicted and reused for a different block. Pinning first and verifying second means a reader either ends up holding what it asked for, or notices it does not and retries — and it never holds a reference to something that has been freed, because frames are not freed.

Evicted keys leave a **tombstone** in the bucket table rather than a hole, so a removed key cannot truncate the probe chain of a key that hashed past it.

25.4.1 The probe stays inside the bucket table

A bucket is a single 32-bit word holding a frame index with an 8-bit **hash tag** packed above it. A bucket whose tag differs cannot hold the key being looked for, so the probe rejects it from that word alone and never touches the frame.

This matters because the bucket table is dense and walked sequentially, while the frame array is large and touched at random: following a bucket to its frame is a likely cache miss, and on a loaded table most of those misses were on buckets that turn out not to match. The tag confines the common case to memory the probe is already streaming through.

The frame's full key is still compared after a tag match, so a tag collision costs one wasted touch and never a wrong answer.

How much this is worth depends entirely on how full the table is. At a load factor around one half it raises lookup throughput by roughly 40% at sixteen threads and 25% at one, and drops `cache_get` from about 30% of CPU to 22%. On a sparsely loaded table it changes nothing at all: probes end on their first bucket, and a bucket that matches was never going to be rejected. A cache sized comfortably above its working set therefore sees none of this, and a cache sized just under one sees most of it.

The tag's bits are cut from a slice of the hash disjoint from both the low bits that pick the shard and the higher slice that picks the probe start, so the three decisions do not correlate. Two index values are reserved for *empty* and *tombstone*, which is why a shard's frame count is capped one power of two below what the index field could otherwise address.

25.5 Eviction

Insertion may need to make room, and the sweep is a **clock**: each frame carries a second-chance bit set on access and cleared by the sweep. The hand advances, clearing bits and taking the first frame whose bit is already clear. Approximate LRU at a fraction of the bookkeeping.

The sweep is bounded by a fixed number of examinations — not by a multiple of the frame count, since that would tie how long the lock is held to how much the shard can hold. The clock exits at the first evictable frame, so the bound is only reached when nearly everything is pinned or freshly referenced, and the hand persists across calls: stopping short paces the work rather than abandoning it, and the next insert resumes where this one stopped.

Eviction happens under the shard lock, so an insert that must evict holds the lock for longer than one that does not. That is why waiters spin only briefly before yielding their core: on an oversubscribed machine, spinning through another thread's clock sweep wastes exactly the capacity the sweep is competing for.

Caution—**Cache sizing is not just a hit-rate knob**

The capacity also bounds the **filter** footprint. Partition range filter blobs are fetched on demand and are meant to live here — see SSTable (ch. 21). A cache too small to hold the working set of filter partitions makes every point lookup re-read its filter, which costs more than the filter saves.

25.6 Exactly-once reclamation

A payload must be released exactly once, and the thread that should do it is not known in advance: it may be the evictor, or a reader that was still inside when eviction happened.

The rule is that **whoever drops the last reference reclaims**. There is no separate reclamation pass, no epoch, no deferred list — the same reference-counting discipline the rest of the engine uses.

Each entry carries its own reclaim function, supplied at insert. That is what lets one cache hold both parsed btree nodes and raw byte blocks: the cache does not know what a payload is, only how to release it.

25.7 What it costs and what it reports

The cache is a pure optimisation — every entry can be dropped and re-read. Nothing depends on it for correctness, which is why it can evict without coordinating with readers.

`tidesdb_get_cache_stats` (ch. 15) reports hits, misses, hit rate, residency, and shard count. The hit rate is cumulative since open, so it moves slowly; compare deltas between samples rather than absolute values when tuning.

25.8 Invariants

Invariant	Why
Frames are never freed while the cache lives	A reader can pin without the frame vanishing under it
The frame count follows the byte budget	A frame ceiling that binds first shrinks the cache silently, while the configured budget still reports as given
The sweep's bound is absolute, not a multiple of the frame count	It limits how long the shard lock is held, which must not grow with how much the shard holds
Pin, then recheck the key	The frame may have been reused between probe and pin

Invariant	Why
An evicted key leaves a tombstone, not a hole	Otherwise it truncates the probe chain of later keys
A payload is reclaimed exactly once, by the last reference dropped	Evictor and reader race; neither may assume it is last
The clock sweep is bounded	An all-pinned shard must not spin forever
Nothing depends on the cache for correctness	It must remain free to evict at any moment

CHAPTER 26

File Descriptor Manager

26.1 The problem

An LSM database accumulates files. A few thousand sstables is ordinary, and every one a reader touches needs an open descriptor. The process has a hard ceiling on those, often 1024 by default, and it is shared with everything else the embedding application is doing.

Left alone this fails in the worst possible way. Descriptors run out during a flush or a compaction — precisely when the database is busiest — and the failure surfaces as `EMFILE` from an `open` deep inside an unrelated code path. Nothing about that error tells the caller the situation is temporary, so it propagates as an I/O failure and a perfectly healthy database reports corruption-shaped errors under load.

The manager exists to turn that into something with a defined meaning: **descriptor exhaustion is backpressure, and backpressure is retryable.**

26.2 A standalone service

It owns no engine and reaches into no global handle. A caller constructs one, opens block managers through it, and gates reader opens against it. It is unit-testable with a stack-allocated instance and no database anywhere — the same composition rule (ch. 16) the rest of the engine follows.

26.3 Labels over one budget

Every tracked descriptor carries a **label** naming its file kind:

Label	File
`FD_LABEL_SSTABLE_KLOG`	An sstable's key log
`FD_LABEL_WAL_LOG`	A write-ahead log
`FD_LABEL_VLOG_SEGMENT`	A value-log segment

Counts are kept per label while **one shared ceiling** bounds the total. The split matters because the kinds are reclaimed differently and by different logic: an idle key log or value-log segment can be closed and reopened on the next read, while an active write-ahead log's descriptor cannot. Labelling lets the right reaper reclaim the right kind without one starving the other.

Value-log segments are held to the same ceiling. The store is a series of segment files rather than the single file it once was, and a large database can hold thousands of them, so leaving them outside the budget would let the value log exhaust the process on its own while the reported figures looked healthy.

The reaper takes segments back **before** key logs. A key log is read on every point lookup that reaches its sstable, while a segment is touched only when a separated value is dereferenced, so of the two the segment's descriptor is more likely to be sitting idle. Neither loses anything by being evicted — both reopen on the next read — and the segment currently taking appends is never taken, since it is written on every spill and would be reopened immediately.

The budget is not `max_open_sstables`. A reserve is held back so a reader can still open a file while the reaper is working: an eighth of the configured maximum, never fewer than 16 descriptors and never more than half of it, subtracted from the maximum and floored at one. At the default `max_open_sstables` of 1024 that reserves 128 and leaves a **budget of 896** — the count the reaper reclaims toward and the point at which readers begin to be gated, both of which arrive before the configured number is reached.

A maximum of **0 means unlimited** to the manager itself: no soft cap, so the reaper never evicts for budget, no reader is ever gated, and resident descriptors are bounded only by the process open-file

limit — where exhaustion surfaces through the `EMFILE` retry path below rather than as backpressure. That state is not reachable by configuring `max_open_sstables` to zero, because `tidesdb_open` resolves a zero there to the default (ch. 36) before the manager is constructed. It is what a caller building a manager directly gets, which is how the unit tests exercise it.

Not everything is labelled. The manifest and stdio are not tracked at all, and neither are temporaries. Those, plus headroom, are what the fixed `TDB_FD_RESERVE_UNTRACKED` of 64 descriptors is held back for.

The budget is cut to fit the process at open. `tidesdb_open` reads the open-file ceiling and calls `fd_manager_budget_for_process`, which lowers `max_open_sstables` until it leaves that reserve free, logging at warn when it does. This matters at the defaults and not only at the extremes: the default `max_open_sstables` is 1024 and the usual POSIX default `RLIMIT_NOFILE` is *also* 1024, so a database left alone would budget every descriptor the process has and leave none for the manifest it must also hold. The ceiling is only ever read — raising it is `tidesdb_raise_open_file_limit` (ch. 10), an operator's call, made before open.

The cut is deliberately conservative about what it will act on. A ceiling that is unlimited, that cannot be read, or that the platform cannot state exactly — Windows, whose low-IO layer permits a large but unqueryable number of handles — reports as *no ceiling*, and the configured figure stands. Cutting against a guess would penalise precisely the process that had no limit at all.

The reader gate therefore asks one question — whether the resident total is under the budget — and not two. An earlier design kept a second, process-wide descriptor count behind it, fed by an observer on every block-manager open and close. It was never armed, and cutting the budget to fit the process is what made it redundant rather than merely unused: the budget now sits at or below `ceiling - reserve`, so the resident check reaches its limit first, always.

What that leaves uncovered is **several databases in one process**. Each cuts against the same ceiling independently, so together they can still exceed it. A shared count would notice, but it would not help: each database's reaper reclaims only its own files, so the one that gets gated would wake a reaper with nothing of the offender's to close, wait out its rechecks and fail regardless. That case belongs to the `EMFILE`/`ENFILE` retry path below, and sizing several databases to share a process is the operator's to do.

Caution — **Every labelled open must pair with a labelled close**

The counts are maintained by the code that takes and releases ownership of a file, not by the block manager underneath it, because only the owner knows which kind of file it is holding. That makes the pairing a standing obligation rather than something the type system enforces.

An unpaired close is worse than it sounds. The two readers of these counts — the reader gate and the reaper — both look at the **total across every label**, so one label drifting drags the total with it. A count that only ever decrements eventually pushes the total below zero, at which point the gate admits every open and the reaper's reclaim loop never executes: the budget silently stops existing while every statistic still reports a sensible number, because the reported figure reads a single label rather than the total.

Key logs are accounted where the sstable adopts and releases them. Write-ahead logs are accounted in `engine_open_wal` and `engine_close_wal`, and the flush path — which retires a generation's log after its data reaches L1 — is the one other place permitted to release one. A new close site that bypasses those is the way this breaks.

26.4 Two ways it applies pressure

A reader gate. Before an sstable read opens a key log, it asks whether the budget allows it. If it does not, the gate does not hand the problem back — it wakes the reaper to close idle files, waits briefly, and rechecks, a bounded number of times. Descriptor pressure under a heavy flush and compaction load outlasts a single recheck, and a caller told to try again has no lever the engine does not already have: its only remedy is to sleep and ask again, which is what the gate does on its behalf, knowing what it is waiting for.

Only when every recheck fails does the read report **busy** — the `TDB_SOURCE_BUSY` of the read path (ch. 18), which the public boundary translates to `TDB_ERR_LOCKED`. That is the exhausted case rather than the ordinary one, and it is still why treating the code as "not found" is a correctness bug. The data exists; only the descriptor was unavailable.

Open retry. Opening a block manager can still hit the real process ceiling under heavy flush and compaction, since not every descriptor in the process is tracked. The open wrappers treat `EMFILE` and

`ENFILE` the same way, and to the same bound: wake the reaper, back off briefly, retry. Both paths absorb the same pressure identically, which matters because they compete for the same descriptors — a reader that gave up sooner than an opener would turn a shortage one of them was about to clear into a failure the caller had to understand.

Both bounds are finite. If the ceiling is genuinely exhausted by something outside the database, retrying forever converts a resource problem into a hang.

26.5 The reaper

A background ticker closes idle descriptors back down toward the budget. Readers signal it when they are gated, so pressure is relieved on demand rather than only on a timer.

Closing a descriptor does **not** evict the sstable. The object stays installed and readable; only its file handle goes. The next read reopens it. This is the distinction drawn in SSTable (ch. 21) — object lifetime and descriptor lifetime are separate — and it is what makes a descriptor safe to reclaim at all.

The reaper must not close a descriptor a reader is inside. An installed sstable rests at one reference — the one its level set holds — and the sweep takes a second on every candidate it collects, so a table with no reader in flight is sitting at **both**. That total is what the evicting window is claimed at, and anything above it means someone is using the file.

Counting only the level set's reference is not a smaller mistake than counting none: the claim is an exact compare-and-exchange, so a resting count named one short simply never matches, and no key log is ever given back while the statistics go on reporting a budget that is quietly not being enforced.

The window itself has to survive whatever runs during it. It is claimed by moving the reference count to a sentinel and released by adding that offset back rather than storing the resting value, so a reference taken or dropped inside the window carries through. Storing it would discard that thread's change — losing an acquire frees a file still being read, losing a release leaks the handle for the life of the process.

The sweep also passes over any table whose descriptor is already closed. A database keeps far more sstables than it keeps open, and one with nothing resident has nothing to give back, so ordering it by age and offering it for eviction is work spent on a table that cannot answer.

26.6 Why this is a module and not a counter

It could have been an atomic counter checked before `open`. It is a module because the behaviour that matters is not counting but **what happens at the limit**: which kind is reclaimed, who is woken, how long a caller waits, how many times it retries, and what it reports when it gives up. Those are policy decisions, and putting them in one testable place is what keeps them consistent across the flush path, the compaction path, and the read path.

26.7 Invariants

Invariant	Why
Every labelled open pairs with a labelled close	Both readers of the counts use the cross-label total, so one drifting label disables the budget entirely
Pressure is waited out before it is reported	The caller's only remedy is to sleep and retry, and the engine can do that better because it knows what it is waiting for
Exhaustion is reported as busy, never as absent	The data exists; only the descriptor is unavailable
A descriptor is reclaimed only at the resting reference count, the level set's plus the sweep's own	Any excess reference means a reader is inside the file. The claim is an exact match, so a resting count named one short never fires at all and the budget stops being enforced silently
The evicting window preserves a reference taken or dropped inside it	The sentinel is a fixed offset, so releasing by adding it back carries the change through; storing the resting value would free a file still being read, or leak the handle
Closing a descriptor never evicts the sstable	The object and its file handle have separate lifetimes

Invariant	Why
Reader-gate rechecks and `EMFILE`/`ENFILE` retries are bounded, and to the same bound	An externally exhausted ceiling must fail, not hang; and a reader that gave up sooner than an opener would fail on a shortage the opener was about to clear
The budget plus the untracked reserve fits the process limit	Untracked files must not be starved by tracked ones. `tidesdb_open` cuts `max_open_sstables` through `fd_manager_budget_for_process` so the reserve is always left free. Bounds one database; several in one process cut against the same ceiling independently and can still exceed it together

Thread Manager

27.1 The problem

The engine runs several kinds of background work: worker pools draining queues, and periodic tickers. Each needs the same lifecycle — create threads, signal shutdown, wake anything blocked, join, and account for the ones still alive.

Hand-rolling that per subsystem is how shutdown bugs are made. Each site invents its own stop flag and its own join, they drift, and a thread that blocks on a queue nobody will ever post to becomes a hang at close. Worse, the *order* of shutdown ends up implicit: whichever pool happens to be stopped first.

So the pthread lifecycle lives in two primitives, and one registry owns every instance.

27.2 Two primitives

A worker pool is a fixed set of threads that each block on a shared queue and hand every dequeued item to a callback. The queue is **caller-owned** — the pool neither creates nor frees it. That separation is what lets the engine shut a pool down and still hold the queue to drain whatever was left in it.

A ticker is a single thread that runs a callback once per interval and can be woken early. The tick runs *first*, then the thread waits — so a ticker started to do something does it immediately rather than after one interval. Wakeability is what lets a reader nudge the fd reaper the moment it is gated instead of waiting out the period.

Neither primitive knows anything about databases. Both are constructed with a callback and a context.

27.3 The registry

Every background process is registered under a **label**:

Kind	Instances
Pool	flush, compaction
Ticker	the transaction clock, the descriptor reaper, deferred frees, and the compaction scheduler — always; idle rotation unless `memtable_idle_flush_seconds` is zero; WAL sync only under `TDB_SYNC_INTERVAL`

The registry owns the handles and stops them. It does **not** own the queues a pool drains — those belong to the engine, which needs them after the workers are gone.

Two properties come out of having one owner:

Shutdown is ordered. Processes are stopped in reverse registration order, so a subsystem started later — and therefore possibly depending on an earlier one — is stopped first. Ad-hoc shutdown gets this right by luck; a registry gets it right by construction.

Background work is enumerable. One place can answer what is running, which is the difference between diagnosing a stuck close and guessing at it.

27.4 Shutdown

Closing a database drains and stops in order:

1. Stop accepting new work.
2. Stop the background processes in reverse registration order — each signalled, woken if blocked, and joined.
3. Drain what remains: sealed memtables still queued are written out, or their logs are closed without unlinking so the data survives to be recovered.

4. Release the shared stores.

Step 3 is where the ordering earns itself. Draining the flush queue requires that no worker is still pulling from it, and that no compaction is still installing into a level set being torn down.

Note—**Close cannot report a failure**

The shutdown path returns no status, so `tidesdb_close` (ch. 10) always succeeds for a live handle. An I/O error while writing the last of the data is logged, not returned. Callers needing a durability guarantee ask for one before closing.

27.5 What runs where

Worth stating plainly, because it explains where latency comes from:

Work	Thread
Buffering, conflict checks, WAL append, memtable apply	The calling thread
Rotation	The committing thread that found the memtable full
Writing a sealed memtable to sstables	Flush pool
Merging sstables	Compaction pool
Writing a buffered file to its descriptor	That file's own flush thread
Closing idle descriptors	Reaper ticker
Freeing memtables and level layouts a reader still held when they were retired	Deferred-free ticker
Deciding which families are due a merge	Compaction-scheduler ticker
Rotating a memtable nothing has written to	Idle-flush ticker
Publishing the second transaction timeouts age against	Transaction-clock ticker
Periodic durability barrier	Sync ticker

27.6 What an idle database costs

An open database taking no traffic should approach zero CPU, and treating that as the signal the engine has quiesced is reasonable — so anything that keeps the process warm is a defect rather than a detail.

The tickers are cheap by construction. Each parks on a condition variable with a deadline rather than polling, and the ones that are conditional are not started at all when nothing asks for them. Four run once a second — the transaction clock, the descriptor reaper, deferred frees and the compaction scheduler — so that is four wakeups a second in the usual configuration and five under `TDB_SYNC_INTERVAL`, plus idle rotation on its own far longer period, thirty seconds by default. None of that registers.

The thing that did register was a **buffered file's flush thread**, which is not a ticker and does not appear in the table above as periodic work. Its park timeout was short enough to wake it two thousand times a second whether or not anything had been written, which is most of an idle process's CPU. The timeout is only insurance against a missed wake, never the mechanism that delivers work, so it now grows while parks come up empty and resets the moment a run drains — see Block Manager (ch. 20).

The lesson generalizes. A periodic *fallback* alongside an event-driven path should be sized for the rare case it exists to catch, not for the latency of the common case, because the common case is already covered by the signal.

27.7 What does not run on a worker

A commit does its own work rather than handing it to a worker, so commit latency is the work itself plus contention — not a queue depth. Rotation is the exception that proves the rule: it runs on a committing thread, which is exactly why a slow one lands in the write latency tail and is logged when it exceeds 20 ms.

27.8 Invariants

Invariant	Why
One registry owns every background process	Otherwise shutdown order is implicit and drifts
Processes stop in reverse registration order	A later subsystem may depend on an earlier one
A pool never owns the queue it drains	The engine needs the queue after the workers are gone
Every stop signals, wakes, and joins	A thread blocked on an empty queue would hang the close
A ticker runs its tick before its first wait	Startup work must not be delayed by a full interval

CHAPTER 28

Transactions and MVCC

28.1 The problem

Readers must not block writers and writers must not block readers, while each reader still sees a coherent view of the database. The standard answer is multi-version concurrency control: never overwrite, always append a new version, and let each reader decide which versions it may see.

That turns concurrency control into a question about **numbers**. Every write gets a sequence; every reader gets a ceiling; a version is visible if its sequence is at or below the ceiling. No locks are involved in a read.

The engine's MVCC core is deliberately pure — it knows sequence numbers and precomputed key hashes, not engine structures — so it builds and tests standalone.

28.2 The clock

Three things live in the clock:

A monotonic counter. Every commit draws a sequence from it. Sequences are never reused; the counter only moves forward, including across restarts (see Recovery (ch. 30)).

A commit-status ring. Drawing a sequence and completing a commit are not the same instant, so a sequence that has been drawn but not finished must be invisible — otherwise a reader could see half a batch. The ring records, for the most recent sequences, whether each one committed.

The ring is bounded, and what happens past its edge is the interesting part. A sequence older than the ring is **treated as committed**, because it must be: it was drawn long enough ago that it cannot still be in flight, and it either applied or was abandoned. That eviction rule is what keeps the visibility check O(1) and the memory fixed, rather than tracking every sequence ever issued.

A reservation table for conflict detection, described below.

28.3 Snapshots

A snapshot is a sequence ceiling, and the isolation level decides what it is:

Level	Ceiling
Read uncommitted	Unbounded — even in-progress sequences are visible
Read committed	The current sequence, re-read for each operation
Repeatable read, snapshot, serializable	Frozen when the transaction begins

Freezing at begin is what makes repeated reads stable: the ceiling does not move, so a version that committed after the transaction started is invisible no matter how many times it looks.

The ceiling is not quite the only filter. A commit whose batch reached the log but failed to enter the memtable leaves entries behind at a sequence that never committed, and those are stepped over in favour of an older visible version at **every** level — including read uncommitted, whose unbounded ceiling would otherwise return them. "Uncommitted" means a sequence still in flight, not one that was given up on.

A snapshot costs nothing to take — it is a number — but holding one is not free. The oldest live snapshot is the floor below which compaction (ch. 24) may drop old versions and tombstones. A transaction left open holds that floor down and keeps garbage alive, which surfaces as `min_snapshot_seq` in the database statistics and as space that will not reclaim.

28.4 Named snapshots

A transaction's snapshot is a number it draws at begin and drops when it ends. A **named snapshot** is the same number held deliberately: `tidesdb_snapshot_create` captures the current sequence and

keeps it, and a transaction opened against it reads as of that point.

The mechanism is already everywhere. Every read resolves at a ceiling, and the memtable and the btree both take the newest version at or below one, so reading the past needs no new path — only a way to say which ceiling. What the handle adds is the **retention**.

A snapshot is registered exactly as a repeatable-read transaction is, so it appears in the minimum the registry publishes and holds the reclamation floor at its own sequence. That is what keeps the versions it resolves to alive: compaction may not discard above the floor, and a flush carries a key's chain into L1 on the same rule.

Caution—**The ceiling is free; the retention is the whole feature**

Setting an older ceiling is a few lines. Below the floor a merge keeps one version of a key — the newest at or below it — and drops the rest, so a read at an older ceiling finds nothing there and reports a key that existed as **absent**. Available and untrue.

So a sequence is readable exactly while something has been holding the floor under it, and the engine tracks where that line is rather than leaving a caller to guess. The **oldest readable sequence** is the highest floor any collection has ever taken: at or above it nothing was ever eligible for collection, and below it the state is gone. `tidesdb_txn_begin_at_seq` compares against that and returns `TDB_ERR_TOO_OLD` instead of answering from what survived.

The watermark is raised where a floor is *taken*, not where the collection finishes. A merge that has already read a floor and is still running has to be visible to a reader deciding whether its sequence is safe, or the check would pass a moment before the versions disappeared.

The transaction registers before the check, in the same order and for the same reason: joining at that sequence caps the floor every later collection takes, so the watermark cannot move above it once it has been read.

A reopened database cannot account for what an earlier run collected — nothing records it, and the sstables hold only whatever survived — so the watermark starts at the sequence recovery resumed from, and points from before a restart are refused.

A snapshot is the stronger of the two. It holds the floor from the moment it is taken, so the point stays readable; a bare sequence is readable only while something else happens to be holding the floor under it, and on an idle database the next merge takes that away.

The cost is the cost of a long-running transaction, because it is the same mechanism: a snapshot left open pins `min_snapshot_seq` and the space below it will not reclaim. It shows up in exactly the place a leaked transaction does, and is diagnosed the same way.

One thing a snapshot does not restore. Expiry is judged against the published clock at the moment of the read, not against the snapshot's sequence, so an entry whose lifetime has elapsed reads as a tombstone through a snapshot taken while it was live. A snapshot travels over versions, not over deadlines.

28.5 Reservations: first-committer-wins

At snapshot and serializable isolation, a commit must not silently overwrite a write it did not see. The check is a reservation over the transaction's write set.

Each key hashes into a slot in a fixed table. A slot packs two things:

```
[ 16-bit key fingerprint ][ 48-bit claiming commit sequence ]
```

The fingerprint is what lets the check stay local on a fixed-size table. Two different keys can land in the same slot; without a fingerprint the commit could not tell a genuine same-key conflict from a hash collision, and would have to read another committer's applied version to disambiguate. With it, **the slot alone answers the question** — no reaching into anyone else's data.

NOTE—**The bound the fingerprint is checked against is a cached minimum**

The fingerprint only decides the outcome for an occupant no newer than the oldest open snapshot; a newer one is treated as a conflict whatever its fingerprint says, because a concurrent writer of the colliding key could still depend on the slot.

Computing that minimum exactly means scanning every registry shard, and a reservation runs once per written key, so the commit path does not compute it. The registry publishes the value instead, refreshed on the compaction scheduler's tick, and the commit path reads it as a single relaxed load hoisted once for the whole write set. A stale reading is only ever *low*, which is the conservative direction: it makes more occupants count as newer, so it can reject a commit that would have been admitted, and can never admit one that should have been rejected.

The empty registry is the case that needs care. The scan answers `UINT64_MAX`, a sentinel meaning nothing constrains you rather than a real minimum, and publishing it would let a transaction beginning afterwards hold a snapshot *below* the published value – the one direction a reader of this may not tolerate. The publisher stores zero for an empty set instead.

The subtlety is what each key is validated *against*. Not the transaction's snapshot, but **the version this transaction actually read for that key**, recorded when `tidesdb_txn_get` (ch. 12) was called. A blind write with no prior read falls back to the snapshot.

This is why the API distinguishes tracking and non-tracking reads. A read that feeds a write must be tracked, so the write is validated against what it was based on. A probe whose answer does not determine what is written should be untracked, because widening the footprint only creates conflicts that are not real.

Losing a reservation is `TDB_ERR_CONFLICT`, raised before anything durable has been written.

28.5.1 An interval has no key to hash

A range delete (ch. 12) writes an interval, not a key, so there is no hash for it to claim and no fingerprint that could describe it. Two transactions — one deleting a range, one writing a key inside it — can never collide in the table above, however they are ordered.

Two things close that, and they cover different windows.

A second, small table holds the intervals themselves. A batch writing one claims it before reserving any of its keys, and every point write checks that table before taking its own slot. The table is almost always empty, so the check is one relaxed load; only when a range delete is actually in flight does a write compare its key against a handful of bounds. That is also what carries a **two-phase** range delete through the window between its prepare and its commit.

The interval goes back **once the batch is visible, and not before**. A key reservation is renamed rather than dropped, because the slot itself is what a later writer of that key reads; an interval has no slot of its own to read, so what replaces it is the committed delete, which a later writer's conflict scan finds. Releasing earlier leaves the gap between the two uncovered, and not releasing at all spends the table: it has a fixed number of slots, so one leaked per commit ends with every later range delete in the database refused as a conflict that no retry could clear.

The slots are a fixed width, which is where the public `TDB_MAX_RANGE_BOUND_SIZE` (ch. 12) limit on a range delete's bounds comes from. A longer bound is turned away at the API rather than narrowed to fit, since a narrowed bound describes a wider range than the caller asked for.

A commit gate covers the rest. The interval table alone still leaves the reverse order open: a point write that has already reserved, and is not yet visible, cannot be seen by a range delete's conflict scan. So a commit carrying one takes the database's commit gate exclusively and runs alone, while every other commit at snapshot or above holds it shared and never waits on another. The gate is held for the whole commit rather than just the reservation, because a batch that has reserved but not yet marked its sequence visible is exactly the write the delete must not miss.

A two-phase transaction takes the gate only for its **prepare**. The in-doubt window that follows has no bound, and holding it there would stall every other committer for as long as the transaction stayed undecided — the held interval covers that window instead.

28.6 The conflict scan, and what it does not read

The reservation catches a transaction that is committing *right now*. It cannot catch one that committed and finished between this transaction's snapshot and its commit — that writer has already released its slot. So a commit at snapshot isolation or above also scans its write set against the data itself, and repeatable-read and serializable scan the read set the same way.

The question that scan asks is narrow. Not *what is the newest version of this key*, but only **does any version of it exist above my snapshot**. That distinction is worth a great deal, because answering the first question means descending a btree in every overlapping sstable at every level, parsing nodes and verifying checksums, purely to discard the answer.

Every sstable records the newest sequence it contains in its footer. An sstable whose newest sequence is at or below the transaction's snapshot **cannot** hold a conflicting version, so the scan skips it on that field alone — no descent, no node read, no checksum. Skipping it stays correct even when that sstable does hold the key, because the only thing being asked is whether something newer exists, and a skipped sstable's answer is no.

Since flush and compaction continuously produce sstables while transactions are short, nearly every sstable predates any live transaction's snapshot, and nearly the whole scan resolves from metadata.

Note—**The memtable is still read**

The skip applies to sstables, which carry the sequence range in their footer. A conflicting commit has usually not been flushed yet, so the memtable genuinely has to be consulted and is the part of the scan that remains.

A range delete (ch. 12) asks the same question over an interval rather than a key: *does any key in these bounds sit above my snapshot*. It prunes on the same footer field — a table whose newest sequence is at or below the snapshot holds nothing newer wherever its keys fall — and stops at the first key it finds, since one is all the commit needs to know.

Two things differ from the point form. Every source is asked rather than the first one holding the key, because a newer key in an interval can be in any of them and a shallower source says nothing about what a deeper one holds elsewhere in the range. And there is **no fallback**: a source that cannot answer an interval reports busy, which the commit retries, rather than being stood in for by a full read the way a missing point probe is. A missing implementation must never read as a clear run.

Busy has to mean *transient*, though. The retry is bounded, and a source that answers busy for a reason that will hold every time — a bound too long for a buffer, say — turns a permanent condition into `TDB_ERR_IO` after the retries run out. A condition the caller cannot clear belongs at the API, as an argument error, not on this path.

28.7 The write and read sets

The **write set** buffers operations in insertion order. Order is what makes savepoints work: a savepoint is a position in the sequence, rolling back to it discards the tail, and releasing it forgets the mark without discarding anything.

The **read set** records what tracking reads observed — key and version — and exists only to feed the reservation check at commit.

Both are per-transaction and single-threaded, which is why a transaction handle is not thread-safe. The one exception is the flag `tidesdb_txn_request_abort` (ch. 12) sets: it is the only field another thread ever writes, and it changes nothing else, so the thread that owns the transaction is still the only one that moves its state.

28.8 The registry

Live transactions are registered so the engine can compute the oldest live snapshot. The registry is on the commit path, so its cost matters: each transaction holds its own slot and leaves in constant time rather than by scanning a list.

It is also **sharded**, because joining and leaving are per-transaction rather than per unit of work — every transaction does both exactly once, whatever else it does — so a single lock made this a database-wide serialization point that got worse with concurrency, not with load.

A transaction takes the shard of the **thread that joins it**, claimed once per thread from a counter. The thread is what actually contends, so it is what the shards have to spread. Deriving the shard from the transaction's own address instead looks equivalent and is not: a caller running one short transaction at a time — a statement-scoped transaction under autocommit, say — frees and reallocates at the same few addresses, so the shards collapse onto a handful of slots and unrelated threads queue on one lock. Measured on sixteen threads that difference is most of the throughput.

The transaction stores the slot it landed in rather than recomputing it, so a leave can never disagree with its add about which lock it needs — which is also what lets a transaction be freed on a thread other than the one that began it.

Note—**The shard count is bounded by the walk, not by the core count**

More shards spread the join and leave further, but the enumeration below holds **every** shard for its whole walk, so a serializable commit acquires that many locks at once. Past about sixty a thread holds more locks than tooling will model — ThreadSanitizer caps a thread there and aborts — and it is a lot of lock traffic for one commit regardless. The count is chosen against that bound rather than against the number of cores.

The two readers of the set treat the sharding differently, and deliberately:

- **The oldest live snapshot** takes one shard at a time. The answer can come out too low but never too high, and too low is the safe direction for a reclamation floor — it keeps a version some reader might still want. It cannot come out too high, because a transaction registering after its shard was read draws its snapshot from a monotonic clock and so is above the minimum already, and one that leaves only raises the true minimum above what was reported.
- **Serializable validation** holds every shard for the whole walk, so it decides against one instant of the live set rather than a smear across shards. A peer appearing between two shards could otherwise be missed by this validation and by its own. The shards are taken in index order, the only order anything takes them in, so the walks cannot deadlock against each other. Holding all of them is affordable precisely because this walk is rare — a serializable commit or a statistics call, never the ordinary write path, which touches one shard.

28.9 Timeouts

A transaction that is begun and never resolved is not merely idle. It holds its snapshot, and at snapshot and serializable isolation its write reservations, so the reclamation floor cannot rise past it – compaction may not drop the versions below that floor, and the value log may not reclaim their bytes. A leaked transaction therefore costs disk for as long as the database is open, and nothing else in the engine can decide it is safe to let go of.

A timeout bounds that. It is off by default (`txn_timeout_seconds` is 0), applies to every transaction when set, and a single transaction overrides it through `tidesdb_txn_set_timeout`.

Two properties are worth knowing:

- **Expiry is lazy.** No background thread aborts anything. The next operation on the transaction notices its deadline has passed, aborts it there, and returns `TDB_ERR_TXN_EXPIRED`. A transaction that is never touched again is never expired – it is resolved at close instead.
- **The clock is cached, not read per check.** A ticker publishes the current second, and a check is one relaxed load against it. That is what keeps a transaction with no timeout paying nothing, and it also means a deadline is accurate to about a second rather than exactly.

28.10 Two-phase commit

Ordinary commit decides and applies in one step. Two-phase commit splits the decision from the application so an external coordinator can gather votes from several participants first.

Phase one — prepare. Runs the same conflict checks as commit and durably logs the write batch under a caller-supplied transaction id, but leaves the writes invisible and unapplied. The transaction holds its snapshot and its reservations until it is resolved — so an undecided prepared transaction applies backpressure to anything contending for its keys, and holds the `min_snapshot_seq` floor down. Deciding promptly is not optional in a busy system.

A read-only transaction prepares with nothing durable and needs no phase two.

A prepared batch that separated any of its values is also the only thing naming them. Its entries are in no memtable and no sstable until phase two decides it, so the value log (ch. 22) counts a staged batch's references alongside the installed tables and holds a floor over their segments — the same reason its write-ahead log generation is pinned, applied to the other store.

Phase two — commit or roll back. Durably logs the decision, then applies the batch and makes it visible.

The critical detail is the sequence. Phase two draws a **fresh** sequence at decision time and carries the batch inside the commit record, rather than applying at the sequence the prepare held.

CAUTION—**Why phase two must re-sequence**

Between a prepare and its decision, other transactions commit. If the prepared batch applied at its original — now older — sequence, a key written in that window would shadow it, even though the prepared transaction decided later. Re-sequencing at decision time makes replay and live application agree: the batch lands where it was decided, not where it voted.

Re-sequencing decides what happens to the reservations. The prepare claimed a slot for every key it wrote, each naming the sequence it drew then — and that sequence is superseded, so it is never marked committed. Phase two therefore **hands each slot over** to the sequence that committed, once the batch is durable and applied.

A slot left naming the superseded sequence would read, to every later writer of that key, as a committer still in flight: they would lose a conflict to a transaction that had already finished, and go on losing it until the ring aged the sequence out — permanently on a quiet database, and not at all on a busy one.

The hold is renamed rather than dropped and retaken. Releasing it would leave the slot empty, and a writer whose snapshot predates the commit would then claim it freely and overwrite the batch with no conflict raised — the precise failure the reservation exists to prevent. Renaming keeps the hold unbroken and leaves the slot naming a sequence that later comparisons can be made against.

The hand-over waits for the batch to be durable because a failed append leaves the transaction prepared for a retry that draws a different sequence, and the slots must still name the prepare's for that retry to find them.

A transient I/O failure in phase two leaves the transaction **prepared**, not aborted, so the coordinator can retry the same decision. Anything else would let a participant unilaterally abandon a transaction the coordinator had already committed elsewhere.

28.10.1 What an undecided prepare keeps alive

A prepared batch never enters a memtable, so it is durable only in the log its PREPARE record was written to. That generation's log therefore cannot be unlinked when it flushes, the way an ordinary generation's is once its data reaches L1 — the flush moved no part of that batch anywhere.

It is that one generation, and no other. The pin is keyed on the generation the record lives in, not on whether some prepare somewhere is undecided. The difference is not academic: an engine driven by a coordinator that always has a transaction in flight — which is what MariaDB's internal two-phase commit between the engine and the binary log looks like under concurrent load — would otherwise keep *every* log it ever wrote, since the count is never zero at the moment a flush asks. The store then grows without bound in a way that looks like a compaction failure and is not one.

Once phase two decides, the prepared generation is free immediately, and its log goes. That follows from the commit record carrying the whole write set: replay applies it inline, so nothing needs the original PREPARE any more. A rollback leaves nothing to undo, so the same holds.

The pin is taken by recovery itself, not by whoever adopts the transaction. A database that reopens with a batch in doubt and simply carries on writing — never asking for its in-doubt list — must still keep that log, because it holds the only copy of a batch someone may yet decide.

28.11 Invariants

Invariant	Why
A sequence is never reused, across restarts included	Reuse would make two different writes indistinguishable
A drawn but uncommitted sequence is invisible	Otherwise a reader sees half a batch
Sequences past the ring's edge count as committed	They cannot still be in flight; this bounds the ring
A write is validated against the version actually read	Validating against the snapshot alone misses read-modify-write races

Invariant	Why
The reservation slot carries a key fingerprint	Distinguishes a real conflict from a hash collision without reading others' data
Compaction may not drop above `min_snapshot_seq`	A live snapshot must still see what it could see
A flush retains against that floor too, not only a merge	A memtable holds the whole chain, so a flush that took the newest version alone would drop what the floor was protecting and a frozen reader would find a live key absent
A named snapshot is registered, not just remembered	The floor is the minimum the registry holds; a sequence kept outside it protects nothing, and reading there answers from whatever a merge happened to leave
Phase two draws a fresh sequence	The batch must land where decided, not where it voted
A decided prepare's reservations move to the sequence that committed	The sequence the prepare reserved with is superseded and never marked committed, so a slot left naming it reads as a committer still in flight and refuses every later writer of that key
A failed phase two leaves the transaction prepared	A participant must not abandon a decision unilaterally
An undecided prepare pins its own log generation, and only that one	Its PREPARE record is the only copy of the batch. Pinning on a database-wide count instead keeps every log ever written whenever a coordinator always has one in flight
A decided prepare frees its generation at once	The commit record carries the write set and replay applies it inline, so nothing needs the original PREPARE
Recovery takes the pin, not the caller who adopts the batch	A database that reopens and never asks for its in-doubt list must still keep the only copy of one

Chapter 29

Iterators and the Merge

29.1 The problem

A key's versions are scattered: some in the active memtable, some in sealed ones, some across several sstables at different levels. A scan has to present them as **one ordered stream of the newest visible version per key**, in either direction, without materialising anything.

This is the same problem compaction solves. A merge reads several sorted sources and resolves each key to what survives — which is a scan whose output happens to be written to a file.

So there is one implementation. The merge is generic over a small source interface, and both user iterators and compaction compose it. What they share is the hard part — keeping many sources positioned, picking the next key across them, and reversing direction correctly — so the ordering cannot diverge between a read and a merge.

What they do not share is what happens to the versions once ordered. A read resolves them against its snapshot; a compaction takes the raw stream and applies its own retention. That split is the subject of the next section.

29.2 The source interface

A source is a small bidirectional cursor over a sorted, versioned key stream:

```
first / last          position at either end
next / prev           step
seek / seek_for_prev  position at the first key >= / last key <= a target
valid                 is it positioned
get                   the entry at the position
```

Entries are byte-ordered by key, and several versions of one key appear adjacent. A memtable adapter and an sstable adapter implement this; so do stub streams in the tests, which is why the merge is testable with neither a memtable nor an sstable in existence.

The contract that matters: **the key and value a `get` returns are valid only until the next positioning call on that source.** The merge copies what it needs before moving on. A caller that held a returned pointer across a step would read freed or reused memory.

29.3 Folding

The merge keeps every source positioned and repeatedly picks the smallest key among them (largest, going backwards). All sources sitting on that key contribute versions, and the merge resolves them to the **newest version at or below the snapshot**, discarding the rest.

Then the outcome depends on what that newest version is:

- A **live value** is emitted.
- An **expired** entry is treated as absent.
- A **tombstone** hides the key — *unless* the iterator was built to emit tombstones.
- A **covering range delete** hides the key too, when it is newer than the version the merge resolved. That decision cannot be made inside one source: an interval sitting in a memtable has to hide a key that arrived from an sstable, so it is applied to the merged stream rather than to any source feeding it. Every positioning call steps past what it covers, so a caller that only asks whether the iterator is valid never sees a deleted key.

All of that describes a **read** scan, which is what snapshot resolution is for. A compaction does not want any of it, and does not merely flip the tombstone switch — it asks for a **raw** merge instead, which yields every version of every key, key ascending and then sequence descending, with no snapshot applied and nothing hidden.

The reason is that a compaction is not answering a question at one point in time. It decides which versions to keep against the oldest snapshot any live reader holds, and it must see the whole version chain to do it. It must also write tombstones into its output, because a tombstone is what shadows older versions in deeper levels the merge did not include — a compaction that dropped one would resurrect the deleted key on disk. So retention and tombstone collection are the compaction's own, applied to the raw stream; the merge supplies the ordering and nothing more.

A raw scan runs in one direction and **refuses to turn round**. The re-seek a resolving flip uses resumes one step past the current key, stepping over that key's remaining versions — which is what a read wants and exactly what a raw scan exists to see. Turning without one is no better: the sources sit one step past the version just emitted, so the first step back re-emits it. Neither answer is right, so `merge_iter_prev` on a forward raw scan returns `TDB_ERR_INVALID_ARGS` rather than a silently repeated version. A raw scan that runs backward from the start never turns and is fine.

Range deletes are the exception to that division of labour: they never enter the merge stream at all, in either mode. They live in the manifest, so a compaction asks the family whether an interval covers each key it is about to emit rather than carrying tombstones through its output.

29.4 Which sources the merge is built from

Every source the merge holds is a source it descends on each seek and compares on each advance, so the cost of a scan is set by how many it has, not by how many keys it returns. An unbounded iterator takes every sstable in the family — correct, and right for a full scan.

A scan that knows its range takes only the sstables whose own key range meets it. Each sstable records its minimum and maximum key, so a level can be asked which of its files overlap a range and the rest are never opened at all. A narrow scan of a family holding hundreds of sstables then costs a handful of descents instead of hundreds, and concurrent narrow scans stop competing for the whole store at once. Measured on a family of 32 disjoint sstables, a four-key scan went from 128 block cache lookups to 4.

Caution — **A short scan is dominated by what it opens, not by what it reads**

Building the merge opens a cursor into every source it kept, and each sstable cursor descends that tree from its root. So the setup is proportional to how many runs overlap the range, while the payoff is proportional to the rows returned — and a lookup that returns a handful of rows pays the whole setup for them. That ratio, not the per-row cost, is what makes a secondary-index lookup expensive on a family with many overlapping runs, and it is why pruning matters more for short scans than for long ones.

It is also why L1 is the tier that hurts. Its files may overlap and, when written from a memtable holding interleaved writes, each one spans the whole key space — so the range check cannot exclude any of them and every scan opens every L1 file whatever range it asked for.

Measured, on a store under live write load, a short index lookup spends roughly **a third of its cycles before it reads an entry at all**: the range prune that picks the sources, the refcount taken on each sstable it kept, and the pin on the active memtable. The prune itself is not a scan of the level — deeper levels are binary-searched — the cost is that it is paid once per level per lookup, along with the epoch enter and exit that make the layout safe to read.

The bound is a **contract, not a fence**: the iterator's results are defined only inside the range it was given, because the sources that could answer outside it were never opened. A caller that seeks past its own bound is asking a question the iterator was not built to answer.

29.5 Direction changes

This is the hard case, and the one most such implementations get wrong.

While iterating forward, a source that runs out is exhausted and drops out of consideration. Reversing does not simply flip the comparison: an exhausted source may hold entries *behind* the current position, and it is no longer positioned anywhere useful. Sources that were never exhausted are positioned one step past what the merge last emitted, which is the wrong place to step backwards from.

So a direction change **re-seeks every source around the current key** rather than trying to reverse in place. It costs a seek per source, and it is the reason the reference warns that flipping repeatedly is not free (ch. 13).

The property being bought is that a flip never drops an entry and never repeats one — which is worth a seek, since both failure modes are silent.

29.6 Stability

An iterator reads at a fixed snapshot, so its view does not change while it runs even as other threads commit, flush, and compact.

Which snapshot depends on the isolation level, and it is the same one a point read in the same transaction resolves against — `txn_read_snapshot`, not the begin sequence unconditionally. Repeatable-read, snapshot and serializable transactions carry a snapshot drawn at begin and the iterator reads at that, so every scan in the transaction sees one instant. A read-committed transaction holds no such snapshot, so the iterator draws the current sequence when it is created: it sees everything committed before it started, and successive scans in one transaction can legitimately differ. Read-uncommitted reads at the maximum sequence, which is what makes another transaction's uncommitted versions visible to it.

Reaching for the begin sequence directly would be wrong rather than merely coarse: a read-committed transaction has none, and filtering a scan against it would hide every live row.

Over that committed view the scan folds the transaction's **own buffered writes**: its uncommitted puts appear, and its deletes hide the rows beneath them, exactly as point reads resolve them through the write set.

Making that true requires holding things alive. An iterator's sources pin what they read: memtables by epoch or reference count, sstables by the reference their level-set layout holds. A compaction may replace a layout while an iterator is inside the old one; the tables it is using are freed only when it lets go.

The cost is that a long-lived iterator holds back reclamation — sstables it pins cannot be deleted, and the transaction behind it holds `min_snapshot_seq` down. A scan left open across a long operation is a real source of space that will not reclaim.

A transaction timeout bounds the second half of that but not the first. Expiry is lazy, so it resolves the transaction at its next operation and releases the snapshot; the iterator's own pins last until it is freed, whatever the transaction behind it did.

29.7 Where the value comes from

The merge yields keys and, for entries whose value is inline, values. A value above the family's threshold lives in the value log, and it is dereferenced **only when asked for**.

That is why a key-only scan is dramatically cheaper than a key-and-value scan over separated values, and why `tidesdb_iter_key` (ch. 13) and `tidesdb_iter_value` (ch. 13) are separate calls rather than one that always returns both.

29.8 Invariants

Invariant	Why
One merge serves reads and compaction	The ordering and direction handling must not diverge between them
A raw scan runs in one direction only	`test_merge_raw_refuses_a_direction_flip` — the resolving flip's re-seek steps over the versions raw exists to emit, and turning without one re-emits the version just delivered; a raw flip is refused rather than answered wrongly
A source's returned key/value dies at the next positioning call	The merge copies before stepping; callers must not hold it
A read resolves against its snapshot; a compaction reads raw	A compaction decides retention across the whole version chain, which a resolved stream has already discarded
A compaction writes tombstones to its output	They shadow older versions in levels the merge did not include; dropping one resurrects the deleted key
A direction change re-seeks every source	Exhausted sources hold entries behind the position
Sources stay pinned for the iterator's life	A compaction may replace a layout mid-scan

Invariant	Why
A bounded iterator answers only inside its bounds	Sources outside the range were never opened, so a key beyond it may have no source that holds it
The snapshot is fixed for the iterator's life	Stability is the guarantee an iterator makes; which snapshot it is comes from the isolation level, but it does not move once chosen

Recovery

30.1 The problem

A database can stop at any instant: mid-append, mid-flush, mid-compaction, mid-manifest-commit. On the next open the on-disk state must be resolved into exactly one answer — **everything acknowledged is present, and nothing else is**.

Recovery is not a special mode. It runs on every open, and a clean shutdown is just the case where there is little to do. That is deliberate: a recovery path exercised only after crashes is a recovery path that does not work.

30.2 Order of operations

```
1. open the manifest         -> rebuild the family registry and level sets
2. open the value log
3. sweep unnamed key logs    -> reclaim what an interrupted build left behind
4. scan NNNNNNN.log          -> the surviving write-ahead log generations
5. replay each generation    -> one sealed memtable per generation, in order
6. install a fresh active memtable above them
7. reseed the sequence clock
8. adopt in-doubt prepared transactions
9. queue the recovered generations for flush
```

The order is forced by dependency. Families must exist before log records naming them can be applied. The clock must be reseeded before anything new is written, or a fresh commit could reuse a sequence that durable data already holds.

The sweep sits where it does for a reason at each end. It has to follow the catalogue, since the catalogue is what says which files are reachable. It has to come before the workers start, because once a flush or compaction is running there is a window in which a file exists and the manifest does not name it yet — and a sweep cannot tell that from an orphan.

30.3 Rebuilding the catalogue

The manifest is replayed to reconstruct the family registry and each family's level set. What it names, exists; what it does not name, does not — so orphaned files from an interrupted flush or compaction are never half-adopted. Having established that, recovery then deletes them: a key log outside the catalogue can never be reached again, so leaving it costs disk and buys nothing. The one exception is a self-healed manifest, where the catalogue was derived from the files and sweeping against it would destroy what the rebuild could not adopt.

A manifest that will not read back does not fail the open. The discarded catalogue is rebuilt from the sstables themselves: the database directory is scanned and every `.klog` whose footer reads back is re-registered at L1, its owning family taken from the family id in its filename. The footer is self-describing enough to adopt a file, but it records neither the family's name nor the level the file sat at – so a rebuilt family is named `cf_` followed by its zero-padded id, carries the default configuration, and its tables land in the one tier that permits overlap. See Manifest (ch. 23) for the two cases and the limits of the rebuild.

The rebuild adopts at most 4096 families, a guard against a corrupt directory rather than a limit a real database approaches. It says how many families and sstables it took at warn either way, and says separately, at error, when it stopped at that bound — a count on its own reads as the whole directory, and the families past the bound are left uncatalogued with their files still on disk.

30.4 Replaying the logs

Each surviving `NNNNNNN.log` is replayed in generation order, and each becomes a sealed memtable in the queue with a fresh active memtable installed above them. The recovered database therefore has

exactly the shape a running one has — no special post-recovery state, no second code path for reads. The recovered generations are then flushed through the ordinary flush machinery.

Replay reads framed blocks forward and stops at the first one it cannot read. A torn final record — a crash mid-append — ends replay there, which is correct: that record was never acknowledged.

Caution—**A hole is not a skipped record**

Because replay stops at the first unreadable block, a gap in the middle of a log discards **everything after it**, silently. Nothing in recovery can detect that; the guarantee has to come from the write side, where only the contiguous completed run is ever written. See Block manager (ch. 20).

Records carry their own sequence, so replay applies each at the sequence it committed with rather than at its position in the file. File order and sequence order need not agree, and correctness comes from the sequence.

Caution—**An entry a durable later write already retired is not applied**

A log can outlive the flush that installed its data. That happens on purpose when an undecided prepare lives in it, since a prepared batch exists nowhere else, and by accident when a crash lands between the install commit and the unlink.

Applying such a log again would be wrong, not merely wasteful. Reads take the **first** source that answers and consult the memtables before any sstable, which is sound only because a memtable holds writes newer than anything on disk. An entry restored from an old log breaks that: a put would sit in front of the tombstone that retired it, and a deleted key would come back.

So replay drops an entry when the sstables already hold a **strictly newer version of that same key**. Dropping it cannot change any answer, since the newer version would have won either way. The question is asked of the sstables directly rather than through the ordinary read stack, which at this point is reading the very memtables being rebuilt. Any answer short of a definite yes keeps the entry: applying one that was already superseded costs a redundant version, dropping one that was not is data loss.

A sequence gate keeps this free in the ordinary case. An entry above the highest sequence any sstable holds cannot be on disk, so it never reaches the probe – and after a clean shutdown the only surviving log is the one the active memtable was never flushed from, whose sequences are all above it.

Not every durable batch is applied. A batch that reached the log but failed to enter the memtable is followed by a `TDB_WAL_KIND_ABORT_SEQ` record naming its sequence, because a write batch's presence in the log is otherwise taken as its commitment — the transaction would come back whole after its caller was told it failed. Replay gathers those sequences before applying anything, so a batch can be skipped by a record that appears after it, and applies the rest.

30.5 Reseeding the clock

The counter must resume above **everything durable**, and no single source knows what that is. Four are consulted, and the highest wins:

Source	Why it can be the highest
WAL replay high-water	The most recent commits, normally
Manifest sequence	Records catalogue progress independently
Recovered sstable maximum	A flushed generation whose log was already unlinked leaves its sstable as the only record of those sequences
Prepared transaction maximum	An in-doubt transaction reserved a sequence nothing applied

The third is the one that is easy to miss. Once a flush retires a memtable and unlinks its log, those sequences exist only inside the resulting sstable's footer. Reseeding from the logs alone would reissue them.

The fourth matters for a different reason: a prepared transaction's sequence was drawn but not applied, so nothing on the data path records it. Skipping it would hand a new writer the same sequence the coordinator may yet commit at.

30.6 In-doubt transactions

A `PREPARE` record with no matching `COMMIT` or `ROLLBACK` after it is **staged**, not applied. Its batch is held, its sequence is accounted for in the reseed, and it surfaces through `tidesdb_recover_prepared` (ch. 12) as a live handle in the prepared state.

A prepare whose decision *was* logged is settled during open and never appears — recovery applies the decision as part of replay rather than leaving it for the caller.

The staging map is carried across every generation, because a prepare and its decision can land in different logs. A transaction prepared before a rotation and committed after it must be resolved, not reported as in doubt.

30.7 Self-healing a torn flush

A crash during a flush can leave a partially written sstable. Its key log is a block-manager file, so the same last-block validation that repairs a torn log applies: the file is truncated back to its last valid block.

The reason this is safe is the ordering established elsewhere — the manifest names a table only after that table is durable. A partially written sstable is therefore not in the catalogue, and whatever is recovered from it is discarded along with the rest of the orphan. The data it was carrying is still in the write-ahead log, which is why the log is unlinked only after its data reaches L1.

30.8 What recovery guarantees

Every acknowledged commit is present. It was in the log before it was acknowledged, and the log is replayed.

Nothing unacknowledged is present. A torn record ends replay; an orphaned file is not in the catalogue.

No sequence is reissued. The clock resumes above the highest of four durable high-water marks.

No decided transaction is left in doubt, and no undecided one is decided for you. Decisions in the log are applied; prepares without them are handed back.

The crash fuzzer tests exactly this: it commits under a sync barrier, kills the process, reopens, and verifies that the recovered state is an **exact prefix** of what was acknowledged — not a superset, not a subset.

30.9 Invariants

Invariant	Why
Recovery runs on every open, not only after a crash	An exercised-only-on-crash path does not work
Families are rebuilt before log records are applied	A record names a family that must already exist
The clock is reseeded before any new write	Otherwise a fresh commit reuses a durable sequence
The reseed consults sstables, not just logs	A flushed generation's log may already be unlinked
The reseed consults prepared transactions	Their sequences were drawn but never applied
Replay applies at the record's own sequence	File order and commit order need not agree
The staging map spans generations	A prepare and its decision can land in different logs
An entry the sstables already superseded is not applied	Reads take the first source that answers and memtables come before sstables, so an entry restored from a log that outlived its flush would shadow the write that retired it
A log is unlinked only after its data reaches L1	Until then it is the only durable copy

Invariant	Why
The orphan sweep runs after the catalogue and before the workers	The catalogue says what is reachable; a running worker legitimately holds a file the manifest has not named yet
A self-healed manifest is never swept against	Its catalogue came from the files, so the sweep would delete exactly what the rebuild could not adopt

Invariants

The preceding chapters each end with the rules their subsystem depends on. This chapter collects them in one place, and adds the column that matters most: **what would catch a violation.**

An invariant is only as good as the check that enforces it. A rule stated in a comment and guarded by nothing is not an invariant — it is a hope, and it will be broken by the next person who has a good reason to restructure the code. So the third column here is not decoration. It is the audit, and where it says *unguarded*, that is a real gap and a place to add a test.

31.1 How to read this

Each entry names the rule, links to the chapter that motivates it, and states what would fail if the rule were broken. The enforcement column distinguishes three cases:

- **A named test or fuzzer** — a violation makes something go red.
- **Indirect** — no test targets the rule, but a violation would likely surface through a broader test that happens to depend on it. Better than nothing, and worse than it sounds: indirect failures point at the symptom, not the rule.
- **Unguarded** — nothing would catch it. These are the entries to act on.

31.2 Durability and ordering

Invariant	From	Enforcement
The write-ahead log must contain no gaps — only the contiguous completed run is written	Block manager (ch. 20)	`fuzz_crash` (exact-prefix oracle); `block_manager` tests cover the ring, not the gap property directly
A range tombstone is retired only once every table in the family records having applied it	Compaction (ch. 24)	`test_engine_range_tombstone_retires_only_when_every_table_applied_it`
A read that misses reports absence only if the level shape stood still while it looked	Life of a read (ch. 18)	`test_engine_read_survives_a_compaction_moving_levels`
Data is durable before the manifest names it	Manifest (ch. 23)	`fuzz_crash`; `test_crash_recovery_torn_flush`, and `test_crash_recovery_torn_wal_append` for the writes a commit itself issues

Invariant	From	Enforcement
A write-ahead log is unlinked only after its data reaches L1	Memtable and WAL (ch. 19)	`test_engine_write_recovery`, `test_engine_prepared_survives_flush`
The batch reaches the log before the memtable	Life of a write (ch. 17)	`fuzz_crash`
A separated value reaches the value log before the record naming it	Life of a write (ch. 17)	`fuzz_crash` — the other order would let a crash leave a record pointing at bytes that are not there, where this one only leaves reclaimable garbage
Every holder of a value log reference is either counted or covered by a floor	Value log (ch. 22)	`test_engine_prepared_batch_keeps_its_separated_values`, `test_engine_separated_value_survives_reopen_and_reclaim`, and the pinned `fuzz_standalone` corpus. There are three holders besides the installed tables — a memtable, an in-flight build, and an undecided prepared batch — and each was found by losing data
Manifest rollover is write-temp-then-rename	Manifest (ch. 23)	`manifest` tests
`ftruncate` is synced explicitly — `O_DSYNC` does not cover it	Block manager (ch. 20)	**Unguarded** — no test kills a process between a truncation and its sync
The value log is durable via `O_DSYNC`, not via a flush-time barrier	Value log (ch. 22)	**Unguarded** — a change to that open would not fail any test, and the comment warning against it is the only guard
Closing a block manager drains its staging ring before closing the descriptor	Memtable and WAL (ch. 19)	`test_l0_wal_ack_on_stage_survives_close` — under `TDB_SYNC_NONE` nothing else makes a staged record durable, so breaking this loses data on a *clean* shutdown, which no crash test would catch
Only `TDB_SYNC_NONE` acknowledges a commit before its record leaves the process	Durability (ch. 5)	`fuzz_crash` runs at `TDB_SYNC_FULL`, so its exact-durable-prefix oracle guards the modes that promise this and deliberately does not exercise the one that does not

31.3 Visibility and ordering of state

Invariant	From	Enforcement
A flush retains a key's version chain against the reclamation floor	Memtable and WAL (ch. 19)	`test_engine_frozen_snapshot_survives_a_flush` — a memtable holds the whole chain while a flush writes one sstable, so a flush taking only the newest version dropped everything under it whatever the floor said. A repeatable-read reader below an overwrite then read a live key as absent, which the default isolation level never sees because it re-reads
A read at a bare sequence refuses below the oldest readable point	Transactions and MVCC (ch. 28)	`test_engine_read_at_a_collected_sequence_is_refused`, `test_engine_oldest_readable_seq_resets_on_open` — below it a merge has kept one version per key, so answering would report a key that existed as absent. The watermark rises where a floor is taken rather than where the collection ends, so a merge already running is visible to the check
A named snapshot holds the floor by being registered	Transactions and MVCC (ch. 28)	`test_engine_snapshot_holds_versions_a_compaction_would_have_collected`, `test_engine_snapshot_holds_and_releases_the_reclamation_floor` — a sequence remembered outside the registry protects nothing, and a read there answers from what a merge happened to leave rather than from what was true
A directory is held by one handle at a time	Database (ch. 10)	`test_engine_second_open_is_refused` — an exclusive lock file plus a recorded owner pid, since `fcntl` locks are per-process and would otherwise let one caller re-lock a directory it already holds. A second open is refused with `TDB_ERR_LOCKED` rather than being allowed to share the store
A rotation takes the prepared log's generation under the same claim the preparer wrote it under	Memtable and WAL (ch. 19)	**Unguarded** — taking the spare empties the slot, which lets the next preparer overwrite the generation field before the taker has read it. The memtable would then be named for a log it does not hold, and the flush that asks whether that generation is pinned would ask about the wrong file. Found by ThreadSanitizer under `fuzz_conc`, not by any test asserting it
The log a rotation installs is opened before the rotation, off the lock	Memtable and WAL (ch. 19)	**Unguarded** — nothing asserts a rotation does no file creation, and opening one inline costs 49 ms with every committer stopped; the engine logs a slow prepare separately from a slow rotation so the two stay distinguishable
The rotation lock is never held across a wait or a device barrier	Memtable and WAL (ch. 19)	**Unguarded** — no test measures lock hold time, and the two violations found were caught only by capturing stalled commits' stacks under load. A reviewer adding a wait under it would see nothing fail
Rotation enqueues the sealed memtable before swapping the active slot	Memtable and WAL (ch. 19)	`fuzz_conc`; `test_l0_rotate_and_read` covers rotation but not the ordering window

Invariant	From	Enforcement
Shallower is newer — flush installs are strictly ordered	Life of a read (ch. 18)	Indirect: `flush`, `test_engine_overwrite_across_recovery`
A drawn but uncommitted sequence is invisible	Transactions and MVCC (ch. 28)	`mvcc` tests
A column family configuration is published whole, never written through	Column family (ch. 11)	`column_family` tests; TSan across the suite — a reader takes an immutable snapshot, so no flush or plan ever sees half of one configuration and half of another
The memtable's reported size counts every resident version, not every key	Memtable and WAL (ch. 19)	`test_skip_list_size_counts_every_version`
The rotation threshold reads memory occupied, not key and value bytes	Memtable and WAL (ch. 19)	`test_skip_list_memory_bytes_counts_structural_overhead` — a node, its pointer arrays and a version struct are about 100 bytes an entry, so counting only the data lets a memtable of small entries hold several times the memory its configuration asks for
A version costs its structure plus its value, so a tombstone is not free	Memtable and WAL (ch. 19)	`test_skip_list_size_counts_tombstone_versions`
`BUSY` short-circuits the source stack rather than falling through	Life of a read (ch. 18)	`source` tests
A level's candidates come from one layout load, not a count then a collect	Life of a read (ch. 18)	**Indirect** — `fuzz_conc` would surface the stale answer. Two loads let a flush install between them, leaving the collected set a subset of what the count promised, so the highest sequence among the candidates was not the highest that existed. The collect reports the true overlap count even when more overlap than fit, which is what lets one load do both
The deep-level candidate window agrees with a scan of the level	Life of a read (ch. 18)	`test_level_set_deep_level_window_matches_a_full_scan` — L2+ runs are non-overlapping and sorted by min_key, so two binary searches bound the files meeting a range. A window starting one file late drops a file that holds the key and the lookup reports it absent; the test checks every boundary and gap differentially against a brute-force overlap

Invariant	From	Enforcement
A tombstone or expired entry is a definitive absence, not a fall-through	Life of a read (ch. 18)	`test_l0_tombstone`, `ttl` tests
Phase two draws a fresh sequence	Transactions and MVCC (ch. 28)	`test_engine_recovered_phase_two_durable_and_ordered`, `test_engine_settled_prepare_lands_above_writes_it_was_in_doubt_through`
An sstable's footer sequence bounds every version it holds	Transactions and MVCC (ch. 28)	**Indirect** — `fuzz_standalone` and `fuzz_conc` would surface a missed conflict, but nothing asserts the footer bound itself. A footer understating its contents would make the commit conflict scan skip an sstable that does hold a newer version, and the write it should have rejected would be accepted

31.4 Reclamation and lifetime

Invariant	From	Enforcement
A memtable is freed only after readers drain	Memtable and WAL (ch. 19)	`test_l0_retire_is_observable_to_a_bracketed_read`; TSan across the suite
A lookup's level-0 walk reports "not yet known", never "absent"	Memtable and WAL (ch. 19)	**Unguarded** – the skip list's descent settles on a predecessor and then walks level 0 to the key, because an insert can splice a smaller key in between. Stopping that walk on a key *below* the target reports a present key as missing, and so does running its budget out and calling the result absent; the walk therefore continues while it is below, is bounded by the list's own entry count plus slack rather than by a flat constant, and hands back a retry that re-descends rather than an absence. `test_skip_list_lookup_never_misses_a_present_key_under_splicing` guards the coarse form of this, but it does **not** reach the narrow window – reinstating a single-load hop still passes it, because landing an insert between a descent finishing and its next load needs an interleaving a test cannot force. `fuzz_conc` is the only other cover
A reclaim never retires the segment taking appends	Value log (ch. 22)	`test_retire_refuses_the_segment_taking_appends` – calls retire on the active slot directly, which is the same question the race asks, and asserts the store keeps writing and reading afterwards. A reclaim reads the active slot per segment rather than once per pass, and the retire re-reads it after claiming the eviction window; deciding against one stale reading would let a roll mid-pass hand it the segment taking appends, whose file it would then unlink
No engine-internal error code reaches a public caller	Error codes (ch. 34)	**Unguarded** – `tdb_public_rc` in db.c is the single choke point that maps `TDB_ERR_BUSY` to `TDB_ERR_LOCKED`, and nothing fails if a new public function forwards an engine result without it; the code db.h does not define would reach a caller's switch and `tidesdb_strerror` would call it unknown
The plan memo is keyed on the shape and the reclamation floor together	Compaction (ch. 24)	`test_engine_scheduler_replans_when_the_gc_floor_moves` – opens a snapshot transaction and asserts the family is planned again with no write having landed

Invariant	From	Enforcement
A subdivided merge writes the same files as an undivided one	Compaction (ch. 24)	`test_compaction_subdivided_merge_matches_one_thread` – runs the same merge with and without permission to split, and requires the same output count and the same value for every key. the split is at the boundaries the sink already rolls on, so the files are the same files; if that stopped holding, a merge's result would depend on how many threads were free, which is not a thing a storage engine may do
A plan that produced jobs is never memoized	Compaction (ch. 24)	**Unguarded** – proving it needs a compaction job to fail while the layout stays put, which the suite has no way to force without fault injection. The failure is silent: the family simply stops being scheduled, and on an idle database that is permanent. Emptiness is read from the plan's job count and from nothing else – deriving it from whether some allocation succeeded would let a failed `malloc` memoize a family that did have work
An idle non-empty memtable is rotated on a timer	Memtable and WAL (ch. 19)	`test_engine_idle_flush_rotates_an_unwritten_memtable` – writes under the rotation threshold and asserts the data reaches L1 with no further write
Only the handle that installed the log sink closes it	Troubleshooting (ch. 9)	`test_engine_log_to_file_writes_into_the_database_directory` – opens a second database without `log_to_file` and asserts the first one's file stopped growing rather than being truncated or reopened
An undecided prepare pins its own log generation and no other	Transactions and MVCC (ch. 28)	`test_engine_undecided_prepare_pins_only_its_own_generation` – one prepare left in doubt across six flushed generations; keyed on a database-wide count instead it retained all seven
A batch recovered in doubt keeps its log across later flushes	Transactions and MVCC (ch. 28)	`test_engine_recovered_in_doubt_batch_survives_later_flushes` – reopens, writes and flushes without ever asking for the in-doubt list, then decides the batch
A snapshot's references are dropped on every path out, including the mismatch	Compaction (ch. 24)	**Unguarded** – a layout that *grew* past the caller's array references nothing, but one that **shrank** between sizing and filling is referenced in full and returns a count that does not match, so a caller treating the mismatch as a failure still owes those references. Forcing that race in a test is not currently possible. Every call site now releases across the whole zeroed array, which is the only form that cannot leak; the one that returned early instead leaked a reference per table on each attempt, pinning it for the life of the process
A family's reported size counts its key logs exactly once	Statistics (ch. 15)	`test_engine_cf_data_size_counts_key_logs_once` – asserts it equals both the sum of `level_sizes` and the real bytes on disk; adding the summed value length on top doubles it for any workload whose values stay inline
A borrowed root is never released by the descent that used it	Life of a read (ch. 18)	`test_get_borrows_a_root_held_across_reads` – reads a tree whose root is internal through a cache too small to hold it, so the root's frame is evicted while pinned and a descent that released the borrowed node would drop the sstable's own reference and free it under the next reader

Invariant	From	Enforcement
A log that will never be written to is unlinked, not just closed	Memtable and WAL (ch. 19)	`test_engine_preparing_a_spare_log_strands_no_file` – releases sixteen preparers together and asserts at most one log joined the directory; closing the loser's descriptor without unlinking its file stranded one log per losing committer per rotation, and each one is scanned, replayed and given a memtable on every later open
An entry lapses wherever it lives, not only in the memtable	Life of a read (ch. 18)	`test_ttl_expires_after_reaching_an_sstable` – flushes while the deadline is still ahead, so the entry reaches the sstable live and only lapses there. Expiry has to be evaluated on every source a version can be read from; confining it to the skip list and to the flush that converts a lapsed version to a tombstone would leave any lifetime longer than the flush interval never expiring at all
Every source ages an entry against the same published second	Life of a read (ch. 18)	`test_get_ages_entries_against_the_injected_clock` – publishes a clock, reads the entry live one second short of its deadline, then advances only that clock and asserts the same read now answers as a tombstone. The memtable's list and every sstable read one value a ticker publishes rather than calling `time(NULL)` per entry; two clocks a second apart would let a key read live from one source and lapsed from another, and the read would fall through to whichever answered
A tombstone is the only thing that means absent	Data model (ch. 3)	`test_engine_an_empty_value_reads_back_as_present` – stores a zero-length value and requires it present out of the memtable, out of an iterator, out of an sstable after a flush and out of the replayed log after a reopen, while a real delete still reads as not-found. the write path once refused it, and refused it only at apply, after the batch was durable – so the caller saw an I/O error from commit for what was a legitimate write. the model fuzzers now generate empty values about one time in sixteen
A lapsed entry is collected by a merge, not by the read that hides it	Compaction (ch. 24)	`test_ttl_expired_entry_is_collected_by_compaction` – asserts the family's live key total falls across a forced compaction, so the bytes go rather than the key merely reading as absent forever
A lapsed entry shadows an older version rather than reading as absent	Life of a read (ch. 18)	**Unguarded** – reading it as absent would let an older version beneath it surface, but the fall-through is not reachable through the public api: a flush triggers a compaction that merges the two versions into one run before any read can observe it, and both mutations of this rule leave the suite green. The sibling case it generalises is guarded at the unit level by `test_compaction_keeps_tombstone_with_sibling`
A scan costs its range, not its family	Iterators (ch. 29)	`test_cf_iter_narrow_range_scan_does_not_open_every_sstable` – 32 disjoint sstables, a four-key scan, and an assertion that fewer than one cache lookup per sstable was taken; unbounded it took 128 for those four keys, which is how concurrent range scans saturated the cores while each wanted almost nothing
Pruning sources never drops one a scan needs	Iterators (ch. 29)	`test_cf_iter_full_scan_spans_every_sstable` – an unbounded scan over the same 32 sstables still returns every key, so a selection rule that dropped a live source would fail rather than silently shorten the stream
A merge never targets the flush tier	Compaction (ch. 24)	`test_planner_merge_always_leaves_the_flush_tier` – a two-level family with its tier over the file trigger, asserting both the chosen target and the emitted job's target sit below L1; targeting L1 consolidates the tier in place and leaves it growing one run per flush

Invariant	From	Enforcement
The tree gains a level when its largest is full	Compaction (ch. 24)	`test_plan_grows_a_level_when_the_largest_is_full` – without it a family stops deepening, the dividing level collapses onto the flush tier, and the tier grows a run per flush with nowhere to drain to
A partitioned merge fans out only when no input spans a boundary	Compaction (ch. 24)	`test_plan_partitioned` covers the aligned case fanning out into disjoint jobs; `test_plan_partitioned_spanning_input` covers the fallback – sharing a spanning input across per-partition jobs lets only the first run, dropping a tombstone without the versions it shadows
Ingestion is paced against merge progress as well as flush progress	Memtable and WAL (ch. 19)	`test_tier_band_paces_against_merge_progress` – asserts the tier band admits below its slow mark, dwells further past it, blocks at the stall mark, and is inert when told to weigh no tier; without it a burst outruns compaction and leaves a tier every later read merges across
A plan's jobs run concurrently and the last one out tears the plan down	Compaction (ch. 24)	**Indirect** – ASan and TSan across the suite. The claim is released only when the final job finishes, which is what a family drop waits on, and a queue drain pays the same count through `engine_job_group_release`
A chunk is sized to the object it holds	Block cache (ch. 25)	**Indirect** – `ENGINE_NODE_ARENA_CHUNK_SIZE` is defined from `TDB_DEFAULT_CF_BTREE_BLOCK_SIZE` so the two cannot drift, but nothing fails if they do. they had already drifted sixteen-fold, and the only symptom was a cache holding a thirty-second of its budget in useful data – no test covers the ratio of a frame's chunk to the node inside it
The cache charges a frame what it reserves, not what it uses	Block cache (ch. 25)	`test_arena_reserved_counts_whole_chunks_not_bytes_used` – asserts one small object still reserves a whole chunk and that reserved never reads below allocated. a cached node is decoded into its own arena, so the memory a frame holds is the chunk it took; charging the bytes the node used let a 64 MiB cache sit resident at 908 MiB, and nothing reported the gap. worst on a small budget, which is why the capacity test – entries near the nominal size, nothing to round – kept passing
The block cache's frame count follows its byte budget	Block cache (ch. 25)	`test_cache_reaches_its_configured_capacity` – fills past the budget and asserts the cache settles within a few percent of it; a frame ceiling that binds first leaves it at a fraction, silently
Each class of file accounts its own device work	Statistics (ch. 15)	`test_engine_io_stats_attribute_device_work` – asserts a fresh database reports zero, that committing moves the log class and flushing moves the sstable class, and that no class reports fewer bytes than writes
Every point a writer can wait at reports count, total and longest	Statistics (ch. 15)	`test_engine_stall_stats_attribute_writer_waits` – asserts a fresh database reports zero, that the log wait counts after writes, and that no total is less than the longest wait inside it

Invariant	From	Enforcement
A finished sstable occupies its data, not its preallocated extent	Block manager (ch. 20)	`test_engine_finished_sstable_is_trimmed_to_its_data` – measures the klog while the database is open, since closing trims it either way and hides the defect
A key log the manifest does not name is swept at open	Recovery (ch. 30)	`test_engine_open_sweeps_orphaned_sstables` – plants an unnamed klog and a staging file, then asserts both are gone and every named klog survived
The orphan sweep is skipped after a self-heal	Recovery (ch. 30)	`test_engine_open_keeps_sstables_when_manifest_self_healed` – plants a klog the rebuild cannot adopt, so the guard is what keeps it; without the guard the sweep deletes it
A merged-away compaction input is unlinked when its last reference drops	Compaction (ch. 24)	`test_engine_compaction_unlinks_merged_inputs`, `test_engine_compaction_leaves_no_orphans_under_load` – both compare the klog files on disk against the live sstable count, which is the only way the leak is visible; nothing else fails when the files pile up
A flush waits for its install ticket before taking the registry read lock, never under it	Architecture (ch. 16)	`test_engine_create_completes_under_sustained_flush` – holding it across the wait both starves family create and deadlocks against the worker holding the earlier ticket
No reader takes the family registry lock again while holding it	Architecture (ch. 16)	**Unguarded** – nothing detects re-entry, and the writer-preferring attribute deadlocks a reader that does it
A flush never waits out a reader to free a memtable	Memtable and WAL (ch. 19)	`test_l0_reclaim_defers_rather_than_blocking` – it holds an immutable pinned across a retire and asserts the retire returns, which hangs outright if the wait is unbounded
A pinned memtable keeps its write-ahead log alive	Memtable and WAL (ch. 19)	`test_memtable_free_leaves_wal_open`
Cache frames are never freed; only payloads are reclaimed	Block cache (ch. 25)	`cache` tests; ASan across the suite

Invariant	From	Enforcement
A cache payload is reclaimed exactly once, by the last reference dropped	Block cache (ch. 25)	`cache` tests; ASan would catch a double free
A sstable is freed by whoever drops the last reference	SSTable (ch. 21)	`level_set`, `sstable` tests; ASan
Sources stay pinned for an iterator's life	Iterators (ch. 29)	`cf_iter`, `iter_api_tests`; TSan
Ingest is paced before the staging ring fills, not when it is full	Memtable and WAL (ch. 19)	`backpressure` tests cover the band, its graduation and its cap; that the append path actually consults it is **unguarded**, and a regression would show only as a latency tail
A copy freezes nothing; it claims what removes files and snapshots the rest	Memtable and WAL (ch. 19)	`fuzz_standalone` and `fuzz_conc` drive backup and clone against concurrent flush and compaction; nothing asserts the absence of a freeze, so a reviewer reintroducing one would see no failure
A caller that mutates the catalogue and cannot finish puts it back	Compaction (ch. 24), Memtable and WAL (ch. 19)	`test_manifest_an_abandoned_half_batch_lands_on_the_next_commit` — buffered records are not discarded when a caller gives up, they wait for the next commit from any path. Both installs name their outputs in a loop and so both must undo a partial one: a compaction that stopped partway would disown its inputs durably with the replacement never named, and the orphan sweep would unlink their files on the next open; a flush would leave the catalogue naming outputs no level set ever took. The clone and the recovery rebuild reach the same end differently — the clone's failure path drops the destination and the drop cascades those records away, and a failed rebuild aborts the open, whose manifest close discards the batch. The undo itself is **unguarded**: reaching it needs a refused allocation inside the install's own window, which `test_alloc_failure_through_a_compaction_keeps_every_commit` walks the path of but cannot land in
An interval reservation is given back once its batch is visible	Transactions and MVCC (ch. 28)	`test_engine_repeated_range_deletes_do_not_exhaust_the_reservations` — a key reservation is renamed rather than released, so an interval delete is the one claim that has to be handed back explicitly. the table has a fixed number of slots, so leaking one per commit ends with every later interval delete refused as a conflict no retry could clear; the test commits far more than there are slots
A copy measures the manifest and reads it inside one hold	Manifest (ch. 23)	`test_manifest_hold_keeps_the_log_still` — a commit past the rollover bound renames a fresh snapshot over the same path, so a length measured outside the hold can be applied to a different file, leaving a catalogue that names klogs the copy does not hold. the test asserts both halves: that the hold blocks every commit, and that those commits really do replace the file once it lifts

Invariant	From	Enforcement
A descriptor is reclaimed only at the idle reference count	fd manager (ch. 26)	`reaper`, `fdmanager` tests
Every labelled descriptor open pairs with a labelled close	fd manager (ch. 26)	`test_engine_wal_descriptor_accounting_balances` — the gate and the reaper both read the cross-label total, so a label that only decrements drives it negative and silently disables the budget while the reported per-label statistic still looks correct
A value-log segment is unlinked only after every reader inside it leaves	Value log (ch. 22)	`test_concurrent_reads_writes_and_reclaim`; TSan across the suite
A sealed value-log segment is never modified	Value log (ch. 22)	`test_reclaim_leaves_the_active_segment_alone`, `test_recover_appends_to_a_fresh_segment`
A value id names one block for its whole life	Value log (ch. 22)	`test_builder_records_vlog_segment_references` — nothing is copied within the store, so an id cannot exist in two segments and a crash leaves no duplicate to resolve
Value-log liveness comes from what the installed tables report, not the index	Value log (ch. 22)	`test_reclaim_drops_dead_values` — the index names every value whose segment has not been dropped, so counting it alone reports garbage as live and reclamation stops silently
A segment a builder may have written to is never reclaimed	Value log (ch. 22)	`test_reclaim_spares_what_a_builder_may_have_written` — values are written before the table naming them is installed, and dropping that segment in the window loses them
A value in a segment being emptied is rewritten, not carried	Value log (ch. 22)	`test_builder_respills_a_value_out_of_a_draining_segment` — carrying the reference leaves the output table holding the old segment, and it could never reach zero
A value is never split across segments	Value log (ch. 22)	`test_a_value_larger_than_a_segment_is_not_split` — the segment target is a roll threshold, so an oversized value overshoots it whole rather than being divided

Invariant	From	Enforcement
A segment with a reader inside it is never closed by the reaper	Value log (ch. 22)	`test_idle_segments_give_back_their_descriptors` and the concurrent reclaim test, which evicts while readers are in flight; TSan across the suite

31.5 Compaction correctness

Invariant	From	Enforcement
Versions are dropped only below `min_snapshot_seq`	Compaction (ch. 24)	`compaction_exec`; `fuzz_standalone` model comparison
A tombstone drops only when every sstable holding the key is in the merge	Compaction (ch. 24)	`compaction_exec`; `fuzz_standalone`
Tombstone safety is evaluated over the whole job, not a sub-range	Compaction (ch. 24)	`compaction_pipeline_tests`; this class of bug has escaped unit tests before and was found by `fuzz_standalone`
Outputs and input removals commit in one manifest batch	Compaction (ch. 24)	`compaction_pipeline_tests`, `manifest`; `test_crash_recovery_torn_compaction` tears each write a merge issues and requires every key back, which is what a half-committed batch would cost
The planner performs no I/O and reads no clock	Compaction (ch. 24)	`compaction_planner` — the tests pass it plain numbers, so an I/O dependency would not link
A read resolves against its snapshot; a compaction reads raw and retains for itself	Iterators (ch. 29)	`merge_iter`, `merge_sources` — both scan the same sources twice, once resolving and once raw, and require the tombstones to appear only in the second

31.6 Recovery

Invariant	From	Enforcement
The clock resumes above the highest of four durable high-water marks	Recovery (ch. 30)	`test_engine_cf_id_not_reused_after_drop_reopen` covers the family-id mark; the sstable and prepared marks are **indirect**
Replay applies at the record's own sequence, not its file position	Recovery (ch. 30)	`test_engine_overwrite_across_recovery`
The prepared-transaction staging map spans generations	Recovery (ch. 30)	`test_engine_prepared_survives_flush`
Families are rebuilt before log records naming them are applied	Recovery (ch. 30)	`test_engine_cf_recovery`
A damaged manifest is rebuilt from the sstables on disk	Manifest (ch. 23)	`manifest` tests cover discarding the log; `test_engine_rebuilds_catalogue_from_sstables` covers the readopt, and asserts the data reads back rather than only that the open succeeded
A torn sstable is truncated to its last valid block	Recovery (ch. 30)	`test_crash_recovery_torn_flush`

31.7 Structural

Invariant	From	Enforcement
Zero-length blocks are rejected at write	Block manager (ch. 20)	`block_manager` tests
The trailing size must match the leading one	Block manager (ch. 20)	`block_manager`, `fuzz_decode`
The footer records the geometry the file was written with	SSTable (ch. 21)	`sstable`, `fuzz_decode`
The footer's encoding pipeline is the one the nodes were encoded with	SSTable (ch. 21)	`fuzz_standalone` — its families draw real pipelines and it reconfigures them mid-run, which is what caught the builder reading the family's pipeline twice and straddling a reconfigure
Every value-log block carries the encoding chain it was written through	Value log (ch. 22)	`fuzz_standalone` — a value carried forward by compaction into an sstable recording a different pipeline decodes by its own chain or not at all
The filter directory holds full first keys — no false negatives	SSTable (ch. 21)	`pr_filter` tests
A snapshot carries `SEQ` and `CF_SEQ`	Manifest (ch. 23)	`test_engine_cf_id_not_reused_after_drop_reopen`
High-water marks are raised, never stored	Manifest (ch. 23)	`manifest` tests
No module below the engine takes a `tidesdb_t`	Architecture (ch. 16)	Structural: every subsystem test constructs its module without a database, so a violation would not compile

31.8 Process and shutdown

Invariant	From	Enforcement
Background processes stop in reverse registration order	Thread manager (ch. 27)	`threadmanager` tests
A pool never owns the queue it drains	Thread manager (ch. 27)	`bg_pool`, `queue`
Every stop signals, wakes, and joins	Thread manager (ch. 27)	`bg_pool`, `bg_ticker` — a missed wake hangs the test
Blocking on backpressure has a ceiling	Memtable and WAL (ch. 19)	`backpressure` tests

Invariant	From	Enforcement
The budget plus the untracked reserve fits the process limit	fd manager (ch. 26)	`test_fd_manager_budget_for_process` – the cut is pure arithmetic so every case is asked directly rather than through whatever ceiling the machine running the suite has, including the default pairing that made it necessary: a 1024 budget against the usual 1024 POSIX ceiling. Bounds one database; several in one process cut against the same ceiling independently and can still exceed it together, which is the operator's to size and the `EMFILE` retry's to absorb
Reader-gate rechecks and `EMFILE`/`ENFILE` retries are bounded, and to the same bound	fd manager (ch. 26)	`test_fd_manager_reader_gate_wait_is_bounded` — drives the gate to its bound with a budget nothing can free, and times it: a gate that refuses without waiting and one that retries past its bound each fail a different assertion. A `_Static_assert` covers the two bounds matching, since when they did not, reads failed on a shortage the open path was about to clear. What is still untested is the real `EMFILE` path, which needs the process ceiling genuinely exhausted

31.9 The gaps

Eighteen entries above are unguarded, incomplete, or only indirectly guarded — thirteen marked unguarded, four only **Indirect**, and one **Partly implemented**. The ones worth stating plainly rather than leaving in a table:

1. **The value log's `O_DSYNC` open.** Nothing fails if someone "optimises" it away. The result would be a durable key log referencing value bytes that never landed — visible only as a rare wrong-value read after a crash. Today the only guard is a comment.
2. **Truncation syncing.** No test kills a process between an `ftruncate` and its sync.
3. **The genuine `EMFILE` path.** The reader gate's bound is now tested directly, but by exhausting the engine's own budget rather than the process ceiling. Nothing drives the process out of descriptors, so the `EMFILE`/`ENFILE` retry in the open path is still only covered by sharing its bound with the gate.
4. **The sstable and prepared high-water marks in the clock reseed.** The family-id mark has a direct test; the other two are covered only by broader recovery tests that would fail for many reasons.
5. **The two rotation rules** — that the log is opened before the rotation and off the lock, and that the lock is never held across a wait or a device barrier. Neither has a test; a breach of either surfaces as stalled commits' stacks under load, which is evidence you have to go looking for rather than a failure that reports itself.
6. **That the append path consults the backpressure policy at all.** The policy's own band, graduation and cap are tested; the call site is not, and a regression there shows only as a latency tail.
7. **That a copy freezes nothing.** Backup and clone run against concurrent flush and compaction in the fuzzers, but nothing asserts the absence of a freeze, so reintroducing one would fail nothing.

The first is the most serious, because the failure is silent, arrives long after the change that caused it, and looks like data corruption rather than a durability bug.

NOTE—**This audit is a snapshot**

It reflects what the suite covered when this chapter was written. Adding an invariant to a subsystem chapter without adding its row here — and without adding the check that fills the third column — is how this chapter rots into the same hopeful comments it exists to replace.

CHAPTER 32

Testing and Tools

Building (ch. 6) covers running these to check a build. This chapter is about what they actually verify, and is written for someone changing the engine rather than someone installing it.

32.1 The test suite

Around fifty test binaries, one per module, built by default and run with `ctest`. They are unit tests in the strict sense: each constructs its module directly, with no database anywhere. That is possible because no module below the engine takes a `tidesdb_t` (ch. 16), and it is the main practical payoff of that rule.

Two suites are integration-level: `engine_tests` drives the public API end to end, and `crash_recovery_tests` covers torn-write repair. That one arms a write to tear at a chosen point, crashes a child the instant it lands, and reopens what was left — swept across every crash point in three phases: a flush, the writes a commit itself issues, and a merge. The child is the same binary run again rather than a fork, so it works where there is no `fork`.

Two are unusual. `doc_samples` compiles every C sample in this manual against the real public header, so a sample that drifts from the API it documents fails the build. It also checks that every backticked identifier in the prose exists somewhere in the sources, which is what catches a chapter still naming a function that has been renamed away.

NOTE—**It builds its corpus by tokenising the sources, escapes included**

A name written only inside a string literal is glued to whatever escape precedes it: a tab followed by a column heading is two characters plus the name, which a tokeniser reads as one word beginning with a *t*. So the escapes are blanked before the split. Without that, prose could not name a tab-separated column that the tool itself prints, and the failure reads as a missing identifier rather than as a quoting artefact.

`portability_create` and `portability_verify` are a pair. The first writes a database, the second reopens it and reads every key back. Locally they run back to back as a round trip; in CI the artifact produced on one architecture is verified on the others, which is what catches a format or endianness divergence. They are built as ordinary targets rather than embedded in the workflow, so a public API change breaks the build: a program that lives inside a CI file is seen by no compiler until the job runs it, and can stop compiling against the API while the jobs keep reporting success. They are also denied the internal include path, so they prove the public header alone is sufficient to use the database.

CAUTION—**Assert on the contract, not on success**

The administrative calls — manual compaction, manual flush, backup, and the column-family rename, drop and reconfigure paths — take an exclusive claim on a family, and answer `TDB_ERR_LOCKED` when a compaction already holds it. Reads and writes never see this; a transaction is unaffected by a compaction running underneath it.

A test that asserts `TDB_SUCCESS` on one of those calls is flaky by construction, and it will usually pass — the window is narrow until a sanitizer widens it. Retry the way a caller must, and pause between attempts: the claim is released by a background thread, so a tight retry loop can exhaust its attempts in the time that thread needs to finish.

32.2 The fuzzers

Four harnesses, none enabled by default:

```
cmake -S . -B build-fuzz -DCMAKE_BUILD_TYPE=RelWithDebInfo -DTIDESDB_BUILD_FUZZERS=ON
cmake --build build-fuzz -j
```

Each is **model-based**: the harness maintains an independent model of what the database should contain and compares the engine against it. That is what makes them able to find wrong answers rather than only crashes.

32.2.1 `fuzz_standalone` — the logical model

Generates random operation sequences against both the engine and a versioned in-memory model, then compares. Catches wrong values, lost writes, resurrected deletes, and visibility errors.

The model is version-aware — it tracks sequences and tombstones, because a model that compares only latest values reports failures constantly, all of them its own fault rather than the engine's. When a model disagrees with the engine, suspect the model first.

It also carries each entry's expiry, converted from the lifetime at the put exactly as the engine converts it, and hides a version that has outlived it. The lifetimes the harness generates are only ever *never* or a whole day, deliberately: expiry is decided by comparing against a second-granular published clock, and a lifetime that elapsed part-way through a run would leave the model and the engine on opposite sides of that boundary, making the oracle the thing that failed. What these lifetimes do catch is the direction that matters — an expiry the engine loses or brings forward turns into a key the model still expects and the database no longer has.

The harness now also generates **empty values**, roughly one in sixteen. An empty value is a value, and the engine must not confuse it with an absent key; this is the oracle that says so under crash and concurrency rather than only in a unit test.

Alongside the value comparison it carries a **structural** oracle that owes nothing to the model. A closed database owns no sstables and no level-set layouts, so both counts have to come back to what they stood at when it opened — and that is checked at **every** close the run performs, not once at the end. The granularity is the point: a balance checked once per run says only that something between the first operation and the last leaked, while one checked per close names the twenty-odd operations since the previous one.

The two counts are worth reading together when it fires. A leaked handle with the layouts balanced is a reference some reader took and never gave back; a leaked layout means the sstables it lists are pinned by a structure nothing will reclaim. They fail in the same way and have entirely different causes, and separating them is what stops an investigation committing to the wrong one.

Note — **Pin a case rather than hope the seed stream finds it again**

A failing run's input is a file, not a seed. Dump it with `TIDESDB_FUZZ_DUMP_LAST` at the iteration count that first fails, replay it alone with `./fuzz_standalone <file>` — seconds rather than minutes — and drop it into `fuzz/corpus/model/` once it is understood. Everything there is replayed by the `fuzz_standalone_corpus` test on every run, so a case found once stays covered whatever the generator does next.

Cutting the repro before instrumenting anything is the order that works. Reading a 200-iteration run's trace to find out which of thousands of operations mattered costs hours and usually settles nothing.

Caution — **The model holds committed state, not per-reader snapshots**

It keeps one latest-committed value per key and has no notion of *which* transaction is asking. A read issued through a transaction at repeatable-read or above is filtered by that transaction's snapshot, so the engine can legitimately hide a value the model holds — and the comparison then reports a lost write that never happened.

The harness avoids this by settling the open transaction before any point where the two views are compared against fresh commits — `fx_op_iter`, `fx_op_flush`, `fx_op_compact`, `fx_op_reopen` and `fx_op_commit_prepared` all call `fx_commit_txn` first. Committing a prepared transaction is the case that makes the omission visible, because it is the only operation that commits a *different* transaction while leaving the reader's own open; every other commit path ends it. A failure of the shape "db missed a key the model has", immediately after a `COMMIT_PREPARED` and with a long-lived transaction open, is this and not an engine fault.

The principled fix is a snapshot-aware model that resolves a read at the asking transaction's sequence. Until then, settling is what keeps the oracle honest, at the cost of not exercising a long-lived reader across a prepared commit.

32.2.2 `fuzz_crash` — durability

Forks a child that commits under a sync barrier, kills it, reopens the database, and checks what survived.

Its oracle is the sharpest of the four: the recovered state must be an **exact durable prefix** of what was acknowledged. Not a superset — a torn final record must be discarded, not half-applied — and not a subset. It fingerprints the model at every plausible prefix and requires the recovered state to match one of them.

This is the harness that guards the ordering rules in the write path (ch. 17).

32.2.3 `fuzz_conc` — concurrency

Worker threads over disjoint key partitions, each with its own exact oracle, while the shared memtable, write-ahead log, flush, compaction, cache, and value log all race underneath.

Partitioning the keyspace is what keeps the oracle exact under concurrency: each worker knows precisely what it wrote, so any disagreement is a real engine fault rather than a race in the checker.

32.2.4 `fuzz_decode` — untrusted input

Feeds mutated bytes to the decoders — sstable footers, column-family configurations, WAL records, partition range filters, btree nodes, and manifest batches.

These are the one place untrusted input enters the engine. Everything else operates on data the engine itself wrote; a decoder operates on whatever is on disk, which after a crash or a bad disk may be anything. A torn or corrupt file must be *rejected*, never over-read.

Several of them sit behind the block manager's checksum, which is why the bytes are handed to them directly rather than written to a file first. Going through the file would only ever exercise the checksum: random damage is caught before the decoder runs. What is left to get wrong is the part a checksum says nothing about — a record loop walking offsets and lengths that are internally consistent and still wrong.

32.2.5 Controlling a run

Variable	Effect	Honoured by
`TIDESDB_FUZZ_ITERS`	Iterations to run	all four
`TIDESDB_FUZZ_SEED`	Seed, for reproducing a failure	all four
`TIDESDB_FUZZ_DIR`	Where databases are created	`fuzz_standalone`, `fuzz_crash`, `fuzz_conc` — `fuzz_decode` opens no database
`TIDESDB_FUZZ_DUMP`	Dump the failing input	`fuzz_standalone`, `fuzz_crash`
`TIDESDB_FUZZ_DUMP_LAST`	Write the last iteration's input to a file and run only that one, for cutting a single-iteration repro	`fuzz_standalone`
`TIDESDB_FUZZ_VERBOSE`	Per-operation tracing	`fuzz_standalone`
`TIDESDB_FUZZ_LOG`, `TIDESDB_FUZZ_LOG_FILE`	Engine log level and destination	`fuzz_standalone`

The last three live in the model harness, which only `fuzz_standalone` and the coverage-guided `fuzz_tidesdb` link; `fuzz_crash` and `fuzz_conc` build against the model alone and ignore them.

```
TIDESDB_FUZZ_DIR=/fast/disk TIDESDB_FUZZ_ITERS=200 ./build-fuzz/fuzz_crash
TIDESDB_FUZZ_SEED=519270 TIDESDB_FUZZ_VERBOSE=1 ./build-fuzz/fuzz_standalone   # reproduce
```

A failure prints its seed. Re-running with that seed reproduces it deterministically, which is the first step of every investigation.

> Caution — **Put fuzz data on fast storage**
>
> Each iteration creates and tears down a database, so the run is dominated by filesystem work. On spinning disks that is slow enough to read as a hang rather than a slow run. Give each concurrent sweep its own directory.

32.2.6 Coverage-guided runs

Under Clang, two additional targets instrument the **whole library** rather than just the harness, so a guided run explores the engine itself: `fuzz_tidesdb` for the logical model and `fuzz_decode_guided` for the decoders. The decoders benefit most — they are small and branch heavily on the input bytes, which is what coverage guidance is good at reaching.

The standalone runners need no instrumentation and build with any compiler, which is why CI uses them.

32.3 The allocation-failure sweep

An allocation being refused is the one failure a caller cannot arrange from outside, which makes the code that runs afterwards — the half-built structure freed, the lock dropped, the handle never published — the least travelled in the engine. `TIDESDB_WITH_ALLOC_FAULT` builds an injector that counts the library's allocations and refuses a chosen one, and `alloc_fault_tests` measures how many a small workload makes and then runs it once per allocation, refusing a different one each time.

Only the armed allocation fails, never the ones after it, so what each round exercises is one site's unwind rather than a cascade in which the cleanup paths are also failing. What a round must show is that the database still opens and still holds every commit it acknowledged — a refusal may cost a write that never returned, and may cost the whole round, but never one the caller was told had landed. Run it under the address sanitizer, which is what turns an abandoned partial structure into a failure rather than a leak nobody sees.

The workload writes to **two** column families rather than one, which is not incidental. A flush demuxes the shared memtable into one output per family, so a single-family workload produces a single output and can never reach the unwind that returns an already-installed output's build reference when a later one cannot be installed. The reachable set of a sweep like this is bounded by what the workload actually does, and a path the workload cannot enter is not covered no matter how many allocations are refused.

It needs a linker that can rewrite a symbol, so it is a build option rather than something always present; see Building (ch. 6).

32.4 What CI enforces

Beyond the platform matrix, four gates are worth knowing about because they fail builds that otherwise look fine.

Zero warnings. `-Wall -Wextra` are always on but nothing makes them fatal, so a warning can accumulate unnoticed. One lane compiles the library, the tests, the fuzz harnesses and the benchmark driver with warnings as errors. It is the only job that fails on one, and it is the only job that compiles the bench driver at all.

Formatting. Every first-party source directory is checked, not just `src` — the public header and the test, fuzz and bench trees are maintained code too. Only vendored sources are exempt.

Fuzzing. All four harnesses run bounded, with the seed drawn from the run number so each build explores new ground while any failure reproduces from the log.

Thread sanitizer. It cannot be combined with the address sanitizer, so without its own lane the data races it finds have no coverage anywhere.

32.5 The benchmark driver

```
cmake -S . -B build-bench -DCMAKE_BUILD_TYPE=Release -DTIDESDB_BUILD_BENCH=ON
cmake --build build-bench -j
./build-bench/tidesdb_bench --benchmarks=fillrandom --threads=8 --num=1000000 --dir=/fast/disk
```

32.5.1 Workloads

Workload	What it does
`fillseq`	Sequential inserts
`fillrandom`	Random inserts
`readrandom`	Point reads of present keys
`readmissing`	Point reads of absent keys — the filter path

Workload	What it does
`readwrite`	Mixed, split by `--read_ratio`
`deleterandom`	Random deletes
`deletewhileread`	Deletes concurrent with reads
`scan`	Range scans of `--scan_length`
`scanrange`	The same scans told their range up front, so the two forms can be compared on one dataset
`scanempty`	Bounded scans of a range holding no keys, in the **gap between two written keys** — what it costs to prove a range empty
`fillindex`	Builds a table family and an index family together, one row and the index entry pointing at it per transaction. Needs `--cfs=2`; it is what `indexscan` and `indexmixed` read
`indexscan`	A secondary-index `ref` join: seek the index family, then a point get on the **table** family per matching entry, which is what the MariaDB plugin's `index_read_secondary` issues. One op per lookup, since the lookup is the unit being measured. `--index_fanout` sets rows per indexed value
`indexmixed`	The same lookups running concurrently with writes, split by `--read_ratio`. The one to use for an index measurement that should mean anything — see the caution below

Several may be given at once: `--benchmarks=fillrandom,readrandom,scan`.

Caution—**`scanempty` probes an interior gap on purpose**

A range past the last key is excluded by every sstable's recorded bounds before a block is read, so it measures the pruning fast path rather than the question. The probe sits between two adjacent written keys, where the bounds of every file spanning that point overlap it and none can be pruned. **Check the cache counters before believing any latency from this workload** — `hits=0 misses=0` means nothing was read and the fast path ate the experiment.

Note—**Size the cache below the data when measuring the scan path**

`indexscan` and `scanempty` are about what a scan opens and reads. A cache large enough to hold the store turns every one of those into a hit and the measurement flattens. Run them at a cache well under the data size, and read `hit_rate` alongside throughput: a change that removes wasted lookups lowers the hit rate while raising throughput, because the lookups it removed were the cheap hits.

Danger—**Read the level table before believing any scan measurement**

Most of what the merge does costs one step per **source** — per sstable whose key range meets the scan — so a change to that loop can only show up on a store that opens several. Range pruning makes that number much smaller than the file count: L2 and beyond hold non-overlapping runs, so a narrow lookup opens about one file per level however many files the level holds. Only L1 may overlap, and only L1 contributes many sources.

So the number that decides whether the measurement can see anything is `read_amp` and the `L1=` count in the per-cf table, **not** `sstables=`. A store at `read_amp 2.00` opens two sources, and against it every per-source change measures as noise — correctly, because there is nothing there to remove.

Worse, the shape does not hold still. Compaction runs on open, so a store measured repeatedly drains towards that state between runs while the benchmark is read-only and looks inert. Setting `--l1_trigger` high at fill time does not prevent it, and neither does passing it again on the read run. Re-read the level table on **each** run rather than assuming the shape you built is the shape you measured.

That is why the index workloads come in two forms. A read-only `indexscan` lets compaction finish, and a finished store has an empty L1 — so it measures the shallow case whatever it was built as. `indexmixed` keeps writing while it reads, which is what holds L1 populated, and it is the one that describes a store under live load.

Three more things an A/B needs to be worth anything:

- **A run of seconds, not tenths.** At a fraction of a second, thread ramp and cache warmth dominate and run-to-run spread reaches 40%. Use `--seconds`.
- **Interleaved variants** (`A B A B`), not all of A then all of B. Throughput drifts over a session and a blocked order aliases that drift onto the variable.
- **An identical starting state per point, restored from a snapshot.** This is the one that bites hardest on a sweep, because a store that is being written degrades as it grows, so whichever point runs first wins. A thread-count sweep run 1→16 reported throughput *falling* with more threads; the same sweep run 16→1 reported it rising fivefold. Copying the store back before each point, and scrambling the order, was what finally produced a curve that meant anything.

And profile only once compaction has drained. A profile taken straight after a fill is full of compaction threads, and it moved the headline number by 30% here.

Danger—**Anything about the store's shape needs `--rate`**

Read amplification, level depth and space amplification are **equilibria** between how fast data arrives and how fast compaction takes it away. A change that drains faster lets the writers go faster too, so an unrated run reaches a new balance at higher throughput and can settle at the very same depth — having bought throughput rather than shallowness.

Comparing the depth of two unrated runs therefore measures nothing: the two ingested different amounts. Pin the input rate with `--rate` so drain is the only variable, and pick the rate deliberately — below both configurations' ceilings they will be identical, because both keep up and there is nothing to see. The interesting rate sits between the two ceilings, where one sustains the target and the other falls short.

Read `achieved` against the target, and the admission stall beside it. A configuration that misses its target is the one that could not keep up, and the stall total says how long its writers were parked for.

And do not compare `read_amp` between runs that were still writing when they ended. It is a snapshot of wherever the compaction cycle happened to be: a run that has just finished a merge shows an almost empty flush tier, one caught mid-merge shows a full one. The same configuration, run three times under saturation, produced `read_amp` of 2, 12 and 2. Depth is comparable only on a settled store, or as a time series through `cfstats.tsv` rather than as the single figure printed at the end.

Under saturation, ask about **write amplification** instead. It is the ratio of work done to data stored, so it is meaningful whatever phase the run ends in, and it is what a change that quietly does more work per byte would show up in. The driver's `write_amp` counts the write-ahead log and the value log alongside flush and compaction, so it holds for a store that separates its values, where the key logs are a rounding error and everything else would report a figure below one.

32.5.2 Options that change what you are measuring

Option	Effect
`--threads`, `--num`	Concurrency and operation count
`--key_size`, `--value_size`	Record shape. **The single most consequential setting** — a value either side of `value_separation_threshold` exercises entirely different paths
`--dist`, `--zipf_theta`	Key distribution: uniform, zipfian, sequential
`--sync`	Durability mode; the largest single lever on write throughput

Option	Effect
`--isolation`	0–4; snapshot and above add conflict detection
`--cfs`, `--txn_ops`	Column families, and operations per transaction
`--memtable_size`, `--cache_size`, `--l0_stall`	The engine knobs most worth sweeping
`--flush_threads`, `--compaction_threads`	Background pool sizes
`--bloom`, `--bloom_fpr`	Filter on or off, and its false-positive target
`--seconds`	Run each workload for a wall-clock duration instead of to its operation count. Prefer it: a run bounded by operations takes whatever time it takes, and at high thread counts that is often a fraction of a second
`--compression`	The column families' encoding pipeline — `none`, `snappy`, `lz4`, `lz4fast`, `zstd`. **A store that compresses is a different engine from one that does not**: every key-log node is encoded on the way out and decoded on every read that misses the cache, so a benchmark left at `none` does not describe a deployment that sets it
`--index_fanout`	Rows sharing one indexed value, for `fillindex` and the index workloads
`--rate`	Hold total throughput to N operations per second. **Required for any question about the store's shape** — see below
`--seed`	Reproducibility

32.5.3 Options that change the tree's shape

These drive the compaction structure itself, which is what a sweep of the layout varies. **Each one left unset keeps the engine's own default**, so an unswept run stays comparable to a swept one.

Option	Effect
`--level_size_ratio`	`T`, the ratio between successive level capacities. Smaller deepens the tree sooner and raises write amplification
`--dividing_level_offset`	How far above the largest level the dividing level sits. 1 means `X = L - 2`
`--min_levels`	Floor on the level count
`--l1_trigger`	Flush-tier runs that make a compaction due
`--tombstone_density`, `--tombstone_min_entries`	The density trigger and the entry count below which it is ignored
`--value_separation_threshold`	Where values spill to the value log. Pairs with `--value_size`
`--btree_block_size`	Target klog node size
`--idle_flush_seconds`	Timer that rotates an idle non-empty memtable
`--skip_list_max_level`, `--skip_list_probability`	Memtable skip-list shape

A worked contrast, on the same 2M-key load: the defaults (`T=10`) settle at two levels with read amplification 2.0 and write amplification near 1.5, while `--level_size_ratio=5 --l1_trigger=2` reaches three levels but pays read amplification 6-7 and write amplification 4.4. Deepening the tree is not free, and at a given data size it may buy nothing.

32.5.4 Time series output

`--sample_interval_ms=N` starts a background sampler writing tab-separated series into the data directory, one file per stream:

File	Contents
`throughput.tsv`	Operations per second over time
`resource.tsv`	Process CPU, resident memory, and live heap
`dbstats.tsv`	Database statistics, including the backpressure counters
`cfstats.tsv`	Per-column-family statistics

File	Contents
`cachestats.tsv`	Block cache hits, misses, residency

Every row is flushed as it is written. A run that is killed – by the OOM killer, by a watchdog, by an operator who has seen enough – is exactly the run whose series matters, and a buffered stream loses the tail of it, which on a short run is the whole file.

> Caution—**`live_mb` is unreliable under threads**
>
> It comes from glibc's `mallinfo2`, which accounts only the main arena. On a multi-threaded run most allocation happens in per-thread arenas and the figure has been observed reporting more than the machine holds. Treat `rss_mb` as the measurement and `live_mb` as a hint at best; to separate live heap from allocator retention, re-run under a different allocator rather than trusting this column.

Each plots directly. The series matter more than the summary for anything involving flush or compaction, because the interesting behaviour is a stall partway through a run and a single average hides it.

32.5.5 Reading the results

Three rules, all learned the hard way:

Interleave your A/B. Run baseline and variant alternately in one session. Comparing against a number from last week measures the machine, not your change.

Take enough samples for the metric. Throughput stabilises within a few runs; tail latency does not. Repeated runs of one unchanged build can spread tail percentiles wide enough that a two-sample comparison tells you nothing, so take several and compare medians.

Run long enough to leave the drive's write cache. A short run on a consumer SSD measures its write cache rather than the drive, and the two differ by orders of magnitude. Decide which regime you care about before choosing a size, and before concluding anything about the engine from a sustained run, write the same volume with `dd` to the same filesystem — that control tells you whether the device was the limit rather than the engine.

Also: benchmark a **Release** build, never a sanitizer build, and keep the data directory off any disk something else is using.

CHAPTER 33

Design Lineage

Two published designs shape TidesDB's storage layer, and a third shapes its write path. This chapter states what each contributes, what TidesDB implements as published, and — more usefully — where it departs and why.

Being explicit about the departures matters. A reader who knows the papers will otherwise assume behaviour TidesDB does not have.

Full citations are in References (ch. 38).

33.1 The papers

Spooky — Niv Dayan, Tamar Weiss, Shmuel Dashesky, Michael Pan, Edward Bortnikov, Moshe Twitto. *Spooky: Granulating LSM-Tree Compactions Correctly.* PVLDB 15(11), 2022, pp. 3071–3084. Shapes compaction.

WiscKey — Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, University of Wisconsin–Madison. *WiscKey: Separating Keys from Values in SSD-Conscious Storage.* FAST '16. Shapes key/value separation.

Aether — Ryan Johnson, Ippokratis Pandis, Radu Stoica, Manos Athanassoulis, Anastasia Ailamaki. *Aether: A Scalable Approach to Logging.* PVLDB 3(1), 2010. Shapes the write-ahead log's append path.

33.2 Spooky, and what TidesDB takes from it

Spooky's problem is that the two classical compaction granularities each fail differently. Full Merge compacts a whole level at once, so "while compacting the LSM-tree's largest level, there must be at least twice as much storage space" — space amplification is exorbitant. Partial Merge picks individual files whose key ranges do not perfectly overlap, so it rewrites non-overlapping data superfluously and interspersing files of different lifetimes worsens SSD garbage collection.

Spooky's answer is to partition the largest level into equal-sized files and partition smaller levels **on the largest level's file boundaries**, so exactly one group of perfectly overlapping files merges at a time.

33.2.1 What TidesDB implements as published

The dividing level. In the paper a compaction into level `L-1` is a *dividing merge*, and its output is partitioned so each output file overlaps at most one file at level `L`. TidesDB computes

```
X = num_levels - 1 - dividing_level_offset      clamped to [1, num_levels - 1]
```

where an offset of 0 gives exactly the paper's `L-1`, and `COMPACTION_SPLIT_BOUNDARIES` is the dividing merge itself. TidesDB **defaults the offset to 1**, putting the dividing level at `L-2` rather than at `L-1` — a departure the paper's own measurements argue for, since at `L-1` a dividing merge rewrites a level holding a tenth of the data in one go. See Compaction (ch. 24).

The full preemptive merge. Before any partitioned merge, the paper picks the smallest level that could absorb everything at and below it without exceeding its capacity, and merges into that; when no level in range can absorb it, the merge goes to the dividing level. TidesDB's target selection is that rule — it accumulates level sizes from the shallowest upward and takes the first whose capacity covers the running total.

Dynamic Capacity Adaptation. DCA is not Spooky's invention — the paper attributes it to RocksDB and *leverages* it, because without it a newly added largest level has far more capacity than data, and durable space amplification "may be two or greater". DCA sets the capacities of levels `1..L-1` from the largest level's **data size** rather than its capacity, bounding durable space amplification to `1/(T-1)`.

TidesDB implements it directly:

```
C_L = base · T^(L-1)              the largest level keeps a geometric capacity
C_i = N_L / T^(L-i)   for i < L   smaller levels track the largest level's actual data
```

33.2.2 What TidesDB adds

A pure planner. Spooky presents an algorithm; TidesDB separates the decisions from the I/O entirely. The planner takes a snapshot of level sizes and file counts and returns decisions with no I/O, no handles, and no clock, so the policy is tested by supplying numbers rather than by building trees on disk. The paper does not prescribe this, and it is the main reason the policy is verifiable at all.

Two triggers the paper does not have, both needed in practice:

- **L1 file count.** TidesDB's L1 holds flush outputs that may overlap, so its file count is direct read amplification and wants an explicit bound.
- **Tombstone density.** A delete-heavy family whose levels are not growing would never compact under a capacity-only rule, and its tombstones would keep costing space and read work.

Per-family policy. Every column family runs its own instance with its own triggers.

33.2.3 Where TidesDB diverges

The SSD garbage-collection model is adopted in part. Part of Spooky's motivation is that flash devices lay data out across erase units, so files of differing lifetimes becoming interspersed makes device-level garbage collection rewrite data repeatedly. Spooky's design deliberately caps the number of files being written simultaneously at three — one from the buffer flush, one from a full preemptive merge, one from a partitioned merge — and merges file pairs *in key order* so data is written and discarded in the same order.

TidesDB keeps the second of those. A partitioned merge is a single streaming pass that rolls to a new output at a boundary as keys go by, so within a job outputs are written in key order with one file open at a time. It does not impose the global cap: flush and compaction are sized independently and may have several outputs open across jobs.

That is a deliberate trade of device-layout tidiness for parallelism, and it is settled rather than outstanding: the cap addresses garbage collection inside the device, and on the storage TidesDB has been characterised against, device-level amplification does not vary with the number of files being written at once. The parallelism it would cost is real and measured; the amplification it would save is not detectable. The flush build in particular was parallelised with an ordered install for exactly that throughput reason.

The output size cap, adapted for separated values. The paper splits an output whose projected size exceeds `N_L / T`. TidesDB applies that rule, derived from the same quantities, on top of boundary alignment rather than instead of it — so a partitioned merge caps each partition's output too.

The adaptation is in what gets counted. The paper has no key/value separation, so an output's size and its data volume are the same quantity. Under separation they are not: a spilled value leaves only a reference in the klog, and what a later merge rewrites is the klog alone. So the cap counts klog bytes. Counting logical value bytes instead would split a run of large values into one file per key while the files it produced were tiny.

The cap also has a floor: a computed cap below one mebibyte is reported as no cap at all. On a small or barely-populated tree `N_L / T` can fall to a few kilobytes, and honouring that literally would roll a new file every few keys — paying a file's worth of overhead to bound something that was never large.

What TidesDB does not implement is the paper's **seamless adaptation** — re-splitting a file that skewed deletes have left larger than the current maximum. The cap applies when a file is written, not retroactively to one already on disk.

33.3 WiscKey, and what TidesDB takes from it

WiscKey's observation is that LSM compaction rewrites values repeatedly even though only keys need sorting. Separating them means compaction moves keys only, so write amplification collapses for large values.

TidesDB takes the core idea: values at or above the database's `value_separation_threshold` are written to a value log, and the key log stores a reference.

It also takes WiscKey's second observation, which is the one about the write-ahead log. **The value log is written by the committing transaction, not by the flush.** A large value written inline would reach the device twice — once in the log record and again in the sstable the flush builds — so the commit puts the bytes in the value log and the record carries only the id. On a 4 KiB-value workload that is the difference between a write amplification of 2.10 and 1.06, and it is also why a memtable of large values holds far more keys before it fills: a version costs an id rather than a value.

It also inherits the cost, and the manual says so where it matters — a point read of a separated value costs a second I/O, and a scan that dereferences values pays it per entry.

33.3.1 Where TidesDB diverges, and this is the substantive part

Values are addressed by opaque logical id, not by file offset.

This is the important difference. In WiscKey a value's address *is* its position in the vLog, so garbage collection — which appends surviving values back to the head — must update the LSM entry for every value it moves. Reclaiming space writes to the tree.

TidesDB stores a logical id. An id names one block for its whole life, so **ordinary compaction carries a value forward by id without reading, re-encoding, or moving it**, and reclaiming a segment that nothing references rewrites no tree at all. The indirection costs an id lookup on read and buys a reclamation whose common case touches only the value log.

Liveness has three holders, not one. Because the commit writes the value, it exists before any tree names it, and it stays that way until the memtable holding the reference is flushed. So the live set is not the installed tables alone: a memtable and an undecided prepared batch hold references too. Tables and prepared batches are counted; memtables are covered by a floor, since their contents move under a reader. Getting that wrong is not a space leak but data loss, and each of the three was found that way.

The key is not stored beside the value. WiscKey stores `(key size, value size, key, value)` in the vLog specifically so garbage collection can determine validity by querying the LSM with the key it just read from the tail. TidesDB does not need the key, and does not need to read the value log to decide validity either: **every sstable records which segments its separated values landed in and how many bytes of each it holds**, in its footer. Summed over the installed tables, that is the live bytes of every segment exactly, computed from metadata alone.

That is the substantive divergence. WiscKey §3.3.2 dismisses scan-based collection as "too heavyweight and only usable for offline garbage collection", and it is right: TidesDB first tried asking the engine for the live id set, which meant scanning every key of every sstable of every family on one shared store, and on a database large enough to need reclaiming that scan does not finish. Making the trees *report* what they hold instead of being *read* is what removed the scan.

Two tiers over sealed segments, not a head/tail circular log. WiscKey keeps valid values in a contiguous range between a tail and a head, reading a chunk from the tail, re-appending survivors to the head, and advancing the tail. TidesDB's store is a series of numbered segments, and it never copies a value itself. A segment nothing references is unlinked whole. A segment holding little enough live data is *marked*, and the next compaction that carries one of its values forward writes that value afresh instead of keeping the reference — so the segment empties through a merge that was already scheduled, and is then unlinked for free.

This is closer to RocksDB's blob garbage collection than to WiscKey's: the rewriting happens inside compaction, where a table is being rebuilt anyway, rather than in a separate pass that must move bytes underneath live readers. What a reclaim leaves behind is a whole file gone rather than a boundary advanced, so there is no partially-reclaimed region to reason about, and because nothing is ever copied within the store, a crash cannot leave the same id in two places.

One database-wide value log, not one per store. Every column family shares it. Per-family logs would shrink the old scan, which is why they were considered — but the scan is gone, and liveness is now metadata a shared store answers as cheaply as a split one would, at fewer files.

No parallel value prefetching. WiscKey's answer to slow range queries is to exploit SSD parallelism: track the iterator's access pattern and prefetch values from the vLog with multiple threads. TidesDB has no such prefetcher. Its mitigation is different in kind — key-only iteration never dereferences a value at all, so a scan built on `tidesdb_iter_key` avoids the cost rather than hiding it, as does an existence check through `tidesdb_txn_contains`. For a scan that genuinely needs every value, WiscKey's prefetching would help and TidesDB does not have it.

Values are encoded in the log, and each describes its own encoding. A value block carries the id, the uncompressed length, the chain of encoding ids the value was written through, and then the stored bytes. The self-description is required rather than decorative: compaction carries a value forward by id without re-encoding it, so the sstable referencing a value may record a different pipeline than the one that wrote it, and one shared store holds values from families configured differently.

33.4 Aether, and the write path

Aether's contribution is that a log's scalability bottleneck is contention on log-space allocation and the serialization of many small writes. Its answers are a consolidation array, so committers combine their space requests, and flush pipelining, so filling and writing overlap.

TidesDB's buffered append ring is that shape: one atomic reservation, parallel fill by each appender into its own slice, and a single flush thread draining the contiguous completed run. The ring's allocation *is* the consolidation, and the flush thread *is* the pipelining.

The divergence is that TidesDB does not implement a consolidation array as a separate structure. The ring's single reserving atomic already serves that purpose, and measurement found no remaining allocation contention to remove.

33.5 What is novel here

Not every part of TidesDB traces to a paper. The combination that does not:

An LSM of B+trees with key/value separation. WiscKey separates values from a conventional LSM whose runs are sorted blocks with a sparse index. TidesDB's key log *is* a B+tree — the sorted store and the index are the same structure, leaves are doubly linked for bidirectional scans, and MVCC version resolution happens natively during the descent rather than as a filter above it.

Composing that with value separation has a compounding effect the papers do not describe. B+tree node density is governed by entry size; moving large values out makes nodes hold far more keys, which shortens the tree and raises cache hit rates for the nodes that remain. Value separation helps a B+tree-based run more than it helps a block-based one, because it improves the index structure itself rather than only the bytes rewritten by compaction.

MVCC inside the run structure. Versions live in the B+tree as version chains resolved against a snapshot ceiling during the descent. Neither paper addresses multi-version visibility; both assume a single current value per key.

Logical-id indirection for reclamation, described above, which makes value log compaction independent of the trees that reference it.

These are design claims, not measured ones. The engine has not been evaluated against the papers' benchmarks, and no claim here should be read as a reproduction of their results.

Part VII

Appendixes

Error Codes

Every public function returns `TDB_SUCCESS` (0) or one of these. They are negative, so `rc < 0` tests for failure and `rc != TDB_SUCCESS` is equivalent.

Code	Value	Meaning
`TDB_SUCCESS`	0	Success
`TDB_ERR_MEMORY`	-1	Allocation failed
`TDB_ERR_INVALID_ARGS`	-2	A `NULL` or out-of-range argument, or a configuration that failed validation
`TDB_ERR_NOT_FOUND`	-3	The key, column family, or savepoint does not exist
`TDB_ERR_IO`	-4	A read, write, or fsync failed
`TDB_ERR_CORRUPTION`	-5	On-disk data did not decode
`TDB_ERR_EXISTS`	-6	A column family with that name already exists
`TDB_ERR_CONFLICT`	-7	Another transaction committed a conflicting write first
`TDB_ERR_TOO_LARGE`	-8	A supplied buffer is too small
`TDB_ERR_MEMORY_LIMIT`	-9	A configured memory limit was reached
`TDB_ERR_INVALID_DB`	-10	The database is closing or otherwise unusable
`TDB_ERR_UNKNOWN`	-11	An error with no more specific code
`TDB_ERR_LOCKED`	-12	Transient contention — retry
`TDB_ERR_READONLY`	-13	A write was attempted against a read-only database
`TDB_ERR_TXN_EXPIRED`	-14	The transaction is no longer active
`TDB_ERR_NO_SPACE`	-15	The device or filesystem is out of space
`TDB_ERR_TXN_ABORTED`	-16	Another thread aborted the transaction through `tidesdb_txn_request_abort`
`TDB_ERR_TOO_OLD`	-17	The sequence asked for is older than the oldest reconstructable point in time

34.1 The two that mean “ask again”

Neither is a failure. Both say the operation did not happen and could succeed if repeated, and the damaging mistake is to record either as a permanent error — above all to mistake one for an absent key.

34.1.1 `TDB_ERR_CONFLICT`

From any level above `TDB_ISOLATION_READ_COMMITTED`, and only from `commit` or `prepare`. Nothing durable was written; the transaction is aborted.

What was violated depends on the level, and the two causes are different events. At `TDB_ISOLATION_REPEATABLE_READ` a key **you read** gained a newer committed version. At `TDB_ISOLATION_SNAPSHOT` a key **you wrote** was written by someone who committed first. `TDB_ISOLATION_SERIALIZABLE` reports either, and additionally the write-skew check.

Retry the whole transaction — begin again, redo the reads, redo the writes. Retrying just the commit is meaningless, since the reads it was validated against are stale.

34.1.2 `TDB_ERR_LOCKED`

Contention the engine could not absorb on your behalf. Where you meet it depends on what you asked for, and the two cases are not equally common.

From a maintenance call — `tidesdb_compact`, `tidesdb_compact_range`, `tidesdb_cf_update_runtime_config`, `tidesdb_rename_column_family`, `tidesdb_clone_column_family`, `tidesdb_backup` — it means another exclusive operation already holds that family.

`tidesdb_flush_memtable`, and `tidesdb_checkpoint` through it, report it for a related but distinct reason: the immutable queue had not drained when their bounded wait expired, which says flush is behind rather than that anything holds a family. This is the ordinary case, and returning immediately is deliberate: these calls never park you behind a compaction that could run for minutes. Retry later, or don't; the work you asked for is in many cases already happening.

From a read or a scan it is rare. Transient pressure — a memtable rotating out from under a reader, a momentarily exhausted descriptor budget — is waited out inside the engine, because a caller told "try again" has no lever the engine does not already have. Reaching a caller means the pressure outlasted every internal recheck.

It never means "not found". A read returning `TDB_ERR_LOCKED` is saying the data may well exist but could not be reached right now. Code that folds it into an absence produces silent wrong answers under load — the single most common way to misuse this API.

Retry with a short backoff.

34.2 Which functions produce what

Code	Typically from
`TDB_ERR_CONFLICT`	`tidesdb_txn_commit`, `tidesdb_txn_prepare`, at any level above `TDB_ISOLATION_READ_COMMITTED`
`TDB_ERR_LOCKED`	`tidesdb_compact`, `tidesdb_compact_range`, `tidesdb_cf_update_runtime_config`, `tidesdb_rename_column_family`, `tidesdb_clone_column_family`, `tidesdb_backup`; `tidesdb_flush_memtable` and `tidesdb_checkpoint` when the immutable queue does not drain in time; `tidesdb_range_stats` when the layout moves mid-scan; rarely, reads, iterator steps and `tidesdb_iter_new` under pressure the engine could not absorb
`TDB_ERR_NOT_FOUND`	`tidesdb_txn_get`, `tidesdb_txn_contains`, family and savepoint lookups
`TDB_ERR_EXISTS`	`tidesdb_create_column_family`, `tidesdb_clone_column_family`
`TDB_ERR_TXN_EXPIRED`	The first operation on a transaction whose timeout has passed. Only when a timeout was set, by `txn_timeout_seconds` or `tidesdb_txn_set_timeout`; it is reported once and the transaction is then aborted
`TDB_ERR_TXN_ABORTED`	Every operation on a transaction after another thread called `tidesdb_txn_request_abort` on it, the commit included. Distinct from `TDB_ERR_CONFLICT` on purpose: a conflict is the engine's own verdict between two writers, where this says an outside authority ruled and the engine never weighed in
`TDB_ERR_TOO_OLD`	`tidesdb_txn_begin_at_seq` for a sequence a collection has already run below. It is not a transient condition and retrying never clears it – the versions that point named are gone, and the boundary only moves further away. Hold a snapshot in advance for a point in time you know you will want
`TDB_ERR_INVALID_DB`	Operations against a closing database
`TDB_ERR_TOO_LARGE`	`tidesdb_recover_prepared` when the buffer is too small
`TDB_ERR_CORRUPTION`	Open and read paths, on undecodable on-disk data
`TDB_ERR_NO_SPACE`	`tidesdb_flush_memtable`, `tidesdb_checkpoint`, and any path that writes an sstable

34.3 `TDB_ERR_NO_SPACE`

The device or filesystem could not take the write. It is separated from `TDB_ERR_IO` because it is the one write failure that says what to do about it: **nothing is damaged, and the operation succeeds once space is freed.** A generic I/O error carries no such promise, and treating a full disk as one is how a filled device gets mistaken for a defect in the engine.

Quota exhaustion (`EDQUOT`, where the platform defines it) reports the same code, since it is the same condition and the same remedy.

Data already committed is unaffected. What fails is the attempt to write new files — a flush, a compaction, a checkpoint — so the memtable and its log keep the data until a flush can land it. Free space and retry.

The engine also logs the condition once per failing file, with the remaining free space, so it is distinguishable in a log without reproducing it.

34.4 Turning a code into text

```
const char *tidesdb_strerror(int code);
```

Returns a short description of any result code, for a log line or an error message. The string is a static literal: never `NULL`, never freed, valid for the life of the process. An unrecognised code describes itself as unknown rather than returning `NULL`, so a result can be passed straight through without being checked first.

```
int rc = tidesdb_txn_commit(txn);
if (rc != TDB_SUCCESS) fprintf(stderr, "commit failed, %s\n", tidesdb_strerror(rc));
```

It describes the code, not the cause. Which key, which family, and which file are yours to add.

34.5 Handling pattern

```
int rc;
int attempts = 0;

do {
    rc = tidesdb_txn_get(txn, cf, key, key_size, &value, &value_size);
} while (rc == TDB_ERR_LOCKED && ++attempts < MAX_RETRIES);

if (rc == TDB_SUCCESS)
{
    /* use value, then free it */
    tidesdb_free(value);
}
else if (rc == TDB_ERR_NOT_FOUND)
{
    /* genuinely absent */
}
else
{
    /* a real failure */
}
```

Note the three-way split. Collapsing “absent” and “failed” into one branch is the shape that hides `TDB_ERR_LOCKED`.

34.6 On `TDB_ERR_CORRUPTION`

It means bytes did not decode — a damaged file, a truncated record, a foreign file in the directory. It does **not** always mean data loss:

- A damaged **manifest** is discarded and rebuilt from the sstables on disk, and does not surface as an error at all — though a rebuilt family comes back named `cf_` followed by its zero-padded id, with the default configuration.
- A torn **final record** in a log is discarded during recovery; it was never acknowledged.
- A corrupt **sstable block** is a genuine problem: restore from a backup.

If corruption appears on a healthy disk, suspect the storage stack before the engine, and check whether anything else is writing into the database directory.

Chapter 35

On-Disk Formats

This appendix documents the layouts as of format version 10.

Five structures carry that version and validate it on read: the block-manager file header, the write-ahead log record, the sstable footer, the manifest record, and a column family's stored configuration. Two carry no version of their own and are governed by whichever of those encloses them — a btree node by its sstable's footer, a value-log block by its segment's file header. The partition range filter directory is the exception: it versions independently, as noted where it is described below.

> Caution—**Descriptive, not a contract**
>
> These layouts are documented so the engine can be understood, debugged, and its files inspected. They are **not** a stability guarantee for third-party readers or writers. What is guaranteed is stated in `VERSIONING.md`: a release reads what prior releases in its major line wrote. Anything beyond that — parsing these files with your own tooling — is at your own risk and may break in a minor release.

All multi-byte integers are **little-endian** unless a table says otherwise. The sstable footer is the exception and is noted where it occurs.

35.1 Files in a database directory

File	Contents
`MANIFEST`	The catalogue: which families and sstables exist
`NNNNNNN.vlog`	One append-only segment of the shared value log holding separated values; exactly one is open for appends and the rest are immutable. Each value's block records the whole chain of encodings it was written through, so one store serves families with different pipelines and a value survives being carried into an sstable configured differently
`NNNNNNN.log`	A write-ahead log generation, zero-padded to 7 digits
`CCCCCCCCCC.SSSSSSS.klog`	An sstable's key log: its family's id zero-padded to ten digits, a dot, then the sstable id zero-padded to seven
`CCCCCCCCCC.SSSSSSS.klog.lstmp`	Transient. Where a build stages uncompressed leaves before encoding them, so it exists only while an sstable is being written and only when the family has an encoding pipeline. It is removed when the build ends, whether it succeeded or failed; only a crash leaves one behind, and the next open sweeps it alongside the orphaned key logs

Every one is a block-manager file and shares the framing below.

Families have no directory of their own. Every file above sits directly in the database directory, and a key log names its owning family in its own filename, by **id** rather than by name. The id is assigned once and never reused, so a family's files never move. That is what makes `tidesdb_rename_column_family` (ch. 11) a change of name and nothing else — nothing on disk moves, and so none of the open sstables whose cached paths a move would invalidate need rebuilding. Mapping an id back to a family name means reading the `MANIFEST`, which is the catalogue for everything else on disk too.

Carrying the family id in the filename is also what lets a database whose `MANIFEST` is unreadable be rebuilt from the key logs alone: the files still say which family each belongs to.

35.2 The block frame

Every record in every file:

```
offset  size  field
------  ----  --------------------------------------------
     0     4  payload size          (uint32, little-endian)
     4     4  payload checksum      (XXH3, truncated to 32 bits)
     8     n  payload
   8+n     4  payload size again    (uint32, little-endian)
  12+n     4  footer magic 0x42445442  ("BTDB" reversed)
```

The size appearing at **both ends**, plus the footer magic, is what makes a partially written record detectable without a separate intentions log. A record whose trailing size disagrees with its leading one, or whose magic is missing, was never completely written.

A payload size of **zero is invalid** and is rejected at write time. Readers treat a zero size field as end-of-file, so permitting one would truncate iteration for everything after it.

35.3 The file header

Every block-manager file begins with 8 bytes:

```
offset  size  field
------  ----  --------------------------------
     0     3  magic 0x544442 ("TDB")
     3     1  format version (10)
     4     4  reserved
```

35.4 Write-ahead log record

One framed block per committed batch. The payload:

```
1 byte    format version (10)
1 byte    record kind
varint    xid length (0 when absent)
n bytes   xid
varint    entry count
then, per entry:
  1 byte  flags
  varint  column family index
  varint  sequence
  varint  key size
  varint  value size
  varint  ttl            (present only when the HAS_TTL flag is set)
  varint  value log id   (present only when the VLOG_REF flag is set)
  n bytes key            (the interval's lower bound for a range delete)
  n bytes value          (absent when the VLOG_REF flag is set; the interval's exclusive upper
                          bound for a range delete, and empty for an open one)
```

Record kinds

Value	Kind	Meaning
0	`TDB_WAL_KIND_WRITE_BATCH`	An ordinary single-phase commit
1	`TDB_WAL_KIND_PREPARE`	Two-phase commit phase one: durable but not applied
2	`TDB_WAL_KIND_COMMIT`	Phase two, commit — carries the batch to apply
3	`TDB_WAL_KIND_ROLLBACK`	Phase two, abandon
4	`TDB_WAL_KIND_ABORT_SEQ`	Names an already-durable sequence that replay must not apply. The sequence rides in the xid slot as 8 big-endian bytes, so the record is the usual framing with an xid length of 8 and an entry count of 0. A distinct kind rather than a rollback, which is keyed by a caller's transaction id and could legitimately be 8 bytes long

`TDB_WAL_KIND_ABORT_SEQ` exists because a write batch's presence in the log *is* its commitment, so a batch that reached the log but failed to enter the memtable would otherwise come back on the next open after its caller was told the transaction failed. Replay collects these first and skips the sequences they name.

Entry flags

Bit	Flag	Meaning
0x01	`TOMBSTONE`	A delete; no value follows
0x02	`SINGLE_DELETE`	The delete supersedes at most one put
0x04	`HAS_TTL`	A ttl field is present for this entry

Bit	Flag	Meaning
0x08	`VLOG_REF`	The value's bytes are in the value log, not in this record. A value log id follows the ttl slot and no value bytes follow the key; `value size` still carries the value's logical length, so a reader knows how large it is without a value log probe. An id of zero is refused rather than read as an empty value
0x10	`TDB_WAL_ENTRY_RANGE_DELETE`	The entry deletes every key in an interval rather than the one key it names. The key is the inclusive lower bound and the **value is the exclusive upper bound**, empty when the interval runs to the end of the column family. Always set alongside `TOMBSTONE`, so a reader that does not know this bit still sees a valueless delete rather than a live put

Sequences are carried per entry, so replay applies each at the sequence it committed with rather than at its position in the file.

Both optional fields are gated on a flag rather than always present, which is why adding the value log reference did not move the format version: a record without the bit encodes and decodes exactly as it always did.

35.5 SSTable footer

The last block of a `.klog`. **Big-endian**, unlike everything else — it is read by a path that predates the little-endian convention elsewhere.

Fixed head, 116 bytes:

```
size  field
----  ------------------------------------------------
   4  magic 0x53535442 ("SSTB")
   4  format version (10)
   8  btree root offset
   8  first leaf offset
   8  last leaf offset
   8  filter directory offset
   4  filter directory size
   8  distinct key count
   8  tombstone count
   8  maximum sequence
   8  total key bytes
   8  total value bytes
   8  klog logical bytes
   8  btree node count
   8  range tombstone applied sequence
   4  btree height
   4  btree node size
```

The *range tombstone applied sequence* is the newest range tombstone this table's build could see at the reclamation floor it ran at, and every one at or below it had already been applied to these contents. The **minimum across a column family's tables** is what says a range tombstone has nothing left to hide and can be retired. A table built before any range tombstone existed records zero and holds that minimum down until it is rewritten, which is the conservative direction: the value has to be a sequence the build actually observed, never the floor, or a table built before a tombstone would claim to have applied it.

Then a variable tail:

```
   4  minimum key length
   n  minimum key
   4  maximum key length
   n  maximum key
   1  encoding count
   n  encoding pipeline ids
   4  value log reference count
  24  per reference: segment(8) + bytes(8) + count(8)
```

The value log references are how an sstable records which segments it draws separated values from and how much of each it holds, which is what lets a reclaim decide a segment is worth draining without reading every key log.

Every length in the tail is bounds-checked against the remaining bytes, so a truncated or lying footer is rejected rather than read past.

Recording the **btree node size** here rather than reading it from configuration is what lets a file be read with the geometry it was written with, after the configuration has changed.

35.6 Partition range filter directory

The resident half of an sstable's filter, written into the key log's aux region and pointed at by the footer's *filter directory offset* and *size*. Little-endian throughout:

```
4 bytes  magic 0x46424254 ("TBBF")
4 bytes  format version
4 bytes  partition count
then, per partition:
  8 bytes  blob offset within the key log
  4 bytes  blob size
  4 bytes  entry count
  4 bytes  first key length
  n bytes  first key
```

The records are in ascending key order, so a lookup binary-searches them to route a query to its partition. The first keys are stored **whole** — an approximate routing could send a query to the wrong partition and produce a false negative, which is worse than no filter at all.

Note—**This directory versions independently of the file holding it**

It carries its own magic and its own format version, validated on read, rather than inheriting the key log's. So its version is **not** the format version at the top of this appendix, and it moves only when this layout changes. The blobs it points at are ordinary bloom filters fetched on demand through the block cache, which is what bounds the filter's resident cost to the directory alone.

35.7 BTree leaf node

```
1 byte     node type
varint     entry count
8 bytes    previous leaf offset   (-1 when first)
8 bytes    next leaf offset       (-1 when last)
2n bytes   key offset table       (uint16 per entry, from the start of the keys section)
varint     base sequence
then, per entry:
  varint   shared prefix length
  varint   suffix length
  varint   value size
  varint   vlog offset            (0 when the value is inline)
  svarint  sequence delta from the base
  svarint  ttl
  1 byte   flags
then:
  keys section     suffixes only, prefix-compressed
  values section   inline values, concatenated
```

Keys store only the suffix not shared with the preceding key. Sequences are deltas from a per-node base. The offset table makes a search within a leaf a binary search rather than a walk.

A compressed node carries a 20-byte header ahead of its compressed payload — original size (4), previous offset (8), next offset (8) — so the sibling links can be patched without decompressing.

35.8 Value log block

```
8 bytes  logical id
8 bytes  value length, with the encoding count in the top byte
c bytes  encoding pipeline ids, in the order they were applied
n bytes  value, as stored
```

A value's length is bounded by the uint32 block frame, so the upper half of the length word was never used; the top byte of it carries how many encodings the value was written through, and that many ids follow the fixed header. A count of zero means the bytes are stored verbatim and no ids follow.

Every value describes its own encoding, and that is load-bearing rather than convenient. Compaction carries a separated value forward by id without re-reading or re-encoding it, so a value written under one pipeline can end up referenced by an sstable whose footer records a different one. A value described by anything other than its own bytes would then be decoded with the wrong chain. It is also what lets one shared store hold values from families that encode differently.

Values are addressed by opaque logical id rather than by file offset, so which segment holds a value is not baked into the sstables referencing it.

The store is a series of numbered segments rather than one file. The store never moves bytes itself: a reclaim unlinks a segment nothing references any more, and a segment that is merely mostly-dead is marked instead, so the next compaction carrying one of its values **writes that value afresh under a new id** rather than keeping the reference. Nothing is ever copied within the store, so an id

names one block for its whole life and a crash cannot leave the same id in two segments — a pass has either unlinked a segment or it has not.

35.9 Manifest record

One framed block per commit, holding a batch:

```
1 byte   format version
then a sequence of records, each starting with a 1-byte opcode
```

Opcode	Record	Size
`MANIFEST_OP_CF_ADD`	op + cf id(8) + name len(2) + blob len(2), then name and blob	13 + variable
`MANIFEST_OP_CF_DROP`	op + cf id(8)	9
`MANIFEST_OP_ADD_P`	op + cf(8) + level(4) + id(8) + entries(8) + bytes(8) + partition(4) + birth level(4)	45
`MANIFEST_OP_MOVE`	op + cf(8) + id(8) + new level(4)	21
`MANIFEST_OP_REMOVE`	op + cf(8) + level(4) + id(8)	21
`MANIFEST_OP_RANGE_DEL`	op + cf(8) + blob len(4), then the blob	13 + variable
`MANIFEST_OP_SEQ`	op + sequence(8)	9
`MANIFEST_OP_CF_SEQ`	op + next family id(8)	9

Records are self-delimiting, so a batch is walked without an index. The column family configuration blob is stored **verbatim and never interpreted** by the manifest.

A rollover writes a single snapshot batch — the family registry, every live sstable, each family's range tombstone set, then `MANIFEST_OP_SEQ` and `MANIFEST_OP_CF_SEQ` — to a temp file that is then atomically renamed over the manifest.

35.10 Compatibility

`VERSIONING.md` holds the policy and the compatibility matrix. In summary: the on-disk format is a first-class contract, a major release may change it and ships migration tooling, a minor release may add a format only if it is opt-in and default-off so downgrade stays possible, and a patch release may not change it at all.

Chapter 36

Configuration Reference

Two structures. Both are filled by a `tidesdb_default_*` call and adjusted; both are borrowed by the call that takes them and may be freed immediately after.

36.1 Database configuration

`tidesdb_config_t`, from `tidesdb_default_config()`.

Field	Default	Effect
`db_path`	none	The database directory. **Must be set.**
`num_flush_threads`	2	Workers writing sealed memtables to sstables
`num_compaction_threads`	2	Workers merging sstables
`log_level`	`TDB_LOG_INFO`	Engine log verbosity
`log_to_file`	0	Write the log to `tidesdb.log` in the database directory rather than stderr
`log_truncation_at`	0	Truncate that file once it passes this size; 0 never truncates
`block_cache_size`	64 MiB	The database-wide block cache budget
`max_open_sstables`	1024	Ceiling on resident descriptors for sstable key logs and value-log segments together. The reaper reclaims toward a slightly lower figure — a reserve of an eighth, at least 16 and at most half, is held back so a reader can still open a file while it works, leaving 896 at the default. **Cut at open to fit the process open-file ceiling**, which at the defaults it does not: 1024 against the usual 1024 `ulimit -n` leaves nothing for the manifest, so it comes down to 960 and says so at warn. Raise the ceiling with `tidesdb_raise_open_file_limit` (ch. 10) to keep the larger figure. See the descriptor manager (ch. 26)
`memtable_write_buffer_size`	64 MiB	Memory the active memtable may occupy before it rotates. A memory budget, not a promise about flush size — see below
`memtable_skip_list_max_level`	12	Skip list height
`memtable_skip_list_probability`	0.25	Skip list level probability
`memtable_sync_mode`	`TDB_SYNC_NONE`	Durability policy — see Durability (ch. 5)
`memtable_sync_interval_us`	1000000	Barrier period under `TDB_SYNC_INTERVAL`
`value_separation_threshold`	1024	Values at or above this size are stored in the shared value log and referenced from the key log. Database level because the value log is one shared structure and the decision keys on the size of a value rather than on which family holds it. A single family can opt out with `keep_values_inline`
`vlog_segment_size`	256 MiB	Size at which the value log seals a segment and opens a fresh one. It does not change the space the store settles at, only what a reclaim costs — smaller segments copy more live data forward, measurably so below the default
`memtable_idle_flush_seconds`	30	Rotate the active memtable after this long with no write; 0 never does

Field	Default	Effect
`memtable_l0_queue_stall_threshold`	16	Sealed-memtable queue depth at which ingest is paced; writers throttle in a band below it and are held at it, always with a ceiling. It governs **one of three** admission signals — the staging ring and L1 depth pace independently, so raising this may not reduce stalling
`txn_timeout_seconds`	0 (no timeout)	How long a transaction may stay active before the next operation on it expires it. An unresolved transaction holds its snapshot and its reservations, which keeps the reclamation floor down, so bound this if callers may leak transactions. A single transaction overrides it with `tidesdb_txn_set_timeout` (ch. 12)

> Caution—**Zero means "choose for me" in most fields**
>
> `num_flush_threads`, `num_compaction_threads`, `block_cache_size`, `max_open_sstables`, `value_separation_threshold`, `vlog_segment_size`, `memtable_write_buffer_size`, `memtable_skip_list_max_level`, and `memtable_skip_list_probability` are resolved to their defaults by `tidesdb_open` when left at zero. You cannot request zero flush threads. `memtable_sync_interval_us` is resolved the same way, but later — the interval-sync ticker substitutes the one-second default when it reads a zero.
>
> The fields where zero is used exactly as given are `log_to_file`, `log_truncation_at`, `memtable_idle_flush_seconds`, `memtable_l0_queue_stall_threshold`, `txn_timeout_seconds`, and `memtable_sync_mode`, where zero is the meaningful value `TDB_SYNC_NONE`.

36.1.1 Log levels

`log_level` is a threshold, not a selection: a message is emitted when its own severity is at least the configured level, so a higher setting emits strictly fewer lines.

Level	Value	Emits
`TDB_LOG_NONE`	0	Nothing — logging is off entirely
`TDB_LOG_TRACE`	1	Everything, including detailed messages meant for debugging the engine
`TDB_LOG_INFO`	2	Engine status and operations, plus warnings and errors
`TDB_LOG_WARN`	3	Conditions worth knowing about that are not yet failures, plus errors
`TDB_LOG_ERROR`	4	Only operations that failed

`TDB_LOG_NONE` is the one value that is not a threshold — it suppresses every line rather than admitting those of severity zero and above.

36.1.2 What to tune first

`memtable_write_buffer_size` trades memory for write amplification. Larger means fewer, bigger flushes and less compaction work, at the cost of memory and a longer recovery replay.

It is a budget on memory occupied, not on data held. An entry costs its key and value plus about a hundred bytes of skip list node, pointer arrays and version struct. That overhead is fixed per entry, so it is most of an entry holding a short value and almost none of one holding a large value. A memtable of 64 byte values reaches a 64 MiB budget holding roughly 28 MB of data; one of 4 KiB values holds very nearly the full 64 MB. If you work with small entries and want the flush sizes a byte-counting engine would give you, raise this figure accordingly — the memory was always being used, it simply was not being counted.

`block_cache_size` is the main read-performance knob. It also bounds the resident cost of partition range filters, so a cache too small to hold the working set of filter partitions makes every point lookup re-read its filter.

Resident memory tracks this figure, but the *number of nodes* it holds is fewer than the budget divided by the node size: each cached node reserves a whole allocator chunk, and the chunk is twice the node size a family is created with. Budget for the memory you want resident, and expect a cache to hold roughly half of it in useful node bytes. A family that raises `btree_klog_block_size` well above the default gets nodes larger than a chunk, which are allocated to their own size rather than rounded up. See the block cache (ch. 25).

`memtable_sync_mode` is the durability decision, and the one with the largest performance consequence.

`num_flush_threads` and `num_compaction_threads` are not free. They compete for the same cores as your application threads; on a saturated machine, raising them takes capacity from the work generating the data.

`num_compaction_threads` helps a **single** column family as well as several. A merge the planner could not divide — the tier drain, whose inputs span the key space — splits its key range across this many threads instead. A database whose write load lands on one hot family is no longer held to one thread's worth of drain. See Compaction (ch. 24).

36.2 Column family configuration

`tidesdb_column_family_config_t`, from `tidesdb_default_column_family_config()`.

Field	Default	Effect
`name`	empty	**Ignored** — the name argument to the create call is authoritative
`level_size_ratio`	10	Each level holds this many times the one above
`min_levels`	1	Levels kept before shape triggers apply
`dividing_level_offset`	1	How far above the largest level the dividing level sits, so 1 means X = L - 2. Selects where a merge writes output partitioned to the largest level's file boundaries
`keep_values_inline`	0	Hold every value in the key log whatever its size, ignoring the database's `value_separation_threshold`
`btree_klog_block_size`	4096	Target btree node size
`encoding_pipeline`	empty	Encodings applied in order to btree klog nodes and to separated values; see the note below
`encoding_count`	0	Entries in the pipeline, up to `TDB_ENCODING_PIPELINE_MAX`
`enable_bloom_filter`	1	Build a partition range filter per sstable
`bloom_fpr`	0.01	Target false positive rate
`default_isolation_level`	`TDB_ISOLATION_READ_COMMITTED`	Level used by `tidesdb_txn_begin_cf`
`l1_file_count_trigger`	4	L1 file count that triggers compaction
`tombstone_density_trigger`	0.5	Tombstone fraction that triggers compaction
`tombstone_density_min_entries`	4096	Entry count below which the density trigger is ignored
`commit_hook_fn`	`NULL`	Commit hook registered at create time
`commit_hook_ctx`	`NULL`	Context passed to the hook

36.2.1 The two that matter most

`value_separation_threshold`, which is database level, decides which values are separated. Below it, values live inline in the btree; at or above it, in the shared value log with a reference in their place.

Lowering it keeps the btree small and scans fast, and costs a second read on every point lookup of a separated value. Raising it does the reverse. Match it to your access pattern: scan-heavy workloads over large values suffer from separation, key-scan workloads benefit from it enormously. A family whose access pattern differs from the rest of the database can set `keep_values_inline` and be left out of separation entirely rather than move the threshold every other family shares.

`btree_klog_block_size` interacts with the block manager's 4 KB first-read window. A node sized just above it costs a second read on every access. 4096 is the default for that reason. Keep `value_separation_threshold` at or under a quarter of it — an inlined value approaching the node size leaves a node holding one entry and spends the btree fan-out that makes a lookup cheap. The pairing is advisory, and a config that breaks it only logs a warning.

36.3 What the encoding pipeline currently does

The field is an array because encodings **stack**: every id is applied in order on write and undone in reverse on read. The btree is handed the resolved transforms rather than an algorithm id, so it has no idea what any of them are — which is what makes a custom registered encoding work exactly like a built-in one.

The ids are resolved to their transforms **once**, when a build starts or an sstable opens, never per node. The registry is a linear scan, and a node read cannot afford one.

Klog nodes carry the whole chain. The footer records the ids, so a reader reconstructs the same chain from the file rather than from whatever the family happens to be configured with now.

Separated values carry the whole chain too, recorded in each value's own block header rather than taken from the sstable that references it. That is not redundancy: compaction carries a separated value forward by id without re-reading or re-encoding it, so a value written under one pipeline can end up referenced by an sstable whose footer records a different one. A value that described itself by anything other than its own bytes would then be decoded with the wrong chain. It is also what lets one shared value log hold values from families that encode differently.

An id that names no compression algorithm at all is rejected outright, whether it arrives at create time or through a runtime update, because the engine would otherwise persist it and then hand it to a codec dispatch that does not recognise it.

An id that names a real algorithm this build was compiled without is **accepted**, so that a database written elsewhere still opens. Reading a node written with a codec the build lacks fails at that read rather than at open.

36.4 Changing configuration at runtime

`tidesdb_cf_update_runtime_config` changes **every** field of a family's configuration while it is open, optionally persisting it. Existing sstables are not rewritten; new settings apply to what is written from then on. This is safe because byte-wise key ordering keeps files written under different settings mergeable.

Database-level configuration is fixed at open.

Glossary

Block — One framed record in a file: size, checksum, payload, size again, magic. Every file the engine writes is a sequence of these.

Block cache — The database-wide bounded pool every on-disk block read passes through. Caches parsed objects as well as raw bytes, so a hit skips both the read and the parse.

Column family — An independent ordered keyspace within one database. Has its own levels and compaction policy; shares the memtable, log, value log, and cache with every other family.

Compaction — Merging sstables into fewer, larger, deeper ones so reads consult fewer files.

DCA — The capacity scheme used by the compaction planner, sizing smaller levels as a fraction of how much data the largest level actually holds, so capacities adapt to the data rather than being fixed in advance.

Dividing level — The shallowest level whose merges partition their output by the level below's file boundaries instead of writing one file.

Output size cap — The largest a compaction output may grow before the merge rolls to a new file, derived as a fraction of the largest level's data. It counts key log bytes, so a separated value costs its reference rather than its length.

Flush — Writing a sealed memtable out as sstables. Because the memtable is shared, a flush covers every column family at once.

Immutable memtable — A memtable that has been sealed by a rotation and is awaiting flush. Still readable; no longer written to.

Isolation level — What a transaction's reads may see and what makes its commit fail. Five, from read-uncommitted to serializable.

Key log (klog) — An sstable's file: a btree holding keys and inline values, plus the filter partitions and the footer.

Level — A tier of sstables within a family. L1 holds flush outputs and may overlap; L2 and below are non-overlapping sorted runs. Shallower is newer.

Manifest — The durable catalogue of what exists. Database-level, so a flush across families is atomic. A file it does not name does not exist.

Memtable — The in-memory write buffer. One per database, shared by every family, with keys namespaced by a four-byte family index prefix.

MVCC — Multi-version concurrency control. Writes append new versions rather than overwriting, so readers never block writers.

Partition range filter — A per-sstable bloom filter split by key range, so its resident cost is bounded by the block cache rather than by the entry count.

Prepared transaction — One that has passed phase one of two-phase commit: durably logged, conflict-checked, but not applied and not visible.

Read amplification — How many sstables a point lookup must consult.

Rotation — Sealing the active memtable and installing a fresh one. Runs on the committing thread that filled it.

Sequence — The monotonic number every commit draws. Visibility is a comparison against it.

Snapshot — A sequence ceiling. A version is visible if it committed at or below it.

Spooky — The compaction policy: a pure deterministic planner plus an executor.

SSTable — An immutable sorted run on disk. One key log file, plus references into the shared value log.

Staging ring — The in-memory buffer a buffered block manager stages appends in, so concurrent appenders do not serialize on the file.

Value separation — Storing a value at or above `value_separation_threshold` in the value log and keeping only its id where the key lives. Done by the committing transaction, so the bytes reach the device once rather than once in the write-ahead log and again in the flush.

Tombstone — A version marking a key deleted. Shadows older versions until a merge can prove none survive beneath it.

Value log (vlog) — The database-wide store holding values too large to keep inline, kept as a series of numbered segment files with one taking appends. A value is addressed by an opaque logical id, so a merge carries it forward by reference rather than rewriting it. The exception is a segment marked for draining, whose values the next merge that touches one writes afresh under a new id so the segment can be dropped.

Write amplification — The ratio of bytes written to the device against bytes written by the application.

Write-ahead log (WAL) — The per-generation log a commit reaches before the memtable, so a crash between the two recovers the batch.

References

Work TidesDB builds on directly. Design lineage (ch. 33) sets out what is implemented as published and where the engine departs.

38.1 Compaction

[Spooky] Niv Dayan, Tamar Weiss, Shmuel Dashesky, Michael Pan, Edward Bortnikov, and Moshe Twitto. *Spooky: Granulating LSM-Tree Compactions Correctly.* Proceedings of the VLDB Endowment, 15(11), 2022, pp. 3071–3084. https://vldb.org/pvldb/vol15/p3071-dayan.pdf

Establishes that Full Merge and Partial Merge each fail differently — the first on space amplification, the second on write amplification and SSD garbage collection — and resolves both by partitioning the largest level into equal files and partitioning smaller levels on those boundaries, so one group of perfectly overlapping files merges at a time. TidesDB takes its dividing level and its capacity model from this paper.

[DCA] Siying Dong, Mark Callaghan, Leonidas Galanis, Dhruba Borthakur, Tony Savor, and Michael Strum. *Optimizing Space Amplification in RocksDB.* CIDR, 2017.

Dynamic Capacity Adaptation. Sizes the capacities of levels `1..L-1` from the largest level's actual data size rather than its capacity, bounding durable space amplification to `1/(T-1)`. Predates Spooky, which cites and leverages it — TidesDB implements it in `compaction_planner_capacities`.

38.2 Key/value separation

[WiscKey] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. *WiscKey: Separating Keys from Values in SSD-Conscious Storage.* USENIX FAST '16. https://www.usenix.org/conference/fast16/technical-sessions/presentation/lu

Observes that LSM compaction rewrites values repeatedly although only keys need sorting, and separates them so compaction moves keys alone. Also sets out the costs separation introduces — range queries become random reads, the value log needs its own garbage collection, and crash consistency spans two structures. TidesDB takes the separation and diverges on all three mitigations.

38.3 Logging

[Aether] Ryan Johnson, Ippokratis Pandis, Radu Stoica, Manos Athanassoulis, and Anastasia Ailamaki. *Aether: A Scalable Approach to Logging.* Proceedings of the VLDB Endowment, 3(1), 2010.

Identifies log-space allocation contention and the serialization of many small writes as the scalability limits of write-ahead logging, and answers them with a consolidation array and flush pipelining. TidesDB's buffered append ring has that shape: the ring's single reserving atomic is the consolidation, and its flush thread is the pipelining.

38.4 Supporting

[xxHash] Yann Collet. *xxHash — Extremely fast non-cryptographic hash algorithm.* https://github.com/Cyan4973/xxHash

XXH3 provides the block checksum every framed record carries.

Set in C059, a Century Schoolbook, with verbatim text in DejaVu Sans Mono.

Generated from the manual sources at version 10.0.0 on 2026-08-26.